

BRAMPTON MANOR ACADEMY - 13202

AQA NEA 7517/C

CrowdAvoider

Junaet Chowdhury - 2438

Analysis.....	9
Project Background.....	9
End user.....	10
1. What journey planning tools do you currently use and what are their strengths and weaknesses?.....	10
2. How do you normally plan your journeys and when do you use these apps?.....	11
3. Have you experienced issues traveling during football match days or large events?.....	11
4. How do you currently adjust your journey when you expect crowding?.....	11
5. What information would be useful to you when planning a journey on busy days?.....	11
6. Would qualitative descriptions of crowding be useful?.....	11
7. How important is accuracy compared to simplicity in journey planning?.....	12
8. Would a station-to-station route be more helpful than a point-to-point route?.....	12
9. Which parameters would you like a journey planner to account for when calculating a route?.....	12
10. Would you use a web-based tool that compares normal routes with crowding-aware routes?.....	12
11. Is there anything else you would like included?.....	12
Summary of end user interview:.....	13
Current solutions.....	13
Solution 1 - TfL Journey Planner (TfL Go):.....	13
Pros:.....	13
Cons:.....	14
Solution 2 - Google Maps:.....	16
Pros:.....	16
Cons:.....	16
Solution 3 - Citymapper:.....	17
Pros:.....	17
Cons:.....	18
Summary of Limitations in Current Solutions:.....	20
Target audience.....	20
Project summary.....	20
Relevant research.....	21
Contextual Research:.....	21
Sources:.....	21
Model of crowding decay:.....	22
Shortest path algorithms:.....	22
Methods and sources.....	23
Use of APIs:.....	23

1. TfL Unified API:.....	23
2. OpenCage Geocoding API:.....	23
Further TfL data:.....	24
Table of data:.....	24
Sources:.....	25
Event data:.....	25
Club and stadium data:.....	25
Sources of default parameters and assumptions used:.....	26
End user survey:.....	26
Table of values that can be modified:.....	28
Supporting Research Sources:.....	30
Other parameters:.....	30
Objectives.....	30
User objectives:.....	30
System objectives:.....	32
Proposed solution.....	35
Modeling.....	35
Prototyping.....	37
First prototype's code:.....	37
pathFinding.py:.....	37
testData.py:.....	39
main.py:.....	40
Feedback on first prototype:.....	40
Second prototype's code (that has been changed):.....	41
pathFinding.py:.....	41
main.py:.....	43
Design.....	44
Design elements from other sections.....	44
High level overview.....	44
Overview of programme logic:.....	47
UI Sketches.....	52
Login page:.....	52
Load page:.....	53
Main page:.....	54
Update page:.....	55
Database error page:.....	56
Data.....	57
SQL:.....	57
Create (Insert):.....	57

Read (Selected):.....	58
Update:.....	58
Delete:.....	59
Data flow diagram:.....	59
Data structures used:.....	62
1. Graph (adjacency list):.....	62
2. Network object:.....	62
3. Priority queue (min heap):.....	63
4. Visited set:.....	63
5. Distances dictionary:.....	63
6. Results object:.....	63
7. Events dictionary:.....	63
Algorithms.....	63
Graph creation:.....	63
Modified Dijkstra's shortest path algorithm:.....	64
Relevant events filtering:.....	65
Crowded network creation algorithm:.....	66
Affected graph generation:.....	71
Crowding severity classification:.....	71
Libraries and languages.....	72
Languages:.....	72
Libraries:.....	72
1. Flask.....	73
2. SQLite.....	73
3. OpenCage Geocoder.....	73
4. Requests.....	73
5. BeautifulSoup.....	73
6. Geopy.....	73
7. Heapq.....	73
8. Abc.....	73
9. Hashlib.....	74
10. Threading.....	74
11. JSON.....	74
12. Datetime.....	74
13. Math.....	74
14. Collections.....	74
15. Copy.....	74
16. Time.....	74
17. Re.....	75

18. Os.....	75
19. Pathlib.....	75
20. jQuery.....	75
21. Select2.....	75
Security.....	75
Login system.....	75
Integrity of data.....	75
Technical Solution.....	77
System overview.....	77
Technical breakdown.....	77
Database component:.....	77
Data Acquisition Component.....	79
Pathfinding Component.....	80
Crowding Algorithm Component:.....	80
Results Component.....	81
Display Component:.....	82
Key code.....	83
BBC Sport event scraper:.....	83
Venue-station relationship mapping:.....	87
TfL network data acquisition:.....	91
Crowding Simulation Algorithm:.....	91
Modified Dijkstra's algorithm with line-change penalty:.....	91
Complex programming techniques.....	91
Complex data model in database:.....	91
Hash Tables, Lists, Stacks, Queues, Graphs, Trees:.....	93
Graph:.....	93
Hash Table:.....	93
Priority Queue:.....	94
Cross-table parameterised SQL:.....	95
Complex User-Defined Use of Object-Oriented Programming:.....	95
Recursive Algorithms:.....	97
Graph Traversal:.....	98
List Operations:.....	98
Queue Operations:.....	99
Hashing:.....	100
Complex Scientific/Mathematical Model:.....	100
Complex User-Defined Algorithms:.....	101
Modified Dijkstra's Algorithm with Line-Change Penalty:.....	101
Crowding Simulation Algorithm:.....	102

Server-Side Scripting Using Request/Response Objects:.....	113
Dynamic Generation of Objects Based on Complex OOP Model:.....	116
Calling Parameterised Web Service APIs and Parsing JSON:.....	117
Defensive programming.....	122
Parameterised Database Queries:.....	122
Secure Password Storage:.....	122
Session-Based Authentication:.....	123
Input Validation:.....	123
Exception Handling:.....	124
API Retry Logic:.....	125
Error Propagation:.....	126
Defensive Database Writes:.....	127
Abstract Base Classes:.....	129
Database Normalisation:.....	130
Mathematical Boundary Guards:.....	132
Recursion Termination:.....	132
Guaranteed Minimum Result:.....	133
Safe Application Startup:.....	133
User Data Preservation During Database Refresh:.....	134
Coding style.....	135
Modules (subroutines) with appropriate interfaces:.....	135
Loosely coupled modules (subroutines) – module code interacts with other parts of the program through its interface only.....	135
Cohesive modules (subroutines) – module code does just one thing.....	135
Modules (collections of subroutines) – subroutines with common purpose grouped.....	138
Defensive programming.....	139
Good exception handling.....	139
Well-designed user interface.....	139
Modularisation of code.....	139
Good use of local variables.....	139
Minimal use of global variables.....	140
Managed casting of types.....	140
Use of constants.....	141
Appropriate indentation.....	142
Self-documenting code.....	142
Consistent style throughout.....	142
File paths parameterised.....	142
Meaningful identifier names.....	143

Annotation used effectively where required.....	143
Testing.....	144
Test strategy.....	144
Test plan.....	144
Login page.....	145
Loading page and database creation.....	146
Main journey page.....	147
Settings page.....	148
Testing evidence.....	150
Unit testing.....	150
Integration testing.....	177
System testing.....	178
Acceptance testing.....	179
Evaluation.....	180
Objectives.....	180
User objectives:.....	180
System objectives:.....	182
End user and target audience.....	184
End user.....	184
Target audience.....	185
Areas for improvement.....	186
Areas for future development.....	187
Conclusion.....	188
Appendices.....	191
Annotated code.....	191
dbError.css.....	191
loading.css.....	192
login.css.....	192
mainPageStyle.css.....	194
update.css.....	196
dbError.js.....	197
loadingScript.js.....	198
mainPageScript.js.....	198
updatePage.js.....	200
databaseError.html.....	202
loading.html.....	202
login.html.....	203
mainPage.html.....	204
updatePage.html.....	207

defaultParameters.json.....	209
leagues.json.....	210
lines.json.....	210
londonClubs.json.....	210
accountClasses.py.....	211
classes.py.....	212
crowdingAlgorithm.py.....	212
customFunctions.py.....	220
databaseCreation.py.....	222
getResults.py.....	225
getTfLData.py.....	227
login.py.....	231
main.py.....	233
networkClasses.py.....	244
pathFinding.py.....	245
scraper.py.....	246
scraperClasses.py.....	249
storeStadiumData.py.....	251
transportClasses.py.....	254
websiteDataClasses.py.....	257

Analysis

Project Background

One of the most significant causes of delays, disruption, and overcrowding on urban transportation networks is large public events, the biggest of which tend to be football matches. These events lead to sudden increases in passenger numbers at nearby stations, building up to, and right after kick-off and end of the game. The naturally resulting overcrowding can lead to slow boarding and alighting, dangerous conditions, and general discomfort for passengers, which propagates across connected services throughout the network. Despite this, most journey planning services don't account for the impact of such scheduled public events when estimating journey times and future busyness.

This issue is especially relevant in London, which aside from being one of the most important world cities, and the official capital of the UK, a G7 country, it can also be considered its transportation and sporting capital. London is home to a large number of professional football clubs in the Premier League, the richest football league in the world, as well as across the English football pyramid, some of whose leagues are more popular than certain national leagues. With several teams regularly, and successfully competing in European and international tournaments, such as the UEFA Champions League, and the Club World Cup, London is also an international hub for sports. Major stadiums such as Wembley Stadium, the Emirates Stadium, Tottenham Hotspur Stadium, Stamford Bridge, and the London Stadium regularly host frequent and high profile fixtures, attracting crowds not only from the city, but nationally and internationally. Many of these thousands of spectators rely on public transport, leading to significant and predictable surges in demand.

Transport for London (TfL) is one of the most extensive and sophisticated urban transport networks in the world, integrating underground, rail, bus, and light rail services across the city. The network is usually very reliable and capable of handling large volumes of passengers. However, like all urban transport systems and networks, it remains susceptible to overcrowding when demand increases sharply, over a short period of time. Event-induced crowding especially puts strain on stations and lines serving major venues, whose effects can cascade across the entire network.

Existing transport or general navigation applications may display qualitative information about crowding, such as “Usually a little busy” or “Not busy”, but these are usually based on historical trends, such as typical weekday or weekend demand. They are not designed to predict crowding caused by specific events that are known in advance, nor do they adjust journey times to reflect the delays that this crowding is likely to cause. As a result, users may be directed along routes that are optimal in theory and in ideal conditions, but inefficient in reality due to event-induced congestion.

This project aims to address this limitation by modelling how large public events influence passenger demand across the transport network and the resulting effects, rather than solely relying on historical trends. Known events occurring on the day of travel are analysed to determine whether they are likely to affect the journey, based on their timing relative to

the user's route and time of travel. The model estimates how passenger numbers increase around affected stations and how this demand spreads through the network, allowing delays caused by overcrowding to be approximated.

These estimated delays then influence adjusted travel times between stations, producing a modified version of the transport network that accounts for event-induced disruption. By recalculating routes using this adjusted network, the system is able to suggest journey options that better reflect the conditions passengers are likely to experience in practice, rather than assuming ideal operating conditions.

By allowing users to compare a standard shortest route (with realistic total journey time) with an alternative route that accounts for predicted congestion, the system provides more practical journey information for passengers travelling on tight schedules, such as commuters and students. Instead of assuming ideal operating conditions, the project shows how modelling passenger demand and its effect on travel times can lead to journey estimates that better reflect real world conditions within a busy transportation network. By allowing users to also change the parameters used in the calculation of crowding and journey times, like how much one extra person quantitatively contributes to delay, they can tweak their preference of how much crowding they are willing to endure, against the extra time a quieter route might take, which is especially useful for users seeking a more comfortable and quiet journey.

End user

A chemistry teacher at my school, Ms Miah, who regularly commutes to and from school across London, asked me to develop a journey planning tool that better accounts for the delays caused by large public events, particularly football matches. She explained to me that existing journey planners often underestimate travel times on match days, making it difficult to plan journeys reliably and forcing commuters to deal unexpectedly with large amounts of crowding on trains and in stations due to these events, making a stressful day of work even more exhausting. Upon receiving this request, I asked her several questions in person, which I recorded to better understand her needs and how the system could support her travel.

1. What journey planning tools do you currently use and what are their strengths and weaknesses?

The most reliable and all-round navigation app I use is Google Maps, which is helpful for basic route planning. I've also recently started using TfL Go, which is helpful to know if there are any disruptions on any train lines on the way to work. Despite these benefits, they lack one critical thing, which is accounting for how busy stations can get on event or match days. Sometimes they'll say a route is fastest, but in reality it's overcrowded, overstimulating and much slower.

2. How do you normally plan your journeys and when do you use these apps?

When planning journeys I usually just put in my start location, destination and time of journey, and pick the first option. Sometimes I only look at the route a couple of days before I travel, so any disruptions that appear later can come as a surprise. Even if I look at a route a couple of hours before I start to travel, it doesn't tell me if there are any disruptions due to scheduled events, which have been scheduled for months by then. Since I already know what is generally the fastest way to work, I usually don't need to check my journey in the mornings as Google Maps always gives me the same route, but sometimes I get to the station and notice on the status board that there are severe delays, so I try to look for another route. However, by then, I'm already running late. If Google Maps had a way of telling me if there would be delays the next day, I would leave earlier, or look for another way to work. I've begun using TfL Go and Citymapper to check the statuses of train lines when I wake up, but they only tell me about disruptions due to engineering works or incidents at stations or on trains. Aside from work, when I travel for other purposes, there isn't really a requirement for me to arrive somewhere on time, but I would love it if there was a way for me to avoid crowding.

3. Have you experienced issues traveling during football match days or large events?

Yes, especially when travelling near stadiums like the Emirates or Stamford Bridge. Trains and stations become extremely busy before and after matches, and even if there aren't official delays, everything just takes longer. The journey planners don't really reflect this, especially ahead of time.

4. How do you currently adjust your journey when you expect crowding?

I usually leave earlier than normal or try to avoid certain stations, but it's mostly guesswork. There's no clear information that tells you how much extra time you should allow or whether a slightly longer route would actually be quicker in practice. Also, there is no way to know if the journey would be comfortable or not.

5. What information would be useful to you when planning a journey on busy days?

I think I would want a more realistic journey time that includes the effects of crowding, along with some indication of how busy stations are expected to be. I really hate overcrowding, so I would love it if I could know if there would be a crowd of fans at my destination and origin, that I would need to mentally prepare myself for, because I tend to get overstimulated by them, especially large groups of football supporters. Being able to compare a route with a less crowded alternative would also be really helpful.

6. Would qualitative descriptions of crowding be useful?

Yes, definitely. Even if it's not exact, knowing whether a station is expected to be very busy would help me decide whether to avoid it, especially if I'm in a rush. Being given raw numbers about how much crowding is expected would not be as useful because

it would be hard to read and slightly overstimulating, which is exactly what I'm trying to avoid when I'm tired in the morning and after work.

7. How important is accuracy compared to simplicity in journey planning?

I do understand that journey times can only be so realistic, so I would prefer if the predictions were slightly more pessimistic (in that the predicted journey time would be slower than reality) because it is almost always better to be safer than sorry. Also, what I consider to be a lot of crowding, might not be for other people, like you, so having a way to change parameters, such as how many people is needed for a station to be considered really busy, would be really helpful and nice for personalisation purposes.

8. Would a station-to-station route be more helpful than a point-to-point route?

The second closest station to me is a forty minute walk away, which under almost no circumstances would I choose, and the second closest station to the school is too far away to be worth it as an alternative route, so I would prefer a station-to-station journey planner. Also, traveling so far across London, taking other forms of transport other than trains, like buses, wouldn't be very helpful, as I would almost always pick a more crowded train over a long and slow bus ride. I think a simple UI and results would be the best so it gets the necessary information across, and nothing more to not be too overstimulating.

9. Which parameters would you like a journey planner to account for when calculating a route?

I think as I've mentioned before, I would like one that mainly just focuses on potential scheduled events which might impact journey times and how comfortable a journey may be. I noticed that of all the events, football games tend to be the most disruptive, so I would like a focus on them. When I pass through the stations to and from work, if there are engineering works coming up, we are told about them long in advance, so that isn't really important. However, one big thing is that different lines have different speeds, for example the same stretch on the DLR is much slower than if I were on the Jubilee line, so I would like the journey planner to not just account for the number of stops, but also how long the train on that line would take.

10. Would you use a web-based tool that compares normal routes with crowding-aware routes?

Yes, especially on days when there are football matches or big events. I think it would be most useful for students and commuters who need to arrive on time and can't afford unexpected delays.

11. Is there anything else you would like included?

Yes, one thing that I would like is a way to save my parameters to my own account. I share my home computer with my husband, who has different travel requirements, so having to change all the parameters every time either of us use the journey

planner after the other would be very inconvenient, and therefore a way to save them to our own accounts would be really helpful.

Summary of end user interview:

After conducting a lengthy in person interview with Ms Miah, here is a broad and simple summary of what she would like in a potential solution:

1. A journey planner which finds the fastest route from Station A to Station B.
2. The journey planner should also give another route which accounts for crowding-induced delays.
 - a. This journey planner should maintain a database of events coming up, especially football matches.
 - b. Stations closer to a stadium should be busier than others.
 - c. The route should overestimate travel times, rather than underestimate them.
 - d. Different train lines have different speeds.
3. Qualitative descriptions of crowding at stations the user will pass through.
4. A way for users to adjust the parameters used in route calculations.
5. A login system, which should ideally be somewhat secure, although as the programme wouldn't store sensitive data, it isn't a big requirement.
6. A simple UI delivering the information the user needs, in a clear manner.

The interview also highlighted the benefits and negatives of different journey planning software she uses, so the solution should take these comments into consideration.

Current solutions

Solution 1 - TfL Journey Planner (TfL Go):

Transport for London provides its own journey planning service through the TfL Journey Planner and the TfL Go mobile application. This service allows users to plan journeys across multiple TfL services, including buses and rail. It also provides live service updates, line status information, and details of planned engineering works or disruptions.

The system benefits from being directly operated by TfL, meaning it has access to accurate real-time operational data. Users can view current delays, service suspensions, and the stated reasons for disruption on individual lines. This information is also displayed on status boards at stations.

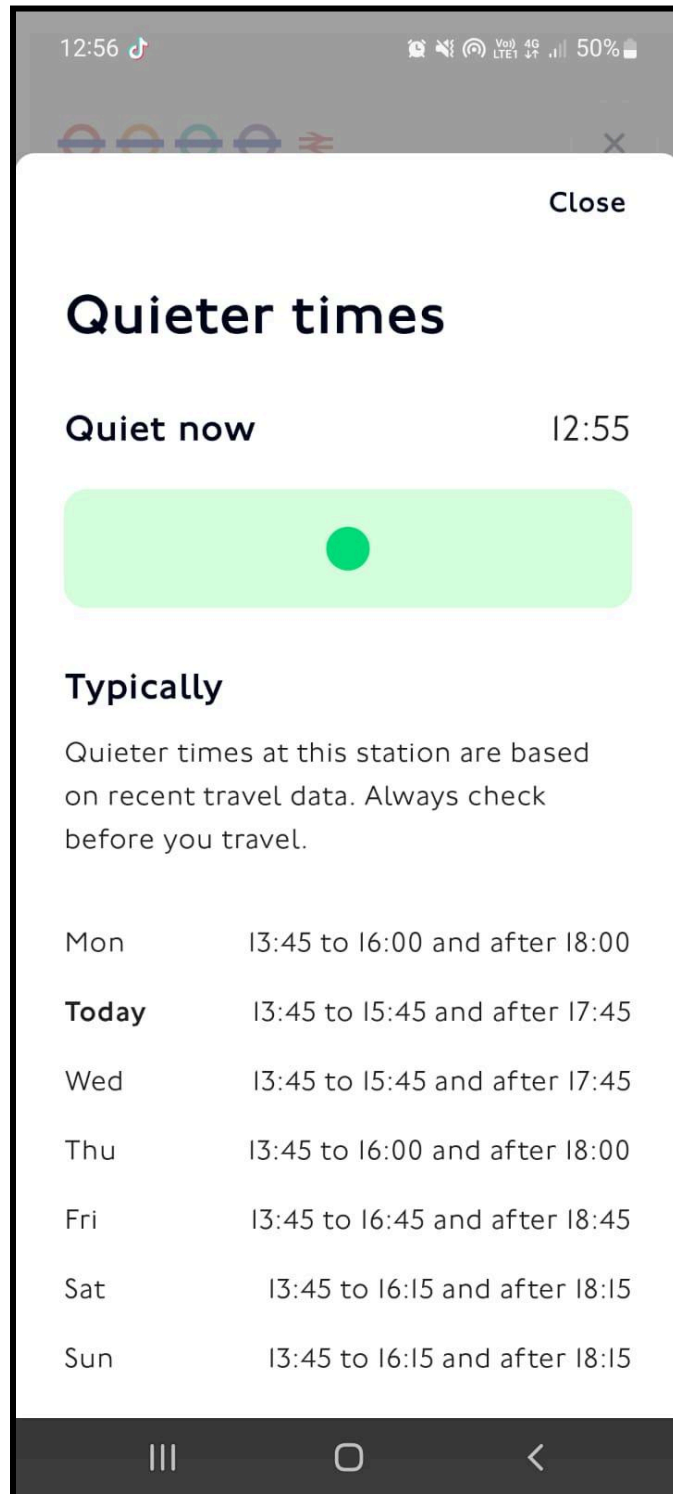
Pros:

- Provides reliable live service updates sourced directly from TfL.
- Covers all TfL services, including rail, in a single platform.
- Allows users to plan routes between stations.
- Displays scheduled future engineering works and major disruptions, known to TfL.
- Provides guidance for each station indicating when it is expected to be less busy.

- Provides a qualitative description of station busyness and expected quieter times based on historical trends.

Cons:

- Information is spread across multiple pages, making the service difficult to navigate quickly.
- Does not quantify how much additional journey time a future disruption or event might cause.
- Crowding-related delays caused by non-TfL events (e.g. football matches) are rarely accounted for, especially on rail services.
- Doesn't allow users to compare alternative routes that explicitly avoid predicted crowding.



A qualitative description of the busyness of a station, and when quiet times are expected on a given day of the week, based on previous trends

Overall, while TfL's journey planner is effective for identifying live disruptions, it provides limited predictive insight into how crowding from future events may affect journey times.

Solution 2 – Google Maps:

Google Maps is one of the most widely used route planning services globally and is commonly used by London commuters. It allows users to plan journeys using multiple transport modes, including walking and TfL rail services. For public transport, it provides estimated journey times, live service updates where available, and forecasts of station busyness.

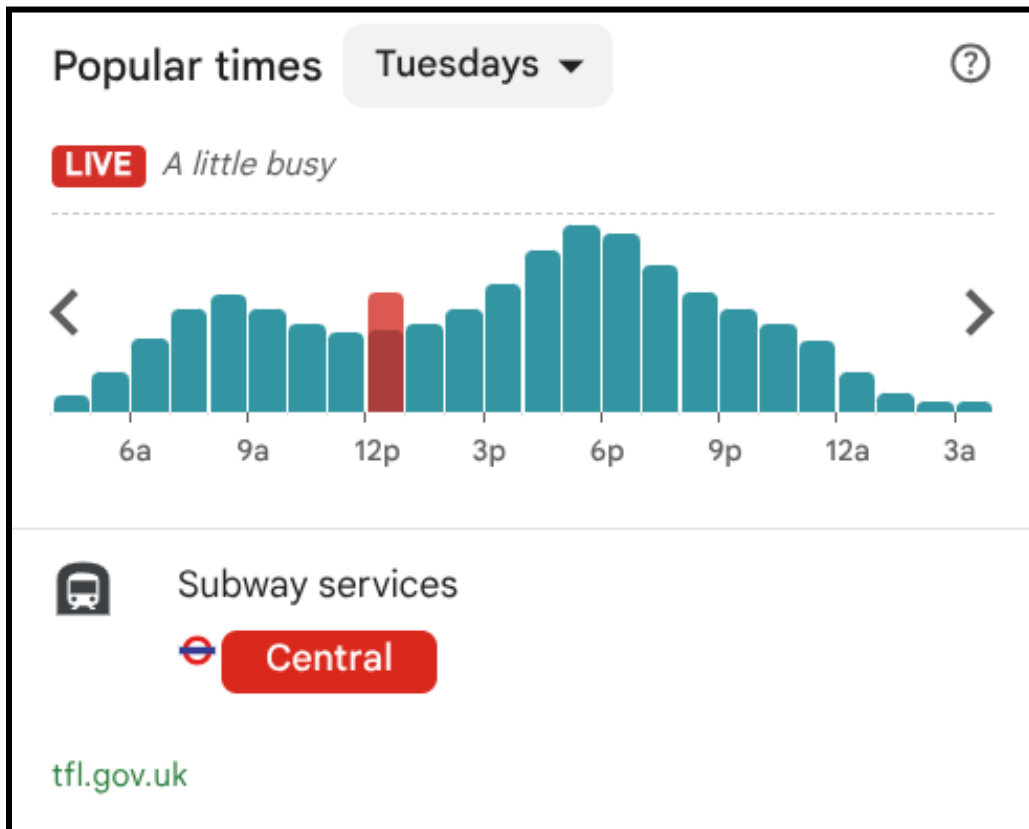
Google Maps also offers multiple route options for the same journey, allowing users to compare speed and convenience. In addition, it includes advanced traffic prediction for road travel, which incorporates congestion caused by traffic jams.

Pros:

- Supports multiple transport modes, including TfL rail services.
- Offers multiple route options for a single journey.
- Provides forecasts of station busyness.
- Displays live delays where available.
- Highlights where delays may occur along a route.

Cons:

- Traffic prediction is primarily focused on road travel, not rail-based public transport.
- Busyness forecasts are trend-based (time of day and day of week) rather than event based.
- Doesn't account for specific public events such as football matches or concerts.
- Crowd related delays are not quantified into additional journey time.
- Busyness is shown only at individual stations, rather than modelling its impact across an entire line.
- No option to request routes that minimise predicted crowding.
- No feature to compare a “normal” journey with a crowding affected alternative.



Shows the expected busyness at specific times of the day based on historical footfall data.

While Google Maps is highly effective for general navigation, its crowding predictions lack the specificity required for accurate journey planning during major events.

Solution 3 – Citymapper:

Citymapper is a public transport focused journey planning application designed for use in major cities, including London. It provides detailed route planning for a range of transport options, including TfL rail services, along with live service updates and disruption alerts. Citymapper is known for its clear interface and user-friendly presentation of route options.

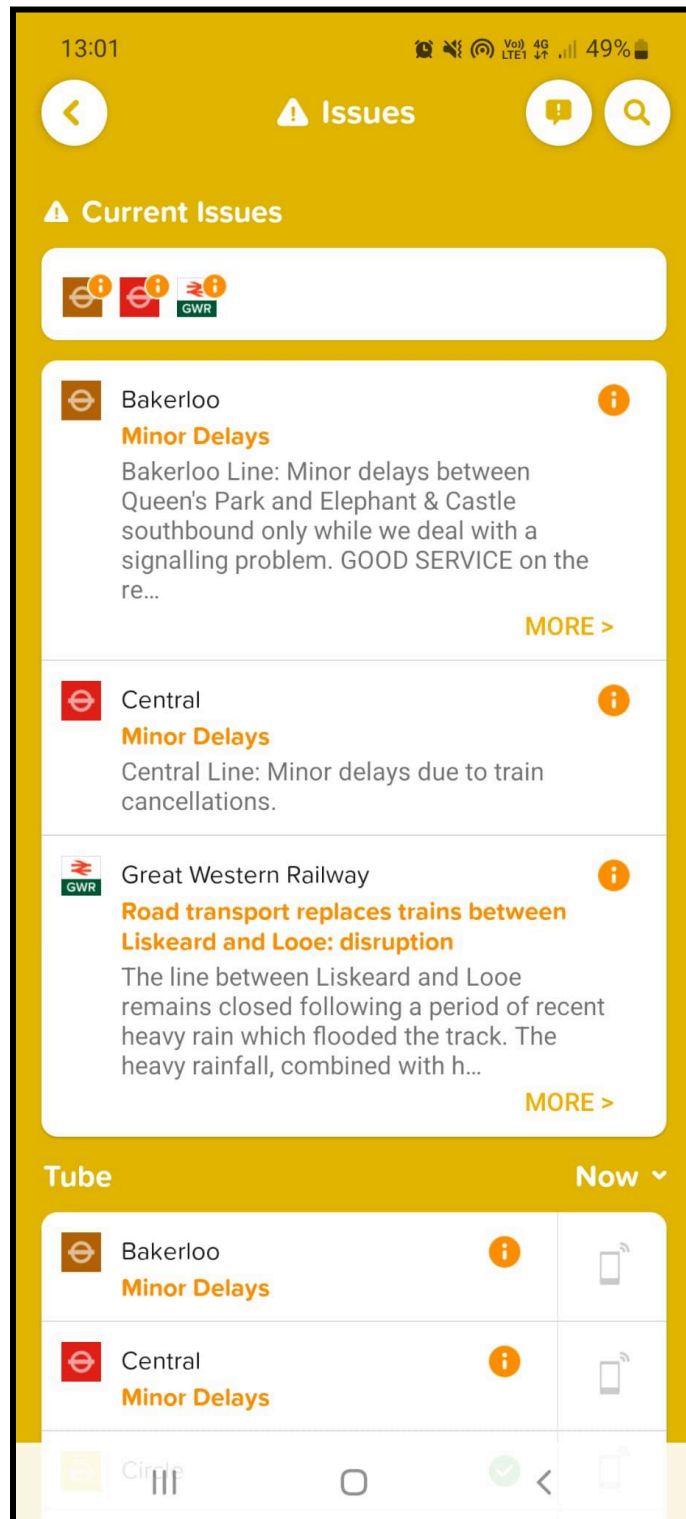
The application also offers multiple route choices and highlights disruptions in real time, making it useful for day-to-day commuting.

Pros:

- Clear and intuitive user interface.
- Accurate live service updates.
- Provides multiple route options for comparison.
- Designed specifically for urban public transport systems, making it tailored to the target audience.

Cons:

- Crowd predictions are reactive rather than predictive.
- Doesn't model the impact of future public events on crowding.
- Doesn't quantify how crowding affects journey duration.
- No functionality to request routes that explicitly avoid predicted congestion.
- Limited insight into how delays at one station affect the wider network.



Notifications about the status of different train lines.

Citymapper is effective for real-time navigation but, like other existing solutions, lacks forward-looking, event-driven crowd modelling.

Summary of Limitations in Current Solutions:

All three existing solutions provide strong live updates and efficient route-finding. However, they share common limitations when it comes to predicting and quantifying crowd-related delays on TfL rail services. In particular:

- Crowd predictions are trend-based rather than event-driven.
- The effect of crowding on overall journey time is not clearly quantified.
- Users cannot easily compare a standard route with a crowding-aware alternative.
- Future disruptions caused by local events are rarely modelled in detail.

These limitations highlight the need for a system that incorporates event-driven crowding predictions and translates qualitative crowding descriptions into practical journey time estimates, which is the aim of this project.

Target audience

The target audience of this system are people who regularly use London's public transport network, specifically TfL rail services, to travel to a specific destination within a limited time frame. This includes commuters, students, and other passengers who rely on journey planners to make time-critical travel decisions.

Unlike casual or leisure travellers, these users are usually more affected by unexpected delays caused by congestion, as even small increases in journey time can have a significant impact on their schedule, especially for those who leave little margin for error when it comes to lateness. Existing journey planning tools typically assume ideal conditions and do not account for sudden surges in passenger numbers caused by large public events, such as football matches. As a result, end users may be directed along routes that appear optimal in theory but are inefficient in practice due to overcrowding. Additionally, there is usually no option to search for routes based on crowding for those wishing for a more comfortable journey.

Project summary

This project is a web-based journey planning system for London's TfL rail network that aims to address the limitation of existing journey planners, which usually at best rely on historical trends and fail to account for congestion caused by specific future events such as football matches. The system uses football fixture data to model how the crowding due to these events impact stations and journey times, which allows users to compare a normal shortest-time route with one that accounts for crowding. At its core, the network is represented as a graph and a modified Dijkstra's algorithm is used to calculate routes while adding penalties for line changes and dynamically adjusting edge weights based on factors such as crowding-induced delays and the average speed of lines. A crowding model estimates additional passenger demand near event venues, distributes the demand using inverse-distance weighting to the nearest stations to the venue, and spreads it across the

network using a recursive propagation algorithm. Gaussian modelling is used to simulate the build up and dispersal of crowds over time, with passenger loads converted into station and line delay factors that affect travel times and informs users about how much crowding they should expect at a station. The system is delivered through a Flask web interface with user authentication, and it allows users to customise model parameters such as attendance rates and crowd propagation.

Relevant research

Contextual Research:

While there are many causes of crowding, scheduled sporting events, and especially football matches, are among the most predictable and intense surges in travel demand. London hosts dozens of professional sports events per month, which attract thousands of spectators (source I). TfL (Transport for London) themselves estimate that on a typical matchday, trains carry tens of thousands of additional passengers to and from stadium areas, with around 60-80% of matchgoers arriving by rail, rather than other modes such as cars. Unlike daily commuter peaks, these surges are highly concentrated in space and time, with most fans arriving in a narrow window before kick-off and departing after the final whistle. This leads to very high localised crowding at specific stations.

Crowding on dense urban rail networks occurs when demand approaches or exceeds the carrying capacity of trains or stations. In such systems, high passenger volumes create congestion that impacts both operational performance and passenger experience. This is due to the fact that:

- Crowding increases the time trains spend at stations because it takes longer for passengers to board and alight when large groups cluster around doors and narrow areas of the platform. This directly slows down train movements and can lead to cascading effects across the network (source II).
- Extremely crowded platforms, trains and passageways increase the risk of safety incidents (source III)
- Beyond operational performance, crowding not only has a negative impact on the quality of the journey, but also increases the perceived journey time for passengers. This has been demonstrated in studies conducted in London, which show that crowding reduces passenger comfort and satisfaction. (source IV)

Sources:

- I. <https://tfl.gov.uk/corporate/transparency/freedom-of-information/foi-request-detail?referenceId=FOI-2371-2324>
- II. <https://wp.wpi.edu/london/projects/2016-projects-summer/underground/>
- III. <https://publications.parliament.uk/pa/cm200203/cmselect/cmtran/201/201we20.htm>
- IV. <https://link.springer.com/article/10.1007/s11116-023-10420-1>

Model of crowding decay:

A Gaussian curve is used to model the distribution of passengers arriving and leaving around football matches because it realistically represents how crowds gradually build up before an event and disperse afterwards. Unlike uniform or linear models, which assume passengers arrive and depart at a constant rate, the Gaussian reflects the natural variability in behavior, with most people arriving near the peak time and fewer arriving very early or very late. In the system, the curve is centred around the estimated peak arrival or departure time, and the number of passengers at each station is scaled according to the station's distance from the venue, using inverse distance weighting.

Shortest path algorithms:

Dijkstra's algorithm was chosen over a heuristic-based alternative such as A* despite them being theoretically faster, because the TfL network's graph is generated dynamically at runtime using data fetched from the TfL API. The structure, connections, and edge weights of the graph are not fixed in advance and can change depending on external data and user-defined parameters, such as crowding and delay factors. While A* relies on a heuristic function, which is a function that estimates the cost from any node to the destination, typically based on simplified information, such as direct geographical distance, defining a reliable and useful heuristic is challenging in this context because travel time is not solely dependent on geographical distance, but is heavily influenced by dynamic factors, such as train speeds, line-wide delays and penalties for line changes. In Dijkstra's shortest path algorithm however, it does not require prior knowledge of the graph's structure or a heuristic estimate, and guarantees the optimal shortest path based purely on the current weights of the edges. This makes it more suitable for a dynamically generated, and frequently modified graph, where edge costs can change significantly.

Below is a pseudocode representation of Dijkstra's shortest path algorithm:

```

DIJKSTRA(Graph, startNode, targetNode)

  FOR each node IN Graph
    distance[node] ← ∞
    previous[node] ← null
  END FOR

  distance[startNode] ← 0

  unvisited ← set of all nodes in Graph

  WHILE unvisited is not empty

    current ← node in unvisited with smallest distance value
    REMOVE current from unvisited

    IF current = targetNode

```

```

    BREAK
  END IF

  FOR each edge connecting current to neighbour

    travelTime ← edge.weight
    newDistance ← distance[current] + travelTime

    IF newDistance < distance[neighbour]
      distance[neighbour] ← newDistance
      previous[neighbour] ← current
    END IF

  END FOR

END WHILE

path ← empty list
node ← targetNode

WHILE node is not null
  INSERT node at start of path
  node ← previous[node]
END WHILE

RETURN path, distance[targetNode]

```

Methods and sources

All external sources used in the projects with links in the document have been accessed in January 2026.

Use of APIs:

1. TfL Unified API:

The TfL Unified API was used as the primary source of data regarding London's transport network, because it provides up-to-date, accurate, and complete information about all London stations, lines, and connections. This allowed the network graph to be generated dynamically and reflected real-world distances and travel times, which would not have been possible using static or secondary datasets. Using the official API improves reliability and removes the need for assumptions about station locations or line structures. Additionally, real-time data is not required for this project, which means the 500 requests per minute call limit is enough for the scope of the project.

2. OpenCage Geocoding API:

The OpenCage Geocoding API was used to obtain the geographic coordinates of stadiums and stations. It was chosen over alternatives such as Google Maps because it provides reliable and accurate location data, while being free to use, making it suitable for the scope and budget of the project (£0.00). Additionally, the free plan's API call limits (2500 per day) were more than enough, considering the fact that the

stadiums would only need to be updated, at most, once a year, when new teams are promoted into the EFL League Two, from London.

Further TfL data:

Aside from the TfL data gathered from the API, the average speeds of the different train lines were also collected. Below is a table of the train lines, their average speeds, and the sources used.

Table of data:

Line name:	Average speed (km/h):	Source:
Bakerloo	27.04	Source A
Central	37.27	Source A
Circle	23.73	Source A
District	29.19	Source A
Hammersmith & City	25.33	Source A
Jubilee	39.11	Source A
Metropolitan	45.61	Source A
Northern	33.28	Source A
Piccadilly	33.11	Source A
Victoria	40.66	Source A
Waterloo & City	33.30	Source A
DLR (All branches)	64.00	Source B
Elizabeth	51.70	Source C
Liberty	36.53	Source A
Lioness	33.80	Source A
Mildmay	26.07	Source A
Windrush	27.36	Source A
Weaver	32.19	Source A

Suffragette	32.19	Source A
-------------	-------	----------

Sources:

- A. <https://tfl.gov.uk/corporate/transparency/freedom-of-information/foi-request-detail?referenceId=FOI-0228-1819>
 - a. Since the FOI request, the individual branches of the London Overground have been upgraded to independent lines, hence the name change on the table.
- B. <https://www.independent.co.uk/travel/news-and-advice/london-dlr-trains-speed-restrictions-b2613480.html>
- C. <https://tfl.gov.uk/corporate/transparency/freedom-of-information/foi-request-detail?referenceId=FOI-1394-2223>

Event data:

A scraper was implemented to collect football match data, including dates, home and away teams, and kick-off times, for the next twelve months. It works by fetching and parsing HTML pages from BBC Sport using the BeautifulSoup library, identifying the relevant data headers and fixture information, and storing the results in structured Python objects and in a database. BBC Sport was chosen as the source of fixture data because its website is well structured and consistent, making it relatively easy to scrape. The HTML markup clearly separates dates and fixture information, which allows for reliable extraction using automated scripts. BBC Sport also provides detailed and up-to-date coverage across multiple leagues, meaning only one data source is required. Furthermore, their robots.txt file does not restrict access to fixture pages, making automated data collection both practical and permissible. The program stores the name of each of the clubs to be considered (all London-based clubs in the professional English leagues) in a JSON file so that they can be matched when scraping. This means the names stored are the names used by BBC Sport.

Club and stadium data:

The capacity of each stadium that hosts a professional English team (teams from the Premier League, Championship, EFL League One, and EFL League Two), were gathered from the Football Ground Guide website (<https://footballgroundguide.com/>). The names of the stadiums stored in the JSON file used by the program, may be slightly different to the one on the website as the OpenCage Geocoding API may use those names. Below is a table of the names and capacities of the stadiums, as used in the program.

Club name:	Stadium name:	Capacity:
Arsenal	Emirates Stadium	60704

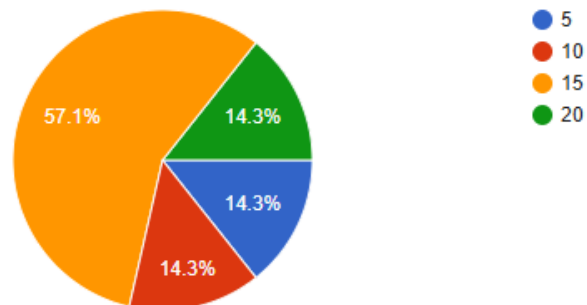
Brentford	Brentford Community Stadium	18250
Chelsea	Stamford Bridge	40343
Crystal Palace	Selhurst Park	25486
Fulham	Craven Cottage	29600
Tottenham Hotspur	Tottenham Hotspur Stadium	62850
West Ham United	London Stadium	62500
Charlton Athletic	The Valley, London	27111
Millwall	The Den	20146
Queens Park Rangers	Loftus Road Stadium	18439
AFC Wimbledon	Plough Lane Stadium	9300
Leyton Orient	Brisbane Road Stadium	9271
Barnet	The Hive Stadium	5100
Bromley	Hayes Lane Stadium	5150

Sources of default parameters and assumptions used:

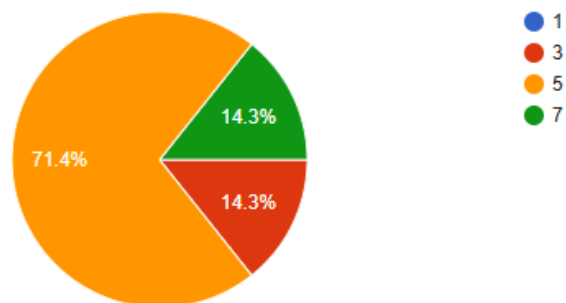
End user survey:

As some of the parameters used are subjective, an end user questionnaire was used to come up with default values. Additionally, values like a propagation factor (percentage of people who get off the train, with every station travelled from the station) don't have concrete sources, meaning this survey was used to gather anecdotal evidence. To do this, I asked Ms Miah to distribute a questionnaire to her sixth form chemistry classes (75 pupils in total) to gather data on the following questions, as a lot of them commute via TfL rail services to school (with the results below):

What is a reasonable amount of time to walk to a station (minutes)?



What is your average walking speed (km/h)?



What proportion of people get off from the train after getting on from a station serving a stadium, per station travelled from the stadium?

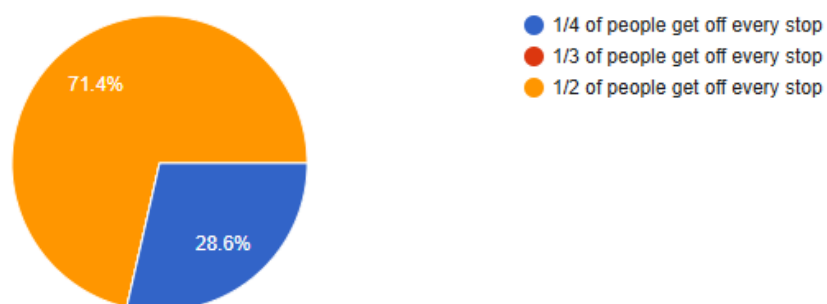


Table of values that can be modified:

Name of parameter:	Value used:	Description:	Justification:
CHANGING_TIME	4.4	Extra time added to each journey (minutes)	Each time a passenger switches train lines, it takes some time for the passenger to get to the next platform and board a new train. The source used to gather this time is Source 1.
MAX_TIME_WINDOW	120	Maximum time from the event peak to consider crowding (minutes)	Beyond a certain time before or after the peak point of crowding, the crowd dies down to the point that it is negligible, or non-existent. At the edge of the time window, the amount of extra people is practically 0. The source used to calculate this time is source 2.
rawArrivalPeakOffset	25	Minutes from event start when crowding peaks on arrival	Crowds typically build up shortly before kick-off, and not at kick-off. The arrival and departure offsets are stored separately because it was initially assumed that they would be different. However, upon further research, it seems that both of the values are the same. The source used for this number is source 3.
rawDeparturePeakOffset	25	Minutes from event end when crowding peaks on departure	Crowds begin to disperse a little after the final whistle, not right at the final whistle. The arrival and

			departure offsets are stored separately because it was initially assumed that they would be different. However, upon further research, it seems that both of the values are the same. The source used for this is source 3.
ATTENDANCE	0.95	Proportion of stadium capacity attending (0-1)	Stadiums tend to operate close to full capacity, which is reflected in the fact that tickets are generally almost always sold out, and their prices increased. However, as it is rare that all those with tickets will attend games, an average number was calculated using the data from source 4.
TRAIN_PROPORTION	0.75	Proportion using TfL rail services (0-1)	This number was gathered from source 5.
SIGMA_FACTOR	4	Controls how tightly people cluster around the peak time — it converts a time window into a Gaussian spread	A sigma value equal to roughly a quarter of the total time window produces a realistic gradual rise and fall rather than a sharp spike, which more accurately reflects natural behavior.
MINIMUM_PEOPLE	50	Minimum people threshold for affecting stations	Small numbers of additional passengers do not measurably affect dwell times or flow.
PERSON_DELAY	0.01	Additional delay per passenger on the journey (minutes)	Calculated using source 6.

PROPAGATION_FACTOR	0.5	Factor for spreading delay across network	Derived from end-user survey responses
WALKING_TIME	15	Default walking time between station and venue (minutes)	Derived from end-user survey responses
WALKING_SPEED	5	Default walking speed (km/h)	Derived from end-user survey responses
NORMAL	1	Crowding thresholds for classification: normal (below this is normal)	N/A
SLIGHTLY	1.5	Crowding thresholds for classification: slightly busy (below this is slightly busy)	N/A
BUSY	2	Crowding thresholds for classification: busy (below this is busy and above is severely busy)	N/A

Supporting Research Sources:

1. <https://www.sciencedirect.com/science/article/pii/S1077291X24000092>
2. https://ops-dr.fhwa.dot.gov/publications/fhwaop04010/chapter5_03.htm#feasibility
3. https://docs.planning.org.uk/20210823/129/DCAPR_156571/s90x7gwknzo7yetv.pdf
4. <https://www.givemesport.com/how-full-each-202425-premier-league-stadium-is-on-match-days/>
5. https://tfl.gov.uk/corporate/transparency/freedom-of-information/foi-request-detail?referenceId=FOI-2371-2324&utm_source=chatgpt.com
6. https://www.scipedia.com/public/Palmqvist_et_al_2019a

Other parameters:

1. The dwell time for a train is assumed to be 0.5 minutes.
 - a. <https://content.tfl.gov.uk/rfli-ccos-tpr2027v1.pdf>

Objectives

User objectives:

1. The system should find and display the fastest route between two TfL rail stations.
 - a. The user should be able to select a start station and a destination station from the full list of TfL rail stations.
 - b. The user should be able to specify a date and time of travel.
 - c. The system should return a route consisting of an ordered list of stations, the lines used, and a total journey time in minutes.
 - d. The total journey time should account for a time penalty when the user must change lines.
2. The system should find and display a second route that accounts for crowding-induced delays caused by nearby football matches.
 - a. The system should maintain an up-to-date database of upcoming football fixtures at London venues, covering at least the next 12 months.
 - b. The system should distribute the modelled passenger demand across nearby stations, so that stations closer to the venue have a greater impact on crowding calculations than those further away.
 - c. The system should propagate crowding outward from stations near the venue to connected stations across the network.
 - d. Journey times on crowded connections should be increased to reflect the estimated delay caused by the additional passengers, meaning the crowding-aware route should never underestimate journey time relative to the base route.
 - e. Lines with higher average speeds should produce shorter base travel times than slower lines on equivalent distances.
 - f. The system should compute an alternative route using these adjusted travel times, which may differ from the fastest base route if a less crowded path exists.
3. The system should display qualitative crowding descriptions for each station along both the base route and the crowding-aware route.
 - a. Each station on both displayed routes should be labelled with one of four descriptions: Normal, Slightly Busy, Busy, or Severely Busy.
 - b. These descriptions should be derived from the estimated passenger load at each station at the time of travel.
4. The system should allow users to adjust the parameters used in crowding and route calculations.
 - a. The user should be able to view all 15 adjustable parameters on a dedicated settings page.
 - b. The user should be able to edit and save any of these parameters, with the updated values taking effect on subsequent route calculations.
 - c. The user should be able to reset all parameters to a set of predefined default values at any time.

- d. When parameters that affect walking distance are updated, the system should recalculate which stations are considered walkable from each venue.
- 5. The system should require users to log in before accessing any functionality, using a secure account system.
 - a. The system should allow users to create an account using a username and password.
 - b. Passwords should never be stored in plain text, and instead they should be hashed using SHA-256 with a unique randomly generated salt per user.
 - c. If a login attempt fails, the system should return a specific message indicating whether the username was not found or the password was incorrect.
 - d. If a username already exists, account creation should fail and return an appropriate message.
 - e. Each user's parameters should be initialised to default values upon account creation.
- 6. The system should present information through a simple, clear interface that delivers what the user needs without unnecessary complexity.
 - a. The interface should consist of no more than five distinct pages: a login page, a loading page, a main journey search page, a parameter settings page, and a database error page.
 - b. The station selection inputs on the main page should support search by name, returning matching results as the user types.
 - c. The results should be presented on the same page as the search form, without navigating away.
 - d. Both routes should be displayed side by side, each showing the lines used, the stations visited, their crowding descriptions, the total journey time, and the estimated arrival time.
 - e. The user should be able to fetch updated fixture and TfL data from a button on the settings page, with a loading screen shown while the update runs in the background.
 - i. The user's saved parameters should be preserved through the update process and remain unchanged once the database has been rebuilt.
 - ii. If the update process fails, the user should be redirected to the database error page with the option to retry.

System objectives:

- 1. A relational database should be created to store all data required by the system.
 - a. The database should be in Third Normal Form, using junction tables to handle many-to-many relationships.

- b. The database should contain the following 9 tables: Stations, Lines, StationLineRelationships, Connections, Venues, VenueStationRelationships, Events, Users, and Teams.
 - c. Station data (name, NaPTAN identifier, latitude, longitude) should be fetched from the TfL API and stored in the Stations table.
 - d. Connection records should be created for every pair of adjacent stations on each line, storing the line used and the base travel time calculated from geographic distance and average line speed.
 - e. Stadium coordinates should be obtained via a geocoding API and stored in the Venues table, with at least one associated walkable station per venue stored in VenueStationRelationships.
 - f. Upcoming football fixture data should be scraped from BBC Sport for all English football league tiers, filtered to London home matches only, and stored in the Events table.
 - g. User credentials and all 15 adjustable parameters should be stored per account in the Users table.
2. A graph-based pathfinding component should be implemented.
 - a. The transport network should be represented as an adjacency list graph, where each station maps to a list of its neighbours along with the connecting line and base travel time.
 - b. A modified Dijkstra's algorithm should be implemented using a min-heap priority queue, where a configurable time penalty is added each time the algorithm transitions between lines.
 - c. The algorithm should return the full ordered path as a list of steps in the form [StationA, StationB, LineUsed, CumulativeTime].
3. A crowding algorithm should be implemented to estimate the impact of football matches on the network.
 - a. The algorithm should identify which events on the travel date fall within a configurable time window of the estimated journey midpoint.
 - i. The journey midpoint should be estimated by running the pathfinding algorithm on the base graph and adding half the total base journey time to the departure time.
 - ii. For each event, the expected peak arrival time should be calculated by applying a configurable offset to the kick-off time, and the expected peak departure time should be calculated by applying a configurable offset to the estimated end time.
 - iii. An event should be included in the crowding calculation if the smaller of the two differences between the journey midpoint and each peak falls within the configurable maximum time window.
 - b. Passenger demand at nearby stations should be calculated using stadium capacity, attendance proportion, TfL usage proportion, and distance-based inverse weighting.
 - c. A Gaussian distribution should be applied to estimate what proportion of the peak demand is present at the time of travel.
 - d. Crowding should be propagated recursively through the graph, with each hop reducing the number of passengers by a configurable propagation factor, stopping when the count falls below a configurable minimum threshold.

- e. A delay factor should be calculated for each affected station based on its estimated passenger load.
 - f. Delay should be spread uniformly across all connections on the same line, so that congestion at one point affects the whole line.
 - g. The algorithm should return a modified network object containing updated delay factors for all stations and connections.
- 4. A secure login and account management component should be implemented.
 - a. Passwords should be hashed using SHA-256 with a unique 16-byte randomly generated salt per user.
 - b. Login should compare the hash of the entered password against the stored hash using the stored salt.
 - c. Account creation should fail silently on duplicate usernames, returning a failure flag to the caller.
 - d. Default parameter values should be loaded from a configuration JSON file and stored in the database on account creation.
- 5. A results component should be implemented to compute and package the three journey outputs.
 - a. The component should produce three route objects: the base fastest route (to be used for internal calculations), the base route re-evaluated under crowded travel times, and a newly calculated crowding-optimised route.
 - b. Each route should be annotated with a qualitative crowding description (Normal, Slightly Busy, Busy, Severely Busy) for every station, derived from user-configurable thresholds.
 - c. The component should return a single results object containing all three routes and their descriptions for use by the display component.
- 6. A Flask web application should be implemented to serve as the interface between the user and the system.
 - a. All file paths, API keys, and configuration values should be stored centrally in Flask's application config dictionary, set at startup, and referenced by all routes.
 - b. The login route should check for the existence of the database file on each request; if absent, it should start database creation in a background thread and redirect to a loading page.
 - c. The loading page should poll a flags endpoint every 2 seconds to check creation status, redirecting to the main page on success or to an error page on failure.
 - d. The main route should be protected by session authentication, redirecting unauthenticated users to the login page.
 - e. The station search endpoint should return a filtered list of matching station names based on a search string, limited to 20 results.
 - f. The save parameters route should update the user's parameters in the database and recalculate venue-station relationships if walking parameters have changed.
 - g. The fetch new data route should back up user data, rebuild the database with fresh external data, and restore user data once complete.

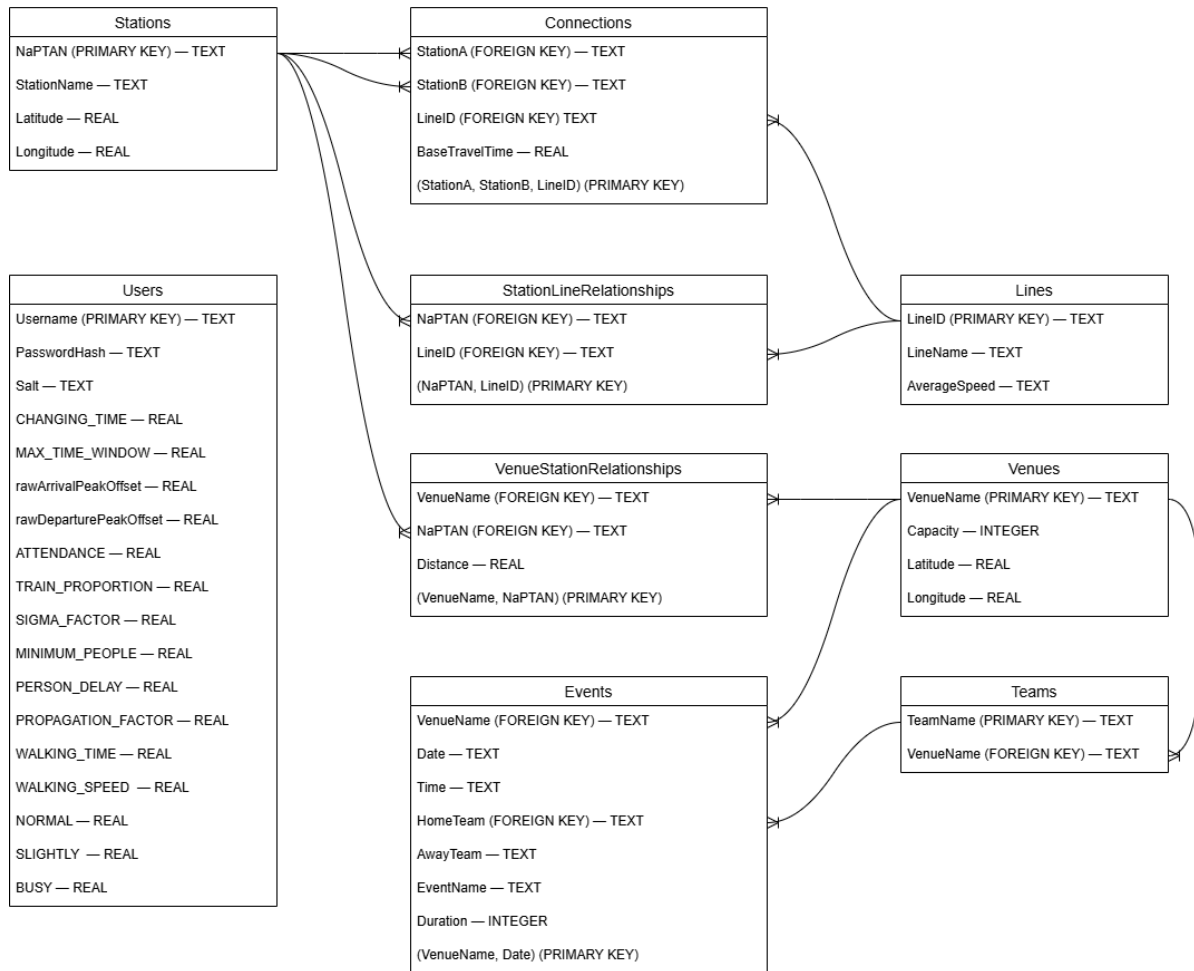
Proposed solution

My proposed solution to the end user's problem is to develop a web-based journey planner for TfL's rail network, implemented in Python using the Flask framework, which explicitly accounts for congestion caused by large scheduled events, with a focus on football matches. A web-based solution allows the system to be accessed from any device with a browser, including laptops, desktops, and mobile phones, making it suitable for commuters, students, and teachers who may use different devices at home and at work. Furthermore, developing a mobile or desktop application would require separate implementations for different operating systems such as Android, iOS, macOS, and Windows.

The system models the TfL rail network as a graph, where stations are represented as nodes and connections between stations are represented by weighted edges. To model congestion, the system maintains a database of upcoming football fixtures at London stadiums. Using stadium capacity, estimated attendance, and the proportion of attendees expected to use TfL rail services, the system estimates additional passenger demand generated by each event. This demand is first distributed to nearby stations using distance-based weighting and then propagated across the rest of the network to simulate how crowds disperse through connected lines and stations. A Gaussian model is used to estimate the gradual build-up and dispersal of crowds before and after matches, allowing the system to estimate congestion levels at the time of a user's journey based on all events occurring that day. These estimated passenger loads are then converted into station-level and line-level delay factors. A modified implementation of Dijkstra's shortest path algorithm is used to calculate routes, incorporating penalties for line interchanges and adjusting edge weights based on estimated delays. This allows the system to compute both a standard shortest-time route and an alternative route that accounts for crowding-induced delays. The system also presents users with qualitative descriptions of expected crowding at stations, such as "normal," "busy," or "severely busy." The parameters used for these calculations can also be adjusted by users.

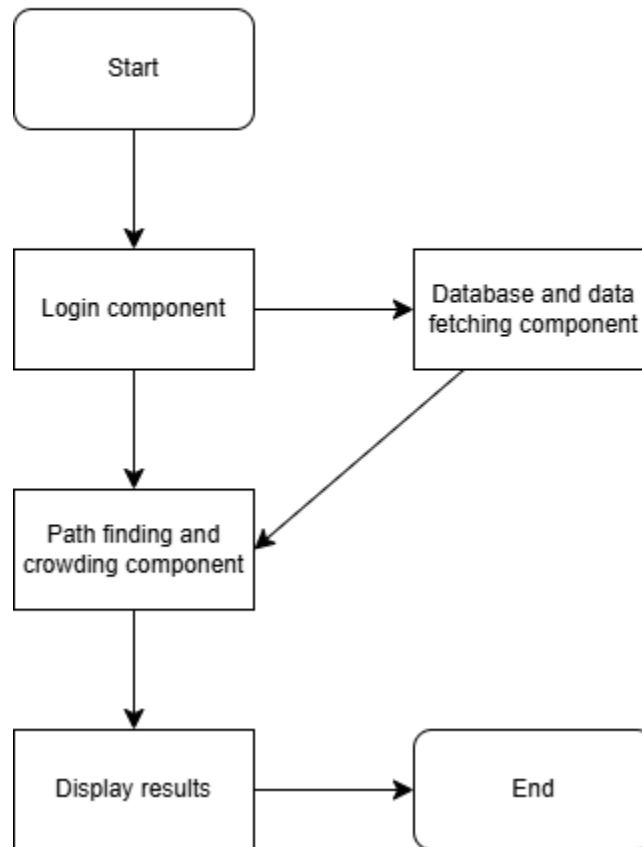
Modeling

Database



This entity-relationship model directly informed the database scheme implemented in the project. In particular, the many-to-many relationships between stations and lines lead to the decision to use junction tables to ensure the database is in third normal form.

Broad Overview



Prototyping

I prototyped the graph creation and path-finding component of the project. This is because the fundamental part of this project is finding the optimal path through a transport network by representing it as a graph. This prototype will help identify considerations for the final program through my own testing and end user feedback.

First prototype's code:

[pathFinding.py](#):

```
import sqlite3
import collections
import math
import heapq
```

```

#Fetches all the unique connections from the database and for every StationA, it adds to a list all StationBs in their own list including Line and travel time
#Forms dictionary in the form {StationA: [[StationB, BaseTravelTime, LineID], [StationC, ...], ...]}
def MakeGraph(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    listOfNeighbourStations = {} # {StationA: [[StationB, BaseTravelTime, LineID], ...]}
    rows = cursor.execute("SELECT StationA, StationB, LineID, BaseTravelTime FROM connections").fetchall()

    for StationA, StationB, LineID, BaseTravelTime in rows: #For every unique connection (direction matters)
        if StationA not in listOfNeighbourStations:
            listOfNeighbourStations[StationA] = []
        if [StationB, BaseTravelTime, LineID] not in listOfNeighbourStations[StationA]:
            listOfNeighbourStations[StationA].append([StationB, BaseTravelTime, LineID])

    connection.close()
    return listOfNeighbourStations

# It is assumed that all stations can be accessed from any other station because the graph is made from the data for me stored in the database which is always complete for the function to be called in the first place
# On the TfL network, every station is eventually connected to every other station
def Dijkstra(graph, startStation, goalStation):
    distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
    cameFrom = {} #To reconstruct path later
    visited = set() #Stores visited stations

    # Start at the given station, with no line yet chosen
    startState = (startStation, None) # This is the state is key because the line change penalty depends on the previous line
    distances[startState] = 0

    frontier = [(0, startState)] #The first item is always the cheapest station to go to next
    goalState = None #Store the final when we reach the goal

    while frontier: #This condition means it runs until all the paths have been explored - till the last one
        currentCost, (currentStation, currentLine) = heapq.heappop(frontier) #Deletes the next route at the top so that the next one to be explored can be at the top

        # Skip if already fully explored this (station, line) - If at the top of the heap, then it means that it is the shortest ever path to the station, which means it is visited
        if (currentStation, currentLine) in visited:
            continue
        visited.add((currentStation, currentLine))

        # Stop if reached the goal station
        if currentStation == goalStation:
            goalState = (currentStation, currentLine)
            break

        # Explore all neighbours of this station
        for nextStation, baseTravelTime, nextLine in graph[currentStation]:
            travelTime = baseTravelTime

            # New total time to reach the neighbour
            newCost = currentCost + travelTime
            neighbourState = (nextStation, nextLine)

            # Only update if we found a faster way
            if newCost < distances[neighbourState]:
                distances[neighbourState] = newCost
                cameFrom[neighbourState] = (currentStation, currentLine, nextLine)
                heapq.heappush(frontier, (newCost, neighbourState))

    # Start from the goal and trace back to the start

```

```

pathSteps = []
currentState = goalState

while currentState != startState:
    prevStation, prevLine, lineUsed = cameFrom[currentState]
    currStation, currLine = currentState
    pathSteps.append((prevStation, currStation, lineUsed, distances[(currStation, currLine)]))
    currentState = (prevStation, prevLine)

pathSteps.reverse()
return pathSteps

```

testData.py:

```

import sqlite3

def CreateTestDatabase(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Create Connections table
    cursor.execute("""
CREATE TABLE IF NOT EXISTS connections (
    StationA TEXT,
    StationB TEXT,
    LineID TEXT,
    BaseTravelTime INTEGER
)
""")

    # Clear old data (for repeated testing)
    cursor.execute("DELETE FROM connections")

    # Fake test data

    testConnections = [
        # Path 1: A->B->C (2 lines)
        ("A", "B", "Line1", 4),
        ("B", "C", "Line2", 5),
        # Path 2: A->D->C (1 line)
        ("A", "D", "Line3", 5),
        ("D", "C", "Line3", 5),
        # Reverse directions
        ("B", "A", "Line1", 5),
        ("C", "B", "Line2", 5),
        ("D", "A", "Line3", 5),
        ("C", "D", "Line3", 5),
    ]

    cursor.executemany("""
INSERT INTO connections (StationA, StationB, LineID, BaseTravelTime)
VALUES (?, ?, ?, ?)
""", testConnections)

    connection.commit()
    connection.close()

```

```
if __name__ == "__main__":
    CreateTestDatabase("test.db")
```

[main.py:](#)

```
import testData
import pathFinding

if __name__ == '__main__':

    # Create the database
    databaseFile = "test.db"
    testData.CreateTestDatabase(databaseFile)

    # Make a graph in the form {StationA: [[StationB, travelTime, Line], ...], ...}
    listOfNeighbourStations = pathFinding.MakeGraph(databaseFile)
    print(listOfNeighbourStations)

    # Example stations
    startStation = "A"
    endStation = "C"

    # The path in the form [(StationA, StationB, Line, timeToStation), ...]
    pathSteps = pathFinding.Dijkstra(listOfNeighbourStations, startStation, endStation)
    print(pathSteps)
```

```
C:\Users\HP\Downloads\Prototype\.venv\Scripts\python.exe C:\Users\HP\Downloads\Prototype\prototype\first\main.py
{'A': [['B', 4, 'Line1'], ['D', 5, 'Line3']], 'B': [['C', 5, 'Line2'], ['A', 5, 'Line1']], 'D': [['C', 5, 'Line3'], ['A', 5, 'Line3']], 'C': [['B', 5, 'Line2'], ['D', 5, 'Line3']]}
[('A', 'B', 'Line1', 4), ('B', 'C', 'Line2', 9)]
```

Process finished with exit code 0

The output shows a path with the total time at 9 units of time (graph's structure in the comments in the code)

Feedback on first prototype:

This prototype initially takes in a list of hard-coded connections that together make up a transport network graph, and stores them in an SQLite table. The MakeGraph function then takes in the database and uses the connections in the Connections table to create a dictionary where each station is a key and the value is a list of tuples which contain the connected station, the line used, and the travel time (weight). This dictionary (the graph) is passed into the Dijkstra function, which implements Dijkstra's shortest path algorithm and outputs in the form [(StationA, StationB, Line, timeToStation), ...].

Upon the creation of this prototype, I showed it to the end user, Ms Miah, to ask for feedback and whether there were any features she might want. I showed her the outputs of the different functions on my laptop on the terminal after school, and wrote down the conversation in my journal, which I've rewritten below:

Me: “Hi Ms, here on the terminal are example outputs of what the path-finding component of the programme should look like. This is an important part of the app, because the core function of the app is route finding. Currently, the programme successfully takes in a set of stations, lines and the time between stations, forms a graph, and uses a shortest path algorithm to find the shortest path”.

Ms Miah: “The routes look correct, but in real life changing lines usually takes extra time. Your algorithm treats staying on the same line and changing lines as equal.”

Me: “That’s true. At the moment, the algorithm only considers travel time between stations, and nothing else”.

Ms Miah: “You should add a time penalty for changing lines so that the route chosen is more realistic”.

Me: “I agree. I will modify the algorithm so that an additional time penalty is applied whenever the user changes from one line to another”.

Upon receiving this feedback, I made a change to my Dijkstra function, so that it became a modified implementation of Dijkstra’s shortest path algorithm, which accounts for the extra time that is required to change trains at a station, below:

Second prototype’s code (that has been changed):

[pathFinding.py](#):

```
import sqlite3
import collections
import math
import heapq

#Fetches all the unique connections from the database and for every StationA, it adds to a list all StationBs in their own list
including Line and travel time
#Forms dictionary in the form {StationA: [[StationB, BaseTravelTime, LineID], [StationC, ...], ...]}
def MakeGraph(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    listOfNeighbourStations = {} # {StationA: [[StationB, BaseTravelTime, LineID], ...]}
    rows = cursor.execute("SELECT StationA, StationB, LineID, BaseTravelTime FROM connections").fetchall()

    for StationA, StationB, LineID, BaseTravelTime in rows: #For every unique connection (direction matters)
        if StationA not in listOfNeighbourStations:
            listOfNeighbourStations[StationA] = []
        if [StationB, BaseTravelTime, LineID] not in listOfNeighbourStations[StationA]:
            listOfNeighbourStations[StationA].append([StationB, BaseTravelTime, LineID])

    connection.close()
    return listOfNeighbourStations

# It is assumed that all stations can be accessed from any other station because the graph is made from the data for me
stored in the database which is always complete for the function to be called in the first place
```

```

#On the TfL network, every station is eventually connected to every other station
def Dijkstra(graph, startStation, goalStation, CHANGING_TIME, penalty=True):
    distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
    cameFrom = {} #To reconstruct path later
    visited = set() #Stores visited stations

    # Start at the given station, with no line yet chosen
    startState = (startStation, None) # This is the state is key because the line change penalty depends on the previous line
    distances[startState] = 0

    frontier = [(0, startState)] #The first item is always the cheapest station to go to next
    goalState = None #Store the final when we reach the goal

    while frontier: #This condition means it runs until all the paths have been explored - till the last one
        currentCost, (currentStation, currentLine) = heapq.heappop(frontier) #Deletes the next route at the top so that the next
        one to be explored can be at the top

        # Skip if already fully explored this (station, line) - If at the top of the heap, then it means that it is the shortest ever path
        to the station, which means it is visited
        if (currentStation, currentLine) in visited:
            continue
        visited.add((currentStation, currentLine))

        # Stop if reached the goal station
        if currentStation == goalStation:
            goalState = (currentStation, currentLine)
            break

        # Explore all neighbours of this station
        for nextStation, baseTravelTime, nextLine in graph[currentStation]:
            travelTime = baseTravelTime

            if penalty: #Can be turned off if needed (for testing purposes)
                if currentLine is not None and currentLine != nextLine:
                    #Initially no line is chosen, so the penalty is not added for changing lines
                    #If the line is switched to travel to the next station, the penalty is added to the cost
                    travelTime += CHANGING_TIME

            # New total time to reach the neighbour
            newCost = currentCost + travelTime
            neighbourState = (nextStation, nextLine)

            # Only update if we found a faster way
            if newCost < distances[neighbourState]:
                distances[neighbourState] = newCost
                cameFrom[neighbourState] = (currentStation, currentLine, nextLine)
                heapq.heappush(frontier, (newCost, neighbourState))

    # Start from the goal and trace back to the start
    pathSteps = []
    currentState = goalState

    while currentState != startState:
        prevStation, prevLine, lineUsed = cameFrom[currentState]
        currStation, currLine = currentState
        pathSteps.append((prevStation, prevLine, lineUsed, distances[(currStation, currLine)]))
        currentState = (prevStation, prevLine)

    pathSteps.reverse()
    return pathSteps

```

[main.py](#):

```
import testData
import pathFinding

if __name__ == '__main__':

    # Create the database
    databaseFile = "test.db"
    testData.CreateTestDatabase(databaseFile)

    # Make a graph in the form {StationA: [[StationB, travelTime, Line], ...], ...}
    listOfNeighbourStations = pathFinding.MakeGraph(databaseFile)
    print(listOfNeighbourStations)

    # Example stations and CHANGING_TIME
    startStation = "A"
    endStation = "C"
    CHANGING_TIME = 4.4

    # The path in the form [(StationA, StationB, Line, timeToStation), ...]
    pathSteps = pathFinding.Dijkstra(listOfNeighbourStations, startStation, endStation, CHANGING_TIME)
    print(pathSteps)
```

```
C:\Users\HP\Downloads\Prototype\.venv\Scripts\python.exe C:\Users\HP\Downloads\Prototype\prototype\second\main.py
{'A': [['B', 4, 'Line1'], ['D', 5, 'Line3']], 'B': [['C', 5, 'Line2'], ['A', 5, 'Line1']], 'D': [['C', 5, 'Line3'], ['A', 5, 'Line3']], 'C': [['B', 5, 'Line2'], ['D', 5, 'Line3']]}
[['A', 'D', 'Line3', 5], ('D', 'C', 'Line3', 10)]

Process finished with exit code 0
```

This now shows the fastest path is the one with more time between stations, but via a single line (total time of 10 units)

This prototype and feedback will help inform my design choices for the final project.

Design

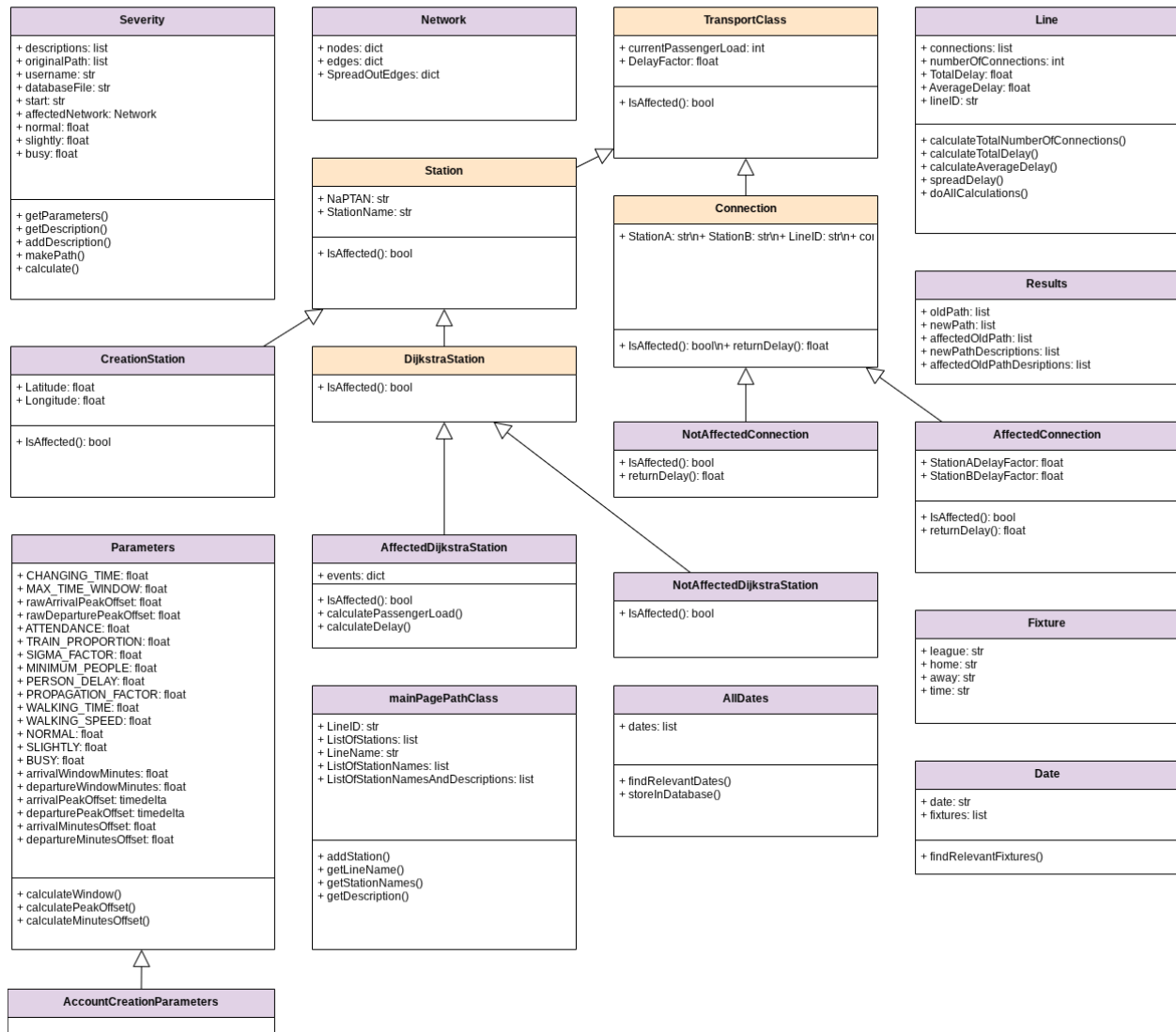
Design elements from other sections

This document is structured to avoid unnecessary repetition. Where content has already been established in the Analysis section, such as the proposed solution, the ER diagram, and the broad system overview, it is not duplicated here but referenced below. Similarly, some design content that is closely tied to its implementation is presented alongside the relevant code in the Technical Solution section rather than repeated here. All content is present in the document and can be found via the reference below:

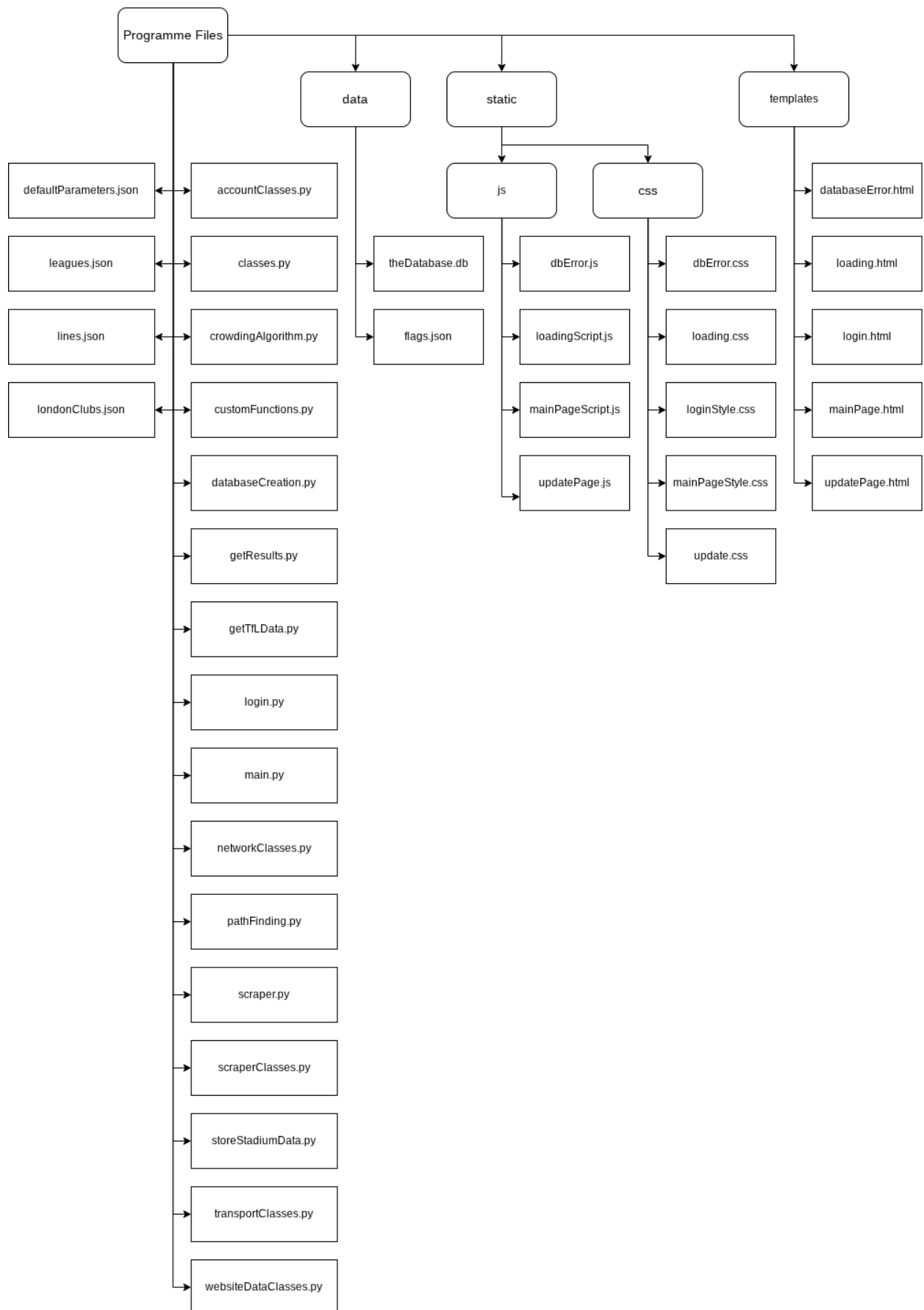
- System objectives are discussed in the System objectives section of the Objectives section of the Analysis section.
- The proposed solution is given in the Analysis section of this document.
- An E-R diagram of the database is given in the Analysis section of this document.
- A broad overview of the programme is given in the Analysis section of this document.
- A description of each system component and how it interacts with others is given in the System Overview section of the Technical Solution.
- Security is discussed in the Techniques section of the Technical Solution.

High level overview

Class Diagram



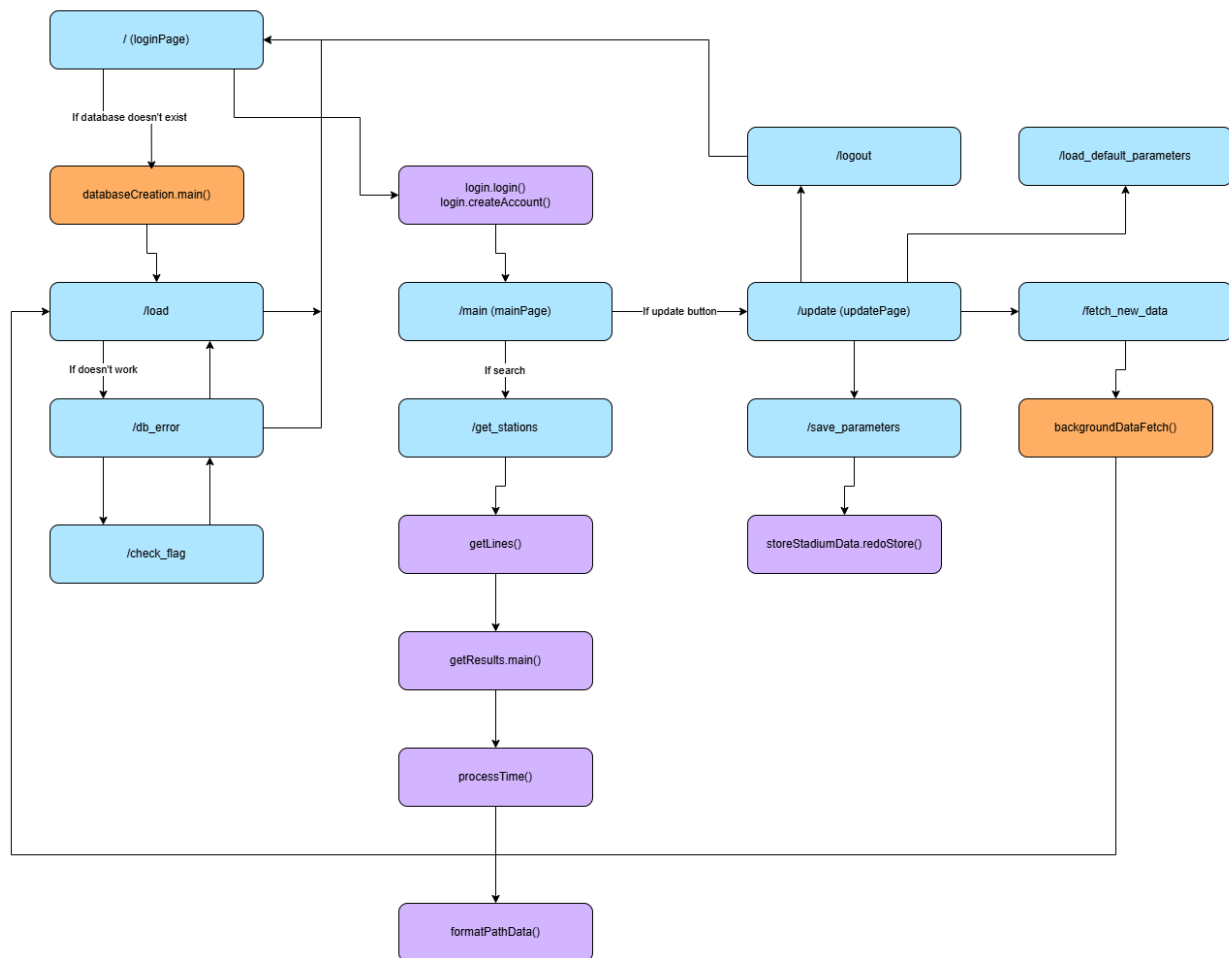
File Structure



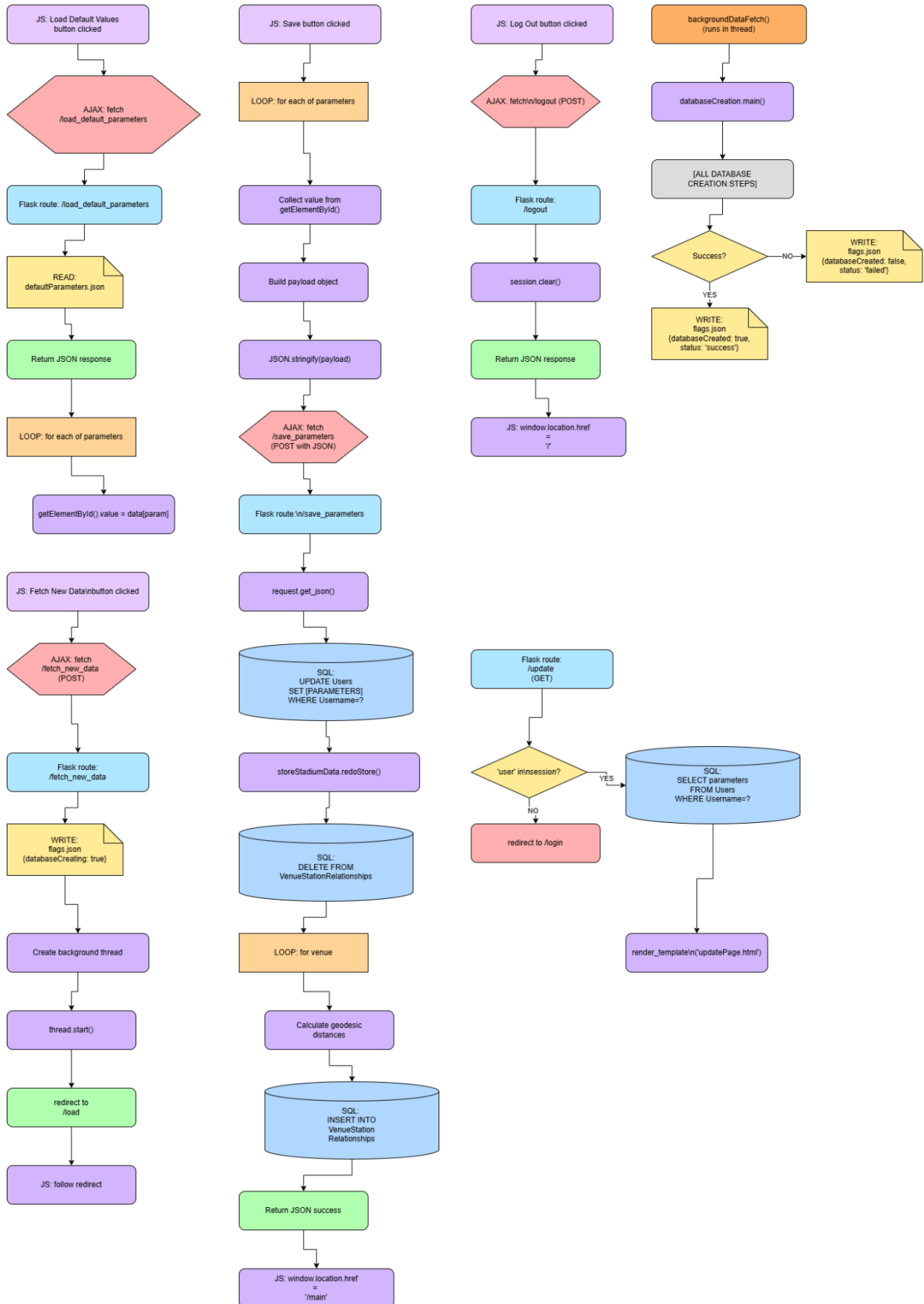
Overview of programme logic:

Due to the large nature of the programme, I've decided to split the overview of the logic into six components so that it is easier to view in this document. This overview shows the order of function and method calls, as well as when files are read and written to. The broad overview of the program's logic has already been shown in this document in the Analysis section. Below are the six components of the programme.

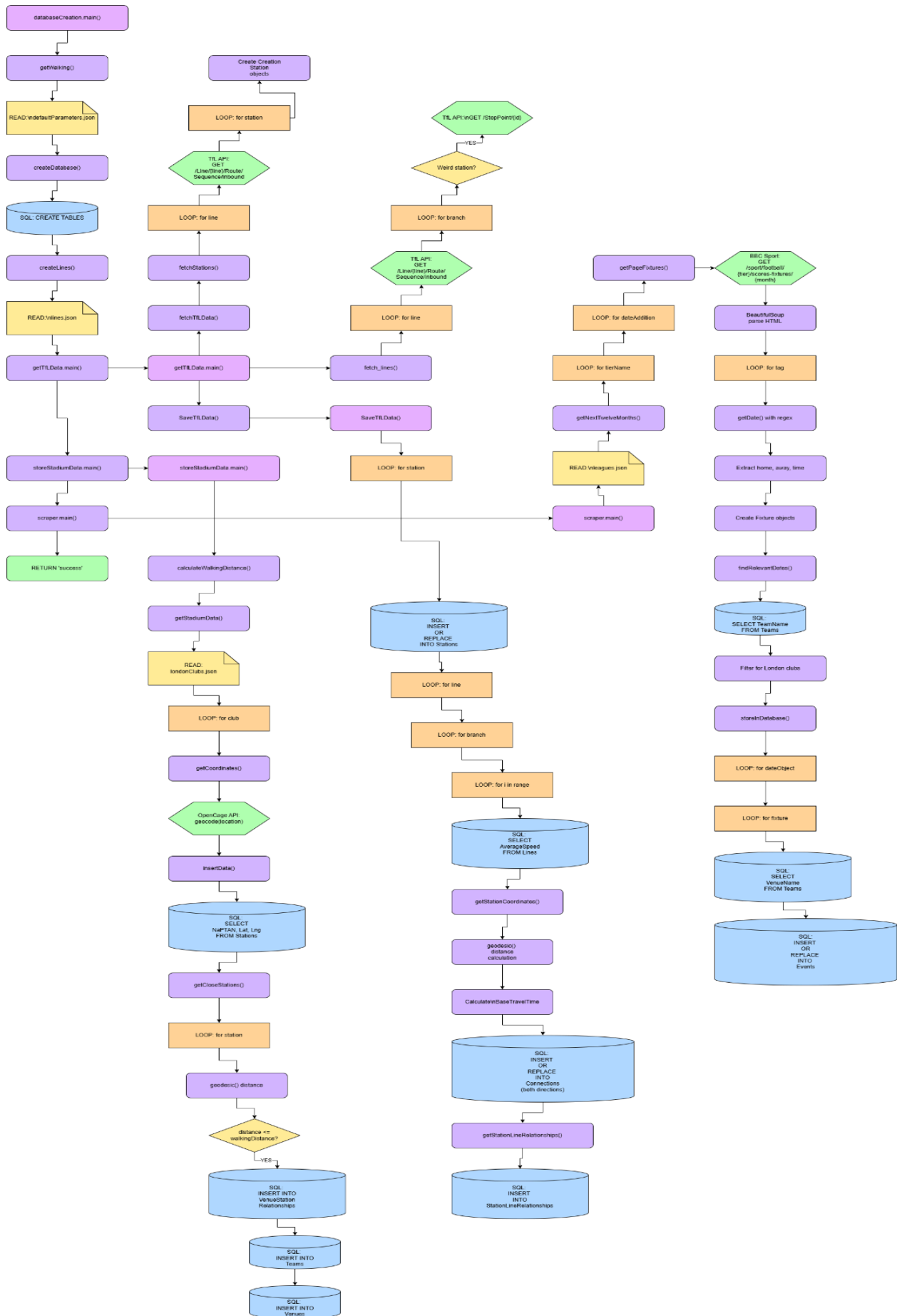
Pages component



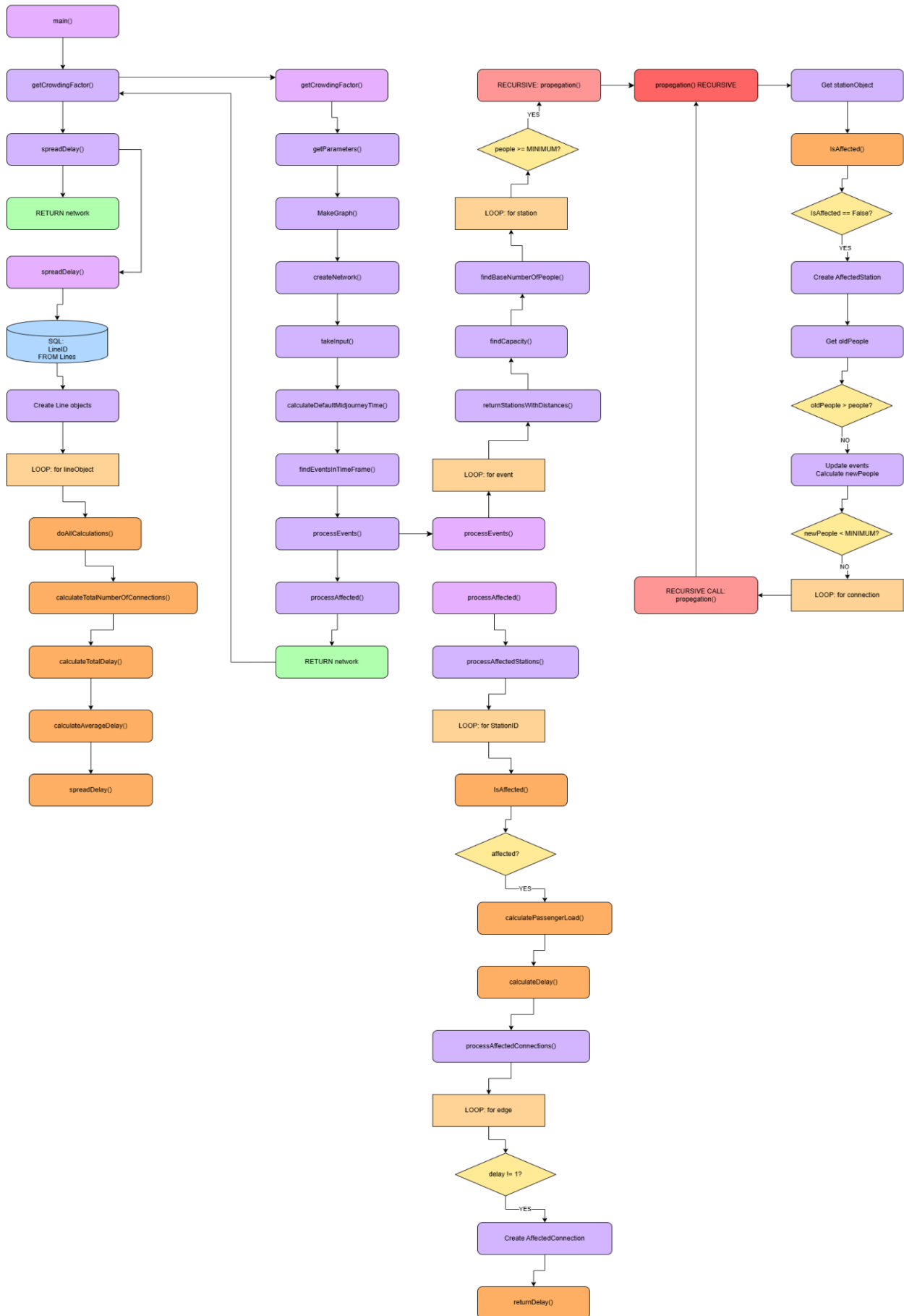
Update page component



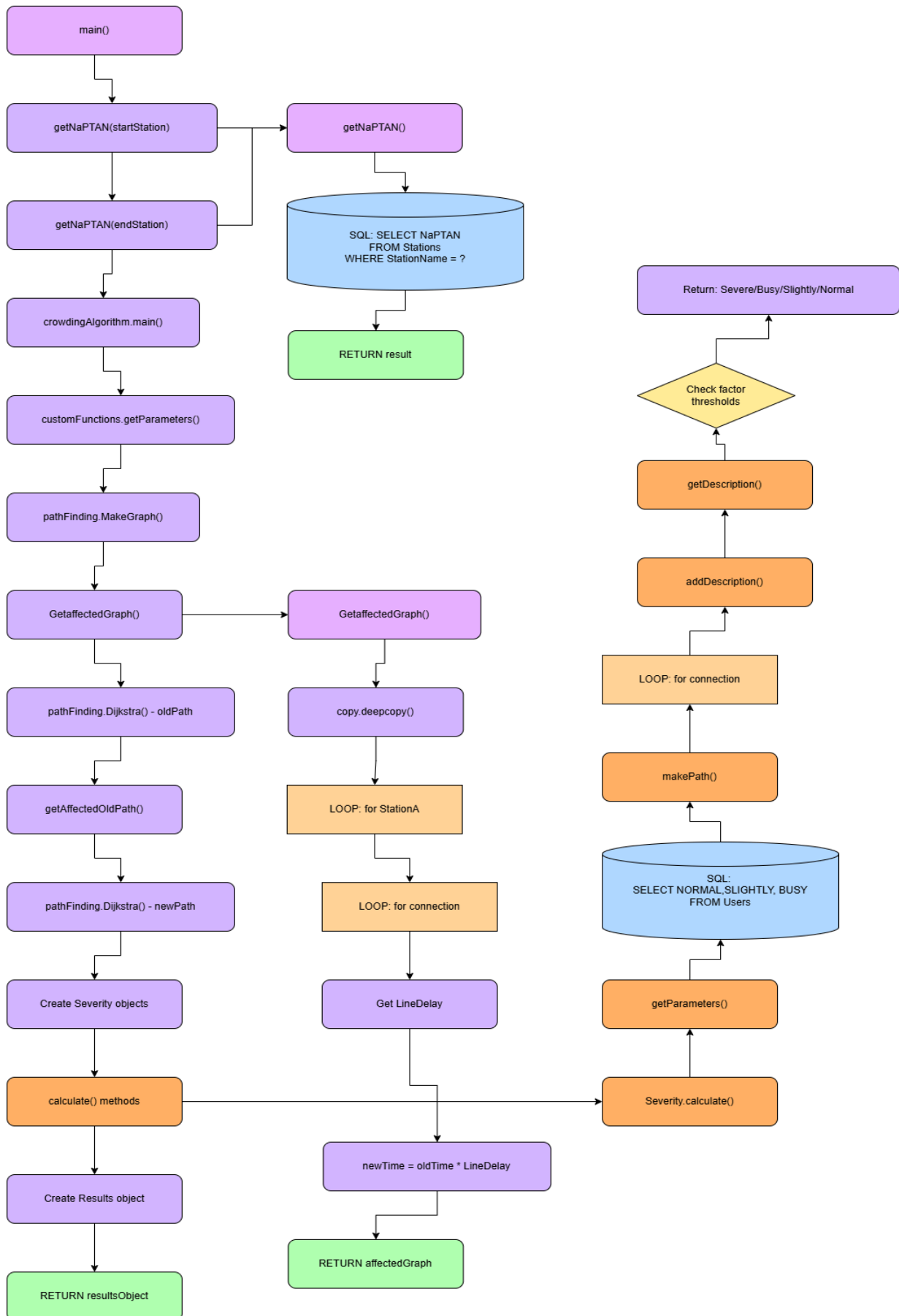
Database creation component

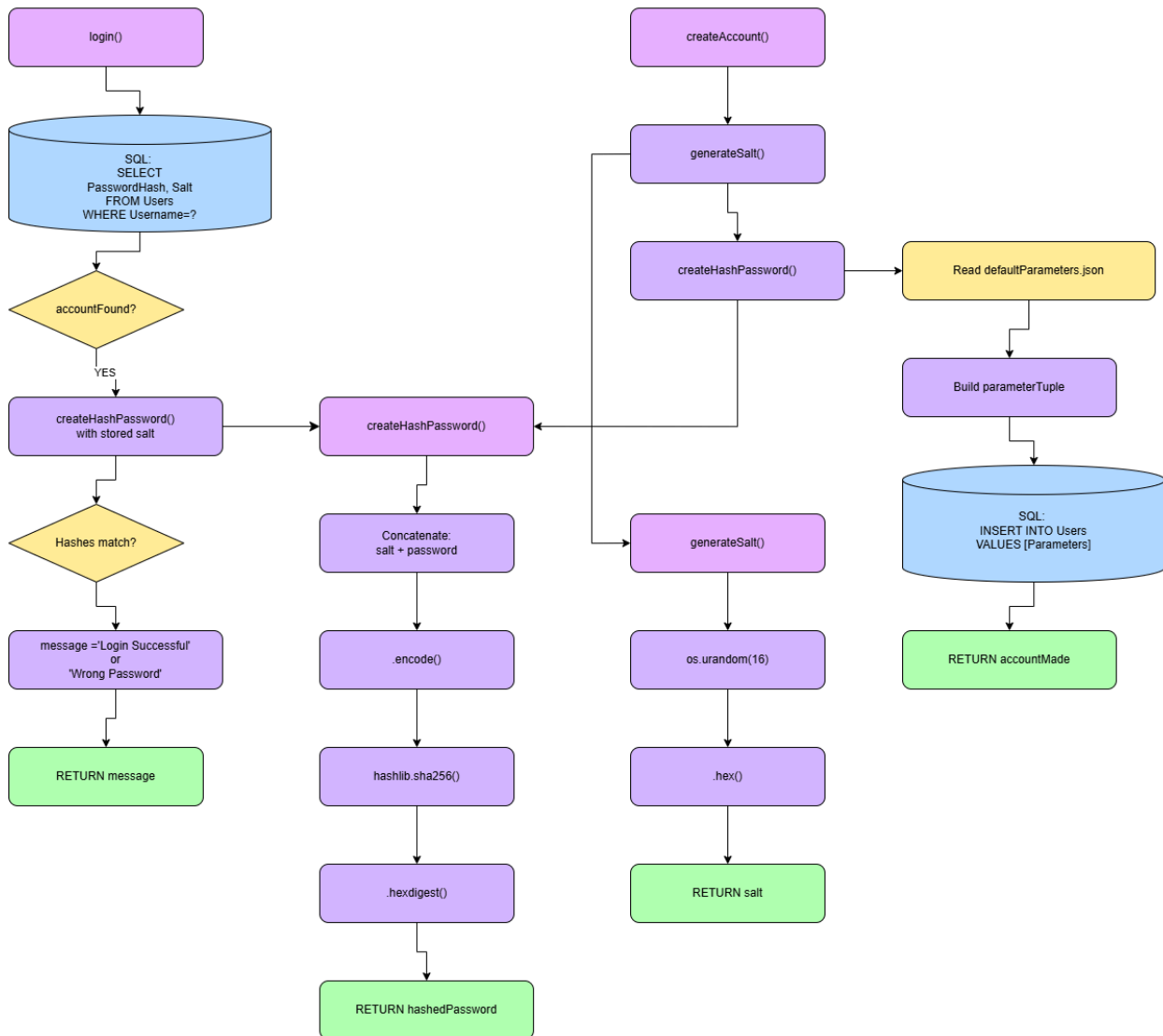


Crowding algorithm



Results fetching component



Login component

UI Sketches

Login page:

Welcome

Username

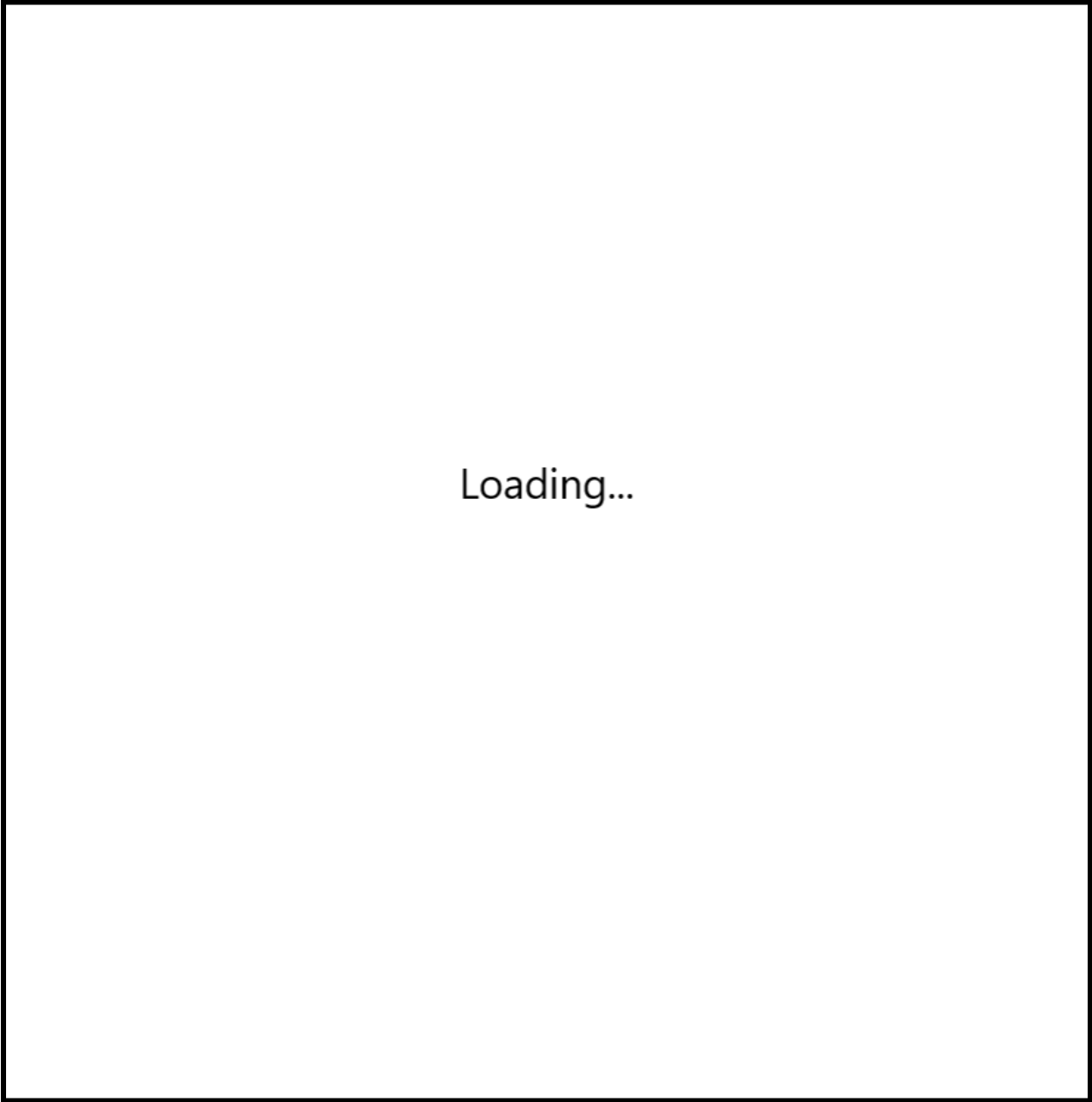
Password

Log in Sign up

A thick red horizontal line is positioned below the buttons, indicating an error message.

The login page where the user submits a username and password to either create an account, or log in. The red line signifies an error message, which would appear once an error would occur, such as “Account not found”, “Wrong password”, or “Account already exists”.

Load page:



Loading...

This page loads when the database is being created or being updated. This takes time, so this page is loaded to show the user that a process is occurring. The “Loading...” message periodically appears and disappears.

[Main page:](#)

Route Search

Original Fastest Path:

Line 1:

Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness

Line 2:

Station - Busyness
Station - Busyness

Total Time:
Arrival Time:

Our Recomendation:

Line 1:

Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness
Station - Busyness

Line 3

Station - Busyness

Total Time:
Arrival Time:

This page is the main page of the programme. The user submits a start and end station, and a time and date for the journey to begin, which the programme takes and uses to calculate the two routes, which it then displays, along with qualitative descriptions of busyness. There is also a list of lines serving the start and end stations. The red line shows where error messages should be displayed for issues such as the two stations being the same or if the time of the journey being in the past. Since the database will only exist if all the data is right, and in the right format, there is no error message for failed calculations, as the database must first exist and be created to reach this page. At the bottom of the page is also an update button, which takes the user to the update page.

Update page:

Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	
Parameter	

Load Default Values

Save

Fetch New Data

Log Out

This page is used to update the parameters used for calculations. The first button fetches the default values from the JSON file, and populates the fields. The second button saves this data and calls the appropriate functions for saving the data. The third button is used to trigger the functions which reset the database (without removing users). The fourth button logs out the user.

[Database error page:](#)

Database Setup Failed

Retry

Data

SQL:

The programme uses a single database whose entity relationships diagram is shown in the Analysis section of the document. Below are some examples of where CRUD was used in the program.

[Create \(Insert\):](#)

```

INSERT INTO Users(
    Username,
    PasswordHash,
    Salt,
    CHANGING_TIME,
    MAX_TIME_WINDOW,
    rawArrivalPeakOffset,
    rawDeparturePeakOffset,
    ATTENDANCE,
    TRAIN_PROPORTION,
    SIGMA_FACTOR,
    MINIMUM_PEOPLE,
    PERSON_DELAY,
    PROPAGATION_FACTOR,
    WALKING_TIME,
    WALKING_SPEED,
    NORMAL,
    SLIGHTLY,
    BUSY
)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

```

Creates a new user account with encrypted login details and the personalised parameters for journey modeling.

Read (Selected):

```

SELECT Lines.LineName
FROM Lines
JOIN StationLineRelationships
    ON Lines.LineID = StationLineRelationships.LineID
JOIN Stations
    ON StationLineRelationships.NaPTAN = Stations.NaPTAN
WHERE Stations.StationName = ?
ORDER BY Lines.LineName

```

Retrieves all tube lines serving a particular station for the web interface's dropdown menus. The two JOINS handle the many-to-many relationships, where one station has multiple lines, and one line passes through multiple stations, through the use of the junction table StationLineRelationships.

Update:

```

INSERT INTO Venues (VenueName, Capacity, Latitude, Longitude)
VALUES (?, ?, ?, ?)
ON CONFLICT (VenueName)
DO UPDATE SET
    Capacity = excluded.Capacity,
    Latitude = excluded.Latitude,
    Longitude = excluded.Longitude

```

If a stadium already exists in the database, this updates its capacity and coordinates rather than failing with a constraint error.

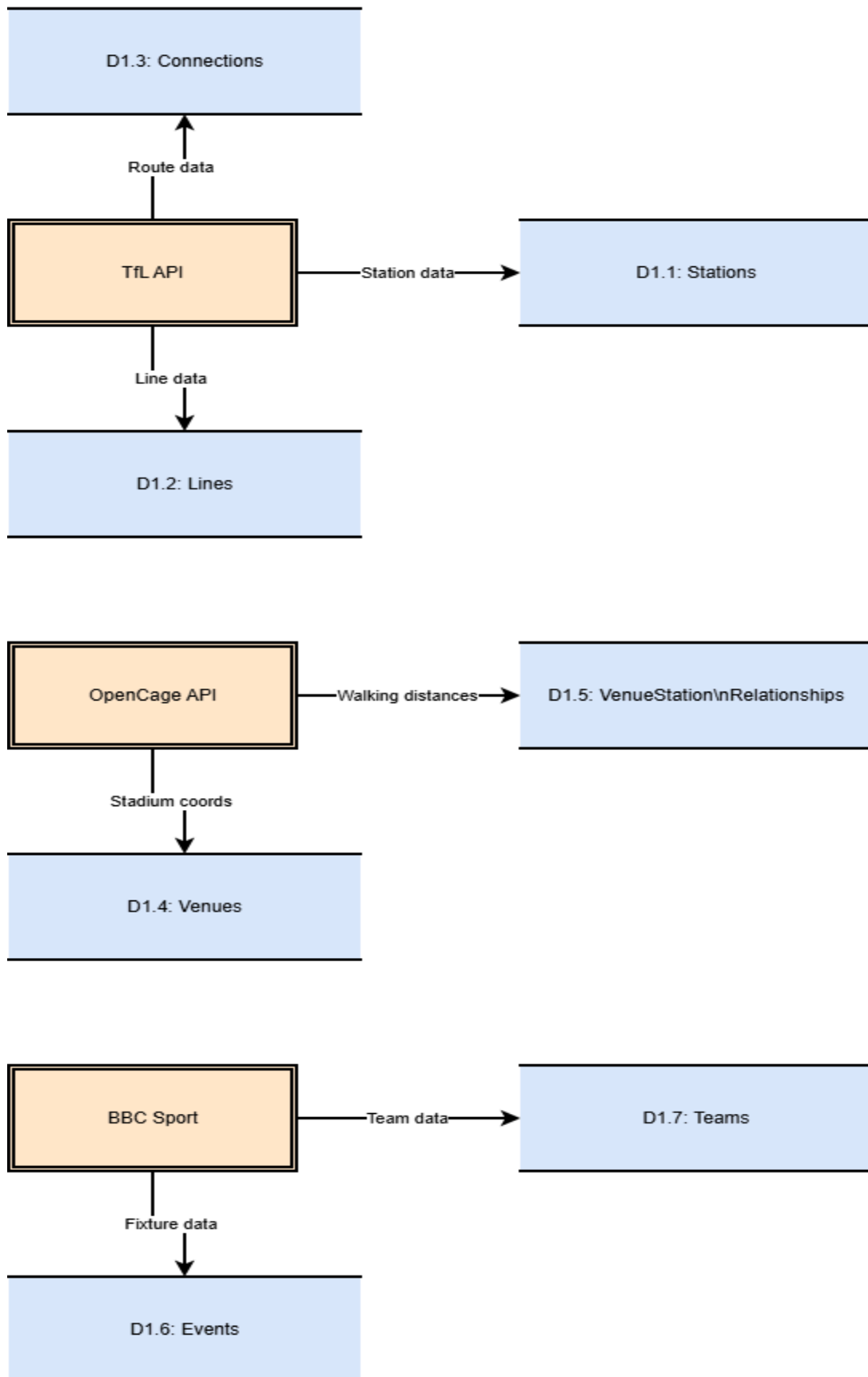
Delete:

```
DELETE FROM VenueStationRelationships
```

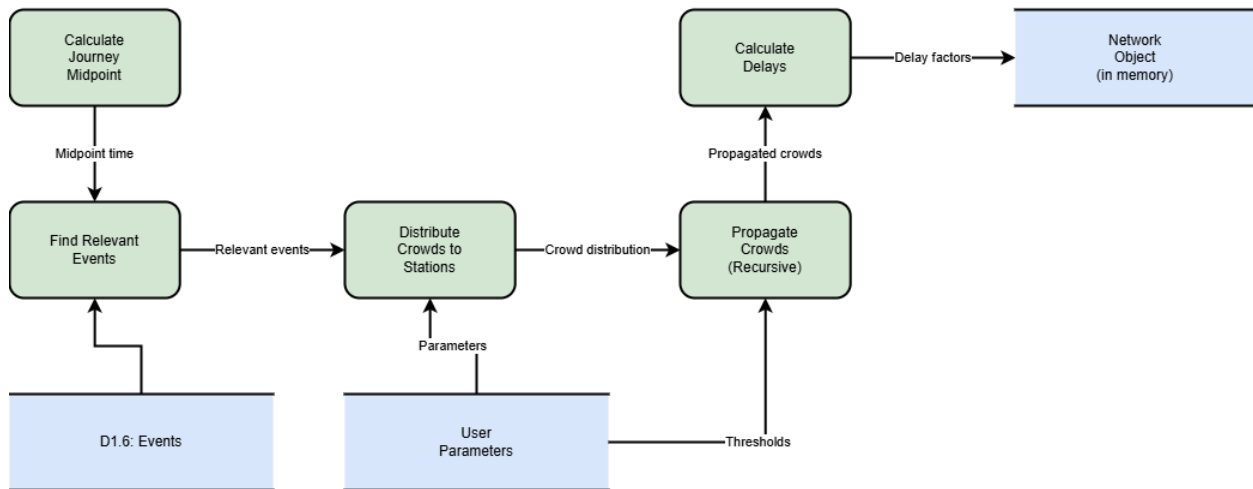
Clears all venue-station relationships before recalculating them with updated user parameters. When users change their walking time or speed settings, the maximum walking distance changes, so all relationships (which stations are within walking distance of which stadiums) must be deleted and recalculated with the new distance threshold.

Data flow diagram:

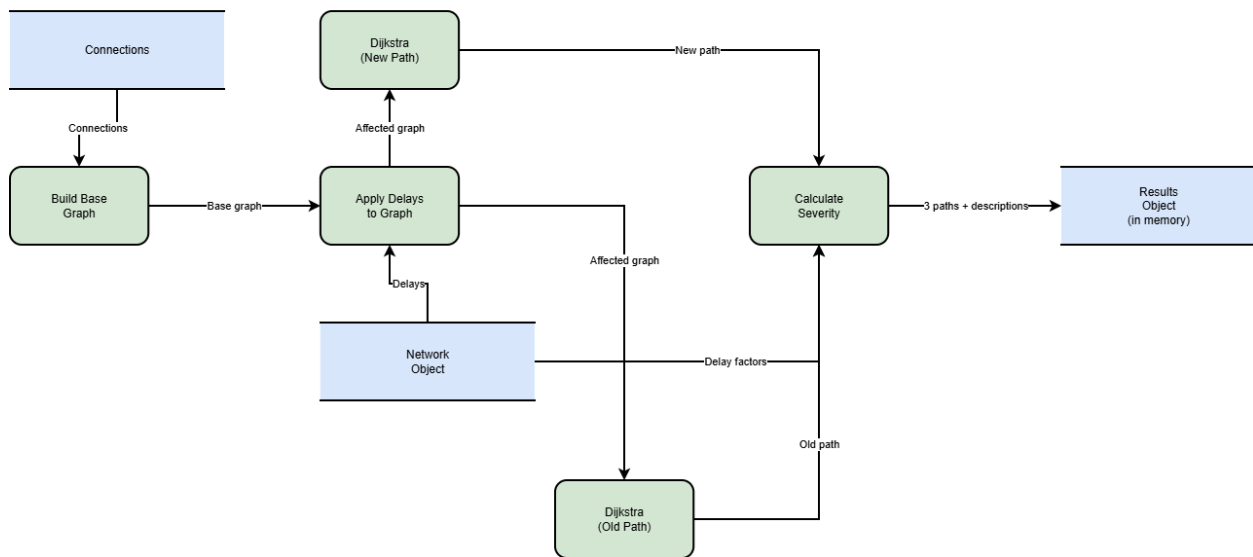
Database Creation



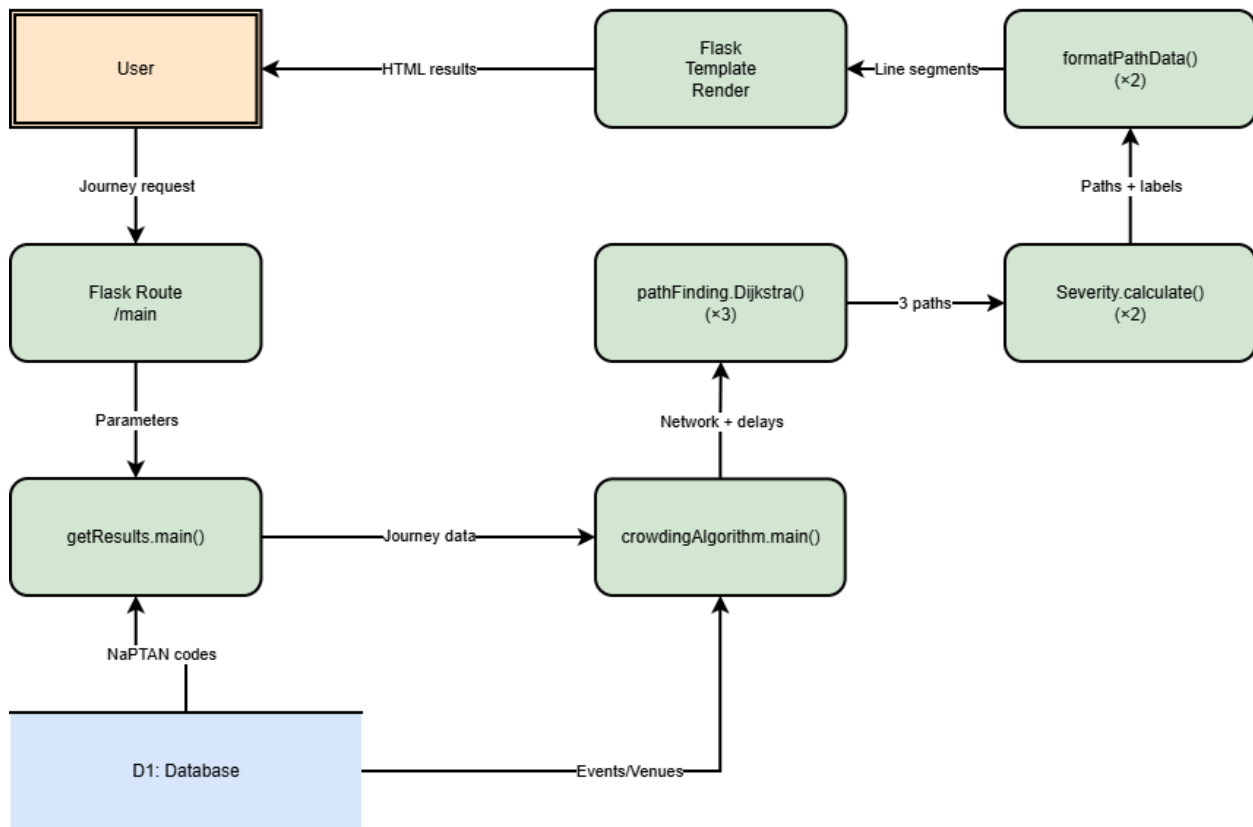
Crowding Simulation



Route Calculation



Complete User Journey Data Flow



Data structures used:

The programme makes use of many built in Python data structures such as lists. It also makes use to custom data structures, made using the primary data structures offered in python. Below are some examples of custom data structures and uses of built in data structures:

1. Graph (adjacency list):

The graph is implemented as a dictionary where each key is a station NaPTAN code and the value is a list of neighboring stations with travel times and line IDs. This adjacency list representation allows $O(1)$ lookup of a station's neighbors, which is needed for efficient pathfinding. In Dijkstra's shortest path algorithm, the system repeatedly fetches `graph[currentStation]` to explore all connected stations.

2. Network object:

The Network object uses two dictionaries to separate station data (nodes) from connection data (edges). Each station object stores its DelayFactor and passenger load from nearby events, while connection objects store delay factors calculated from their stations.

3. Priority queue (min heap):

Dijkstra's shortest path algorithm uses Python's `heapq` to create a priority queue of unexplored paths, ordered by cumulative travel time. The heap results in a $O(\log n)$ insertion and removal of paths, compared to $O(n)$ for a sorted list. Each element is a tuple of (cost, (stationID, lineID)). The heap guarantees that `heappop` always returns the cheapest unexplored path, making it essential for finding optimal routes efficiently.

4. Visited set:

A set tracks which (station, line) pairs have been fully explored to prevent reprocessing. Sets provide $O(1)$ membership testing compared to $O(n)$ for lists, which is essential when the same station can be reached via different lines. Before exploring a path, the algorithm checks if the state is in visited to skip already optimised routes, reducing the need for unnecessary calculations, especially in a large network.

5. Distances dictionary:

This dictionary stores the best known travel time to reach each (station, line) state, with unvisited states defaulting to infinity. Using `defaultdict` gets rid of `KeyError` exceptions and the need for existence checks, which results in clean code like `if newCost < distances[state]` without pre-initialisation. This structure is essential to Dijkstra's shortest path algorithm, storing the optimal cost to reach each state as the search progresses.

6. Results object:

The Results object groups all journey calculation outputs into a single container with named attributes. It stores three different paths (original, optimised, and original with delays) along with crowding severity descriptions for each station on the paths. This structure provides clean access.

7. Events dictionary:

Each affected station object contains a dictionary mapping (venue, date) tuples to passenger counts. The composite key prevents event overwrites when the same venue has matches on different dates or when multiple venues affect the same station. When calculating station delays, the algorithm iterates through `station.events.values()` to sum the total passenger load, which is then multiplied by a delay factor to determine how much crowding is to occur at that station.

Algorithms

Below are some of the key algorithms in the program.

Graph creation:

```

FUNCTION make_graph(database_file):
CONNECT to database_file
CREATE empty dictionary graph

GET all rows from Connections table
(FIELDS: StationA, StationB, LineID, BaseTravelTime)

FOR each row in rows:
  GET StationA
  GET StationB
  GET LineID
  GET BaseTravelTime

  IF StationA not in graph:
    graph[StationA] = empty list

  IF [StationB, BaseTravelTime, LineID] not in graph[StationA]:
    ADD [StationB, BaseTravelTime, LineID] to graph[StationA]

CLOSE database connection
RETURN graph

```

The purpose of this component is to construct a graph representation of the transport network using the data stored in the database. Each station acts as a node, and each connection between stations acts as a weighted directed edge. The algorithm fetches all the connections from the Connections table, where each record represents a valid movement between one station to another, via a specific line, with a base travel time (which acts as the initial weight of the graph). For each connection, the algorithm ensures that a list exists for the starting station (Station A). It then appends the neighbouring station (Station B), along with the base travel time and the line used. A check is performed to avoid duplicate connections being added, resulting in a dictionary in the form {StationA : [[StationB, BaseTravelTime, LineID], ...]}, which means the graph is represented in a form such that the other parts of the programme can use it, such as the modified Dijkstra's shortest path algorithm component.

Modified Dijkstra's shortest path algorithm:

```

FUNCTION dijkstra(graph, start_station, goal_station, CHANGING_TIME, penalty):
CREATE distances dictionary with default value INFINITY
CREATE empty dictionary came_from
CREATE empty set visited

start_state = (start_station, NONE)
distances[start_state] = 0

CREATE priority_queue frontier
ADD (0, start_state) to frontier

goal_state = NONE

WHILE frontier is not empty:
  GET current_cost, (current_station, current_line) from frontier

  IF (current_station, current_line) in visited:
    CONTINUE

```

```

ADD (current_station, current_line) to visited

IF current_station == goal_station:
    goal_state = (current_station, current_line)
    BREAK

FOR each neighbour in graph[current_station]:
    GET next_station
    GET base_travel_time
    GET next_line

    travel_time = base_travel_time

    IF penalty == TRUE:
        IF current_line is not NONE AND current_line != next_line:
            travel_time = travel_time + CHANGING_TIME

    new_cost = current_cost + travel_time
    neighbour_state = (next_station, next_line)

    IF new_cost < distances[neighbour_state]:
        distances[neighbour_state] = new_cost
        came_from[neighbour_state] = (current_station, current_line, next_line)
        ADD (new_cost, neighbour_state) to frontier

CREATE empty list path_steps
current_state = goal_state

WHILE current_state != start_state:
    GET previous_station, previous_line, line_used from came_from[current_state]
    GET current_station, current_line from current_state
    ADD (previous_station, current_station, line_used,
        distances[(current_station, current_line)]) to path_steps
    current_state = (previous_station, previous_line)

REVERSE path_steps
RETURN path_steps

```

This algorithm is a modified version of Dijkstra's shortest path algorithm. In this implementation, each node is represented as a pair in the form (station, current line), rather than just the station. This allows the algorithm to account for the additional time required when a passenger changes train lines. A priority queue is used to always explore the next state with the lowest total travel time. When moving between stations, the base travel time of the connection is added to the total cost (within the program, this base travel time may actually be a time calculated by considering crowding). If the passenger switches to a different line, a fixed penalty is added to simulate the extra time required to change trains, which is where this implementation of Dijkstra's shortest path algorithm differs from a conventional one the most. The algorithm keeps track of the shortest known time to each state and records the previous state in a dictionary. Once the goal station is reached, the algorithm backtracks through these states, to reconstruct the full route, including the stations visited, the lines used, and the total time to reach that step.

Relevant events filtering:

```

FUNCTION find_events_in_timeframe(midjourney_time, database_file, date,
                                MAX_TIME_WINDOW, arrival_peak_offset,
                                departure_peak_offset):

CONNECT to database_file
CREATE empty dictionary important_events

GET all events from Events table WHERE Date = date

FOR each event in events:
    GET venue
    GET event_start_time
    GET event_duration

    event_end_time = event_start_time + event_duration

    start_time_difference =
        ABSOLUTE((event_start_time + arrival_peak_offset) - midjourney_time)

    end_time_difference =
        ABSOLUTE((event_end_time + departure_peak_offset) - midjourney_time)

    IF start_time_difference > end_time_difference:
        time_difference = end_time_difference
    ELSE:
        time_difference = start_time_difference

    minutes_difference = CONVERT time_difference to minutes

    IF minutes_difference <= MAX_TIME_WINDOW:
        start_difference_minutes =
            CONVERT start_time_difference to minutes
        end_difference_minutes =
            CONVERT end_time_difference to minutes

        important_events[(venue, date)] =
            [minutes_difference,
             start_difference_minutes,
             end_difference_minutes]

CLOSE database connection
RETURN important_events

```

The algorithm finds which events are likely to impact a journey by comparing the time of travel with the expected peaks of crowding for events on the same day. Rather than assuming crowding occurs exactly at the start, or end of an event, peak offsets are applied to more accurately reflect real world behaviour. For each event, the difference between the midpoint of the journey (passed in), and the two peaks of the event (arrival and departure), is calculated. The smaller of the two differences is chosen as events closer in time to the journey are likely to have a greater impact. Only events within a time-window are included, which prevents unnecessary processing of irrelevant events. The result is a dictionary of events, which also includes the two differences, which may affect the journey, which is later used by the crowd propagation algorithm.

Crowded network creation algorithm:

```

FUNCTION gaussian_factor(relative_peak, sigma, relative_time_difference):
  x = (relative_peak - relative_time_difference) / sigma
  RETURN  $e^{(-0.5 \times x^2)}$ 

```

```

FUNCTION inverse_weight(stations_with_distances):
  // stations_with_distances is a dictionary {NaPTAN: distance_km}
  extra = 1
  // extra is added to every distance to prevent division by zero
  // if a station is at the same location as the venue
  weights = empty dictionary
  total_weight = 0

```

```

  FOR each station IN stations_with_distances:
    distance = stations_with_distances[station]
    weight = 1 / (distance + extra)
    weights[station] = weight
    total_weight = total_weight + weight

```

```

  proportions = empty dictionary
  FOR each station IN weights:
    proportions[station] = weights[station] / total_weight

```

```

  RETURN proportions
  // proportions is {NaPTAN: proportion}, all values sum to 1.0
  // stations closer to the venue receive a higher proportion

```

```

FUNCTION propagate(graph, station_id, people, event, network,
  MINIMUM_PEOPLE, PROPAGATION_FACTOR):

```

```

  station = network.nodes[station_id]

```

```

  IF station.IsAffected() == FALSE:
    // Replace the unaffected station object with an affected one
    new_station = AffectedStation(station_id, station.name)
    new_station.events[event] = people
    network.nodes[station_id] = new_station
    station = new_station

```

```

  ELSE:
    // Station already affected — check if this event has reached it before
    IF event IN station.events:
      old_people = station.events[event]
    ELSE:
      old_people = 0

```

```

  IF old_people >= people:
    RETURN
    // A larger or equal wave already propagated through here
    // No update needed and no further propagation required

```

```

  station.events[event] = people
  network.nodes[station_id] = station

```

```

  new_people = people × PROPAGATION_FACTOR

```

```

  IF new_people < MINIMUM_PEOPLE:
    RETURN // Base case: crowd too small to affect further stations

```

```

  FOR each neighbour IN graph[station_id]:
    neighbour_id = neighbour[0]
    CALL propagate(graph, neighbour_id, new_people, event,
      network, MINIMUM_PEOPLE, PROPAGATION_FACTOR)

```

```
FUNCTION calculate_crowding(username, date, journey_start, start_station,
                           end_station, database):
```

```
// — Step 1: Load user parameters —————
```

```
CONNECT to database
```

```
GET parameters for username FROM Users table:
```

```
    CHANGING_TIME, MAX_TIME_WINDOW,
    rawArrivalPeakOffset, rawDeparturePeakOffset,
    ATTENDANCE, TRAIN_PROPORTION,
    SIGMA_FACTOR, MINIMUM_PEOPLE,
    PERSON_DELAY, PROPAGATION_FACTOR
```

```
arrivalWindowMinutes = MAX_TIME_WINDOW
```

```
departureWindowMinutes = MAX_TIME_WINDOW
```

```
arrivalPeakOffset = rawArrivalPeakOffset // in minutes
```

```
departurePeakOffset = rawDeparturePeakOffset // in minutes
```

```
// — Step 2: Build base unaffected network —————
```

```
base_graph = CALL make_graph(database)
```

```
network.nodes = empty dictionary
```

```
network.edges = empty dictionary
```

```
FOR each station_id IN base_graph:
```

```
    GET station_name FROM Stations table WHERE NaPTAN = station_id
```

```
    network.nodes[station_id] = NotAffectedStation(station_id, station_name)
```

```
    FOR each connection IN base_graph[station_id]:
```

```
        station_b = connection[0]
```

```
        travel_time = connection[1]
```

```
        line_id = connection[2]
```

```
        key = (station_id, station_b, line_id)
```

```
        network.edges[key] = NotAffectedConnection(station_id, station_b,
                                                    line_id, travel_time)
```

```
// — Step 3: Estimate journey midpoint time —————
```

```
base_path = CALL dijkstra(base_graph, start_station, end_station,
                           CHANGING_TIME)
```

```
total_minutes = base_path[-1][3] // cumulative cost of last step
```

```
half_minutes = total_minutes / 2
```

```
midjourney_time = journey_start + half_minutes // datetime value
```

```
// — Step 4: Find relevant events —————
```

```
relevant_events = empty dictionary
```

```
// {(venue, date): [closest_diff_mins, start_diff_mins, end_diff_mins]}
```

```
GET all events FROM Events table WHERE Date = date
```

```
FOR each event IN events:
```

```
    event_start = PARSE event.Time as datetime
```

```
    event_end = event_start + event.Duration // Duration in minutes
```

```
    arrival_peak = event_start + arrivalPeakOffset
```

```
    departure_peak = event_end + departurePeakOffset
```

```
    start_diff_mins = ABS(arrival_peak - midjourney_time) in minutes
```

```
    end_diff_mins = ABS(departure_peak - midjourney_time) in minutes
```

```
    closest_diff_mins = MIN(start_diff_mins, end_diff_mins)
```

```
IF closest_diff_mins <= MAX_TIME_WINDOW:
```

```
    relevant_events[(event.venue, date)] =
```

```
        [closest_diff_mins, start_diff_mins, end_diff_mins]
```

```

// — Step 5: Distribute passengers to nearby stations —————
FOR each (venue, date) IN relevant_events:
    start_diff_mins = relevant_events[(venue, date)][1]
    end_diff_mins = relevant_events[(venue, date)][2]

    capacity = GET Capacity FROM Venues WHERE VenueName = venue

    total_attendance = capacity × ATTENDANCE
    rail_users = total_attendance × TRAIN_PROPORTION

    stations_with_distances = GET all rows FROM VenueStationRelationships
        WHERE VenueName = venue
    // returns {NaPTAN: distance_km}

    proportions = CALL inverse_weight(stations_with_distances)

    // Enforce minimum sigma of 1 to prevent division by zero inside
    // gaussian_factor if the user sets a very large SIGMA_FACTOR value
    sigma_arrival = MAX(1, arrivalWindowMinutes / SIGMA_FACTOR)
    sigma_departure = MAX(1, departureWindowMinutes / SIGMA_FACTOR)

    // Evaluate how much of peak demand is present at the journey midpoint
    // relative_time_difference = 0 because we evaluate AT the midpoint
    arrival_factor = CALL gaussian_factor(
        relative_peak = start_diff_mins - arrivalPeakOffset,
        sigma = sigma_arrival,
        relative_time_difference = 0)

    departure_factor = CALL gaussian_factor(
        relative_peak = end_diff_mins + departurePeakOffset,
        sigma = sigma_departure,
        relative_time_difference = 0)

    FOR each station_id IN proportions:
        arrival_people = rail_users × proportions[station_id] × arrival_factor
        departure_people = rail_users × proportions[station_id] × departure_factor
        people = MAX(arrival_people, departure_people)

        IF people >= MINIMUM_PEOPLE:
            CALL propagate(base_graph, station_id, people,
                (venue, date), network,
                MINIMUM_PEOPLE, PROPAGATION_FACTOR)

// — Step 6: Calculate station delay factors —————
FOR each station_id IN base_graph:
    station = network.nodes[station_id]

    IF station.IsAffected() == TRUE:
        total_people = 0
        FOR each event IN station.events:
            total_people = total_people + station.events[event]

        station.currentPassengerLoad = total_people
        station.DelayFactor = 1 + (total_people × PERSON_DELAY)
    // Unaffected stations retain their default DelayFactor of 1.0

// — Step 7: Calculate connection delay factors —————
FOR each key IN network.edges:
    station_a = key[0]
    station_b = key[1]
    line_id = key[2]

    delay_a = network.nodes[station_a].DelayFactor

```

```

delay_b = network.nodes[station_b].DelayFactor

IF delay_a != 1.0 OR delay_b != 1.0:
    // At least one endpoint is affected by crowding
    base_travel_time = network.edges[key].BaseTravelTime
    connection = AffectedConnection(station_a, station_b, line_id,
                                     base_travel_time, delay_a, delay_b)
    connection.DelayFactor = (delay_a + delay_b) / 2
    network.edges[key] = connection
// Unaffected connections retain their default DelayFactor of 1.0

// — Step 8: Spread delay uniformly across each line —————
line_connections = empty dictionary // {line_id: [connection, ...]}

GET all LineIDs FROM Lines table
FOR each line_id:
    line_connections[line_id] = empty list

FOR each key IN network.edges:
    line_id = key[2]
    line_connections[line_id].APPEND(network.edges[key])

FOR each line_id IN line_connections:
    connections_on_line = line_connections[line_id]
    n = LENGTH(connections_on_line)

    total_delay = 0
    FOR each connection IN connections_on_line:
        total_delay = total_delay + connection.DelayFactor

    average_line_delay = total_delay / n

    FOR each connection IN connections_on_line:
        connection.LineDelay = average_line_delay

CLOSE database connection
RETURN network

```

The algorithm begins by constructing an unaffected base representation of the transport network, where every station is initialised as a node with a neutral delay factor of 1.0 and every connection is stored with its base travel time. To determine when crowding should be evaluated, the midpoint of the journey is estimated by running Dijkstra's algorithm on the base graph, halving the total journey time, and adding that to the journey start time. This midpoint acts as the single point in time around which all crowding calculations are centred.

For each event on the travel date, the expected peak arrival time and peak departure time are calculated using configurable offsets. The smaller of the two differences between these peaks and the journey midpoint is taken, and only events where this difference falls within the user's maximum time window are included in the calculation.

For each relevant event, the expected number of passengers using TfL rail services is estimated from the stadium capacity, an attendance proportion, and a rail usage proportion. These passengers are distributed across nearby stations using inverse distance weighting. Each station receives a share proportional to $1 / (\text{distance} + 1)$, normalised so all proportions add to 1. A Gaussian function is then applied to scale this figure based on how far in time the journey midpoint is from the nearest demand peak.

Crowding is then propagated recursively outward through the graph. Each station that receives people above the minimum threshold passes a reduced count to its neighbours, multiplied by the propagation factor on each hop. A station is only updated and only propagates further if the incoming wave is strictly larger than any previously recorded wave for the same event, which both prevents redundant recursion and guarantees termination.

Once propagation is complete, each affected station's total passenger load is summed across all contributing events and converted into a delay factor using $\text{DelayFactor} = 1 + (\text{totalPassengers} \times \text{PERSON_DELAY})$. Each connection then receives a delay factor equal to the average of its two endpoint stations' delay factors. Finally, all connections on the same line are grouped and assigned the average delay factor across the whole line.

Affected graph generation:

```

FUNCTION create_affected_graph(base_graph, affected_network):
  CREATE affected_graph as a deep copy of base_graph

  FOR each StationA in affected_graph:
    GET connections from affected_graph[StationA]

    FOR each connection in connections:
      GET StationB
      GET BaseTravelTime
      GET LineID

      key = (StationA, StationB, LineID)
      GET LineDelay from affected_network.edges using key

      new_travel_time = BaseTravelTime * LineDelay
      UPDATE connection travel time to new_travel_time

  RETURN affected_graph

```

The algorithm generates a modified version of the transport network that accounts for delays due to crowding. To make sure the original network's data is preserved, a deep copy of the base graph is created. For every connection in the graph, the corresponding delay factor is retrieved from the affected network object which contains crowding information calculated earlier. Each connection's base travel time is multiplied by its line delay factor, producing a new travel time that reflects the impact of crowding. The resulting affected graph can then be used by the pathfinding algorithm to determine routes that minimise travel time.

Crowding severity classification:

```

FUNCTION calculate_path_severity(path, start_station,
                                affected_network, NORMAL,
                                SLIGHTLY, BUSY):

```

```

CREATE empty list descriptions

ADD [start_station,
    classify(affected_network.nodes[start_station].DelayFactor)]
    to descriptions

FOR each step in path:
    GET current_station from step
    GET DelayFactor from affected_network.nodes[current_station]

    IF DelayFactor > BUSY:
        severity = "Severe"
    ELSE IF DelayFactor > SLIGHTLY:
        severity = "Busy"
    ELSE IF DelayFactor > NORMAL:
        severity = "Slightly"
    ELSE:
        severity = "Normal"

    ADD [current_station, severity] to descriptions

RETURN descriptions

```

The algorithm processes the stations along a journey path, starting with the origin station, and then iterating through each subsequent station. For each station, the delay factor is compared against the thresholds (user-defined) to classify crowding as Normal, Slightly, Busy or Severe. The output is a list of stations paired with their crowding descriptions.

Libraries and languages

Languages:

Python was chosen as the main backend language because of its flexibility, readability, and strong support for object oriented programming, and algorithmic problem solving. This made it particularly suitable for implementing complex components, and interaction with external APIs. While alternatives such as Java or C# could have been used, Python allowed for faster development with less boilerplate code, which simplifies debugging and maintenance.

The system is implemented as a web application, allowing it to be accessed on any device with a web browser. The front end was therefore developed using HTML, CSS, and JavaScript, which are standard, widely supported web technologies. HTML and CSS are used to structure the interface, while JavaScript is used to provide interactivity, and asynchronous behaviors such as checking database creation status without requiring page reloads. This approach was chosen over a platform specific application, to maximise accessibility and ease of deployment.

Libraries:

Below are the libraries that were used to write the program. As majority of the programme was written in Python, and the back-end is in Python, most of the libraries are Python libraries, however, there are also a couple JavaScript libraries used.

1. [Flask](#)

I decided to use the web framework Flask to develop the application. Flask is a lightweight Python web framework that provides the core functionality required to handle routing, user sessions, and communication between the front end and backend logic. Unlike larger frameworks such as Django, Flask doesn't impose a strict structure, making it easier to integrate complex backend logic and background threads.

2. [SQLite](#)

The sqlite3 Python library was used to manage the project's relational database. SQLite was chosen because it is lightweight, requires no separate server, and stores the entire database in a single file.

3. [OpenCage Geocoder](#)

The OpenCage Geocoder Python library was used to convert stadium names into geographic coordinates. This was used over methods such as using the Google Maps API, due to its simplicity and cost benefit.

4. [Requests](#)

The requests Python library was used to communicate with external APIs. It gives a simple and readable interface for sending HTTP requests and handling JSON responses.

5. [BeautifulSoup](#)

The BeautifulSoup Python library was used to scrape football fixture data from BBC Sport webpages. This allowed the system to automatically gather data, without manual input. BeautifulSoup was chosen because it is designed for parsing and navigating HTML documents in a robust and readable way.

6. [Geopy](#)

The geopy Python library was used to calculate distances between coordinates. This was used to determine what number of people are allocated to stations based on distance from a stadium.

7. [Heapq](#)

The heapq Python library was used to implement priority queues within the modified Dijkstra's shortest path algorithm, which allows the program to efficiently select the next station with the lowest current travel time.

8. [Abc](#)

The abc Python library was used to define abstract base classes. This enforces consistency between classes, particularly for methods like delay calculation, more

easily allowing for polymorphism to be used when handling affected and non-affected elements (objects) in the network.

9. Hashlib

The hashlib Python library was used to securely hash user passwords together with a randomly generated salt before storing them in the database. This was done to improve security and prevent passwords from being stored in plain text.

10. Threading

The threading Python library was used to perform long running tasks, such as database creation and data fetching, in the background. This prevents the web application from freezing while these tasks are running and improves the overall user experience.

11. JSON

The json Python library was used to parse and handle JSON data from external APIs, and from local JSON files (some of which contain hard coded data).

12. Datetime

The datetime Python library was used to handle date and time related functionality, such as conversion of inputted or stored date and time data, into a format to be outputted or used in calculations. It is also used to get the current date and time.

13. Math

The math Python library provided access to mathematical functions, especially relevant in the calculation of the Gaussian factor, where an exponential function was used, and the implementation of Dijkstra's shortest path algorithm, where the infinity object was used.

14. Collections

The collections Python library was used to simplify the management of complex data structures. Specifically, defaultdict was used to initialise a dictionary that stores the best known travel time to each station, with a default value of infinity. This ensures that any missing station data would automatically be assigned a value of infinity, which helped to simplify the implementation of Dijkstra's shortest path algorithm.

15. Copy

The copy Python library was used to create deep copies of data structures, specifically when generating an affected graph from the base graph. This ensures that modifications made to the affected graph did not alter the original base graph.

16. Time

The time Python library was used to control execution timing, particularly through the use of the sleep functionality, which was especially used in request reattempts. This ensured that too many and too frequent requests weren't being made.

17. Re

The re (regular expressions) Python library was used in the web scraping component to extract specific elements in the scraped HTML content.

18. Os

The os Python library was used for interacting with the operating system, primarily to delete the database file when rebuilding the database from scratch. This ensured that the database could be safely created without leaving corrupted or outdated files.

19. Pathlib

The pathlib Python library was used to handle file system paths. It was primarily used to check whether files exist before attempting to access them, preventing runtime errors and improving the reliability of the application.

20. jQuery

jQuery is a lightweight JavaScript library. It was primarily used to make sure JavaScript code was only used once the DOM was fully loaded, which helped to prevent errors due to elements not being available yet. It also served as a dependency for the Select2 library.

21. Select2

Select2 was used to implement a searchable dropdown menu for selecting start and end stations.

Security

Login system

User passwords were not in plain text within the database. Instead, each password is protected using cryptographic hashing, with a unique salt. When a user account is created, a random salt is generated, using `os.urandom(16)`. This ensures that each user has a unique salt, which protects against rainbow table and pre-computer hash attacks. The salt is stored alongside the hashed password in the database. The user's password is combined with the salt and then hashed using the SHA-256 hashing algorithm. Only the resulting hash and the salt are stored in the database, along with the username, and the associated user values to be stored on the same row. As hashing is a one way process the original password can't be retrieved.

During login, the password entered by the user is hashed again using the stored salt and compared to the stored hash. Authentication will only be successful if the two hashes match.

Integrity of data

The database is designed in third normal form, to maintain data integrity, and reduce redundancy. Each table represents a single entity, all non-key attributes depend fully on the primary key, and there are no transitive dependencies. This ensures that data is stored consistently and can't become contradictory across the database.

Individual records are not deleted from the database. When transport or event data is updated, the entire database is rebuilt, preventing inconsistencies caused by partial updates or orphaned relational data. User accounts are preserved during this process, by storing all their data in memory temporarily, and as the Users table doesn't reference any other tables.

Additionally, data is validated before being written into the database, and any anomalous or unrealistic data is disregarded to prevent it from negatively affecting the accuracy of the system's outputs. If when updating the data to be fetched from external sources (APIs and scraping), the data fetching fails or the formatting is not in the right manner, the database will refuse to be created until the issue is resolved, and the corrupted data will not be inputted. Instead, only when the data is correct will the data be stored and if this is not achieved, the database will be continuously wiped or deleted.

Technical Solution

System overview

UML diagrams for the programme can be found in the Design section of this document, along with example SQL queries. An entity relationships diagram for the database can be found in the Analysis section of this document.

The implementation of Dijkstra's shortest path algorithm used in the final programme is built off from the prototype in the Analysis section.

The UI of the programme is mostly similar to the UI sketches in the Design section of this document, and actual evidence of the UI can be found in the testing videos.

Technical breakdown

The system is broken down into six interlinked components using the principles of procedural decomposition.

Database component:

The system uses a single SQLite database file stored locally on the user's machine. The database is designed to be in Third Normal Form, which means that each table stores information about exactly one entity and all non-key attributes depend only on the primary key. Junction tables are used wherever many-to-many relationships exist between entities, which avoids data duplication and keeps the schema clean and consistent. All foreign key relationships are declared explicitly to maintain referential integrity.

The schema consists of nine tables:

- **Stations** stores every station served by TfL rail services. Its fields are NaPTAN (nationally unique identifier used by TfL to identify each station), StationName (the official name of the station as returned by the TfL API), Latitude, and Longitude. NaPTAN is the primary key, which ensures every station has a unique identifier that can be referenced by other tables.
- **Lines** stores each line with its internal TfL identifier (LineID), its display name (LineName), and its average train speed (AverageSpeed) in km/h. This data is not fetched dynamically but is instead loaded from a local JSON file, since average line speeds do not change frequently enough to require regular updates from an API.
- **StationLineRelationships** is a junction table that handles the many-to-many relationship between stations and lines, since a station can be served by multiple lines and a line serves multiple stations. Its composite primary key is (NaPTAN, LineID), and it holds foreign keys referencing both the Stations and Lines tables.

- **Connections** stores every directed edge in the transport graph. Each record represents a valid move from one station (StationA) to another (StationB) via a specific line (LineID), and also stores the base travel time (BaseTravelTime) for that connection. Because direction matters for pathfinding, travelling from A to B is not necessarily the same as travelling from B to A, and both directions of each connection are stored as separate rows. The composite primary key is (StationA, StationB, LineID), and the table holds foreign keys referencing both stations and the line.
- **Venues** stores each London football stadium, including its name (VenueName), its official capacity (Capacity), and its geographic coordinates (Latitude, Longitude). VenueName is the primary key and is referenced by the VenueStationRelationships and Events tables.
- **VenueStationRelationships** is a junction table that links venues to their nearby stations and stores the walking distance between each pair. Its composite primary key is (VenueName, NaPTAN). The reason for this design of the table is further explained below.
- **Events** stores upcoming football fixtures, with fields for the venue (VenueName), date (Date), kick-off time (Time), home team (HomeTeam), away team (AwayTeam), league name (EventName), and match duration (Duration). The composite primary key is (VenueName, Date), which reflects the assumption that at most one fixture per venue per day is relevant. VenueName is a foreign key referencing Venues, and HomeTeam is a foreign key referencing Teams.
- **Teams** links each London based professional football club to its home stadium, with fields for the team name (TeamName) and the name of its home venue (VenueName). This foreign key relationship means that when the scraper stores a fixture under a home team, the corresponding venue can always be resolved.
- **Users** stores the login credentials and all fifteen user-adjustable parameters for each account. The credential fields are Username (the primary key), PasswordHash, and Salt. The parameter fields cover all the values used by the crowding and pathfinding algorithms, including CHANGING_TIME, MAX_TIME_WINDOW, rawArrivalPeakOffset, rawDeparturePeakOffset, ATTENDANCE, TRAIN_PROPORTION, SIGMA_FACTOR, MINIMUM_PEOPLE, PERSON_DELAY, PROPAGATION_FACTOR, WALKING_TIME, WALKING_SPEED, NORMAL, SLIGHTLY, and BUSY.

The parameters WALKING_TIME and WALKING_SPEED, which together determine how far a user is willing to walk from a station to reach a venue, are stored individually in the Users table because different users may have different walking speeds or preferences. This means that two users with different walking parameters would expect the system to use a different set of nearby stations when calculating how crowds spread from a venue across the network.

One solution would be to add a Username column to VenueStationRelationships and store a separate set of relationships for every user. However, this would mean duplicating the entire set of venue-station pairs for each account in the database, which is unnecessary because the system is an application where each installation runs with its own local database file, meaning there is never any concurrent access from multiple users. Taking advantage of this, VenueStationRelationships is treated as a single shared resource that is always configured

for the currently active user. On every login and every account creation, the system retrieves the active user's WALKING_TIME and WALKING_SPEED from the Users table, deletes all existing rows from VenueStationRelationships, and repopulates the table using those parameters. This ensures the table always reflects the correct set of nearby stations for whoever is logged in, without storing any duplicate data or adding unnecessary columns to the schema.

Another solution would be to calculate venue-station relationships on the fly during each journey search rather than storing them in the database. However, this requires calculating the distances between every venue and station in the network on every search, and adding this would add a noticeable delay, particularly on lower-end hardware. By running the calculation once at login and caching the results in the database, the system avoids this entirely during journey searches. Instead, there is an additional delay, when creating an account or logging in, which is acceptable as this is only done once, till another user logs in.

Data Acquisition Component

The TfL Unified API is used to fetch station and network data. The API is called once per line to retrieve the ordered sequence of stations along each branch of that line. From this, Station objects are created and stored in the Stations table. Connections are then derived by iterating through each consecutive pair of stations on a branch, calculating the geodesic distance between their coordinates using the geopy library, and dividing by the line's average speed to produce a base travel time in minutes. A fixed dwell time constant of 0.5 minutes is added to every connection to represent the time a train waits at a platform. Because the TfL API occasionally returns stations under different identifiers depending on the context, a dictionary of corrected NaPTAN mappings is maintained during the fetch to ensure each station is stored under its correct hub identifier.

The OpenCage Geocoding API is used to convert each stadium name into geographic coordinates. For each London club in the local JSON configuration file, the API is queried with the stadium name, and the resulting latitude and longitude are stored in the Venues table. Using these coordinates and the coordinates of every station in the database, the geodesic distance from each stadium to every station is calculated. Stations are sorted by distance from the stadium, and the closest station is always included in VenueStationRelationships regardless of distance, to ensure every venue always has at least one associated station. Every other station within the user's calculated walking distance is also added, along with its distance value.

The BBC Sport website is used to collect upcoming fixture data. The next twelve months of fixtures are fetched by constructing a URL for each English football league and each monthly page, then parsing the HTML using BeautifulSoup. For each date heading found on the page, all fixtures listed beneath it are extracted, capturing the home team, away team, kick-off time, and league. If a kick-off time has not yet been confirmed, the time field is stored as "TBC" rather than being left empty, which prevents downstream code from failing when trying to parse the time. Fixtures whose home team is not a London club are then filtered out, and only the relevant matches are stored in the Events table.

All three data sources are accessed through a shared `safeGet` utility function, which wraps every HTTP request in a try/except block and implements retry logic.

Pathfinding Component

The `MakeGraph` function fetches all rows from the `Connections` table and builds an adjacency list, implemented as a Python dictionary, where each key is a station NaPTAN and the corresponding value is a list of neighbouring stations. Each neighbour entry contains the connected station's NaPTAN, the base travel time to reach it, and the line used. Because both directions of each connection are stored in the database, the resulting graph is bidirectional. This dictionary is then passed into the pathfinding algorithm as the representation of the network.

The implementation of Dijkstra's shortest path algorithm treats the search state not simply as a station, but as a pair in the form (station, current line). This is necessary because the cost of reaching a station depends on which line was used to arrive there. If the next step of the journey requires switching to a different line, a fixed time penalty (`CHANGING_TIME`) is added to the travel time for that step. The algorithm uses a min-heap priority queue to always explore from the lowest-cost known state first, and a visited set to avoid re-exploring states that have already been finalised. Once the goal station is reached, the algorithm backtracks through a dictionary of parent states to reconstruct the full path. The algorithm returns the path as an ordered list of steps, where each step is in the form (StationA, StationB, LineUsed, CumulativeTime), and CumulativeTime is the total journey time from the start to reach StationB.

Crowding Algorithm Component:

The algorithm begins by running the pathfinding component on the base uncrowded graph to find the fastest route between the user's start station and destination. Half of the total journey time is then added to the journey start time to produce an estimated midpoint time. This midpoint is used as the reference point for all crowding calculations that follow, since it represents when the user is most likely to be in transit and therefore most affected by crowding.

For each fixture on the travel date, the algorithm calculates two time references, which are the expected arrival peak, which is the time shortly before kick-off when most fans are arriving at the stadium, and the expected departure peak, which is the time shortly after the final whistle. If either peak falls within the user's `MAX_TIME_WINDOW` of the journey midpoint, the event is included in the crowding calculation. This ensures that only events which realistically overlap with the user's journey are considered.

For each relevant event, the maximum number of passengers expected to use TfL rail is calculated by multiplying the stadium's capacity by the `ATTENDANCE` proportion and then by the `TRAIN_PROPORTION`. These passengers are distributed across the venue's nearby

stations using inverse distance weighting, so that stations closer to the stadium receive a larger proportion than those further away. A Gaussian function is then applied to scale the number of passengers based on how far the journey midpoint is in time from the nearest demand peak, producing an estimate of how many additional passengers are actually present on the network at the time the user is travelling.

Starting from each venue-adjacent station, crowd numbers are propagated outward through the graph. Each neighbouring station in the graph receives a proportion of the originating station's passenger count, determined by the `PROPAGATION_FACTOR` parameter. This process repeats recursively until the number of people being propagated falls below the `MINIMUM_PEOPLE` threshold, at which point the recursion stops.

Once propagation is complete, each affected station's total passenger load is calculated by adding the contributions from all relevant events. This load is multiplied by the `PERSON_DELAY` parameter to produce a station-level delay factor, which represents the proportion by which travel times involving that station are increased. Stations that were not affected by any event retain a delay factor of 1.0, which means that their travel times are unchanged.

The individual delay factors of connections on the same line are averaged together, and this average is applied uniformly to all connections on that line as a line-level delay factor. This reflects how rail delays tend to propagate along an entire line in practice, rather than remaining localised to a single station or connection.

The result of the algorithm is a `Network` object, which contains all station nodes with their delay factors and all connection edges with their line-level delay factors. This object is passed to the results component.

Results Component

The base graph is deep copied to ensure the original is not modified, and each connection's travel time is then multiplied by its line's delay factor from the `Network` object. This produces an affected graph where every edge reflects the estimated impact of crowding on travel time.

The pathfinding algorithm is run three times. First, on the base graph to find the original fastest route without crowding, which is used for calculation purposes and not displayed. Second, on a stripped version of the affected graph that contains only the connections used by the original route, which produces the journey time a user on their usual route would experience under crowded conditions to show how much slower the original route becomes on a match day. Third, on the full affected graph to find a new optimised route that accounts for delays, which may differ from the original if a less delayed alternative exists.

For each of the two displayed routes, every station along the path is assigned a qualitative crowding description. The station's delay factor is compared against the user's `NORMAL`, `SLIGHTLY`, and `BUSY` threshold parameters to classify it as Normal, Slightly Busy, Busy, or

Severely Busy. This gives the user a readable indication of what to expect at each station along their journey.

The component returns a Results object, which acts as a container for all three route objects and their associated crowding descriptions. This object is then passed to the display component for rendering.

Display Component:

When the application starts, all file paths and API keys are stored in Flask's app.config dictionary. This includes the path to the database file, the paths to all JSON configuration and flag files, and both the TfL and OpenCage API keys. Every route handler retrieves these values from the config dictionary rather than using hardcoded paths, which means that if any file location needs to change, it only needs to be updated in one place.

The login page is the default entry point for the application. On each request, the system checks whether the database file exists using `pathlib.Path.exists()`. If it does not exist, database creation is started in a background thread so that the application remains responsive, and the user is redirected to a loading page. If the database exists, the user can either log in with an existing account or create a new one. On successful authentication, the username is stored in Flask's server-side session and the user is redirected to the main page. On every subsequent page, the session is checked before any processing takes place, and users without an active session are immediately redirected back to the login page.

Because database creation involves multiple API calls and can take a significant amount of time, the loading page polls a status endpoint every two seconds rather than making the user wait on a blank screen. The status endpoint reads a flags JSON file that the background thread writes to when it finishes. If the status is "success," the user is redirected to the main page. If the status is failed, the user is redirected to an error page where they are given the option to retry.

In the main page, the user selects a start station and destination station using a searchable dropdown, which is populated dynamically via an AJAX call to a station search endpoint that queries the database and returns matching station names based on the user's input. The user also enters a date and time of travel. On submission, the server validates that the two stations are different and that the selected time is in the future before passing the inputs to the results component. The three route outputs are then rendered on the same page, showing the line segments, station names, qualitative crowding descriptions, total journey time, and estimated arrival time for each route.

The settings page displays all fifteen of the user's adjustable parameters in an editable table. The user can save changes, which updates the database and, if the walking parameters have changed, triggers a recalculation of the `VenueStationRelationships` table for that user. The user can also load the default parameter values from the configuration file without navigating away from the page, or trigger a data refresh which backs up the current

user data, rebuilds the database with freshly fetched external data, and then restores the user data once the rebuild is complete.

Key code

BBC Sport event scraper:

```
import bs4
import datetime
from customFunctions import safeGet
import re
import classes
import json

#So that 12 months of fixtures can be fetched from the date of request
def getNextTwelveMonths():
    currentDate = datetime.datetime.now()
    months = []
    year = currentDate.year
    month = currentDate.month

    for monthNumber in range(12):
        monthString = str(month)
        if len(monthString) == 1:
            monthString = "0" + monthString
        yearMonth = f"{year}-{monthString}"
        months.append(yearMonth)
        month += 1
    if month > 12:
        month = 1
        year += 1

    return months #results in [yyyy-mm, ...]

#To get the actual numerical date of the month from the string fetched because it is originally in a form like "Saturday 12th December"
def getDate(monthYear, restOfDate):
    try:
        # Extract the day number from the string
        dayNumber = re.search(r"\d{1,2}", restOfDate).group()
        if len(dayNumber) == 1:
            dayNumber = "0" + dayNumber
        year, month = monthYear.split("-")
        date = f"{dayNumber}-{month}-{year}"
        return date
    except:
        # Returned when the text does not represent a valid date
        return "Error"

# Fetches and parses fixtures from a BBC Sport fixtures page
def getPageFixtures(url, monthYear, tierName, fixtures):
    response = safeGet(url)
    if type(response) is int:
        return response
    soup = bs4.BeautifulSoup(response.text, "html.parser")

    # Check if the page contains a link to today's fixtures
```

```

# If so, ensure today's date exists in the fixtures dictionary
# This is because they are represented slightly differently in the page (no date, just the word "today")
today_link = soup.find("a", id="today")
if today_link:
    currentDate = datetime.datetime.now().strftime("%d-%m-%Y") #Today's date in dd-mm-yyyy format
    try:
        test = fixtures[currentDate] #Basically to check if it exists
    except:
        fixtures[currentDate] = []

#On the webpage, the fixtures are below each date, so fetch the date first and find all fixtures until the next date, so that
these fixtures can be stored under the stored date
for tag in soup.find_all(["h2", "span"]):
    if tag.name == "h2": #This is where the date can be found
        text = tag.get_text(strip=True)
        currentDateText = text
        currentDate = getDate(monthYear, currentDateText)
        # Creates a list to hold fixtures of the current date. If already made by another league, this prevents it being
        overwritten
        try:
            test = fixtures[currentDate]
            if currentDate == "Error":
                raise Exception("Not Date")
        except:
            fixtures[currentDate] = []

    elif tag.name == "span" and "versus" in tag.get_text(): #Where fixtures are stored
        if currentDate != "Error":
            text = tag.get_text(strip=True)
            parts = text.split("versus")
            home = parts[0].strip()
            away_time = parts[1].strip()

            # Extract time if present
            if "kick off" in away_time:
                away, time = away_time.split(" kick off ")
                time = time.strip()
            else:
                away, time = away_time, "TBC"

            # Store fixture data for me
            fixtures[currentDate].append({
                "league": tierName,
                "home": home,
                "away": away.strip(),
                "time": time
            })

return fixtures

# Main function for scraping fixtures and storing them in the database
def main(databaseFile, leaguesFile):
    fixtures = {}

    # Load league tiers and their BBC identifiers
    with open(leaguesFile, "r") as leaguesFileObject:
        tiers = json.load(leaguesFileObject)
        dateAdditions = getNextTwelveMonths()

    #Results in a dictionary of lists containing dictionaries in the format : {Date: [{"league": "A", "home": "B", "away": "C",
    "time": "D"}]}
    for tierName in tiers.keys():
        tierID = tiers[tierName]

```

```

for dateAddition in dateAdditions:
    url = f"https://www.bbc.co.uk/sport/football/{tierID}/scores-fixtures/{dateAddition}?filter=fixtures"
    fixtures = getPageFixtures(url, dateAddition, tierName, fixtures)

    # Catch errors returned by safeGet
    if type(fixtures) is int:
        return fixtures

# Convert raw fixture dictionaries into class objects
# This allows filtering and database storage using class methods
dateObjectList = []
for dateString in fixtures.keys():
    listOfFixtureObjects = []
    for fixtureDictionary in fixtures[dateString]:
        league = fixtureDictionary["league"]
        home = fixtureDictionary["home"]
        away = fixtureDictionary["away"]
        time = fixtureDictionary["time"]

        fixtureObject = classes.Fixture(league, home, away, time)
        listOfFixtureObjects.append(fixtureObject)

    dateObject = classes.Date(dateString, listOfFixtureObjects)
    dateObjectList.append(dateObject)

# AllDates object contains all Date objects and applies filtering logic
AllDatesObject = classes.AllDates(dateObjectList)

# Remove non-London fixtures using the Teams table
AllDatesObject.findRelevantDates(databaseFile)
AllDatesObject.storeInDatabase(databaseFile)

return True

```

```

import sqlite3

#GLOBAL PARAMETERS
#Duration of a match including breaks
DURATION = 120 # Minutes

# Fixture object acts as a container for scraped fixture data for me
class Fixture():
    def __init__(self, league, home, away, time):
        self.league = league # League the fixture belongs to (e.g. Premier League)
        self.home = home # Home team
        self.away = away # Away team
        self.time = time # Kick-off time (or TBC)

#Date object contains the date and a list of fixture objects
#Date object's findRelevantFixtures method deletes all fixtures not in London
class Date():
    def __init__(self, date, fixtures):
        self.date = date # Date in dd-mm-yyyy format
        self.fixtures = fixtures # List of Fixture objects on this date

# Removes all fixtures whose home team is not a London club

```

```

def findRelevantFixtures(self, londonClubs):
    relevantFixtures = []

    for fixture in self.fixtures:
        home = fixture.home
        London = False

        #Check if home is in London
        if home in londonClubs:
            London = True

        if London:
            relevantFixtures.append(fixture)

    # Replace fixtures list with only relevant fixtures
    self.fixtures = relevantFixtures

#Contains a list of date objects
class AllDates():
    def __init__(self, dates):
        self.dates = dates # List of Date objects

# Filters out dates that do not contain any London fixtures and the fixtures on that date that are not relevant
def findRelevantDates(self, databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    londonClubs = []

    cursor.execute("""
        SELECT TeamName
        FROM Teams
    """)
    rows = cursor.fetchall()
    for clubTuple in rows:
        londonClubs.append(clubTuple[0])

    relevantDates = []
    for dateObject in self.dates:
        # Filter fixtures within the date
        dateObject.findRelevantFixtures(londonClubs) #Filters out all non-relevant

        # Only keep dates that still contain at least one fixture
        if len(dateObject.fixtures) != 0:
            relevantDates.append(dateObject) #This means at least one event on this date is in London
    self.dates = relevantDates

    connection.close()

# Stores all remaining fixtures into the Events table
def storeInDatabase(self, databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Convert date format from dd-mm-yyyy to dd/mm/yyyy
    for dateObject in self.dates:
        Date = dateObject.date
        day, month, year = Date.split("-")
        Date = f"{day}/{month}/{year}"
        for fixture in dateObject.fixtures:
            HomeTeam = fixture.home

```

```

AwayTeam = fixture.away
Time = fixture.time
Name = fixture.league

# Fetch the venue associated with the home team
cursor.execute("""
SELECT VenueName
FROM Teams
WHERE TeamName = ?
""", (HomeTeam,))
VenueName = cursor.fetchall()[0][0]

# Store the event in the Events table
# Duration is fixed as an assumption
cursor.execute("""
INSERT OR REPLACE INTO Events (VenueName, Date, Time, HomeTeam, AwayTeam, EventName, Duration)
VALUES (?, ?, ?, ?, ?, ?, ?)
""", (VenueName, Date, Time, HomeTeam, AwayTeam, Name, DURATION),)

connection.commit()

connection.close()

```

The scraper first loads a JSON file which maps the names of the leagues, to BBC Sport's internal identifiers. It then generates a list of year-month strings, starting from the current date to allow for the scrapping of upcoming fixtures over the next twelve months. For each combination of league and month, it constructs a URL to BBC Sports's fixtures page (pattern can be found above in the code). This page is then parsed using BeautifulSoup and extract fixture data from the page's DOM structure.

BBC Sport's HTML layout is organised such that <h2> tags containing dates in the format which can be found in the code above, is followed by individual fixtures in tags containing the word "versus" with the tags containing home team, away team and time of kick off. This means the function iterates through all tags finding <h2> elements to establish the current date, and captures all subsequent fixture elements until the next date header is encountered. Regular expressions are used to extract the numeric day from the date strings and combined with the year-month from the URL, to get the date.

The scraper handles edge cases like "today" fixtures which BBC marks with an anchor tag called "today" by using Python's datetime.now() to get the current date. Fixtures without a confirmed kick-off time default to "TBC."

All the extracted data are accumulated in a dictionary, with the structure found above. After all API pages are scraped across all leagues and months, the data are converted into a three level object hierarchy, where the AllDates object contains Date objects, which contain Fixture objects. This allows for methods to be run to filter out non-London based clubs and store the events in the database.

Venue-station relationship mapping:

```
import sqlite3
```

```

from opencage.geocoder import OpenCageGeocode
import json
from geopy.distance import geodesic

# Use of the API to get coordinates
def getCoordinates(key, location):
    geocoder = OpenCageGeocode(key)

    # The first result is used as it is assumed to be the most relevant and correct
    results = geocoder.geocode(location)[0]["geometry"] # Where you can get the coordinates
    return results

# Finds all stations within walking distance of a venue
# Ensures at least one station is always returned (the closest one)
def getCloseStations(allStations, venueCoordinates, walkingDistance):
    stationWithDistance = []
    CloseStationDictionary = {}

    # Calculate distance from the venue to every station
    for stationTuple in allStations:
        stationCoordinates = (stationTuple[1], stationTuple[2])
        distanceKM = geodesic(venueCoordinates, stationCoordinates).km
        stationWithDistance.append((stationTuple[0], distanceKM))

    # Sort stations by distance from the venue
    stationWithDistance.sort(key=lambda x: x[1])

    # Always include the closest station to guarantee at least one station per venue
    firstStation = stationWithDistance.pop(0)
    CloseStationDictionary[firstStation[0]] = firstStation[1]

    # Add all stations within the walking distance threshold
    for station in stationWithDistance:
        if station[1] <= walkingDistance:
            CloseStationDictionary[station[0]] = station[1]

    return CloseStationDictionary

# Inserts stadium, team, and venue-station relationship data for me into the database
def insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    latitude = coordinates["lat"]
    longitude = coordinates["lng"]
    venueCoordinates = (latitude, longitude)

    cursor.execute("""
        SELECT NaPTAN, Latitude, Longitude
        FROM Stations
    """)
    allStations = cursor.fetchall()

    # Find stations that are within walking distance of the stadium
    closeStations = getCloseStations(allStations, venueCoordinates, walkingDistance)

    # Store the venue-station relationships and walking distances
    for stationID in closeStations.keys():
        distance = closeStations[stationID]
        cursor.execute("""
            INSERT INTO VenueStationRelationships

```

```

VALUES (?, ?, ?)
ON CONFLICT (VenueName, NaPTAN) DO
UPDATE SET
    Distance = excluded.Distance
"""', (stadiumName, stationID, distance))

# Store team and home venue relationship
cursor.execute("""
INSERT INTO Teams (TeamName, VenueName)
VALUES (?, ?)
ON CONFLICT (TeamName)
DO UPDATE SET
    VenueName = excluded.VenueName
"""', (club, stadiumName))

# Store venue details
cursor.execute("""
INSERT INTO Venues (VenueName, Capacity, Latitude, Longitude)
VALUES (?, ?, ?, ?)
ON CONFLICT (VenueName)
DO UPDATE SET
    Capacity = excluded.Capacity,
    Latitude = excluded.Latitude,
    Longitude = excluded.Longitude
"""', (stadiumName, capacity, latitude, longitude))

connection.commit()
connection.close()

# Uses JSON file of club data for me
#{ "ClubName": [StadiumName, Capacity] }
def getStadiumData(londonClubsFile, databaseFile, key, walkingDistance):
    with open(londonClubsFile, "r") as file:
        londonClubs = json.load(file)

    # Process every London club and its stadium
    for club, data in londonClubs.items():
        stadiumName, capacity = data
        try:
            coordinates = getCoordinates(key, stadiumName)
        except:
            # Return error message if OpenCage API fails
            return "OpenCage Geocoder API"
        insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance)

    return True

# Calculates the maximum walking distance using walking speed and walking time
def calculateWalkingDistance(walkingTime, walkingSpeed):
    # walkingSpeed is in km/h and walkingTime is in minutes
    distance = (walkingSpeed * walkingTime) / 60
    return distance

# Main function for fetching and storing stadium related data for me
def main(key, londonClubsFile, databaseFile, walkingTime, walkingSpeed):
    # Calculate what is considered a reasonable walking distance
    walkingDistance = calculateWalkingDistance(walkingTime, walkingSpeed)

    done = getStadiumData(londonClubsFile, databaseFile, key, walkingDistance)
    print(done)

```

```

return done

# Recalculates and reinserts venue-station relationships when walking parameters change
def redoStore(walkingTime, walkingSpeed, londonClubsFile, databaseFile):
    walkingDistance = calculateWalkingDistance(walkingTime, walkingSpeed)

    with open(londonClubsFile, "r") as file:
        londonClubs = json.load(file)

    # Clear existing venue-station relationships
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("DELETE FROM VenueStationRelationships")
    connection.commit()
    connection.close()

    # Reinsert relationships using updated walking distance
    for club, data in londonClubs.items():
        stadiumName, capacity = data

        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()

        # Fetch stored venue coordinates to avoid unnecessary API calls
        cursor.execute("""
            SELECT Latitude, Longitude
            FROM Venues
            WHERE VenueName = ?
            """, (stadiumName, ))

        coordinates = {}
        coordinates["lat"], coordinates["lng"] = cursor.fetchall()[0]

        insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance)

    connection.close()

```

The venue-station relationship mapping element of the programming takes the coordinates of the venue and a walking distance threshold, which is previously calculated using user parameters.

The code creates a list of (stationID, distance) tuples, sorts the list in ascending order to find the nearest stations. One safety feature is that the top item is always added to the list of accepted stations, regardless of distance, to ensure that every venue has at least one associated station. After guaranteeing the closest station, the function iterates through remaining stations and includes any whose distance is less than or equal to the walking distance threshold, returning a dictionary mapping station NaPTAN codes to their walking distances from the venue. This data is then inserted into the database.

To ensure data consistency with user-specific walking parameters, the system calls the `redoStore()` function whenever a user logs in, signs up, or updates their walking parameters. This function fetches that user's walking time and speed parameters from the Users table and completely recalculates all venue-station relationships by deleting all rows from `VenueStationRelationships` and reinserting them based on the current user's walking distance threshold. This approach is appropriate for this application as each installation has

its own local database file, which ensures that the VenueStationRelationships table always contains data configured for the currently logged in user without requiring user-specific columns in the table or storing duplicate relationship data for every user.

TfL network data acquisition:

This part of the programme is explained below in the Calling Parameterised Web Service APIs and Parsing JSON subsection of the Complex programming techniques section.

Crowding Simulation Algorithm:

This section is also explained in the Complex programming techniques section. It has also been explained in the Design section of the document.

Modified Dijkstra's algorithm with line-change penalty:

As with the two above, this can be found in the Complex programming techniques section. It has also been explained and covered in the Design and Analysis sections of the document.

Complex programming techniques

Complex data model in database:

```
# Creates the SQLite database and all required tables
# If tables already exist, they will not be recreated
def createDatabase(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.executescript("""
        CREATE TABLE IF NOT EXISTS Stations (
            NaPTAN TEXT PRIMARY KEY,
            StationName TEXT,
            Latitude REAL,
            Longitude REAL
        );

        CREATE TABLE IF NOT EXISTS Lines (
            LineID TEXT PRIMARY KEY,
            LineName TEXT,
            AverageSpeed TEXT
        );

        CREATE TABLE IF NOT EXISTS StationLineRelationships (
            NaPTAN TEXT,
            LineID TEXT,
            PRIMARY KEY (NaPTAN, LineID),
            FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN),
```

```
FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);
```

```
CREATE TABLE IF NOT EXISTS Connections (
    StationA TEXT,
    StationB TEXT,
    LineID TEXT,
    BaseTravelTime REAL,
    PRIMARY KEY (StationA, StationB, LineID),
    FOREIGN KEY (StationA) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (StationB) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);
```

```
CREATE TABLE IF NOT EXISTS Venues (
    VenueName TEXT PRIMARY KEY,
    Capacity INTEGER,
    Latitude REAL,
    Longitude REAL
);
```

```
CREATE TABLE IF NOT EXISTS VenueStationRelationships (
    VenueName TEXT,
    NaPTAN TEXT,
    Distance REAL,
    PRIMARY KEY (VenueName, NaPTAN),
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
    FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN)
);
```

```
CREATE TABLE IF NOT EXISTS Events (
    VenueName TEXT,
    Date TEXT,
    Time TEXT,
    HomeTeam TEXT,
    AwayTeam TEXT,
    EventName TEXT,
    Duration INTEGER,
    PRIMARY KEY (VenueName, Date),
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
    FOREIGN KEY (HomeTeam) REFERENCES Teams(TeamName)
);
```

```
CREATE TABLE IF NOT EXISTS Users (
    Username TEXT PRIMARY KEY,
    PasswordHash TEXT,
    Salt TEXT,
    CHANGING_TIME REAL,
    MAX_TIME_WINDOW REAL,
    rawArrivalPeakOffset REAL,
    rawDeparturePeakOffset REAL,
    ATTENDANCE REAL,
    TRAIN_PROPORTION REAL,
    SIGMA_FACTOR REAL,
    MINIMUM_PEOPLE REAL,
    PERSON_DELAY REAL,
    PROPAGATION_FACTOR REAL,
    WALKING_TIME REAL,
    WALKING_SPEED REAL,
    NORMAL REAL,
    SLIGHTLY REAL,
    BUSY REAL
);
```

```
CREATE TABLE IF NOT EXISTS Teams (
    TeamName TEXT PRIMARY KEY,
    VenueName TEXT,
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName)
);

"""

connection.commit()
connection.close()
```

The system uses a relational database with 9 interlinked tables connected via foreign key relationships. The relationships are best illustrated on the SQL Entity Relationship diagram, which can be found in the Modelling section of this document. The database makes use of junction tables and composite keys, enabling many-to-many relationships while maintaining a structure consistent with Third Normal Form. The structured design supports the graph-based network representation, event-to-station mapping, and user-specific parameter storage necessary for the crowding simulation algorithm.

Hash Tables, Lists, Stacks, Queues, Graphs, Trees:

Graph:

```
#Fetches all the unique connections from the database and for every StationA, it adds to a list all StationBs in their own list including Line and travel time
#Forms dictionary in the form {StationA: [[StationB, BaseTravelTime, LineID], [StationC, ...], ...]}
def MakeGraph(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    listOfNeighbourStations = {} # {StationA: [[StationB, BaseTravelTime, LineID], ...]}
    rows = cursor.execute("SELECT StationA, StationB, LineID, BaseTravelTime FROM connections").fetchall()

    for StationA, StationB, LineID, BaseTravelTime in rows: #For every unique connection (direction matters)
        if StationA not in listOfNeighbourStations:
            listOfNeighbourStations[StationA] = []
        if [StationB, BaseTravelTime, LineID] not in listOfNeighbourStations[StationA]:
            listOfNeighbourStations[StationA].append([StationB, BaseTravelTime, LineID])

    connection.close()
    return listOfNeighbourStations
```

The graph is implemented using an adjacency list representation where each station (NaPTAN code) maps to a list of its neighboring stations with edge weights (travel times) and metadata (line IDs). This structure enables $O(1)$ neighbor lookups, improving efficiency and allowing it to represent the large TfL rail network as a weighted and directed graph.

Hash Table:

```
distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
```

A hash table implemented using Python's defaultdict with a lambda function stores the best known travel time to each (station, line) state, and automatically initialises unvisited states to infinity, eliminating KeyError exceptions, and proving O(1) lookups, being more efficient than storing the data in simple lists or other data structures.

Priority Queue:

```
distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
cameFrom = {} #To reconstruct path later
visited = set() #Stores visited stations

# Start at the given station, with no line yet chosen
startState = (startStation, None) # This is the state is key because the line change penalty depends on the previous line
distances[startState] = 0

frontier = [(0, startState)] #The first item is always the cheapest station to go to next
goalState = None #Store the final when we reach the goal

while frontier: #This condition means it runs until all the paths have been explored - till the last one
    currentCost, (currentStation, currentLine) = heapq.heappop(frontier) #Deletes the next route at the top so that the next
    one to be explored can be at the top

    # Skip if already fully explored this (station, line) - If at the top of the heap, then it means that it is the shortest ever path to
    the station, which means it is visited
    if (currentStation, currentLine) in visited:
        continue
    visited.add((currentStation, currentLine))

    # Stop if reached the goal station
    if currentStation == goalStation:
        goalState = (currentStation, currentLine)
        break

    # Explore all neighbours of this station
    for nextStation, baseTravelTime, nextLine in graph[currentStation]:
        travelTime = baseTravelTime

        if penalty: #Can be turned off if needed (for testing purposes)
            if currentLine is not None and currentLine != nextLine:
                #Initially no line is chosen, so the penalty is not added for changing lines
                #If the line is switched to travel to the next station, the penalty is added to the cost
                travelTime += CHANGING_TIME

        # New total time to reach the neighbour
        newCost = currentCost + travelTime
        neighbourState = (nextStation, nextLine)

        # Only update if we found a faster way
        if newCost < distances[neighbourState]:
            distances[neighbourState] = newCost
            cameFrom[neighbourState] = (currentStation, currentLine, nextLine)
            heapq.heappush(frontier, (newCost, neighbourState))
```

A priority queue implemented using Python's heapq module maintains unexplored paths ordered by cumulative travel time. The min-heap structure provides O(log n) insertion and removal operations compared to O(n) for maintaining a sorted list.

Cross-table parameterised SQL:

```
cursor.execute("""
SELECT Lines.LineName
FROM Lines
JOIN StationLineRelationships
    ON Lines.LineID = StationLineRelationships.LineID
JOIN Stations
    ON StationLineRelationships.NaPTAN = Stations.NaPTAN
WHERE Stations.StationName = ?
ORDER BY Lines.LineName
""", (stationName,))
```

This cross-table parameterised query performs two JOIN operations across three tables (Lines, StationLineRelationships, Stations) to retrieve all tube lines serving a given station.

Complex User-Defined Use of Object-Oriented Programming:

```
from abc import ABC, abstractmethod

# Abstract base class for all transport-related components
# Holds common attributes used for crowding and delay simulation
class TransportClass(ABC):
    def __init__(self):
        self.currentPassengerLoad = 0 #initially set to 0 and is the extra number of people present due to the events
        self.DelayFactor = 1 # initially set to 1 and is the factor by which the time to travel would be multiplied by to simulate crowding

    @abstractmethod
    def IsAffected(self):
        pass

# Represents a transport line consisting of multiple connections
# Responsible for spreading delay across all connections on the line
class Line():
    def __init__(self, connectionObjects, lineID):
        self.connections = connectionObjects
        self.numberOfConnections = 0
        self.TotalDelay = 0
        self.AverageDelay = 0
        self.lineID = lineID

    # Counts how many connections exist on this line
    def calculateTotalNumberOfConnections(self):
        self.numberOfConnections = len(self.connections)

    # Calculates the total delay factor across all connections
    def calculateTotalDelay(self):
        totalDelay = 0
        for connection in self.connections:
            totalDelay += connection.DelayFactor
```

```

self.TotalDelay = totalDelay

# Computes the average delay factor for the line
def calculateAverageDelay(self):
    averageDelay = self.TotalDelay / self.numberOfConnections
    self.AverageDelay = averageDelay

# Applies the average delay back onto each connection
def spreadDelay(self):
    for connection in self.connections:
        connection.LineDelay = self.AverageDelay

# Performs all line-level delay calculations in sequence
def doAllCalculations(self):
    self.calculateTotalNumberOfConnections()
    self.calculateTotalDelay()
    self.calculateAverageDelay()
    self.spreadDelay()

# Base class for all station types
class Station(TransportClass):
    def __init__(self, NaPTAN, StationName):
        super().__init__()
        self.NaPTAN = NaPTAN
        self.StationName = StationName

    def IsAffected(self):
        pass

# Station class used during database creation
# Includes geographic coordinates
class CreationStation(Station):
    def __init__(self, NaPTAN, StationName, Latitude, Longitude):
        super().__init__(NaPTAN, StationName)
        self.Latitude = Latitude
        self.Longitude = Longitude

    def IsAffected(self):
        pass

#Serves as the parent class of the affected and not affected dijkstra station classes
class DijkstraStation(Station):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)

    def IsAffected(self):
        pass

# Represents a station affected by nearby events
class AffectedDijkstraStation(DijkstraStation):
    def __init__(self, NaPTAN, StationName):

```

```

super().__init__(NaPTAN, StationName)
self.events = {} # {(VenueName, Date) : NumberOfPeople}

def IsAffected(self):
    return True

# Calculates the total passenger load caused by all events
def calculatePassengerLoad(self):
    for event in self.events.keys():
        numberOfPeople = self.events[event]
        self.currentPassengerLoad += numberOfPeople
    return self.currentPassengerLoad

# Calculates the station delay factor based on passenger load
def calculateDelay(self, PERSON_DELAY):
    extraDelay = PERSON_DELAY * self.currentPassengerLoad
    self.DelayFactor += extraDelay

# Represents a station not affected by any events
class NotAffectedDijkstraStation(DijkstraStation):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)

def IsAffected(self):
    return False

```

The programme makes use of OOP throughout the code, with different groups of classes defined in different files, as they have different general roles within the programme. One such group of classes are the transport related classes, where complex user-defined use of OOP can be clearly seen. Here, there is a complex four level inheritance hierarchy, with TransportClass as an abstract base class, defined using Python's ABC module. This defines common attributes and abstract methods. Subclasses then inherit from this, and further subclasses inherit from them. Polymorphism is also implemented and can be demonstrated via the abstract IsAffected() method, as the different subclasses implement it differently. Composition is demonstrated in the Severity class which contains a Network object that itself contains dictionaries of Station and Connection objects, and the Network class aggregates multiple transport objects into nodes and edges dictionaries.

Recursive Algorithms:

```

def propegation(graph, stationID, people, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR):
    stationObject = network.nodes[stationID] #Fetches the Dijkstra station object stored in the network

    # If station is not yet affected, create a new affected object
    if stationObject.IsAffected() == False:
        NaPTAN = stationID
        StationName = stationObject.StationName
        #Above lines fetch the information stored in the old object
        newStationObject = classes.AffectedDijkstraStation(NaPTAN, StationName) #Creates a new affected object
        newStationObject.events[event] = people #Adds the number of people at this station because of this event

```

```

stationObject = newStationObject
network.nodes[stationID] = stationObject #Replaces old object

# Check previous people count for this event
try:
    oldPeople = stationObject.events[event] #This will work if and only if the station was already affected by this event
except:
    oldPeople = 0 #This will run if the station has been already affected but not by this event so the event can't be found in the object, so the old population will be set as 0

if oldPeople > people:
    pass #No need to update because a larger wave of people have passed through already. Propagation will not be needed because a larger wave existed which has been propagated already
else:
    stationObject.events[event] = people
    network.nodes[stationID] = stationObject #Updates the people at this station due to this event at this station
    newPeople = people * PROPEGATION_FACTOR

# Stop recursion if propagated population is too small
if newPeople < MINIMUM_PEOPLE:
    pass
else:
    connections = graph[stationID]
    for connection in connections:
        newStationID = connection[0]
        propegation(graph, newStationID, newPeople, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)

```

The propagation function uses recursion to spread crowd counts through the tube network from event venues, with each recursive call visiting neighboring stations and multiplying the people count by a decay factor ($\text{PROPAGATION_FACTOR} < 1$). The base case stops recursion when the propagated population falls below the `MINIMUM_PEOPLE` threshold, preventing infinite recursion and ensuring the algorithm terminates after crowds dissipate enough through the network.

Graph Traversal:

```

for connection in connections:
    newStationID = connection[0]
    propegation(graph, newStationID, newPeople, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)

```

Graph traversal is an integral part of this programme as it is fundamentally a path finding app, which utilises Dijkstra's shortest path algorithm (custom implementation). One example where graph traversal is also found in the programme is the recursive propagation function, which implements a depth-first traversal of the station network graph, visiting each station's neighbors and recursively exploring their neighbors until the decay factor reduces the crowd below the minimum threshold. This traversal updates the states of the station during by converting `NotAffectedDijkstraStation` objects to `AffectedDijkstraStation` objects and recording event-specific crowd counts at each visited station.

List Operations:

```

# Start from the goal and trace back to the start
pathSteps = []
currentState = goalState

while currentState != startState:
    prevStation, prevLine, lineUsed = cameFrom[currentState]
    currStation, currLine = currentState
    pathSteps.append((prevStation, currStation, lineUsed, distances[(currStation, currLine)]))
    currentState = (prevStation, prevLine)

pathSteps.reverse()
return pathSteps

```

Although mostly basic, list operations are used throughout the project for data collection and manipulation. The example given here is when path reconstruction in Dijkstra's shortest path algorithm appends station tuples to a list during backtracking, then reverses the entire list to produce the correct start-to-goal order.

Queue Operations:

```

distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
cameFrom = {} #To reconstruct path later
visited = set() #Stores visited stations

# Start at the given station, with no line yet chosen
startState = (startStation, None) # This is the state is key because the line change penalty depends on the previous line
distances[startState] = 0

frontier = [(0, startState)] #The first item is always the cheapest station to go to next
goalState = None #Store the final when we reach the goal

while frontier: #This condition means it runs until all the paths have been explored - till the last one
    currentCost, (currentStation, currentLine) = heapq.heappop(frontier) #Deletes the next route at the top so that the next
    one to be explored can be at the top

    # Skip if already fully explored this (station, line) - If at the top of the heap, then it means that it is the shortest ever path to
    the station, which means it is visited
    if (currentStation, currentLine) in visited:
        continue
    visited.add((currentStation, currentLine))

    # Stop if reached the goal station
    if currentStation == goalStation:
        goalState = (currentStation, currentLine)
        break

    # Explore all neighbours of this station
    for nextStation, baseTravelTime, nextLine in graph[currentStation]:
        travelTime = baseTravelTime

        if penalty: #Can be turned off if needed (for testing purposes)
            if currentLine is not None and currentLine != nextLine:
                #Initially no line is chosen, so the penalty is not added for changing lines
                #If the line is switched to travel to the next station, the penalty is added to the cost
                travelTime += CHANGING_TIME

        # New total time to reach the neighbour
        newCost = currentCost + travelTime

```

```

neighbourState = (nextStation, nextLine)

# Only update if we found a faster way
if newCost < distances[neighbourState]:
    distances[neighbourState] = newCost
    cameFrom[neighbourState] = (currentStation, currentLine, nextLine)
    heapq.heappush(frontier, (newCost, neighbourState))

```

A priority queue is implemented using Python's `heapq` module to manage the frontier of unexplored paths in Dijkstra's algorithm, where paths are ordered by cumulative travel time. The `heappop` operation removes and returns the minimum-cost path for exploration, while `heappush` inserts new paths, ensuring the algorithm always processes the most promising path next for optimal route finding.

Hashing:

```

# Generates a cryptographically secure random salt
# A unique salt is used per user to protect against rainbow table attacks
def generateSalt():
    return os.urandom(16).hex() #Creates a random 16-byte salt and returns it as hex

# Creates a SHA-256 hash of the password combined with a salt
# Passwords are never stored in plain text
def createHashPassword(password, salt):
    saltedPassword = salt + password
    byteForm = saltedPassword.encode()
    hashObject = hashlib.sha256(byteForm)
    hashedPassword = hashObject.hexdigest()
    return hashedPassword

```

Although not implemented via a custom hashing function, the programme makes use of hashing via SHA-256 from Python's `hashlib` library to securely store user passwords in the database. Each password is combined with a randomly generated salt before hashing to prevent rainbow table attacks, and only the hash and salt are stored in the Users table, ensuring passwords cannot be recovered even if the database is compromised.

Complex Scientific/Mathematical Model:

```

# Gaussian factor at peak for crowding
# relativePeak = expected peak
# relativeTimeDifference = time offset from peak
# sigma = standard deviation for the Gaussian curve
# Returns a value between 0 and 1 representing relative crowding
def gaussianFactorAtPeak(relativePeak, sigma, relativeTimeDifference):
    x = (relativePeak - relativeTimeDifference) / sigma
    returnFactor = math.exp(-0.5 * x * x)
    return returnFactor

```

```

# Adjust for peak offsets
relativeArrivalPeakTime = startDifferenceMinutes - arrivalPeakOffset

```

```

relativeDepartPeakTime = endDifferenceMinutes + departurePeakOffset

sigmaArrival = max(1, arrivalWindowMinutes / SIGMA_FACTOR) #max used to make sure sigma is at least bigger or equal to 1
sigmaDeparture = max(1, departureWindowMinutes / SIGMA_FACTOR)

#Realistically there isn't going to be a difference between arrival and departure rates because if you came by train, you will probably leave by train so will not affect, and vice versa, so the factors will be set to one

arrivalsFactorAtMid = gaussianFactorAtPeak(relativeArrivalPeakTime, sigmaArrival, 0)
depratureFactorAtMid = gaussianFactorAtPeak(relativeDepartPeakTime, sigmaDeparture, 0)

```

The system uses a custom Gaussian distribution function to model crowd distribution around event times, using a custom mathematical formula to calculate the proportion of event attendees traveling at the journey midpoint. The model calculates sigma (standard deviation) from the time window and SIGMA_FACTOR parameter, applies the Gaussian function separately for arrival and departure, and multiplies the result by the expected crowd (capacity × attendance × train proportion) distributed across nearby stations using inverse-distance weighting. This means that crowds peak at the start and end of the events according to the bell curve. The above evidence is split into two as the code using this technique is spread over multiple functions.

Complex User-Defined Algorithms:

Modified Dijkstra's Algorithm with Line-Change Penalty:

```

# It is assumed that all stations can be accessed from any other station because the graph is made from the data for me stored in the database which is always complete for the function to be called in the first place
#On the TfL network, every station is eventually connected to every other station
def Dijkstra(graph, startStation, goalStation, CHANGING_TIME, penalty=True):
    distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
    cameFrom = {} #To reconstruct path later
    visited = set() #Stores visited stations

    # Start at the given station, with no line yet chosen
    startState = (startStation, None) # This is the state is key because the line change penalty depends on the previous line
    distances[startState] = 0

    frontier = [(0, startState)] #The first item is always the cheapest station to go to next
    goalState = None #Store the final when we reach the goal

    while frontier: #This condition means it runs until all the paths have been explored - till the last one
        currentCost, (currentStation, currentLine) = heapq.heappop(frontier) #Deletes the next route at the top so that the next one to be explored can be at the top

        # Skip if already fully explored this (station, line) - If at the top of the heap, then it means that it is the shortest ever path to the station, which means it is visited
        if (currentStation, currentLine) in visited:
            continue
        visited.add((currentStation, currentLine))

        # Stop if reached the goal station
        if currentStation == goalStation:
            goalState = (currentStation, currentLine)
            break

```

```

# Explore all neighbours of this station
for nextStation, baseTravelTime, nextLine in graph[currentStation]:
    travelTime = baseTravelTime

    if penalty: #Can be turned off if needed (for testing purposes)
        if currentLine is not None and currentLine != nextLine:
            #Initially no line is chosen, so the penalty is not added for changing lines
            #If the line is switched to travel to the next station, the penalty is added to the cost
            travelTime += CHANGING_TIME

    # New total time to reach the neighbour
    newCost = currentCost + travelTime
    neighbourState = (nextStation, nextLine)

    # Only update if we found a faster way
    if newCost < distances[neighbourState]:
        distances[neighbourState] = newCost
        cameFrom[neighbourState] = (currentStation, currentLine, nextLine)
        heapq.heappush(frontier, (newCost, neighbourState))

# Start from the goal and trace back to the start
pathSteps = []
currentState = goalState

while currentState != startState:
    prevStation, prevLine, lineUsed = cameFrom[currentState]
    currStation, currLine = currentState
    pathSteps.append((prevStation, currStation, lineUsed, distances[(currStation, currLine)]))
    currentState = (prevStation, prevLine)

pathSteps.reverse()
return pathSteps

```

This is a modified implementation of Dijkstra's shortest path algorithm that uses (station, line) tuples as states instead of just stations, allowing the algorithm to apply a time penalty when the journey requires changing between lines. The algorithm maintains a priority queue of states ordered by cumulative cost, uses a visited set to avoid reprocessing states, and reconstructs the optimal path by backtracking through the cameFrom dictionary from goal to start.

Crowding Simulation Algorithm:

```

from pathFinding import MakeGraph, Dijkstra
import datetime
import sqlite3
import math
import classes
import customFunctions

#Kinda useless but kept it because annoying to remove now
def takeInput(date, journeyStart, startStation, endStation):
    data = {
        "date": date,
        "journeyStart": journeyStart,
        "startStation": startStation,

```

```

    "endStation": endStation
}

return data

#Returns the approximate time of the middle of the journey (without consideration of delays and things)
def calculateDefaultMidjourneyTime(baseInputData, databaseFile, CHANGING_TIME):
    journeyStart = baseInputData["journeyStart"]
    startStation = baseInputData["startStation"]
    endStation = baseInputData["endStation"]

    graph = MakeGraph(databaseFile)
    # Run Dijkstra to get the shortest path steps from start to end and the time for the journey (if there was no crowding)
    steps = Dijkstra(graph, startStation, endStation, CHANGING_TIME)
    totalMinutes = steps[-1][3] #steps is in the form [[PreviousStationNaPTAN, NextStationNaPTAN, lineID,
    LengthOfTimeToPoint],...]
    halfMinutes = totalMinutes // 2

    # Convert journey start to datetime object
    timeObject = datetime.datetime.strptime(journeyStart, "%H:%M")

    # Add half journey duration to get midpoint
    timeDelta = datetime.timedelta(minutes=halfMinutes)
    timeObject += timeDelta

    return timeObject

# Returns events that might impact the journey based on a given timeframe
# Takes into account the peak offsets for arrivals and departures
# Returns dictionary of events: {(venue, date): [minuteDifference, startDiff, endDiff]}
def findEventsInTimeFrame(midjourneyTime, databaseFile, baseInputData, MAX_TIME_WINDOW, arrivalPeakOffset,
departurePeakOffset):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    date = baseInputData["date"]
    importantEvents = {} # (venue, date) : minuteDifference - The events which might affect the journey

    #Fetches all the events in that day
    eventsInDay = cursor.execute("""
        SELECT *
        FROM Events
        WHERE Date = ?
        """, (date,)).fetchall()

    #Event is in the form [VenueName, Date, Time, HomeTeam, AwayTeam, EventName, Duration]
    for event in eventsInDay:
        duration = event[6]
        eventTime = datetime.datetime.strptime(event[2], "%H:%M")
        endTime = eventTime + datetime.timedelta(minutes=duration)

        # Calculate difference from journey midpoint to arrival and departure peaks
        startTimeDifference = abs(eventTime - arrivalPeakOffset - midjourneyTime) #Peaks are unlikely to be immediately at
        the start or end of an event
        endTimeDifference = abs((endTime + departurePeakOffset) - midjourneyTime)

        # Choose the smaller of the two differences for maximum impact (the closer events are in time, the bigger their impacts
        are)
        if startTimeDifference > endTimeDifference:
            timeDifference = endTimeDifference
        else:
            timeDifference = startTimeDifference
        minutesDifference = timeDifference.total_seconds() // 60

```

```

# Only consider events within the MAX_TIME_WINDOW
if minutesDifference <= MAX_TIME_WINDOW:
    venue = event[0]
    startDifferenceMinutes = startTimeDifference.total_seconds() // 60
    endDifferenceMinutes = endTimeDifference.total_seconds() // 60
    importantEvents[(venue, date)] = [minutesDifference, startDifferenceMinutes, endDifferenceMinutes]

connection.close()
return importantEvents

#This function assigns each station a weight so that the number of people at the stations will be more accurately assigned
#stationDistances dictionary format - {NaPTAN: distance}
#Returns dictionary format - {NaPTAN: proportion}
def inverseWeight(stationDistances):
    extra = 1 #Avoid divide by zero errors - if the distance between the station and the venue is 0
    weights = {}
    proportions = {}
    totalWeight = 0

    # Calculate inverse-distance weight for each station
    for station in stationDistances.keys():
        distance = stationDistances[station]
        weight = 1 / (distance + extra)
        totalWeight += weight
        weights[station] = weight

    # Convert weights to proportions of total
    for station in weights.keys():
        proportion = weights[station] / totalWeight
        proportions[station] = proportion

    return proportions

# Fetches stations near a venue and their distance from the venue
# Returns dictionary {NaPTAN: distance}
def returnStationsWithDistances(VenueName, databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    stationsWithDistances = {}

    cursor.execute("""
        SELECT NaPTAN, Distance
        FROM VenueStationRelationships
        WHERE VenueName = ?
    """, (VenueName,))
    rows = cursor.fetchall()

    for stationTuple in rows:
        stationsWithDistances[stationTuple[0]] = stationTuple[1]

    connection.close()
    return stationsWithDistances

# Gaussian factor at peak for crowding
# relativePeak = expected peak
# relativeTimeDifference = time offset from peak
# sigma = standard deviation for the Gaussian curve
# Returns a value between 0 and 1 representing relative crowding
def gaussianFactorAtPeak(relativePeak, sigma, relativeTimeDifference):
    x = (relativePeak - relativeTimeDifference) / sigma

```

```

returnFactor = math.exp(-0.5 * x * x)
return returnFactor

#For each event, finds the base number of people at the nearby stations at the middle of the journey
# Returns dictionary {NaPTAN: [arrivalPeople, departurePeople]}
def findBaseNumberOfPeople(
    capacity,
    ATTENDANCE, #The proportion of seats that will actually be filled
    TRAIN_PROPORTION, #The proportion of people using TfL rail
    startDifferenceMinutes, #minutes from journey midpoint to event start
    endDifferenceMinutes, #minutes from journey midpoint to event end
    arrivalWindowMinutes, #Length of time of arrival window where the journey may be affected
    departureWindowMinutes, #Length of time of departure window where the journey may be affected
    arrivalPeakOffset, #Offset because peaks are usually not at exactly the start or end
    departurePeakOffset,
    stationsWithDistances, #Dictionary of stations with the values being the distance between the station and the station
    and the venue
    SIGMA_FACTOR
):

    totalAttendance = capacity * ATTENDANCE #Gets the expected attendance at the venue
    tfIUsageTotal = totalAttendance * TRAIN_PROPORTION #Gets the expected total usage of TfL rail

    # Convert distances into proportional distribution of passengers
    proportions = inverseWeight(stationsWithDistances)
    StationsWithPeopleAtPeak = {}
    for stationID in proportions.keys():
        StationsWithPeopleAtPeak[stationID] = proportions[stationID] * tfIUsageTotal

    # Adjust for peak offsets
    relativeArrivalPeakTime = startDifferenceMinutes - arrivalPeakOffset
    relativeDepartPeakTime = endDifferenceMinutes + departurePeakOffset

    sigmaArrival = max(1, arrivalWindowMinutes / SIGMA_FACTOR) #max used to make sure sigma is at least bigger or equal
    to 1
    sigmaDeparture = max(1, departureWindowMinutes / SIGMA_FACTOR)

    #Realistically there isn't going to be a difference between arrival and departure rates because if you came by train, you
    will probably leave by train so will not affect, and vice versa, so the factors will be set to one

    arrivalsFactorAtMid = gaussianFactorAtPeak(relativeArrivalPeakTime, sigmaArrival, 0)
    depratureFactorAtMid = gaussianFactorAtPeak(relativeDepartPeakTime, sigmaDeparture, 0)

    # Compute expected arrivals and departures at each station
    stationsAtMid = {} #Format - {NaPTAN: ArrivalNumbers, DepartureNumbers}
    for stationID in StationsWithPeopleAtPeak.keys():
        arrivalPeople = StationsWithPeopleAtPeak[stationID] * arrivalsFactorAtMid
        departurePeople = StationsWithPeopleAtPeak[stationID] * depratureFactorAtMid
        stationsAtMid[stationID] = [arrivalPeople, departurePeople]

    return stationsAtMid

def findCapacity(databaseFile, VenueName):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("""
        SELECT Capacity
        FROM Venues
        Where VenueName = ?
        """, (VenueName,))
    capacity = cursor.fetchall()[0][0]
    connection.close()

```

```

return capacity

# Propagates people counts recursively through the network
# Updates affected stations using AffectedDijkstraStation objects
def propagation(graph, stationID, people, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR):
    stationObject = network.nodes[stationID] #Fetches the Dijkstra station object stored in the network

    # If station is not yet affected, create a new affected object
    if stationObject.IsAffected() == False:
        NaPTAN = stationID
        StationName = stationObject.StationName
        #Above lines fetch the information stored in the old object
        newStationObject = classes.AffectedDijkstraStation(NaPTAN, StationName) #Creates a new affected object
        newStationObject.events[event] = people #Adds the number of people at this station because of this event
        stationObject = newStationObject
        network.nodes[stationID] = stationObject #Replaces old object

    # Check previous people count for this event
    try:
        oldPeople = stationObject.events[event] #This will work if and only if the station was already affected by this event
    except:
        oldPeople = 0 #This will run if the station has been already affected but not by this event so the event can't be found in
        the object, so the old population will be set as 0

    if oldPeople > people:
        pass #No need to update because a larger wave of people have passed through already. Propagation will not be
        needed because a larger wave existed which has been propegated already
    else:
        stationObject.events[event] = people
        network.nodes[stationID] = stationObject #Updates the people at this station due to this event at this station
        newPeople = people * PROPEGATION_FACTOR

    # Stop recursion if propagated population is too small
    if newPeople < MINIMUM_PEOPLE:
        pass
    else:
        connections = graph[stationID]
        for connection in connections:
            newStationID = connection[0]
            propagation(graph, newStationID, newPeople, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)

#Creates a parameter object to store the parameters used for calculation
def getParameters(username, databaseFile):
    parameterObject = customFunctions.getParameters(username, databaseFile)
    return parameterObject

# Creates NotAffectedStation and NotAffectedConnection objects which are then stored in the dictionaries
# All this is stored and contained in a network object
def createNetwork(baseGraph, databaseFile, stations, connections, network):
    sqlConnection = sqlite3.connect(databaseFile)
    cursor = sqlConnection.cursor()

    # Create station objects
    for StationA in baseGraph.keys():
        cursor.execute("""
            SELECT StationName
            FROM Stations
            WHERE NaPTAN = ?
            """, (StationA,))
        row = cursor.fetchall()
        StationName = row[0][0]

```

```

NaPTAN = StationA
baseStationClass = classes.NotAffectedDijkstraStation(NaPTAN, StationName)
stations[StationA] = baseStationClass

# Create connection objects
for connection in baseGraph[StationA]:
    StationA = StationA
    StationB = connection[0]
    BaseTravelTime = connection[1]
    LineID = connection[2]
    baseConnectionClass = classes.NotAffectedConnection(StationA, StationB, LineID, BaseTravelTime)
    key = (StationA, StationB, LineID)
    connections[key] = baseConnectionClass

sqlConnection.close()

network.nodes = stations
network.edges = connections

#Starts off propagation of populations from each close station - then calls propegation() which then runs recursively
def processEvents(events, databaseFile, parameterObject, baseGraph, network):
    ATTENDANCE = parameterObject.ATTENDANCE
    TRAIN_PROPORTION = parameterObject.TRAIN_PROPORTION
    arrivalWindowMinutes = parameterObject.arrivalWindowMinutes
    departureWindowMinutes = parameterObject.departureWindowMinutes
    arrivalMinutesOffset = parameterObject.arrivalMinutesOffset
    departureMinutesOffset = parameterObject.departureMinutesOffset
    SIGMA_FACTOR = parameterObject.SIGMA_FACTOR
    MINIMUM_PEOPLE = parameterObject.MINIMUM_PEOPLE
    PROPEGATION_FACTOR = parameterObject.PROPEGATION_FACTOR

    for event in events.keys():
        venue = event[0]
        timings = events[event]
        startDifferenceMinutes = timings[1]
        endDifferenceMinutes = timings[2]
        stationsWithDistances = returnStationsWithDistances(venue, databaseFile)
        capacity = findCapacity(databaseFile, venue)
        baseNumberOfPeopleAtStations = findBaseNumberOfPeople(
            capacity,
            ATTENDANCE,
            TRAIN_PROPORTION,
            startDifferenceMinutes,
            endDifferenceMinutes,
            arrivalWindowMinutes,
            departureWindowMinutes,
            arrivalMinutesOffset,
            departureMinutesOffset,
            stationsWithDistances,
            SIGMA_FACTOR
        )

        # Propagate people through the network recursively
        for station in baseNumberOfPeopleAtStations.keys():
            arrivals = baseNumberOfPeopleAtStations[station][0]
            departures = baseNumberOfPeopleAtStations[station][1]
            people = max(arrivals, departures)
            if people >= MINIMUM_PEOPLE:
                propegation(baseGraph, station, people, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)

#Calls methods to calculate the delay of each station

```

```

def processAffectedStations(baseGraph, parameterObject, network):
    PERSON_DELAY = parameterObject.PERSON_DELAY
    for StationID in baseGraph.keys():
        stationObject = network.nodes[StationID]
        if stationObject.IsAffected():
            stationObject.calculatePassengerLoad()
            stationObject.calculateDelay(PERSON_DELAY)

#Calls methods to calculate the delay of each connection
def processAffectedConnections(network):
    connectionObjects = network.edges
    for edgeReference in connectionObjects.keys():
        edge = connectionObjects[edgeReference]
        StationA = edge.StationA
        StationB = edge.StationB
        LineID = edge.LineID
        BaseTravelTime = edge.BaseTravelTime
        StationADelay = network.nodes[StationA].DelayFactor
        StationBDelay = network.nodes[StationB].DelayFactor
        if StationADelay or StationBDelay != 1:
            newConnectionObject = classes.AffectedConnection(StationA, StationB, LineID, BaseTravelTime, StationADelay,
            StationBDelay)
            newConnectionObject.returnDelay()
            network.edges[(StationA, StationB, LineID)] = newConnectionObject

#Delay of each station in the network is calculated, and then the delay of each connection is calculated
def processAffected(baseGraph, parameterObject, network):
    processAffectedStations(baseGraph, parameterObject, network)
    processAffectedConnections(network)

# Main function to compute crowding factors for a journey
# Builds the network, processes events, and applies delays
# Returns network object with updated delays
def getCrowdingFactor(username, databaseFile, date, journeyStart, startStation, endStation):
    stations = {}
    connections = {}

    parameterObject = getParameters(username, databaseFile)
    baseGraph = MakeGraph(databaseFile)
    network = classes.Network()

    createNetwork(baseGraph, databaseFile, stations, connections, network) #Creates NotAffectedStation and
    NotAffectedConnection objects which are then stored in the dictionaries which are then stored in the network object
    baseInputData = takeInput(date, journeyStart, startStation, endStation)
    midTimeObject = calculateDefaultMidjourneyTime(
        baseInputData,
        databaseFile,
        parameterObject.CHANGING_TIME)
    events = findEventsInTimeFrame(
        midTimeObject,
        databaseFile,
        baseInputData,
        parameterObject.MAX_TIME_WINDOW,
        parameterObject.arrivalPeakOffset,
        parameterObject.departurePeakOffset)
    processEvents(events, databaseFile, parameterObject, baseGraph, network)
    processAffected(baseGraph, parameterObject, network)

#This results in a network object
#The network object has attributes edges and stations
#Stations contain a type of dijkstraStation objects which store delays

```

```

#Edges are connections which also store delays
#Network doesn't have to be returned in the above functions since the reference isn't changed

return network

# Adjusts delays across lines to make them consistent along a line
def spreadDelay(network, databaseFile):
    listOfLines = {}
    listOfLinesWithConnectionObjects = {}

    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    #Fetches a list of lines
    cursor.execute("""
        SELECT LineID
        FROM Lines""")
    rows = cursor.fetchall()
    lineIDs = []
    for row in rows:
        lineIDs.append(row[0])
    connection.close()

    #Edits the dictionary so that the keys are the lineIDs
    for lineID in lineIDs:
        listOfLines[lineID] = []

    #Gets all the connections using a line and stores that connection object (memory reference) in the dictionary
    for edgeKey in network.edges.keys():
        lineID = edgeKey[2] #Contains the lineID of the connection
        listOfLines[lineID].append(network.edges[edgeKey])

    #Creates a line object for each line which would contain the list of connection objects belonging to that line, so that
    #methods may be called
    for lineID in listOfLines:
        lineObject = classes.Line(listOfLines[lineID], lineID)
        listOfLinesWithConnectionObjects[lineID] = lineObject

    #Calls the methods to calculate the average delay across the entire line, so that the delay is consistent across the line
    # (delays tend to affect the entire line as opposed to being local to a station or few)
    for lineObject in listOfLinesWithConnectionObjects.values():
        lineObject.doAllCalculations()

    #Nothing needs to be returned as the objects themselves are not being changed to different objects, only their attributes
    # are being changed and therefore their memory reference is the same

# Main for processing a journey
# Returns network object with stations and connections updated with delays
def main(date, journeyStart, startStation, endStation, databaseFile, username):
    affectedNetwork = getCrowdingFactor(username, databaseFile, date, journeyStart, startStation, endStation)
    spreadDelay(affectedNetwork, databaseFile) #For line, the connections belonging to that line will have the same delay
    stored in the LineDelay attribute (local delay is stored in DelayFactor)

    #For testing purposes
    for edgeKey in affectedNetwork.edges.keys():
        connection = affectedNetwork.edges[edgeKey]
        #print(connection.connectionReference, connection.DelayFactor, connection.LineDelay)
    pass

    # For testing purposes
    for nodeKey in affectedNetwork.nodes.keys():
        node = affectedNetwork.nodes[nodeKey]
        #print(node.StationName, node.NaPTAN, node.DelayFactor)

```

```
pass
```

```
return affectedNetwork
```

```
from abc import ABC, abstractmethod
```

```
# Abstract base class for all transport-related components
```

```
# Holds common attributes used for crowding and delay simulation
```

```
class TransportClass(ABC):
```

```
    def __init__(self):
```

```
        self.currentPassengerLoad = 0 #initially set to 0 and is the extra number of people present due to the events
```

```
        self.DelayFactor = 1 # initially set to 1 and is the factor by which the time to travel would be multiplied by to simulate crowding
```

```
@abstractmethod
```

```
def IsAffected(self):
```

```
    pass
```

```
# Represents a transport line consisting of multiple connections
```

```
# Responsible for spreading delay across all connections on the line
```

```
class Line():
```

```
    def __init__(self, connectionObjects, lineID):
```

```
        self.connections = connectionObjects
```

```
        self.numberOfConnections = 0
```

```
        self.TotalDelay = 0
```

```
        self.AverageDelay = 0
```

```
        self.lineID = lineID
```

```
# Counts how many connections exist on this line
```

```
def calculateTotalNumberOfConnections(self):
```

```
    self.numberOfConnections = len(self.connections)
```

```
# Calculates the total delay factor across all connections
```

```
def calculateTotalDelay(self):
```

```
    totalDelay = 0
```

```
    for connection in self.connections:
```

```
        totalDelay += connection.DelayFactor
```

```
    self.TotalDelay = totalDelay
```

```
# Computes the average delay factor for the line
```

```
def calculateAverageDelay(self):
```

```
    averageDelay = self.TotalDelay / self.numberOfConnections
```

```
    self.AverageDelay = averageDelay
```

```
# Applies the average delay back onto each connection
```

```
def spreadDelay(self):
```

```
    for connection in self.connections:
```

```
        connection.LineDelay = self.AverageDelay
```

```
# Performs all line-level delay calculations in sequence
```

```
def doAllCalculations(self):
```

```
    self.calculateTotalNumberOfConnections()
```

```
    self.calculateTotalDelay()
```

```

    self.calculateAverageDelay()
    self.spreadDelay()

# Base class for all station types
class Station(TransportClass):
    def __init__(self, NaPTAN, StationName):
        super().__init__()
        self.NaPTAN = NaPTAN
        self.StationName = StationName

    def IsAffected(self):
        pass

# Station class used during database creation
# Includes geographic coordinates
class CreationStation(Station):
    def __init__(self, NaPTAN, StationName, Latitude, Longitude):
        super().__init__(NaPTAN, StationName)
        self.Latitude = Latitude
        self.Longitude = Longitude

    def IsAffected(self):
        pass

#Serves as the parent class of the affected and not affected dijkstra station classes
class DijkstraStation(Station):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)

    def IsAffected(self):
        pass

# Represents a station affected by nearby events
class AffectedDijkstraStation(DijkstraStation):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)
        self.events = {} # {(VenueName, Date) : NumberOfPeople}

    def IsAffected(self):
        return True

# Calculates the total passenger load caused by all events
def calculatePassengerLoad(self):
    for event in self.events.keys():
        numberOfPeople = self.events[event]
        self.currentPassengerLoad += numberOfPeople
    return self.currentPassengerLoad

# Calculates the station delay factor based on passenger load
def calculateDelay(self, PERSON_DELAY):
    extraDelay = PERSON_DELAY * self.currentPassengerLoad

```

```

        self.DelayFactor += extraDelay

# Represents a station not affected by any events
class NotAffectedDijkstraStation(DijkstraStation):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)

    def IsAffected(self):
        return False

# Parent class for all connection types used in Dijkstra's algorithm
class Connection(TransportClass):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime):
        super().__init__()
        self.StationA = StationA
        self.StationB = StationB
        self.LineID = LineID
        self.connectionReference = (self.StationA, self.StationB, self.LineID) #Unique Identifier
        self.BaseTravelTime = BaseTravelTime
        self.LineDelay = 0

    @abstractmethod
    def IsAffected(self):
        pass

    @abstractmethod
    def returnDelay(self):
        pass

# Connection that is not affected by events or crowding
class NotAffectedConnection(Connection):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime):
        super().__init__(StationA, StationB, LineID, BaseTravelTime)

    def IsAffected(self):
        return False

    def returnDelay(self): #Because it isn't affected
        self.DelayFactor = 1
        return 1

# Connection affected by delays at one or both stations
class AffectedConnection(Connection):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime, StationADelayFactor, StationBDelayFactor):
        super().__init__(StationA, StationB, LineID, BaseTravelTime)
        self.StationADelayFactor = StationADelayFactor
        self.StationBDelayFactor = StationBDelayFactor

    def IsAffected(self):
        return True

```

```

# Calculates delay as the average of both station delay factors -- use of polymorphism as the two connection types are
handled differently
def returnDelay(self):
    averageDelayFactor = (self.StationADelayFactor + self.StationBDelayFactor) / 2
    self.DelayFactor = averageDelayFactor
    return averageDelayFactor

# Massive container for all the data for me about the graph
class Network():
    def __init__(self):
        self.nodes = {} #Dictionary of station objects with the key being the NaPTAN
        self.edges = {} #Dictionary of connection objects with the key being (StationA, StationB, LineID)
        self.SpreadOutEdges = {} # Used to store edges after delay propagation

```

The algorithm begins by loading the user parameters and creating a base graph. It then initialises a Network object and populates it with NotAffectedDijkstraStation and NotAffectedConnection objects, for all the stations and unique connections on the TfL rail network. The system calculates the journey midpoint time and finds all the events from the database occurring within a window of time centred around this time. For each event, it finds valid stations within a wallaby distance of the stadium the event will occur at and finds the crowd sizes at each station using a Gaussian distribution centred around the event's peak times, weighted by inverse walking distance from the venue. For each station with crowds exceeding the minimum people threshold, for each event, it calls a recursive function (explained in this section under the subheading "Recursive Algorithms") to spread crows to neighbouring stations. After all the events have been processed, the number of passengers at each station is converted into a DelayFactor, by calling methods of AffectedDijkstraStation objects. AffectedConnection objects are created for any connection whose endpoint stations have delays, calculating each connection's delay as the average of its two endpoint station delays via the polymorphic returnDelay() method. Finally, spreadDelay() groups all connections by LineID, creates Line objects for each line containing references to all its connection objects, and calls doAllCalculations() which computes the average delay across all connections on that line and updates each connection's LineDelay attribute to ensure delays are consistent along the entire line rather than being localised to individual stations.

To summarise, this algorithm takes in a base graph of the transport network, user parameters and the events at the time of travel, to create a weighted graph that reflects the conditions of the network.

Server-Side Scripting Using Request/Response Objects:

```

# Main Journey Planning Page
@app.route('/main', methods=['GET', 'POST']) #Need to uncomment out the validation for correct time
def mainPage():
    # Redirect to login if not logged in
    if "username" not in session:
        return redirect(url_for("loginPage"))

```

```

databaseFile = app.config["DATABASE_FILE"]
username = session["username"]

# Initialise variables for template
error = None
startStationLines = None
endStationLines = None
formattedOldPath = None
formattedNewPath = None
oldPathTime = None
oldPathArrival = None
newPathTime = None
newPathArrival = None

validDateCheck = True #Toggle on and off for testing

if request.method == 'POST':
    if "updateButton" in request.form:
        # Redirect to update parameters page
        return redirect(url_for("updatePage"))
    else:
        # Get form input
        start = request.form.get('start')
        end = request.form.get('end')
        date = request.form.get('date')
        time = request.form.get('time')
        selectedTime = datetime.strptime(f"{date} {time}", "%Y-%m-%d %H:%M")

        if start == end:
            error = "Start and end stations must be different"
        elif validDateCheck and selectedTime < datetime.now():
            error = "Selected time must be in the future"
        else:
            # Format date for getResults
            dateObject = datetime.strptime(date, "%Y-%m-%d")
            passInDate = dateObject.strftime("%d/%m/%Y")

            # Get line information for dropdowns
            startStationLines = getLines(start, databaseFile)
            endStationLines = getLines(end, databaseFile)

            # Calculate journey paths
            resultsObject = getResults.main(passInDate, time, start, end, databaseFile, username)

            # Old Path (Affected)
            affectedOldPath = resultsObject.affectedOldPath
            affectedOldPathDescriptions = resultsObject.affectedOldPathDescriptions
            preFormataffectedOldPathMinutes = affectedOldPath[-1][3] # The time taken to get to the last station on the path,
            # which is stored as the fourth element [prevStation, currStation, LineUsed, TimeToStation]
            oldPathTime, oldPathArrival = processTime(preFormataffectedOldPathMinutes, time)
            formattedOldPath = formatPathData(affectedOldPath, affectedOldPathDescriptions, start, databaseFile)

            # New Path (Optimised)
            newPath = resultsObject.newPath
            newPathDescriptions = resultsObject.newPathDescriptions
            preFormataffectedNewPathMinutes = newPath[-1][3] # The time taken to get to the last station on the path, which
            # is stored as the fourth element [prevStation, currStation, LineUsed, TimeToStation]
            newPathTime, newPathArrival = processTime(preFormataffectedNewPathMinutes, time)
            formattedNewPath = formatPathData(newPath, newPathDescriptions, start, databaseFile)

return render_template(
    "mainPage.html",
    error=error,

```

```

startLines=startStationLines,
endLines=endStationLines,
formattedOldPath=formattedOldPath,
formattedNewPath=formattedNewPath,
oldPathTime=oldPathTime,
oldPathArrival=oldPathArrival,
newPathTime=newPathTime,
newPathArrival=newPathArrival
)

```

```

# Save User Parameters
@app.route("/save_parameters", methods=["POST"])
def saveParameters():
    """
    Saves user-modified parameters and updates stadium data.
    """
    databaseFile = app.config["DATABASE_FILE"]
    londonjsonFile = app.config["LONDONJSON_FILE"]

    username = session.get("username")
    data = request.get_json()

    values = (
        float(data["CHANGING_TIME"]),
        float(data["MAX_TIME_WINDOW"]),
        float(data["rawArrivalPeakOffset"]),
        float(data["rawDeparturePeakOffset"]),
        float(data["ATTENDANCE"]),
        float(data["TRAIN_PROPORTION"]),
        float(data["SIGMA_FACTOR"]),
        float(data["MINIMUM_PEOPLE"]),
        float(data["PERSON_DELAY"]),
        float(data["PROPAGATION_FACTOR"]),
        float(data["WALKING_TIME"]),
        float(data["WALKING_SPEED"]),
        float(data["NORMAL"]),
        float(data["SLIGHTLY"]),
        float(data["BUSY"]),
        username # WHERE clause
    )

    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("""
        UPDATE Users SET
            CHANGING_TIME = ?,
            MAX_TIME_WINDOW = ?,
            rawArrivalPeakOffset = ?,
            rawDeparturePeakOffset = ?,
            ATTENDANCE = ?,
            TRAIN_PROPORTION = ?,
            SIGMA_FACTOR = ?,
            MINIMUM_PEOPLE = ?,
            PERSON_DELAY = ?,
            PROPAGATION_FACTOR = ?,
            WALKING_TIME = ?,
            WALKING_SPEED = ?,
            NORMAL = ?,
            SLIGHTLY = ?,
            BUSY = ?
        WHERE Username = ?
    """, values)
    connection.commit()

```

```

connection.close()

# Recalculate stadium data
walkingTime = float(data["WALKING_TIME"])
walkingSpeed = float(data["WALKING_SPEED"])

storeStadiumData.redoStore(walkingTime, walkingSpeed, londonjsonFile, databaseFile)

return jsonify({"success": True})

```

The Flask application implements a client-server model with 11 routes handling authentication, journey calculation, parameter updates, and data fetching. The routes use session objects to maintain user state across requests, extract data from POST request forms and return either rendered HTML templates or JSON responses for AJAX calls, allowing parameter updates without page reloads. Above are two examples.

Dynamic Generation of Objects Based on Complex OOP Model:

```

# If station is not yet affected, create a new affected object
if stationObject.IsAffected() == False:
    NaPTAN = stationID
    StationName = stationObject.StationName
    #Above lines fetch the information stored in the old object
    newStationObject = classes.AffectedDijkstraStation(NaPTAN, StationName) #Creates a new affected object
    newStationObject.events[event] = people #Adds the number of people at this station because of this event
    stationObject = newStationObject
    network.nodes[stationID] = stationObject #Replaces old object

```

```

# Creates NotAffectedStation and NotAffectedConnection objects which are then stored in the dictionaries
# All this is stored and contained in a network object
def createNetwork(baseGraph, databaseFile, stations, connections, network):
    sqlConnection = sqlite3.connect(databaseFile)
    cursor = sqlConnection.cursor()

    # Create station objects
    for StationA in baseGraph.keys():
        cursor.execute("""
            SELECT StationName
            FROM Stations
            WHERE NaPTAN = ?
            """, (StationA,))
        row = cursor.fetchall()
        StationName = row[0][0]

        NaPTAN = StationA
        baseStationClass = classes.NotAffectedDijkstraStation(NaPTAN, StationName)
        stations[StationA] = baseStationClass

    # Create connection objects
    for connection in baseGraph[StationA]:
        StationA = StationA
        StationB = connection[0]
        BaseTravelTime = connection[1]
        LineID = connection[2]
        baseConnectionClass = classes.NotAffectedConnection(StationA, StationB, LineID, BaseTravelTime)
        key = (StationA, StationB, LineID)

```

```

connections[key] = baseConnectionClass

sqlConnection.close()

network.nodes = stations
network.edges = connections

```

```

#Calls methods to calculate the delay of each connection
def processAffectedConnections(network):
    connectionObjects = network.edges
    for edgeReference in connectionObjects.keys():
        edge = connectionObjects[edgeReference]
        StationA = edge.StationA
        StationB = edge.StationB
        LineID = edge.LineID
        BaseTravelTime = edge.BaseTravelTime
        StationADelay = network.nodes[StationA].DelayFactor
        StationBDelay = network.nodes[StationB].DelayFactor
        if StationADelay or StationBDelay != 1:
            newConnectionObject = classes.AffectedConnection(StationA, StationB, LineID, BaseTravelTime, StationADelay,
            StationBDelay)
            newConnectionObject.returnDelay()
            network.edges[(StationA, StationB, LineID)] = newConnectionObject

```

The system dynamically generates and replaces objects at runtime in various locations, with two examples given above (with the second one split in two). During crowd propagation, NotAffectedDijkstraStation objects are replaced with AffectedDijkstraStation objects when they become affected by events, preserving the station's NaPTAN and name while adding event tracking capability. When building the affected network, the system instantiates either AffectedConnection or NotAffectedConnection objects for each connection based on whether their endpoint stations are affected, with AffectedConnection objects having additional delay factor parameters and implementing polymorphic returnDelay() behavior to calculate delays from endpoint stations.

Calling Parameterised Web Service APIs and Parsing JSON:

```

import sqlite3
import classes
from customFunctions import safeGet
from geopy.distance import geodesic

#GLOBAL PARAMETERS
#How long a train waits at a station
WAITING_TIME = 0.5

# Creates a dictionary with the format - {NaPTAN : Station object}
# Fetches all stations served by the given LineIDs from the TfL API
def fetchStations(TfL_API_KEY, lineIDs):
    stationDictionary = {}
    params = {
        "app_key": TfL_API_KEY,
    }

```

```

# Loop through every line stored in the database
for line in lineIDs:
    url = f"https://api.tfl.gov.uk/Line/{line}/Route/Sequence/inbound"
    response = safeGet(url, params=params)
    if type(response) is int: #Checks if there has been an error
        return response
    data = response.json()
    stations = data["stations"]

    # Create a CreationStation object for every unique station
    for station in stations:
        if station["id"] not in stationDictionary: # So every entry is unique
            NaPTAN = station['id']
            StationName = station['name']
            Latitude = station['lat']
            Longitude = station['lon']

            # CreationStation extends Station and stores geographic coordinates
            stationDictionary[station["id"]] = classes.CreationStation(NaPTAN, StationName, Latitude, Longitude)

return stationDictionary

# Uses a list of LineIDs and a list of Station objects to create a dictionary in the form {LineID : [[Branch1], [Branch2], ..., [Branchn]]}
def fetch_lines(stationDictionary, TfL_API_KEY, lineIDs):
    linesDictionary = {} # Final dictionary that will be returned
    # Some stations are returned strangely by the API (different IDs for different branches)
    # This dictionary maps incorrect IDs to their correct hub NaPTAN IDs
    weirdStations = {} # {incorrectID : correctID}
    params = {
        "app_key": TfL_API_KEY,
    }

    # Loop through each LineID
    for line in lineIDs:
        url = f"https://api.tfl.gov.uk/Line/{line}/Route/Sequence/inbound"
        response = safeGet(url, params=params)
        if type(response) is int: #Checks if there has been an error
            return response
        data = response.json()
        branches = data["orderedLineRoutes"]
        listOflistsOfNaPTAN = []

        # Each branch represents one possible ordered path of stations on the line
        for branchAllData in branches:
            # branchAllData contains additional metadata which is not needed
            branch = branchAllData["naptanids"] # List of all stations (NaPTAN IDs) of the branch
            for i, id in enumerate(branch):
                if id not in stationDictionary: # All possible unique stations already exist in stationDictionary, but some stations
                    # have other IDs for different modes, which means they need to be filtered out so that there is only one ID per station
                    if id not in weirdStations: # Store of all the stations that have different IDs
                        # Fetch station details to find its hub NaPTAN ID
                        newURL = f"https://api.tfl.gov.uk/StopPoint/{id}" # Fetches all data for me regarding that particular station
                        newResponse = safeGet(newURL, params=params)
                        newData = newResponse.json()
                        newId = newData["hubNaptanCode"] # Collects the ID that represents the entire station and not just that
mode/line
                        # Store mapping so future lookups don't require another API call
                        weirdStations[id] = newId

            # Replace incorrect ID with correct hub ID
            id = weirdStations[id]
            branch[i] = id # Changes the ID of the station in the branch to the correct ID

```

```

        # Add the cleaned branch to the list of branches for this line
        listOflistsOfNAPTAN.append(branch)

    linesDictionary[line] = listOflistsOfNAPTAN

    return linesDictionary

# Fetches all TfL data for me required to populate the database (that can be gotten from the TfL API)
# Returns stationDictionary and linesDictionary
def fetchTfLData(databaseFile, TfL_API_KEY):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Creates a list of LineIDs
    linesCursorObject = cursor.execute("SELECT LineID FROM Lines")
    lineIDs = []
    for lineTuple in linesCursorObject: # SQL returns data for me in the form (LineID,)
        lineIDs.append(lineTuple[0])

    # Fetch all stations served by the stored lines
    stationDictionary = fetchStations(TfL_API_KEY, lineIDs)
    print("Fetched Stations")

    # Fetch ordered branches for each line
    linesDictionary = fetch_lines(stationDictionary, TfL_API_KEY, lineIDs)
    print("Fetched Lines")

    connection.close()
    return stationDictionary, linesDictionary

# Puts every possible Station, Line pair into the StationLineRelationships table
# Acts as the joining table between Stations and Lines
def getStationLineRelationships(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Use the Connections table to get which stations are served by which lines
    rows = cursor.execute("SELECT StationA, LineID from Connections").fetchall()
    for row in rows:
        StationA = row[0]
        LineID = row[1]

        # Ignore as well because station might have more than 1 connection to another station via the same line -- different
        # branch lines are given start to end, which therefore means sometimes multiple of the same connection are attempted to be
        # stored
        cursor.execute("""
            INSERT OR REPLACE INTO StationLineRelationships (NaPTAN, LineID)
            VALUES (?, ?)
            """, (StationA, LineID))

    connection.commit()
    connection.close()

# Fetches the latitude and longitude of a station from the database
def getStationCoordinates(databaseFile, StationID):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("""
        SELECT Latitude, Longitude
        FROM Stations
    """)

```

```

WHERE NaPTAN = ?
"""', (StationID, ))
coordinates = cursor.fetchall()[0]
connection.close()
return coordinates

# Stores all fetched TfL data for me into the database
# Populates Stations, Connections, and StationLineRelationships tables
# Print statements for testing purposes
def SaveTfLData(databaseFile, TfL_API_KEY):
    stationDictionary, linesDictionary = fetchTfLData(databaseFile, TfL_API_KEY)

    # Stop execution if an API error occurred
    if type(stationDictionary) is int or type(linesDictionary) is int:
        print(f"A {stationDictionary} error has occurred")
        return stationDictionary

    print("TfL Data Fetched")
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Store all stations in the Stations table
    for station in stationDictionary.values():
        NaPTAN = station.NaPTAN
        StationName = station.StationName
        Latitude = station.Latitude
        Longitude = station.Longitude
        cursor.execute("""
            INSERT OR REPLACE INTO Stations (NaPTAN, StationName, Latitude, Longitude)
            VALUES (?, ?, ?, ?)
            """, (NaPTAN, StationName, Latitude, Longitude))
        connection.commit()
    print("Stored Stations")

    # Store all connections between adjacent stations on each line
    for line in linesDictionary.keys():
        LineID = line
        branches = linesDictionary[line]
        for branch in branches:
            # Adjacent stations form edges in the transport graph
            for i in range(len(branch) - 1):
                StationA = branch[i]
                StationB = branch[i + 1]

            # Fetch average speed of the line from the database
            cursor.execute("""
                SELECT AverageSpeed
                From Lines
                WHERE LineID = ?
                """, (LineID,))
            AverageSpeed = float(cursor.fetchall()[0][0])

            # Fetch station coordinates
            stationACoordinates = getStationCoordinates(databaseFile, StationA)
            stationBCoordinates = getStationCoordinates(databaseFile, StationB)

            # Calculate distance and base travel time (no crowding)
            distanceKM = geodesic(stationACoordinates, stationBCoordinates).km

            # Add a default minute penalty to represent the time trains wait at a station
            BaseTravelTime = ((distanceKM / AverageSpeed) * 60) + WAITING_TIME

            # Store connections in both directions

```

```

        cursor.execute("""
            INSERT OR REPLACE INTO Connections (StationA, StationB, LineID, BaseTravelTime)
            VALUES (?, ?, ?, ?)
            """, (StationA, StationB, LineID, BaseTravelTime))
        cursor.execute("""
            INSERT OR REPLACE INTO Connections (StationA, StationB, LineID, BaseTravelTime)
            VALUES (?, ?, ?, ?)
            """, (StationB, StationA, LineID, BaseTravelTime))

    print("Stored connections")

    connection.commit()
    connection.close()

    # Populate StationLineRelationships using stored connections
    getStationLineRelationships(databaseFile)

    return True

#Main function
def main(TFL_API_KEY, databaseFile):
    code = SaveTfLData(databaseFile, TFL_API_KEY)
    if type(code) is int:
        print(f"A {code} error has occurred")
        return code
    return True

```

```

#Use of the API to get coordinates
def getCoordinates(key, location):
    geocoder = OpenCageGeocode(key)

    # The first result is used as it is assumed to be the most relevant and correct
    results = geocoder.geocode(location)[0][["geometry"]] #Where you can get the coordinates
    return results

```

The system uses multiple external APIs including Transport for London (TfL) and OpenCage Geocoding, using parameterised requests with API keys and query parameters. The TfL API returns nested JSON structures containing the required data. The OpenCage API uses parameterised queries to convert stadium names to geographic coordinates, parsing the JSON response to extract longitude and longitude data from calculating walking distances to nearby stations. Two examples are given above.

The TfL network data acquisition module works by first iterating through each tube line and making API calls which return JSON arrays containing all the stations served by that line including their NaPTAN code (unique station identifiers), display names and geographic coordinates. CreateStation objects are created for each station to encapsulate the data. Ordered branch sequences are fetched from the same API endpoint, and it handles the complexity of TfL's inconsistent station identification system, where some stations return different NaPTAN codes for different modes or platforms. When an unknown station ID is encountered, the function makes an additional API call to the StopPoint endpoint to fetch that station's hubNaptanCode which represents the entire station hub rather than individual platforms, caching these mappings in the weirdStations dictionary to avoid redundant API calls. The fetched data is then put into the database, by first storing the stations, and then creating bidirectional connections, in the connections table. The BaseTravelTime for each

connection is calculated using the coordinates of the two stations, to calculate distance, and the average speed of the line of the connection.

Defensive programming

Parameterised Database Queries:

```
cursor.execute("""
    SELECT PasswordHash, Salt
    FROM Users
    WHERE Username = ?
""", (username,))
```

Every single SQL query across the entire codebase uses parameterised placeholders (the ? syntax) rather than string interpolation, meaning user-supplied values are never directly embedded into query strings. This is the standard defence against SQL injection attacks. By passing values as a separate tuple, the SQLite driver treats them purely as data rather than executable SQL, so a malicious username or other input cannot manipulate the query logic. This pattern is used consistently and an example is given above.

Secure Password Storage:

```
# Generates a cryptographically secure random salt
# A unique salt is used per user to protect against rainbow table attacks
def generateSalt():
    return os.urandom(16).hex() #Creates a random 16-byte salt and returns it as hex

# Creates a SHA-256 hash of the password combined with a salt
# Passwords are never stored in plain text
def createHashPassword(password, salt):
    saltedPassword = salt + password
    byteForm = saltedPassword.encode()
    hashObject = hashlib.sha256(byteForm)
    hashedPassword = hashObject.hexdigest()
    return hashedPassword
```

Passwords are never stored in plain text. Instead, a unique 16-byte cryptographically random salt is generated for each user using `os.urandom()`, which relies on the operating system's secure random number generator. The salt is prepended to the password before hashing with SHA-256, meaning two users with identical passwords will have completely different stored hashes. This defends against rainbow table attacks (pre-computed hash lookups), since an attacker would need to brute-force each user's password individually using their unique salt. During login, the stored salt is retrieved and the same process is repeated, comparing hashes rather than raw passwords.

Session-Based Authentication:

```
# Redirect to login if not logged in
if "username" not in session:
    return redirect(url_for("loginPage"))
```

```
# Logout
@app.route("/logout", methods=["POST"])
def logout():
    """
    Logs out the user by removing session.
    """
    session.pop("username", None) # Remove username from session
    return redirect(url_for("loginPage"))
```

The main journey planning page checks for the presence of a username key in Flask's server-side session object before processing any request. If no session exists, meaning the user has not successfully logged in, they are immediately redirected to the login page rather than being shown an error or partial content. Flask sessions are cryptographically signed using the app's secret key, preventing clients from forging or tampering with session data.

Logout is handled by `session.pop("username", None)` which safely removes the key without raising an error if it is absent.

Input Validation:

```
if start == end:
    error = "Start and end stations must be different"
elif validDateCheck and selectedTime < datetime.now():
    error = "Selected time must be in the future"
```

```
<!-- Username input field -->
<input
  class="textInput"
  type="text"
  name="username"
  placeholder="Username"
  required
>
```

```
/* Numeric only enforcement */
document.querySelectorAll("input[type='number']").forEach(input => {
  // Block e, E, +, -
  input.addEventListener("keydown", e => {
    if (["e", "E", "+", "-"].includes(e.key)) {
      e.preventDefault();
    }
  });
});

// Clean pasted input
input.addEventListener("input", () => {
```

```

    input.value = input.value.replace(/^[^0-9.]/g, '');
  });
});

```

Input validation is applied at multiple layers throughout the application. On the server side, main.py validates that the selected journey datetime is not in the past and that the start and end stations are not identical, returning an error message to the user, rather than crashing if either check fails.

On the client side, all critical form fields use the HTML required attribute to prevent submission with empty values.

The parameter update page applies JavaScript event listeners to every numeric input to block characters like e, E, +, and - (valid in HTML number inputs but can't be used) and strip non-numeric characters from pasted content. Together these layers reduce the chance of invalid data reaching the back-end algorithm.

Exception Handling:

```

cursor.execute("""
    SELECT PasswordHash, Salt
    FROM Users
    WHERE Username = ?
""", (username,))

try:
    result = cursor.fetchall()[0]
    accountFound = True
except:
    accountFound = False

```

```

try:
    # Insert user credentials and modelling parameters into Users table
    cursor.execute(
        """
        INSERT INTO Users(
            Username,
            PasswordHash,
            Salt,
            CHANGING_TIME,
            MAX_TIME_WINDOW,
            rawArrivalPeakOffset,
            rawDeparturePeakOffset,
            ATTENDANCE,
            TRAIN_PROPORTION,
            SIGMA_FACTOR,
            MINIMUM_PEOPLE,
            PERSON_DELAY,
            PROPAGATION_FACTOR,
            WALKING_TIME,
            WALKING_SPEED,
            NORMAL,
            SLIGHTLY,
            BUSY

```

```

    )
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """
    parameterTuple
)
accountMade = True
except:
    # Likely occurs if username already exists
    accountMade = False

```

```

#To get the actual numerical date of the month from the string fetched because it is originally in a form like "Saturday 12th December"
def getDate(monthYear, restOfDate):
    try:
        # Extract the day number from the string
        dayNumber = re.search(r"\d{1,2}", restOfDate).group()
        if len(dayNumber) == 1:
            dayNumber = "0" + dayNumber
        year, month = monthYear.split("-")
        date = f"{dayNumber}-{month}-{year}"
        return date
    except:
        # Returned when the text does not represent a valid date
        return "Error"

```

```

# Check previous people count for this event
try:
    oldPeople = stationObject.events[event] #This will work If and only if the station was already affected by this event
except:
    oldPeople = 0 #This will run if the station has been already affected but not by this event so the event can't be found in the object, so the old population will be set as 0

```

try/except blocks are used throughout the codebase to handle cases where an operation may fail due to circumstances outside the program's control. In login.py, indexing into an empty query result and PRIMARY KEY constraint violations are both caught and converted into boolean flags which lead to error messages being displayed to the user, rather than crashes.

In the scraper, the regex match on BBC Sport HTML may return None if a tag is not actually a date, so the entire date-parsing logic is wrapped in a try/except that returns "Error." Fixtures under an "Error" date are skipped.

In the crowding algorithm's recursive propagation, a KeyError is caught when a station has been affected by other events but not yet by the current one, defaulting the prior count to zero. In total, try/except is used in over 10 locations across the codebase.

These are just three examples, but there are more in the code.

API Retry Logic:

```

# Performs a safe HTTP GET request with retry logic

```

```

# Handles rate limiting, server errors, and lack of internet connection
def safeGet(url, params=None):
    waitTime = 65 # Wait time in seconds when rate-limited or server error occurs
    attempts = 3 # Maximum number of retry attempts
    for attempt in range(attempts):
        try:
            response = requests.get(url, params=params, timeout=5)

            # Handle API rate limiting
            if response.status_code == 429:
                print(f"Rate limit reached: Attempt {attempt + 1} of {attempts}")
                time.sleep(waitTime)
                continue

            # Handle server-side errors
            if response.status_code >= 500 and response.status_code < 600:
                print(f"Server error: Attempt {attempt + 1} of 2")
                time.sleep(waitTime)
                continue

            # Successful or client-side response returned
            return response

        except:
            # Covers timeouts, connection errors, etc.
            pass

    # If all attempts fail, return the status code if available
    try:
        return response.status_code
    except:
        # Occurs if no response was ever received (e.g. no internet)
        return 0

```

The `safeGet` function wraps every external HTTP request in exception handling and implements automatic retry logic for transient failures. Upon encountering errors, the programme waits 65 seconds before retrying, up to three times. If all attempts are exhausted, it returns the HTTP status code if one was received, or 0 if no response was ever obtained. Callers in `getTfLData.py`, `scraper.py`, and `storeStadiumData.py` then check if `type(response)` is `int` to detect failure.

Error Propagation:

```

# Fetch and store TfL transport data for me
tfIDone = getTfLData.main(TfL_API_KEY, databaseFile)
if type(tfIDone) == int: #When an error has occurred a status code will be returned
    return tfIDone
pass

```

```

# Geocode stadiums and link them to stations with default walking parameters
geocodingMessage = storeStadiumData.main(OPENCAGE_API_KEY, londonjsonFile, databaseFile, walkingTime,
walkingSpeed)
if type(geocodingMessage) != bool:
    return geocodingMessage

```

```

if message == "success":
    status = "success"
else:
    # Remove database if creation failed
    status = "failed"
    databaseFile = Path(app.config["DATABASE_FILE"])
    if databaseFile.exists():
        databaseFile.unlink()

```

Rather than allowing exceptions to bubble up uncontrolled through the call stack, the database creation pipeline uses return values as error signals. Functions return an integer HTTP status code or error string on failure, and True or "success" otherwise. Each step checks the return type and propagates the error upward rather than continuing with potentially incomplete data. Here are a couple examples.

If creation fails, main.py deletes the partially created database file to avoid leaving it in a broken state that could cause errors on the next run.

Defensive Database Writes:

```

# Creates the SQLite database and all required tables
# If tables already exist, they will not be recreated
def createDatabase(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.executescript("""
        CREATE TABLE IF NOT EXISTS Stations (
            NaPTAN TEXT PRIMARY KEY,
            StationName TEXT,
            Latitude REAL,
            Longitude REAL
        );

        CREATE TABLE IF NOT EXISTS Lines (
            LineID TEXT PRIMARY KEY,
            LineName TEXT,
            AverageSpeed TEXT
        );

        CREATE TABLE IF NOT EXISTS StationLineRelationships (
            NaPTAN TEXT,
            LineID TEXT,
            PRIMARY KEY (NaPTAN, LineID),
            FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN),
            FOREIGN KEY (LineID) REFERENCES Lines(LineID)
        );

        CREATE TABLE IF NOT EXISTS Connections (
            StationA TEXT,
            StationB TEXT,
            LineID TEXT,
            BaseTravelTime REAL,
            PRIMARY KEY (StationA, StationB, LineID),
            FOREIGN KEY (StationA) REFERENCES Stations(NaPTAN),
            FOREIGN KEY (StationB) REFERENCES Stations(NaPTAN),

```

```
FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);
```

```
CREATE TABLE IF NOT EXISTS Venues (
    VenueName TEXT PRIMARY KEY,
    Capacity INTEGER,
    Latitude REAL,
    Longitude REAL
);
```

```
CREATE TABLE IF NOT EXISTS VenueStationRelationships (
    VenueName TEXT,
    NaPTAN TEXT,
    Distance REAL,
    PRIMARY KEY (VenueName, NaPTAN),
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
    FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN)
);
```

```
CREATE TABLE IF NOT EXISTS Events (
    VenueName TEXT,
    Date TEXT,
    Time TEXT,
    HomeTeam TEXT,
    AwayTeam TEXT,
    EventName TEXT,
    Duration INTEGER,
    PRIMARY KEY (VenueName, Date),
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
    FOREIGN KEY (HomeTeam) REFERENCES Teams(TeamName)
);
```

```
CREATE TABLE IF NOT EXISTS Users (
    Username TEXT PRIMARY KEY,
    PasswordHash TEXT,
    Salt TEXT,
    CHANGING_TIME REAL,
    MAX_TIME_WINDOW REAL,
    rawArrivalPeakOffset REAL,
    rawDeparturePeakOffset REAL,
    ATTENDANCE REAL,
    TRAIN_PROPORTION REAL,
    SIGMA_FACTOR REAL,
    MINIMUM_PEOPLE REAL,
    PERSON_DELAY REAL,
    PROPAGATION_FACTOR REAL,
    WALKING_TIME REAL,
    WALKING_SPEED REAL,
    NORMAL REAL,
    SLIGHTLY REAL,
    BUSY REAL
);
```

```
CREATE TABLE IF NOT EXISTS Teams (
    TeamName TEXT PRIMARY KEY,
    VenueName TEXT,
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName)
);
```

```
""")
```

```
connection.commit()
connection.close()
```

```

# Store the venue-station relationships and walking distances
for stationID in closeStations.keys():
    distance = closeStations[stationID]
    cursor.execute("""
        INSERT INTO VenueStationRelationships
        VALUES (?, ?, ?)
        ON CONFLICT (VenueName, NaPTAN) DO
        UPDATE SET
            Distance = excluded.Distance
    """, (stadiumName, stationID, distance))

```

All nine table creation statements use CREATE TABLE IF NOT EXISTS, which means the function can be safely rerun, without raising errors if tables already exist.

Throughout the programme, INSERT OR REPLACE and ON CONFLICT DO UPDATE is used, which avoids needing separate queries for checking existence, and it means that when rerunning these sections of code, existing rows are updated where necessary, rather than failing with UNIQUE constraint violations. Above is an example.

Abstract Base Classes:

```

from abc import ABC, abstractmethod

```

```

# Abstract base class for all transport-related components
# Holds common attributes used for crowding and delay simulation
class TransportClass(ABC):
    def __init__(self):
        self.currentPassengerLoad = 0 #initially set to 0 and is the extra number of people present due to the events
        self.DelayFactor = 1 # initially set to 1 and is the factor by which the time to travel would be multiplied by to simulate crowding

    @abstractmethod
    def IsAffected(self):
        pass

```

```

# Parent class for all connection types used in Dijkstra's algorithm
class Connection(TransportClass):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime):
        super().__init__()
        self.StationA = StationA
        self.StationB = StationB
        self.LineID = LineID
        self.connectionReference = (self.StationA, self.StationB, self.LineID) #Unique identifier
        self.BaseTravelTime = BaseTravelTime
        self.LineDelay = 0

    @abstractmethod
    def IsAffected(self):
        pass

    @abstractmethod

```

```
def returnDelay(self):
    pass
```

TransportClass is defined as an Abstract Base Class using Python's abc module, with `IsAffected()` marked as an abstract method. Connection adds a second abstract method `returnDelay()`. This means neither class can be instantiated directly and any attempt raises a `TypeError` at runtime. All subclasses are forced to provide their own implementations of these methods, guaranteeing that every object in the network can be safely queried for its affected status and delay factor. This prevents the error of accidentally using a base class object that has no real implementation of a critical method.

Database Normalisation:

```
# Creates the SQLite database and all required tables
# If tables already exist, they will not be recreated
def createDatabase(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.executescript("""
CREATE TABLE IF NOT EXISTS Stations (
    NaPTAN TEXT PRIMARY KEY,
    StationName TEXT,
    Latitude REAL,
    Longitude REAL
);

CREATE TABLE IF NOT EXISTS Lines (
    LineID TEXT PRIMARY KEY,
    LineName TEXT,
    AverageSpeed TEXT
);

CREATE TABLE IF NOT EXISTS StationLineRelationships (
    NaPTAN TEXT,
    LineID TEXT,
    PRIMARY KEY (NaPTAN, LineID),
    FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);

CREATE TABLE IF NOT EXISTS Connections (
    StationA TEXT,
    StationB TEXT,
    LineID TEXT,
    BaseTravelTime REAL,
    PRIMARY KEY (StationA, StationB, LineID),
    FOREIGN KEY (StationA) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (StationB) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);

CREATE TABLE IF NOT EXISTS Venues (
    VenueName TEXT PRIMARY KEY,
    Capacity INTEGER,
    Latitude REAL,
```

```

        Longitude REAL
    );

    CREATE TABLE IF NOT EXISTS VenueStationRelationships (
        VenueName TEXT,
        NaPTAN TEXT,
        Distance REAL,
        PRIMARY KEY (VenueName, NaPTAN),
        FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
        FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN)
    );

    CREATE TABLE IF NOT EXISTS Events (
        VenueName TEXT,
        Date TEXT,
        Time TEXT,
        HomeTeam TEXT,
        AwayTeam TEXT,
        EventName TEXT,
        Duration INTEGER,
        PRIMARY KEY (VenueName, Date),
        FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
        FOREIGN KEY (HomeTeam) REFERENCES Teams(TeamName)
    );

    CREATE TABLE IF NOT EXISTS Users (
        Username TEXT PRIMARY KEY,
        PasswordHash TEXT,
        Salt TEXT,
        CHANGING_TIME REAL,
        MAX_TIME_WINDOW REAL,
        rawArrivalPeakOffset REAL,
        rawDeparturePeakOffset REAL,
        ATTENDANCE REAL,
        TRAIN_PROPORTION REAL,
        SIGMA_FACTOR REAL,
        MINIMUM_PEOPLE REAL,
        PERSON_DELAY REAL,
        PROPAGATION_FACTOR REAL,
        WALKING_TIME REAL,
        WALKING_SPEED REAL,
        NORMAL REAL,
        SLIGHTLY REAL,
        BUSY REAL
    );

    CREATE TABLE IF NOT EXISTS Teams (
        TeamName TEXT PRIMARY KEY,
        VenueName TEXT,
        FOREIGN KEY (VenueName) REFERENCES Venues(VenueName)
    );

    """
    connection.commit()
    connection.close()

```

The database schema is designed to approximately Third Normal Form: each table stores attributes that depend only on its primary key, with no transitive dependencies. Many-to-many relationships are handled via junction tables with composite primary keys, and foreign key constraints are declared throughout, so the database enforces referential integrity itself

Mathematical Boundary Guards:

```
#This function assigns each station a weight so that the number of people at the stations will be more accurately assigned
#stationDistances dictionary format - {NaPTAN: distance}
#Returns dictionary format - {NaPTAN: proportion}
def inverseWeight(stationDistances):
    extra = 1 #Avoid divide by zero errors - if the distance between the station and the venue is 0
    weights = {}
    proportions = {}
    totalWeight = 0

    # Calculate inverse-distance weight for each station
    for station in stationDistances.keys():
        distance = stationDistances[station]
        weight = 1 / (distance + extra)
        totalWeight += weight
        weights[station] = weight

    # Convert weights to proportions of total
    for station in weights.keys():
        proportion = weights[station] / totalWeight
        proportions[station] = proportion

    return proportions
```

```
sigmaArrival = max(1, arrivalWindowMinutes / SIGMA_FACTOR) #max used to make sure sigma is at least bigger or equal to 1
sigmaDeparture = max(1, departureWindowMinutes / SIGMA_FACTOR)
```

Two mathematical guard patterns are used to prevent invalid inputs from reaching core calculations. In `inverseWeight()`, a constant of 1 is added to every distance before the $1 / \text{distance}$ calculation. This means that if a station is located exactly at a venue ($\text{distance} = 0$), the division would cause a `ZeroDivisionError`, and the $+1$ prevents this while having negligible effect on the weighting since distances are in kilometres.

In `findBaseNumberOfPeople()`, `max(1, ...)` enforces a minimum standard deviation of 1 for the Gaussian distribution. If a user configures an extreme `SIGMA_FACTOR`, the division could produce zero, which would also cause a `ZeroDivisionError` inside the Gaussian formula. Both guards make the mathematical model unconditionally safe regardless of user parameter values.

Recursion Termination:

```
# Stop recursion if propagated population is too small
if newPeople < MINIMUM_PEOPLE:
    pass
else:
    connections = graph[stationID]
    for connection in connections:
        newStationID = connection[0]
```

```
propegation(graph, newStationID, newPeople, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)
```

The MINIMUM_PEOPLE threshold acts as the termination condition. The propagated count decreases with each hop, and once it drops below the minimum the recursion stops. This prevents a stack overflow and protects against infinite recursion. Crowds too small to cause meaningful delays do not need to be tracked through the network.

Guaranteed Minimum Result:

```
# Always include the closest station to guarantee at least one station per venue
firstStation = stationWithDistance.pop(0)
CloseStationDictionary[firstStation[0]] = firstStation[1]

# Add all stations within the walking distance threshold
for station in stationWithDistance:
    if station[1] <= walkingDistance:
        CloseStationDictionary[station[0]] = station[1]
```

If the walking distance threshold is very small, it is possible that no stations would fall within the walking radius, leaving a venue with no associated stations causing errors further down. By unconditionally popping and inserting the single closest station before applying the distance filter, the function guarantees that every venue always has at least one station regardless of the threshold.

Safe Application Startup:

```
# Check if database exists
databaseFilePath = Path(databaseFile)
if not databaseFilePath.exists():
    # Create flags file if it doesn't exist
    flagsPath = Path(flagsFile)

    if not flagsPath.exists():
        with open(flagsFile, "w") as file:
            json.dump({"databaseCreated": False}, file)

# Create database in a separate thread to avoid blocking and to allow for a waiting page to load
threading.Thread(target=createDatabase).start()
return redirect(url_for("loadPage"))
```

```
@app.route('/check_flag', methods=['GET'])
def checkFlag():
    """
    Returns current flags status (database creation, error, previous page).
    """
    flagsFile = app.config["FLAGS_FILE"]

    response = {
        "status": "creating",
        "error": None,
        "previousPage": session.get("previousPage")
```

```

}

try:
    with open(flagsFile, "r") as file:
        response.update(json.load(file))
except:
    pass

return jsonify(response)

```

Before attempting any database operations, the login route checks whether the database file exists using `pathlib.Path.exists()`. If not, it starts a background thread to create it and redirects to a loading page, so the user sees a screen with a message rather than a crash. The loading page polls the `/check_flag` endpoint every two seconds to check creation status. This endpoint reads a JSON flags file written by the background thread.

User Data Preservation During Database Refresh:

```

# Background Database Update
def backgroundDataFetch():
    """
    Updates the database in the background while preserving user data.
    """
    databaseFile = app.config["DATABASE_FILE"]

    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM Users")
    users_data = cursor.fetchall() # List of tuples with all user rows
    connection.close()

    # Delete old database
    databasePath = Path(databaseFile)
    databasePath.unlink()

    # Recreate database
    createDatabase()

    # Restore user data
    if users_data:
        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()
        cursor.executemany("""
            INSERT OR REPLACE INTO Users (
                Username,
                PasswordHash,
                Salt,
                CHANGING_TIME,
                MAX_TIME_WINDOW,
                rawArrivalPeakOffset,
                rawDeparturePeakOffset,
                ATTENDANCE,
                TRAIN_PROPORTION,
                SIGMA_FACTOR,
                MINIMUM_PEOPLE,
                PERSON_DELAY,
                PROPAGATION_FACTOR,
                WALKING_TIME,

```

```

        WALKING_SPEED,
        NORMAL,
        SLIGHTLY,
        BUSY
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """
    users_data)
    connection.commit()
    connection.close()

```

When the user triggers a data refresh, the entire database must be deleted and recreated to pull in fresh fixture and TfL data. Rather than losing all user accounts and custom parameters in this process, `backgroundDataFetch` first reads and stores all rows from the Users table in memory before deletion, then reinserts them after the new database is created.

Coding style

Modules (subroutines) with appropriate interfaces:

Throughout the code, subroutines with well defined inputs and outputs have been used, with their names appropriately describing their functions. An example is:

```

#Use of the API to get coordinates
def getCoordinates(key, location):
    geocoder = OpenCageGeocode(key)

    # The first result is used as it is assumed to be the most relevant and correct
    results = geocoder.geocode(location)[0]["geometry"] #Where you can get the coordinates
    return results

```

Loosely coupled modules (subroutines) – module code interacts with other parts of the program through its interface only

Each module has a clean interface and callers interact only with those functions. For example, `getResults.py` doesn't directly access the internal functions of `crowdingAlgorithm.py`, but rather the defined entry point.

```

# Generate the affected network (crowding and delays applied)
affectedNetwork = crowdingAlgorithm.main(date, journeyStart, startStation, endStation, databaseFile, username)

```

Cohesive modules (subroutines) – module code does just one thing

This way of coding has been used multiple times in the code. For example, in `storeStadiumData.py`, each function performs exactly one defined task.

```

import sqlite3
from opencage.geocoder import OpenCageGeocode
import json
from geopy.distance import geodesic

# Use of the API to get coordinates
def getCoordinates(key, location):
    geocoder = OpenCageGeocode(key)

    # The first result is used as it is assumed to be the most relevant and correct
    results = geocoder.geocode(location)[0][0] #Where you can get the coordinates
    return results

# Finds all stations within walking distance of a venue
# Ensures at least one station is always returned (the closest one)
def getCloseStations(allStations, venueCoordinates, walkingDistance):
    stationWithDistance = []
    CloseStationDictionary = {}

    # Calculate distance from the venue to every station
    for stationTuple in allStations:
        stationCoordinates = (stationTuple[1], stationTuple[2])
        distanceKM = geodesic(venueCoordinates, stationCoordinates).km
        stationWithDistance.append((stationTuple[0], distanceKM))

    # Sort stations by distance from the venue
    stationWithDistance.sort(key=lambda x: x[1])

    # Always include the closest station to guarantee at least one station per venue
    firstStation = stationWithDistance.pop(0)
    CloseStationDictionary[firstStation[0]] = firstStation[1]

    # Add all stations within the walking distance threshold
    for station in stationWithDistance:
        if station[1] <= walkingDistance:
            CloseStationDictionary[station[0]] = station[1]

    return CloseStationDictionary

# Inserts stadium, team, and venue-station relationship data for me into the database
def insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    latitude = coordinates["lat"]
    longitude = coordinates["lng"]
    venueCoordinates = (latitude, longitude)

    cursor.execute("""
        SELECT NaPTAN, Latitude, Longitude
        FROM Stations
    """)
    allStations = cursor.fetchall()

    # Find stations that are within walking distance of the stadium
    closeStations = getCloseStations(allStations, venueCoordinates, walkingDistance)

    # Store the venue-station relationships and walking distances
    for stationID in closeStations.keys():
        distance = closeStations[stationID]
        cursor.execute("""

```

```

        INSERT INTO VenueStationRelationships
        VALUES (?, ?, ?)
        ON CONFLICT (VenueName, NaPTAN) DO
        UPDATE SET
            Distance = excluded.Distance
        """ , (stadiumName, stationID, distance))

# Store team and home venue relationship
cursor.execute("""
    INSERT INTO Teams (TeamName, VenueName)
    VALUES (?, ?)
    ON CONFLICT (TeamName)
    DO UPDATE SET
        VenueName = excluded.VenueName
    """ , (club, stadiumName))

# Store venue details
cursor.execute("""
    INSERT INTO Venues (VenueName, Capacity, Latitude, Longitude)
    Values (?, ?, ?, ?)
    ON CONFLICT (VenueName)
    DO UPDATE SET
        Capacity = excluded.Capacity,
        Latitude = excluded.Latitude,
        Longitude = excluded.Longitude
    """ , (stadiumName, capacity, latitude, longitude))

connection.commit()
connection.close()

#Uses JSON file of club data for me
#{ "ClubName": [StadiumName, Capacity]}
def getStadiumData(londonClubsFile, databaseFile, key, walkingDistance):
    with open(londonClubsFile, "r") as file:
        londonClubs = json.load(file)

    # Process every London club and its stadium
    for club, data in londonClubs.items():
        stadiumName, capacity = data
        try:
            coordinates = getCoordinates(key, stadiumName)
        except:
            # Return error message if OpenCage API fails
            return "OpenCage Geocoder API"
        insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance)

    return True

# Calculates the maximum walking distance using walking speed and walking time
def calculateWalkingDistance(walkingTime, walkingSpeed):
    # walkingSpeed is in km/h and walkingTime is in minutes
    distance = (walkingSpeed * walkingTime) / 60
    return distance

# Main function for fetching and storing stadium related data for me
def main(key, londonClubsFile, databaseFile, walkingTime, walkingSpeed):
    # Calculate what is considered a reasonable walking distance
    walkingDistance = calculateWalkingDistance(walkingTime, walkingSpeed)

    done = getStadiumData(londonClubsFile, databaseFile, key, walkingDistance)
    print(done)

```

```

return done

# Recalculates and reinserts venue-station relationships when walking parameters change
def redoStore(walkingTime, walkingSpeed, londonClubsFile, databaseFile):
    walkingDistance = calculateWalkingDistance(walkingTime, walkingSpeed)

    with open(londonClubsFile, "r") as file:
        londonClubs = json.load(file)

    # Clear existing venue-station relationships
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("DELETE FROM VenueStationRelationships")
    connection.commit()
    connection.close()

    # Reinsert relationships using updated walking distance
    for club, data in londonClubs.items():
        stadiumName, capacity = data

        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()

        # Fetch stored venue coordinates to avoid unnecessary API calls
        cursor.execute("""
            SELECT Latitude, Longitude
            FROM Venues
            WHERE VenueName = ?
            """, (stadiumName,))

        coordinates = {}
        coordinates["lat"], coordinates["lng"] = cursor.fetchall()[0]

        insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance)

    connection.close()

```

Modules (collections of subroutines) – subroutines with common purpose grouped.

The codebase is split into multiple files, each grouping subroutines with a shared purpose. For example:

File:	Purpose:
login.py	Account creation and authentication
pathFinding.py	Graph construction and Dijkstra's algorithm
crowdingAlgorithm.py	Crowd modelling
scraper.py	Web scraping of fixture data
getTfLData.py	TfL API data acquisition

storeStadiumData.py	Geocoding and venue-station linking
customFunctions.py	Shared utility functions used across modules
databaseCreation.py	Database schema creation and population

Defensive programming.

Already covered in Defensive programming,

Good exception handling.

Already covered in Defensive programming,

Well-designed user interface

The UI sketches accurately display the final programme, and demonstrations of the UI can be found in the testing videos.

Modularisation of code

The file structure of the programme can be found in the Design section of this document, which clearly shows the code being split across multiple files, rather than large single files. The different modules handle different tasks, with the program ultimately being orchestrated by main.py.

Good use of local variables

Intermediate calculations and other steps are broken into named local variables, rather than chained into a single expression. This makes each step of the calculation readable, and that all variables are scoped to the function they are needed in. For example:

```
#For each event, finds the base number of people at the nearby stations at the middle of the journey
# Returns dictionary {NaPTAN: [arrivalPeople, departurePeople]}
def findBaseNumberOfPeople(
    capacity,
    ATTENDANCE, #The proportion of seats that will actually be filled
    TRAIN_PROPORTION, #The proportion of people using TfL rail
    startDifferenceMinutes, #minutes from journey midpoint to event start
    endDifferenceMinutes, #minutes from journey midpoint to event end
    arrivalWindowMinutes, #Length of time of arrival window where the journey may be affected
    departureWindowMinutes, #Length of time of departure window where the journey may be affected
    arrivalPeakOffset, #Offset because peaks are usually not at exactly the start or end
    departurePeakOffset,
```

```

    statonsWithDistances, #Dictionary of stations with the values being the distance between the station and the station
    and the venue
    SIGMA_FACTOR
):

    totalAttendance = capacity * ATTENDANCE #Gets the expected attendance at the venue
    tflUsageTotal = totalAttendance * TRAIN_PROPORTION #Gets the expected total usage of TfL rail

    # Convert distances into proportional distribution of passengers
    proportions = inverseWeight(statonsWithDistances)
    StationsWithPeopleAtPeak = {}
    for stationID in proportions.keys():
        StationsWithPeopleAtPeak[stationID] = proportions[stationID] * tflUsageTotal

    # Adjust for peak offsets
    relativeArrivalPeakTime = startDifferenceMinutes - arrivalPeakOffset
    relativeDepartPeakTime = endDifferenceMinutes + departurePeakOffset

    sigmaArrival = max(1, arrivalWindowMinutes / SIGMA_FACTOR) #max used to make sure sigma is at least bigger or equal
    to 1
    sigmaDeparture = max(1, departureWindowMinutes / SIGMA_FACTOR)

    #Realistically there isn't going to be a difference between arrival and departure rates because if you came by train, you
    will probably leave by train so will not affect, and vice versa, so the factors will be set to one

    arrivalsFactorAtMid = gaussianFactorAtPeak(relativeArrivalPeakTime, sigmaArrival, 0)
    depratureFactorAtMid = gaussianFactorAtPeak(relativeDepartPeakTime, sigmaDeparture, 0)

    # Compute expected arrivals and departures at each station
    stationsAtMid = {} #Format - {NaPTAN: ArrivalNumbers, DepartureNumbers}
    for stationID in StationsWithPeopleAtPeak.keys():
        arrivalPeople = StationsWithPeopleAtPeak[stationID] * arrivalsFactorAtMid
        departurePeople = StationsWithPeopleAtPeak[stationID] * depratureFactorAtMid
        stationsAtMid[stationID] = [arrivalPeople, departurePeople]

    return stationsAtMid

```

Minimal use of global variables

The global keyword is not used anywhere in the codebase, as there are no shared global variables. There are capitalised constants, but they are defined and used only in their respective modules, with them never being reassigned after definition, and clearly being labeled with capital letters. All variables are passed as function arguments or stored in Flask's app.config dictionary.

Managed casting of types

This has been done consistently through the code, with an example below:

```

# Save User Parameters
@app.route("/save_parameters", methods=["POST"])
def saveParameters():
    """
    Saves user-modified parameters and updates stadium data.
    """

```

```

databaseFile = app.config["DATABASE_FILE"]
londonjsonFile = app.config["LONDONJSON_FILE"]

username = session.get("username")
data = request.get_json()

values = (
    float(data["CHANGING_TIME"]),
    float(data["MAX_TIME_WINDOW"]),
    float(data["rawArrivalPeakOffset"]),
    float(data["rawDeparturePeakOffset"]),
    float(data["ATTENDANCE"]),
    float(data["TRAIN_PROPORTION"]),
    float(data["SIGMA_FACTOR"]),
    float(data["MINIMUM_PEOPLE"]),
    float(data["PERSON_DELAY"]),
    float(data["PROPAGATION_FACTOR"]),
    float(data["WALKING_TIME"]),
    float(data["WALKING_SPEED"]),
    float(data["NORMAL"]),
    float(data["SLIGHTLY"]),
    float(data["BUSY"]),
    username # WHERE clause
)

connection = sqlite3.connect(databaseFile)
cursor = connection.cursor()
cursor.execute("""
    UPDATE Users SET
        CHANGING_TIME = ?,
        MAX_TIME_WINDOW = ?,
        rawArrivalPeakOffset = ?,
        rawDeparturePeakOffset = ?,
        ATTENDANCE = ?,
        TRAIN_PROPORTION = ?,
        SIGMA_FACTOR = ?,
        MINIMUM_PEOPLE = ?,
        PERSON_DELAY = ?,
        PROPAGATION_FACTOR = ?,
        WALKING_TIME = ?,
        WALKING_SPEED = ?,
        NORMAL = ?,
        SLIGHTLY = ?,
        BUSY = ?
    WHERE Username = ?
""", values)
connection.commit()
connection.close()

# Recalculate stadium data
walkingTime = float(data["WALKING_TIME"])
walkingSpeed = float(data["WALKING_SPEED"])

storeStadiumData.redoStore(walkingTime, walkingSpeed, londonjsonFile, databaseFile)

return jsonify({"success": True})

```

Use of constants

Values derived from assumptions which will not change are defined as constants in uppercase on the module level, making their purpose immediately clear from their name and can easily be updated. An example is below:

```
#How long a train waits at a station
WAITING_TIME = 0.5
```

Appropriate indentation

Indentation is mandatory in Python and therefore has been used.

Self-documenting code

Function and variable names are chosen to describe their purpose clearly enough that the logic is largely readable without needing comments.

Consistent style throughout

A consistent naming convention is applied throughout. Functions use camelCase, classes use PascalCase, and constants use UPPER_CASE. Database connections and cursors are always named connection and cursor. All SQL queries are written using triple-quoted strings with consistent indentation. All files follow the same structure of imports at the top, constants next, then function or class definitions.

File paths parameterised

No file paths are hardcoded within route handlers or algorithm functions. All paths are passed in as parameters and stored in Flask's app.config dictionary at startup. This means changing the location of any file only requires updating the single call to setUpConfig() in the if __name__ == '__main__' block, rather than finding every reference scattered across the codebase.

```
# Makes these accessible anywhere in the app
def setUpConfig(
    app,
    databaseFile,
    LinesjsonFile,
    TfL_API_KEY,
    OPENCAGE_API_KEY,
    londonjsonFile,
    leaguesjsonFile,
    flagsFile,
    defaultParametersFile
):
    """
```

Sets up all configuration variables in Flask's config dictionary.

```
"""
app.config["DATABASE_FILE"] = databaseFile
app.config["LINES_JSON_FILE"] = LinesjsonFile
app.config["TFL_API_KEY"] = TfL_API_KEY
app.config["OPENCAGE_API_KEY"] = OPENCAGE_API_KEY
app.config["LONDONJSON_FILE"] = londonjsonFile
app.config["LEAGUES_JSON_FILE"] = leaguesjsonFile
app.config["FLAGS_FILE"] = flagsFile
app.config["DEFAULT_PARAMETERS_FILE"] = defaultParametersFile
```

Meaningful identifier names

Variable and function names across the codebase describe their content or purpose precisely.

```
listOfNeighbourStations = {} # {StationA: [[StationB, BaseTravelTime, LineID], ...]}
```

Annotation used effectively where required

Comments are used where the code alone does not make the reasoning clear. Docstrings are used on Flask route functions in main.py to describe each route's purpose.

```
#Fetches all the unique connections from the database and for every StationA, it adds to a list all StationBs in their own list including Line and travel time
#Forms dictionary in the form {StationA: [[StationB, BaseTravelTime, LineID], [StationC, ...], ...]}
def MakeGraph(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    listOfNeighbourStations = {} # {StationA: [[StationB, BaseTravelTime, LineID], ...]}
    rows = cursor.execute("SELECT StationA, StationB, LineID, BaseTravelTime FROM connections").fetchall()

    for StationA, StationB, LineID, BaseTravelTime in rows: #For every unique connection (direction matters)
        if StationA not in listOfNeighbourStations:
            listOfNeighbourStations[StationA] = []
        if [StationB, BaseTravelTime, LineID] not in listOfNeighbourStations[StationA]:
            listOfNeighbourStations[StationA].append([StationB, BaseTravelTime, LineID])

    connection.close()
    return listOfNeighbourStations
```

```
# Loading Page
@app.route('/load', methods=['GET', 'POST'])
def loadPage():
    """
    Simple page to show when database is being created or updated.
    """
    return render_template("loading.html")
```

Testing

Test strategy

To test the programme, I employed the following testing strategies across different stages of development:

- A. Before full development began, I prototyped the pathfinding component and presented it to Ms Miah for feedback. Her response helped inform the final design of the algorithm, in particular, the introduction of the line change penalty. This early prototype testing helped to clarify the direction of this project before significant development time was invested.
- B. While developing each module, I included an `if __name__ == "__main__"` block at the end of every file containing sample test data. This allowed me to run individual modules in isolation and verify their outputs before integrating them with the rest of the system. This not only means that testing during future development will be easier, but is also evidence of iterative testing, and therefore has been left in the annotated code.
- C. I created a suite of automated unit tests using Python's `unittest` library, which can be found in the Unit testing section below. These tests cover every function across all the programme files and were run regularly during development to catch errors when changes were made to existing code.
- D. After verifying that the individual functions worked as intended through the unit tests, I carried out structured manual testing as laid out in the test plan below. It is further explained in the Test plan section.
- E. I then tested the interfaces between components through integration testing, verifying that modules which depend on each other, produce correct results when working together. This is described in the Integration testing section below.
- F. I carried out system testing by recording a full walkthrough of the application from start to finish as a standard user. The videos can be found in the System testing section below (playlist on Youtube).
- G. Finally, the completed programme was given to my end user Ms Miah, who tested it over several weeks and confirmed that her original requirements had been met. This is described in the Acceptance testing section.

Test plan

The following test plan covers all major components of the programme. Tests are grouped by component and page, mirroring the structure of the application. Each test is classified as Normal (N), Boundary (B), or Erroneous (E). The Actual Outcome column contains video timestamps where evidence is recorded.

Login page

<https://youtu.be/o1U4TzhishE>

Test Number	Test	Purpose	Test Data	Expected outcome	Actual outcome
1	Login with empty username and password	Test that partial data inputs are rejected (E)	Both fields left empty	Cannot log in. HTML required attribute prevents submission	00:00-00:29
2	Login with empty password only	Test that partial data inputs are rejected (E)	Valid username, empty password	Cannot log in. HTML required attribute prevents submission	00:29-00:50
3	Login with username that does not exist	Test erroneous data is handled gracefully (E)	"doesnotexist" as username	"Account Not Found" message displayed on login page	00:50-01:16
4	Login with correct username but wrong password	Test that wrong password is caught (E)	"testuser" with incorrect password	"Wrong Password" message displayed on login page	01:16-01:46
5	Login with correct username and correct password	Test that expected valid data works (N)	Valid credentials	User is logged in and redirected to main journey page	01:46-02:21
6	Sign up with empty fields	Test that empty sign-up inputs are rejected (E)	Both fields empty	HTML required attribute prevents submission	02:21-02:42
7	Sign up with a username that already exists	Test that duplicate usernames are caught (E)	Username already in the database	"Account already created" message displayed	02:42-03:16

8	Sign up with a new unique username and password	Test that valid sign-up works (N)	New username and password	Account is created, user is logged in and redirected to main page	03:16-03:44
9	Verify that new account is initialised with default parameters	Test that default parameters are assigned on account creation (N)	Newly created account	All 15 parameters match the values in defaultParameters.json	03:44-04:20
10	Access main page without logging in	Test that unauthenticated access is blocked (E)	Navigate directly to /main without session	User is redirected to login page	04:20-04:37
11	Access settings page without logging in	Test that unauthenticated access is blocked (E)	Navigate directly to /update without session	User is redirected to login page	04:37-05:00

Loading page and database creation

<https://youtu.be/Baq3aLqgXs0>

Test Number	Test	Purpose	Test Data	Expected outcome	Actual outcome
12	First launch with no database file present	Test that database creation starts and loading page is shown (N)	Delete database file, relaunch application	Loading page is displayed while database is created in background	00:00-00:48
13	Loading page polling updates when creation completes	Test that loading page redirects correctly on success (N)	Wait for database creation to finish	User is automatically redirected to login page	00:48-01:43
14	Retry button on database error page	Test that retry resets flags and restarts the process	Click Retry on database error page	Flags are reset, creation restarts, loading page	01:43-04:20

		(N)		shown again	
--	--	-----	--	-------------	--

Main journey page

<https://youtu.be/vFfK7rwYsF4>

Test Number	Test	Purpose	Test Data	Expected outcome	Actual outcome
15	Search for a station by typing partial name	Test that the station autocomplete search works (N)	Type "King" into start station box	Dropdown shows matching stations including King's Cross St. Pancras	00:00-00:44
16	Select the same station for start and end	Test that identical start and end is caught (E)	Select "Angel Underground Station" for both start and end	Error message: "Start and end stations must be different"	00:44-01:29
17	Submit with a date and time in the past	Test that past datetime is rejected (E)	Select yesterday's date and a time in the past	Error message: "Selected time must be in the future"	01:29-02:13
18	Submit with today's date but a time 1 minute ago	Test boundary case for datetime validation (B)	Today's date, time 1 minute ago	Error message: "Selected time must be in the future"	02:13-02:57
19	Submit a valid journey on a date with no events	Test that system produces a result when no crowding applies (N)	Two different stations, future date with no fixtures	Both routes displayed, journey times identical (no crowding delay)	02:57-04:54
20	Submit a valid journey on a date with a nearby fixture	Test that crowding is applied when an event is present (N)	Two stations near a London stadium, date of a fixture	Both routes displayed; crowded route shows higher journey time or alternative route	04:54-07:05

21	Verify both routes are displayed	Test that both paths are shown to the user (N)	Any valid journey search	Two path containers visible, each showing line segments, station names and crowding descriptions	07:05-07:42
22	Verify qualitative crowding descriptions appear on stations	Test that each station has a crowding label (N)	Journey on a date with a nearby fixture	Each station listed shows one of: Normal, Slightly, Busy, Severe	07:42-08:13
23	Verify total journey time and arrival time are shown	Test that time outputs are formatted correctly (N)	Any valid journey search	Both Total Time and Arrival Time fields populated on both route displays	08:13-08:35
24	Click the Update button on main page	Test that Update button redirects correctly (N)	Click Update button	User is redirected to settings page	08:35-09:00

Settings page

<https://youtu.be/MjDlmwrnUr8>

Test Number	Test	Purpose	Test Data	Expected outcome	Actual outcome
25	Settings page displays current parameter values	Test that stored values are loaded correctly (N)	Navigate to /update while logged in	All 15 parameter fields populated with the user's stored values	00:00-01:03
26	Enter a non-numeric character into a numeric field	Test that input sanitisation blocks invalid characters (E)	Type "abc" into CHANGING_TIME field	Non-numeric characters are stripped by JavaScript input listener	01:03-01:38

27	Enter a negative value into a numeric field	Test that negative inputs are blocked (E)	Type "-5" into SIGMA_FACT OR field	The minus sign is stripped; only the digit "5" remains	01:38-02:17
28	Enter scientific notation character "e" into a numeric field	Test that "e" is blocked in number inputs (E)	Press "e" key in any parameter field	Key is blocked; no character is entered	02:17-03:08
29	Click Load Default Values	Test that defaults are loaded from configuration file (N)	Click Load Default Values button	All 15 fields update to match the values in defaultParameters.json	03:08-03:45
30	Save modified parameters	Test that updated parameters are written to the database (N)	Change CHANGING_TIME to 6.0, click Save	Navigating back to settings shows CHANGING_TIME as 6.0	03:45-04:33
31	Change walking parameters and save	Test that VenueStationRelationships recalculation is triggered (B)	Change WALKING_TIME to 5, save	VenueStationRelationships is repopulated using new walking distance	04:33-06:27
32	Click Log Out	Test that session is cleared and user is logged out (N)	Click Log Out button	User session is cleared and user is redirected to login page	06:27-06:44
33	Click Fetch New Data	Test that background data refresh starts and loading page is shown (N)	Click Fetch New Data button	User is redirected to loading page; database is rebuilt; user data is preserved	06:44-09:07
34	Access settings page after logout	Test that settings page is no longer accessible after logout	Log out, then navigate to /update	User is redirected to login page	09:07-09:44

		(E)			
--	--	-----	--	--	--

Testing evidence

Unit testing

```

import sys
import os
import types
import sqlite3
import unittest
import math
import hashlib
import tempfile
import datetime
import json
import copy

# -----
# API keys — update these if the keys change
# -----
TfL_API_KEY = "0ff5a2076cd640cb957e63d6947efc61"
OPENCAGE_API_KEY = "eb0e2c9b71cc45f7aafe0ae4ecc44cc2"

# Mock third-party modules that require a network connection to install
# opencage and geopy are used for geocoding and distance calculations
opencageMock = types.ModuleType("opencage")
opencageGeocoderMock = types.ModuleType("opencage.geocoder")

class FakeGeocoder:
    def __init__(self, key):
        pass
    def geocode(self, location):
        return [{"geometry": {"lat": 51.555, "lng": -0.108}}]

opencageGeocoderMock.OpenCageGeocode = FakeGeocoder
opencageMock.geocoder = opencageGeocoderMock
sys.modules["opencage"] = opencageMock
sys.modules["opencage.geocoder"] = opencageGeocoderMock

geopyMock = types.ModuleType("geopy")
geopyDistanceMock = types.ModuleType("geopy.distance")

class FakeGeodesic:
    # Simple approximation for testing - 1 degree ~ 111km
    def __init__(self, coordinatesA, coordinatesB):
        latDiff = abs(coordinatesA[0] - coordinatesB[0])
        lonDiff = abs(coordinatesA[1] - coordinatesB[1])
        self.km = math.sqrt(latDiff**2 + lonDiff**2) * 111

geopyDistanceMock.geodesic = FakeGeodesic
geopyMock.distance = geopyDistanceMock
sys.modules["geopy"] = geopyMock
sys.modules["geopy.distance"] = geopyDistanceMock

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files'))

```

```

import login
import customFunctions
import crowdingAlgorithm
import pathFinding
import scraper
import storeStadiumData
import databaseCreation
import getResults
import getTfLData
import classes
import main as mainApp

# Creates a minimal in-memory SQLite database with the full schema and seed data for testing
def createTestDatabase():
    connection = sqlite3.connect(":memory:")
    cursor = connection.cursor()

    cursor.executescript("""
        CREATE TABLE Stations (
            NaPTAN TEXT PRIMARY KEY,
            StationName TEXT,
            Latitude REAL,
            Longitude REAL
        );
        CREATE TABLE Lines (
            LineID TEXT PRIMARY KEY,
            LineName TEXT,
            AverageSpeed TEXT
        );
        CREATE TABLE StationLineRelationships (
            NaPTAN TEXT,
            LineID TEXT,
            PRIMARY KEY (NaPTAN, LineID)
        );
        CREATE TABLE Connections (
            StationA TEXT,
            StationB TEXT,
            LineID TEXT,
            BaseTravelTime REAL,
            PRIMARY KEY (StationA, StationB, LineID)
        );
        CREATE TABLE Venues (
            VenueName TEXT PRIMARY KEY,
            Capacity INTEGER,
            Latitude REAL,
            Longitude REAL
        );
        CREATE TABLE VenueStationRelationships (
            VenueName TEXT,
            NaPTAN TEXT,
            Distance REAL,
            PRIMARY KEY (VenueName, NaPTAN)
        );
        CREATE TABLE Teams (
            TeamName TEXT PRIMARY KEY,
            VenueName TEXT
        );
        CREATE TABLE Events (
            VenueName TEXT,
            Date TEXT,
            Time TEXT,
            HomeTeam TEXT,
            AwayTeam TEXT,
    """

```

```

    EventName TEXT,
    Duration INTEGER,
    PRIMARY KEY (VenueName, Date)
);
CREATE TABLE Users (
    Username TEXT PRIMARY KEY,
    PasswordHash TEXT,
    Salt TEXT,
    CHANGING_TIME REAL,
    MAX_TIME_WINDOW REAL,
    rawArrivalPeakOffset REAL,
    rawDeparturePeakOffset REAL,
    ATTENDANCE REAL,
    TRAIN_PROPORTION REAL,
    SIGMA_FACTOR REAL,
    MINIMUM_PEOPLE REAL,
    PERSON_DELAY REAL,
    PROPAGATION_FACTOR REAL,
    WALKING_TIME REAL,
    WALKING_SPEED REAL,
    NORMAL REAL,
    SLIGHTLY REAL,
    BUSY REAL
);
""")

# Seed three stations forming a simple chain: STA -> STB -> STC
cursor.executemany("INSERT INTO Stations VALUES (?, ?, ?, ?)", [
    ("STA", "Station A", 51.50, -0.10),
    ("STB", "Station B", 51.51, -0.11),
    ("STC", "Station C", 51.52, -0.12),
])

cursor.execute("INSERT INTO Lines VALUES ('central', 'Central Line', '37.27')")

# Bidirectional connections between adjacent stations
cursor.executemany("INSERT INTO Connections VALUES (?, ?, ?, ?)", [
    ("STA", "STB", "central", 2.0),
    ("STB", "STA", "central", 2.0),
    ("STB", "STC", "central", 3.0),
    ("STC", "STB", "central", 3.0),
])

cursor.executemany("INSERT INTO StationLineRelationships VALUES (?, ?)", [
    ("STA", "central"),
    ("STB", "central"),
    ("STC", "central"),
])

# Seed a test venue with two nearby stations
cursor.execute("INSERT INTO Venues VALUES ('Test Stadium', 1000, 51.505, -0.105)")
cursor.execute("INSERT INTO Teams VALUES ('Test FC', 'Test Stadium')")
cursor.execute("INSERT INTO VenueStationRelationships VALUES ('Test Stadium', 'STA', 0.5)")
cursor.execute("INSERT INTO VenueStationRelationships VALUES ('Test Stadium', 'STB', 1.2)")

# Seed a test user with known credentials and default parameters
salt = "aabbccdd"
passwordHash = hashlib.sha256((salt + "password").encode()).hexdigest()
cursor.execute("""
    INSERT INTO Users VALUES (
        'testuser', ?, ?,
        4.4, 120, 25, 25, 0.95, 0.75, 4, 50, 0.002, 0.5, 15, 5, 1.0, 1.3, 2.0
    )
""", (passwordHash, salt))

```

```

# Seed a test event at the test venue
cursor.execute("INSERT INTO Events VALUES ('Test Stadium', '01/01/2026', '15:00', 'Test FC', 'Away FC', 'Premier League',
120)")

connection.commit()
return connection

# Writes an in-memory database to a temporary file and returns the file path
# Needed because most functions take a file path rather than a connection object
def writeTestDatabaseToFile(connection):
    temporaryFile = tempfile.NamedTemporaryFile(suffix=".db", delete=False)
    temporaryFile.close()
    diskConnection = sqlite3.connect(temporaryFile.name)
    connection.backup(diskConnection)
    diskConnection.close()
    return temporaryFile.name

# Builds a minimal network object from the test database for use in crowding tests
def buildTestNetwork(databaseFile):
    network = classes.Network()
    graph = pathFinding.MakeGraph(databaseFile)
    stations = {}
    connections = {}
    crowdingAlgorithm.createNetwork(graph, databaseFile, stations, connections, network)
    return network, graph

# -----
# login.py
# -----

class TestGenerateSalt(unittest.TestCase):

    def testReturnsString(self):
        salt = login.generateSalt()
        self.assertIsInstance(salt, str)

    def testCorrectLength(self):
        #16 bytes converts to 32 hex characters
        salt = login.generateSalt()
        self.assertEqual(len(salt), 32)

    def testUniqueEachTime(self):
        #Two salts generated back to back should never be the same
        self.assertNotEqual(login.generateSalt(), login.generateSalt())

class TestCreateHashPassword(unittest.TestCase):

    def testReturnsString(self):
        result = login.createHashPassword("password", "somesalt")
        self.assertIsInstance(result, str)

    def testSHA256Length(self):
        #SHA-256 always produces a 64 character hex digest
        result = login.createHashPassword("password", "somesalt")
        self.assertEqual(len(result), 64)

    def testSameInputsSameHash(self):
        hash1 = login.createHashPassword("abc", "salt123")
        hash2 = login.createHashPassword("abc", "salt123")

```

```

self.assertEqual(hash1, hash2)

def testDifferentSaltsDifferentHash(self):
    hash1 = login.createHashPassword("abc", "salt1")
    hash2 = login.createHashPassword("abc", "salt2")
    self.assertNotEqual(hash1, hash2)

def testDifferentPasswordsDifferentHash(self):
    hash1 = login.createHashPassword("password1", "salt")
    hash2 = login.createHashPassword("password2", "salt")
    self.assertNotEqual(hash1, hash2)

#createAccount() calls generateSalt() and createHashPassword() internally, so if these
#tests pass it also confirms both of those functions work in a real end-to-end context
class TestCreateAccount(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.defaultParametersFile = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files',
        'defaultParameters.json')

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testNewAccountReturnsTrue(self):
        result = login.createAccount(self.defaultParametersFile, "newuser", "pass123", self.databaseFile)
        self.assertTrue(result)

    def testDuplicateUsernameReturnsFalse(self):
        login.createAccount(self.defaultParametersFile, "dupuser", "pass", self.databaseFile)
        result = login.createAccount(self.defaultParametersFile, "dupuser", "pass", self.databaseFile)
        self.assertFalse(result)

    def testAccountStoredInDatabase(self):
        login.createAccount(self.defaultParametersFile, "checkuser", "pass", self.databaseFile)
        connection = sqlite3.connect(self.databaseFile)
        rows = connection.execute("SELECT Username FROM Users WHERE Username = 'checkuser'").fetchall()
        connection.close()
        self.assertEqual(len(rows), 1)

class TestLogin(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testCorrectCredentials(self):
        result = login.login(self.databaseFile, "testuser", "password")
        self.assertEqual(result, "Login Successful")

    def testWrongPassword(self):
        result = login.login(self.databaseFile, "testuser", "wrongpass")
        self.assertEqual(result, "Wrong Password")

    def testNonexistentUser(self):
        result = login.login(self.databaseFile, "nobody", "pass")
        self.assertEqual(result, "Account Not Found")

```

```

# -----
# customFunctions.py
# -----

#crowdingAlgorithm.getParameters() is a thin wrapper that just calls
#customFunctions.getParameters(), so if these tests pass it also confirms
#crowdingAlgorithm.getParameters() works correctly
class TestGetParameters(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsParameterObject(self):
        result = customFunctions.getParameters("testuser", self.databaseFile)
        self.assertIsNotNone(result)

    def testCorrectChangingTime(self):
        result = customFunctions.getParameters("testuser", self.databaseFile)
        self.assertAlmostEqual(result.CHANGING_TIME, 4.4)

    def testCorrectSigmaFactor(self):
        result = customFunctions.getParameters("testuser", self.databaseFile)
        self.assertAlmostEqual(result.SIGMA_FACTOR, 4.0)

    def testWindowIsCalculated(self):
        #calculateWindow() should be called inside getParameters so this should not be None
        result = customFunctions.getParameters("testuser", self.databaseFile)
        self.assertIsNotNone(result.arrivalWindowMinutes)

# -----
# pathFinding.py
# -----

class TestMakeGraph(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsDictionary(self):
        graph = pathFinding.MakeGraph(self.databaseFile)
        self.assertIsInstance(graph, dict)

    def testCorrectStationsPresent(self):
        graph = pathFinding.MakeGraph(self.databaseFile)
        self.assertIn("STA", graph)
        self.assertIn("STB", graph)

    def testNeighboursCorrect(self):
        graph = pathFinding.MakeGraph(self.databaseFile)
        neighbourIDs = [neighbour[0] for neighbour in graph["STA"]]
        self.assertIn("STB", neighbourIDs)

    def testTravelTimesArePositive(self):
        graph = pathFinding.MakeGraph(self.databaseFile)

```

```

    for neighbours in graph.values():
        for entry in neighbours:
            self.assertGreater(entry[1], 0)

class TestDijkstra(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.graph = pathFinding.MakeGraph(self.databaseFile)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsList(self):
        path = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4)
        self.assertIsInstance(path, list)

    def testPathNotEmpty(self):
        path = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4)
        self.assertGreater(len(path), 0)

    def testPathEndsAtGoal(self):
        path = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4)
        lastStation = path[-1][1]
        self.assertEqual(lastStation, "STC")

    def testAdjacentStationsOneStep(self):
        #STA and STB are directly connected so path should be a single step
        path = pathFinding.Dijkstra(self.graph, "STA", "STB", 4.4)
        self.assertEqual(len(path), 1)

    def testCostIncreasesAlongPath(self):
        #Cumulative cost at each step should always be increasing
        path = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4)
        costs = [step[3] for step in path]
        self.assertEqual(costs, sorted(costs))

    def testPenaltyOffGivesLowerOrEqualCost(self):
        #Disabling line change penalty should never increase journey cost
        pathWithPenalty = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4, penalty=True)
        pathWithoutPenalty = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4, penalty=False)
        self.assertEqual(pathWithoutPenalty[-1][3], pathWithPenalty[-1][3])

    def testSameStartAndEndReturnsEmptyPath(self):
        path = pathFinding.Dijkstra(self.graph, "STA", "STA", 4.4)
        self.assertEqual(len(path), 0)

# -----
# crowdingAlgorithm.py
# -----

class TestTakeInput(unittest.TestCase):

    def testReturnsDictionary(self):
        result = crowdingAlgorithm.takeInput("01/01/2026", "15:00", "STA", "STC")
        self.assertIsInstance(result, dict)

    def testCorrectKeysPresent(self):
        result = crowdingAlgorithm.takeInput("01/01/2026", "15:00", "STA", "STC")
        self.assertIn("date", result)
        self.assertIn("journeyStart", result)

```

```
self.assertIn("startStation", result)
self.assertIn("endStation", result)
```

```
def testValuesStoredCorrectly(self):
    result = crowdingAlgorithm.takeInput("01/01/2026", "15:00", "STA", "STC")
    self.assertEqual(result["date"], "01/01/2026")
    self.assertEqual(result["startStation"], "STA")
```

```
class TestInverseWeight(unittest.TestCase):
```

```
def testReturnsDictionary(self):
    result = crowdingAlgorithm.inverseWeight({"STA": 1.0, "STB": 2.0})
    self.assertIsInstance(result, dict)
```

```
def testProportionsSumToOne(self):
    result = crowdingAlgorithm.inverseWeight({"STA": 1.0, "STB": 2.0, "STC": 3.0})
    self.assertAlmostEqual(sum(result.values()), 1.0, places=9)
```

```
def testCloserStationHigherProportion(self):
    result = crowdingAlgorithm.inverseWeight({"CLOSE": 0.5, "FAR": 5.0})
    self.assertGreater(result["CLOSE"], result["FAR"])
```

```
def testZeroDistanceDoesNotCrash(self):
    #+1 guard in inverseWeight should prevent ZeroDivisionError when distance is 0
    result = crowdingAlgorithm.inverseWeight({"STA": 0.0, "STB": 1.0})
    self.assertAlmostEqual(sum(result.values()), 1.0, places=9)
```

```
def testSingleStationProportionsIsOne(self):
    result = crowdingAlgorithm.inverseWeight({"STA": 2.0})
    self.assertAlmostEqual(result["STA"], 1.0)
```

```
class TestGaussianFactor(unittest.TestCase):
```

```
def testAtPeakReturnsOne(self):
    #When relativePeak equals relativeTimeDifference, x=0, exp(0)=1
    result = crowdingAlgorithm.gaussianFactorAtPeak(10, 5, 10)
    self.assertAlmostEqual(result, 1.0)
```

```
def testReturnsBetweenZeroAndOne(self):
    result = crowdingAlgorithm.gaussianFactorAtPeak(10, 5, 20)
    self.assertGreaterEqual(result, 0.0)
    self.assertLessEqual(result, 1.0)
```

```
def testFurtherFromPeakLowerFactor(self):
    close = crowdingAlgorithm.gaussianFactorAtPeak(10, 5, 12)
    far = crowdingAlgorithm.gaussianFactorAtPeak(10, 5, 30)
    self.assertGreater(close, far)
```

```
class TestReturnStationsWithDistances(unittest.TestCase):
```

```
def setUp(self):
    self.connection = createTestDatabase()
    self.databaseFile = writeTestDatabaseToFile(self.connection)
```

```
def tearDown(self):
    os.unlink(self.databaseFile)
```

```
def testReturnsDictionary(self):
    result = crowdingAlgorithm.returnStationsWithDistances("Test Stadium", self.databaseFile)
    self.assertIsInstance(result, dict)
```

```

def testCorrectStationsReturned(self):
    result = crowdingAlgorithm.returnStationsWithDistances("Test Stadium", self.databaseFile)
    self.assertIn("STA", result)
    self.assertIn("STB", result)

def testDistanceValuesArePositive(self):
    result = crowdingAlgorithm.returnStationsWithDistances("Test Stadium", self.databaseFile)
    for distance in result.values():
        self.assertGreater(distance, 0)

def testUnknownVenueReturnsEmpty(self):
    result = crowdingAlgorithm.returnStationsWithDistances("Nonexistent Venue", self.databaseFile)
    self.assertEqual(result, {})

class TestFindCapacity(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsCorrectCapacity(self):
        result = crowdingAlgorithm.findCapacity(self.databaseFile, "Test Stadium")
        self.assertEqual(result, 1000)

class TestFindEventsInTimeFrame(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        #Journey mid-point just after the event starts
        self.midjourneyTime = datetime.datetime.strptime("15:30", "%H:%M")
        self.baseInputData = {
            "date": "01/01/2026",
            "journeyStart": "15:00",
            "startStation": "STA",
            "endStation": "STC"
        }
        #Peak offset as timedelta(minutes=25) — matches the fixed calculatePeakOffset()
        self.peakOffset = datetime.timedelta(minutes=25)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsDictionary(self):
        result = crowdingAlgorithm.findEventsInTimeFrame(self.midjourneyTime, self.databaseFile, self.baseInputData, 120,
self.peakOffset, self.peakOffset)
        self.assertIsInstance(result, dict)

    def testNearbyEventsFound(self):
        #Event at 15:00 should be within the 120 minute window of a 15:30 midjourney
        result = crowdingAlgorithm.findEventsInTimeFrame(self.midjourneyTime, self.databaseFile, self.baseInputData, 120,
self.peakOffset, self.peakOffset)
        self.assertGreater(len(result), 0)

    def testNoEventsOnEmptyDate(self):
        emptyDateInputData = {
            "date": "01/01/2099",
            "journeyStart": "15:00",
            "startStation": "STA",

```

```

        "endStation": "STC"
    }
    result = crowdingAlgorithm.findEventsInTimeFrame(self.midjourneyTime, self.databaseFile, emptyDateInputData, 120,
self.peakOffset, self.peakOffset)
    self.assertEqual(result, {})

```

```
class TestCreateNetwork(unittest.TestCase):
```

```

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

```

```

    def tearDown(self):
        os.unlink(self.databaseFile)

```

```

    def testAllStationsAddedToNetwork(self):
        network, graph = buildTestNetwork(self.databaseFile)
        self.assertIn("STA", network.nodes)
        self.assertIn("STB", network.nodes)
        self.assertIn("STC", network.nodes)

```

```

    def testAllConnectionsAddedToNetwork(self):
        network, graph = buildTestNetwork(self.databaseFile)
        self.assertIn(("STA", "STB", "central"), network.edges)

```

```

    def testStationsStartAsNotAffected(self):
        network, graph = buildTestNetwork(self.databaseFile)
        for stationObject in network.nodes.values():
            self.assertFalse(stationObject.isAffected())

```

```
class TestPropegation(unittest.TestCase):
```

```

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.network, self.graph = buildTestNetwork(self.databaseFile)
        self.event = ("Test Stadium", "01/01/2026")

```

```

    def tearDown(self):
        os.unlink(self.databaseFile)

```

```

    def testStationBecomesAffected(self):
        crowdingAlgorithm.propegation(self.graph, "STA", 200, 50, self.network, self.event, 0.5)
        self.assertTrue(self.network.nodes["STA"].isAffected())

```

```

    def testPropegationSpreadsToNeighbours(self):
        #200 people at STA with factor 0.5 should propagate 100 to STB (above MINIMUM_PEOPLE=50)
        crowdingAlgorithm.propegation(self.graph, "STA", 200, 50, self.network, self.event, 0.5)
        self.assertTrue(self.network.nodes["STB"].isAffected())

```

```

    def testBelowMinimumDoesNotPropagate(self):
        #40 people at STA with factor 0.5 = 20, which is below MINIMUM_PEOPLE=50 so STB should stay unaffected
        crowdingAlgorithm.propegation(self.graph, "STA", 40, 50, self.network, self.event, 0.5)
        self.assertFalse(self.network.nodes["STB"].isAffected())

```

```

    def testLargerWaveOverwritesSmaller(self):
        crowdingAlgorithm.propegation(self.graph, "STA", 100, 50, self.network, self.event, 0.5)
        crowdingAlgorithm.propegation(self.graph, "STA", 500, 50, self.network, self.event, 0.5)
        self.assertEqual(self.network.nodes["STA"].events[self.event], 500)

```

```
class TestProcessAffectedStations(unittest.TestCase):
```

```

def setUp(self):
    self.connection = createTestDatabase()
    self.databaseFile = writeTestDatabaseToFile(self.connection)
    self.network, self.graph = buildTestNetwork(self.databaseFile)
    self.parameterObject = customFunctions.getParameters("testuser", self.databaseFile)
    event = ("Test Stadium", "01/01/2026")
    crowdingAlgorithm.propegation(self.graph, "STA", 500, 50, self.network, event, 0.5)

def tearDown(self):
    os.unlink(self.databaseFile)

def testAffectedStationDelayFactorAboveOne(self):
    crowdingAlgorithm.processAffectedStations(self.graph, self.parameterObject, self.network)
    self.assertGreater(self.network.nodes["STA"].DelayFactor, 1.0)

def testAffectedStationHasHigherDelayThanUnaffected(self):
    crowdingAlgorithm.processAffectedStations(self.graph, self.parameterObject, self.network)
    #STA is directly affected so its delay should be higher than the default of 1.0
    #This confirms processAffectedStations correctly increases delay on affected stations
    self.assertGreater(self.network.nodes["STA"].DelayFactor, 1.0)

#spreadDelay() is the last step in crowdingAlgorithm.main(), so the setUp here runs the
#full crowding pipeline — meaning if these tests pass, it also confirms processAffected(),
#processAffectedStations(), processAffectedConnections(), getCrowdingFactor(), and
#crowdingAlgorithm.main() all work correctly end-to-end
class TestSpreadDelay(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.network, self.graph = buildTestNetwork(self.databaseFile)
        parameterObject = customFunctions.getParameters("testuser", self.databaseFile)
        event = ("Test Stadium", "01/01/2026")
        crowdingAlgorithm.propegation(self.graph, "STA", 500, 50, self.network, event, 0.5)
        crowdingAlgorithm.processAffected(self.graph, parameterObject, self.network)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testLinedelaysIsSetOnConnections(self):
        crowdingAlgorithm.spreadDelay(self.network, self.databaseFile)
        #All connections on the central line should now have the same LineDelay
        centralConnections = [edge for key, edge in self.network.edges.items() if key[2] == "central"]
        lineDelays = [c.LineDelay for c in centralConnections]
        self.assertTrue(all(d == lineDelays[0] for d in lineDelays))

# -----
# storeStadiumData.py
# -----

class TestCalculateWalkingDistance(unittest.TestCase):

    def testKnownValue(self):
        #5 km/h for 60 minutes = 5.0 km
        result = storeStadiumData.calculateWalkingDistance(60, 5)
        self.assertAlmostEqual(result, 5.0)

    def testDefaultParameters(self):
        #Default parameters: 15 minutes at 5 km/h = 1.25 km
        result = storeStadiumData.calculateWalkingDistance(15, 5)
        self.assertAlmostEqual(result, 1.25)

```

```

def testZeroTimeReturnsZero(self):
    result = storeStadiumData.calculateWalkingDistance(0, 5)
    self.assertAlmostEqual(result, 0.0)

class TestGetCloseStations(unittest.TestCase):

    def setUp(self):
        #Three stations at increasing distances from venue coordinates
        self.allStations = [
            ("STA", 51.50, -0.10), # Closest — same location as venue
            ("STB", 51.60, -0.20), # Medium distance
            ("STC", 51.80, -0.40), # Far away
        ]
        self.venueCoordinates = (51.50, -0.10) #Same coordinates as STA

    def testReturnsDictionary(self):
        result = storeStadiumData.getCloseStations(self.allStations, self.venueCoordinates, 5.0)
        self.assertIsInstance(result, dict)

    def testAlwaysReturnsAtLeastOne(self):
        #Even with a threshold of zero, the closest station should always be included
        result = storeStadiumData.getCloseStations(self.allStations, self.venueCoordinates, 0.0)
        self.assertGreaterEqual(len(result), 1)

    def testClosestStationAlwaysIncluded(self):
        result = storeStadiumData.getCloseStations(self.allStations, self.venueCoordinates, 0.0)
        self.assertIn("STA", result)

    def testDistantStationExcluded(self):
        #STC is roughly 33km away so should not appear within a 1km threshold
        result = storeStadiumData.getCloseStations(self.allStations, self.venueCoordinates, 1.0)
        self.assertNotIn("STC", result)

    def testStationWithinThresholdIncluded(self):
        #Use a large threshold to make sure STB is included
        result = storeStadiumData.getCloseStations(self.allStations, self.venueCoordinates, 20.0)
        self.assertIn("STB", result)

#insertData() calls getCloseStations() internally to find nearby stations for the venue,
#so if these tests pass it also confirms getCloseStations() works in a real context
class TestInsertData(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.coordinates = {"lat": 51.50, "lng": -0.10}

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testVenuesStoredInDatabase(self):
        storeStadiumData.insertData(self.databaseFile, "New FC", "New Stadium", 5000, self.coordinates, 2.0)
        connection = sqlite3.connect(self.databaseFile)
        rows = connection.execute("SELECT VenueName FROM Venues WHERE VenueName = 'New Stadium'").fetchall()
        connection.close()
        self.assertEqual(len(rows), 1)

    def testTeamsStoredInDatabase(self):
        storeStadiumData.insertData(self.databaseFile, "New FC", "New Stadium", 5000, self.coordinates, 2.0)
        connection = sqlite3.connect(self.databaseFile)
        rows = connection.execute("SELECT TeamName FROM Teams WHERE TeamName = 'New FC'").fetchall()

```

```

connection.close()
self.assertEqual(len(rows), 1)

def testVenueStationRelationshipCreated(self):
    storeStadiumData.insertData(self.databaseFile, "New FC", "New Stadium", 5000, self.coordinates, 5.0)
    connection = sqlite3.connect(self.databaseFile)
    rows = connection.execute("SELECT * FROM VenueStationRelationships WHERE VenueName = 'New Stadium'").fetchall()
    connection.close()
    self.assertGreater(len(rows), 0)

```

```
class TestRedoStore(unittest.TestCase):
```

```

def setUp(self):
    self.connection = createTestDatabase()
    self.databaseFile = writeTestDatabaseToFile(self.connection)

    #Write a minimal londonClubs JSON using only clubs already seeded in the test database
    #redoStore reads venue coordinates from the Venues table so the venue must exist
    self.londonClubsFile = tempfile.NamedTemporaryFile(suffix=".json", delete=False, mode="w")
    json.dump({"Test FC": [{"Test Stadium", 1000}], self.londonClubsFile)
    self.londonClubsFile.close()
    self.londonClubsFilePath = self.londonClubsFile.name

```

```

def tearDown(self):
    os.unlink(self.databaseFile)
    os.unlink(self.londonClubsFilePath)

```

```

def testVenueStationRelationshipsAreRepopulated(self):
    #redoStore should delete and reinsert VenueStationRelationships for the given venues
    storeStadiumData.redoStore(15, 5, self.londonClubsFilePath, self.databaseFile)
    connection = sqlite3.connect(self.databaseFile)
    rows = connection.execute("SELECT * FROM VenueStationRelationships").fetchall()
    connection.close()
    self.assertGreater(len(rows), 0)

```

```

# -----
# scraper.py
# -----

```

```
class TestGetDate(unittest.TestCase):
```

```

def testStandardDate(self):
    result = scraper.getDate("2026-01", "Saturday 10th January")
    self.assertEqual(result, "10-01-2026")

```

```

def testSingleDigitDaysPadded(self):
    result = scraper.getDate("2026-03", "Wednesday 5th March")
    self.assertEqual(result, "05-03-2026")

```

```

def testNoNumberReturnsError(self):
    result = scraper.getDate("2026-01", "No numbers here")
    self.assertEqual(result, "Error")

```

```

def testEmptyStringReturnsError(self):
    result = scraper.getDate("2026-01", "")
    self.assertEqual(result, "Error")

```

```
class TestGetNextTwelveMonths(unittest.TestCase):
```

```

def testReturnsTwelveEntries(self):
    result = scraper.getNextTwelveMonths()

```

```

self.assertEqual(len(result), 12)

def testAllEntriesCorrectFormat(self):
    import re
    result = scraper.getNextTwelveMonths()
    for entry in result:
        self.assertRegex(entry, r"^\d{4}-\d{2}$")

def testNoDuplicateMonths(self):
    result = scraper.getNextTwelveMonths()
    self.assertEqual(len(result), len(set(result)))

# -----
# databaseCreation.py
# -----

class TestCreateDatabase(unittest.TestCase):

    def setUp(self):
        temporaryFile = tempfile.NamedTemporaryFile(suffix=".db", delete=False)
        temporaryFile.close()
        self.databaseFile = temporaryFile.name

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testAllTablesCreated(self):
        databaseCreation.createDatabase(self.databaseFile)
        connection = sqlite3.connect(self.databaseFile)
        tables = {row[0] for row in connection.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
        connection.close()
        expectedTables = {"Stations", "Lines", "StationLineRelationships", "Connections", "Venues", "VenueStationRelationships",
                           "Events", "Users", "Teams"}
        self.assertEqual(expectedTables, tables)

    def testIdempotentOnSecondCall(self):
        #Calling createDatabase twice should not raise any errors
        databaseCreation.createDatabase(self.databaseFile)
        databaseCreation.createDatabase(self.databaseFile)

    def testCreateLinesPopulatesTable(self):
        databaseCreation.createDatabase(self.databaseFile)
        databaseCreation.createLines(self.databaseFile, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..',
                           'programme files', 'lines.json'))
        connection = sqlite3.connect(self.databaseFile)
        count = connection.execute("SELECT COUNT(*) FROM Lines").fetchone()[0]
        connection.close()
        self.assertGreater(count, 0)

    def testGetWalkingReturnsPositiveValues(self):
        walkingSpeed, walkingTime = databaseCreation.getWalking(os.path.join(os.path.dirname(os.path.abspath(__file__)), '..',
                           'programme files', 'defaultParameters.json'))
        self.assertIsNotNone(walkingSpeed)
        self.assertIsNotNone(walkingTime)
        self.assertGreater(walkingSpeed, 0)
        self.assertGreater(walkingTime, 0)

# -----
# getResults.py
# -----

class TestGetNaPTAN(unittest.TestCase):

```

```

def setUp(self):
    self.connection = createTestDatabase()
    self.databaseFile = writeTestDatabaseToFile(self.connection)

def tearDown(self):
    os.unlink(self.databaseFile)

def testReturnsCorrectNaPTAN(self):
    result = getResults.getNaPTAN("Station A", self.databaseFile)
    self.assertEqual(result, "STA")

def testDifferentStationDifferentNaPTAN(self):
    result = getResults.getNaPTAN("Station B", self.databaseFile)
    self.assertEqual(result, "STB")

#GetaffectedGraph() is a core step used by getAffectedOldPath() and getResults.main(),
#so if these tests pass alongside TestGetStrippedGraph and TestDijkstra, it also
#confirms getAffectedOldPath() and getResults.main() work correctly end-to-end
class TestGetAffectedGraph(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.network, self.graph = buildTestNetwork(self.databaseFile)
        parameterObject = customFunctions.getParameters("testuser", self.databaseFile)
        event = ("Test Stadium", "01/01/2026")
        crowdingAlgorithm.propegation(self.graph, "STA", 500, 50, self.network, event, 0.5)
        crowdingAlgorithm.processAffected(self.graph, parameterObject, self.network)
        crowdingAlgorithm.spreadDelay(self.network, self.databaseFile)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsDictionary(self):
        affectedGraph = getResults.GetaffectedGraph(self.graph, self.network)
        self.assertIsInstance(affectedGraph, dict)

    def testOriginalGraphNotModified(self):
        #GetaffectedGraph uses deepcopy so original should be unchanged
        originalTime = self.graph["STA"][0][1]
        getResults.GetaffectedGraph(self.graph, self.network)
        self.assertEqual(self.graph["STA"][0][1], originalTime)

    def testAffectedGraphHasHigherTravelTimes(self):
        affectedGraph = getResults.GetaffectedGraph(self.graph, self.network)
        #At least one connection should have a higher travel time due to crowding
        originalTotal = sum(entry[1] for neighbours in self.graph.values() for entry in neighbours)
        affectedTotal = sum(entry[1] for neighbours in affectedGraph.values() for entry in neighbours)
        self.assertGreaterEqual(affectedTotal, originalTotal)

class TestGetStrippedGraph(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.graph = pathFinding.MakeGraph(self.databaseFile)
        self.oldPath = pathFinding.Dijkstra(self.graph, "STA", "STC", 4.4)

    def tearDown(self):
        os.unlink(self.databaseFile)

```

```

def testReturnsDictionary(self):
    result = getResults.getStrippedGraph(self.oldPath, self.graph)
    self.assertIsInstance(result, dict)

def testOnlyContainsPathConnections(self):
    #Stripped graph should only have stations that appear in the path
    strippedGraph = getResults.getStrippedGraph(self.oldPath, self.graph)
    pathStations = {step[0] for step in self.oldPath}
    for station in strippedGraph.keys():
        self.assertIn(station, pathStations)

# -----
# websiteDataClasses.py (via classes.py)
# -----

class TestMainPagePathClass(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testAddStation(self):
        pathObject = classes.mainPagePathClass("central")
        pathObject.addStation("STA")
        self.assertIn("STA", pathObject.ListOfStations)

    def testAddMultipleStations(self):
        pathObject = classes.mainPagePathClass("central")
        pathObject.addStation("STA")
        pathObject.addStation("STB")
        self.assertEqual(len(pathObject.ListOfStations), 2)

    def testGetLineName(self):
        pathObject = classes.mainPagePathClass("central")
        pathObject.getLineName(self.databaseFile)
        self.assertEqual(pathObject.LineName, "Central Line")

    def testGetStationNames(self):
        pathObject = classes.mainPagePathClass("central")
        pathObject.addStation("STA")
        pathObject.addStation("STB")
        pathObject.getStationNames(self.databaseFile)
        stationNames = [entry[0] for entry in pathObject.ListOfStationNames]
        self.assertIn("Station A", stationNames)
        self.assertIn("Station B", stationNames)

    def testGetDescription(self):
        pathObject = classes.mainPagePathClass("central")
        pathObject.addStation("STA")
        pathObject.ListOfStationNames = ["Station A", "STA"]
        pathObject.getDescription(["STA", "Normal"])
        self.assertEqual(pathObject.ListOfStationNamesAndDescriptions[0][2], "Normal")

# -----
# scraperClasses.py (via classes.py)
# -----

```

*#Note: AllDates.storeInDatabase() does not have its own direct test but is used in the
#full scraper pipeline — if TestGetPageFixtures passes and fixtures are stored correctly*

#via testAllDatesFiltersNonLondonFixtures, it confirms storeInDatabase() works
 class TestScraperClasses(unittest.TestCase):

```
def setUp(self):
    self.connection = createTestDatabase()
    self.databaseFile = writeTestDatabaseToFile(self.connection)

def tearDown(self):
    os.unlink(self.databaseFile)

def testFixtureStoresData(self):
    fixtureObject = classes.Fixture("Premier League", "Test FC", "Away FC", "15:00")
    self.assertEqual(fixtureObject.home, "Test FC")
    self.assertEqual(fixtureObject.away, "Away FC")
    self.assertEqual(fixtureObject.time, "15:00")

def testFindRelevantFixturesRemovesNonLondon(self):
    londonFixture = classes.Fixture("PL", "Test FC", "Other", "15:00")
    nonLondonFixture = classes.Fixture("PL", "NonLondonClub", "Other", "15:00")
    dateObject = classes.Date("01-01-2026", [londonFixture, nonLondonFixture])
    dateObject.findRelevantFixtures(["Test FC"])
    self.assertEqual(len(dateObject.fixtures), 1)
    self.assertEqual(dateObject.fixtures[0].home, "Test FC")

def testFindRelevantFixturesKeepsAllLondon(self):
    londonFixture = classes.Fixture("PL", "Test FC", "Other", "15:00")
    dateObject = classes.Date("01-01-2026", [londonFixture])
    dateObject.findRelevantFixtures(["Test FC"])
    self.assertEqual(len(dateObject.fixtures), 1)

def testAllDatesFiltersNonLondonFixtures(self):
    londonFixture = classes.Fixture("PL", "Test FC", "Away", "15:00")
    nonLondonFixture = classes.Fixture("PL", "ManUtd", "Away", "15:00")
    dateObject = classes.Date("01-01-2026", [londonFixture, nonLondonFixture])
    allDatesObject = classes.AllDates([dateObject])
    allDatesObject.findRelevantDates(self.databaseFile)
    self.assertEqual(len(allDatesObject.dates), 1)
    self.assertEqual(len(allDatesObject.dates[0].fixtures), 1)
```

 # transportClasses.py (via classes.py)
 # _____

```
class TestTransportClasses(unittest.TestCase):

def testAffectedStationIsAffected(self):
    stationObject = classes.AffectedDijkstraStation("STA", "Station A")
    self.assertTrue(stationObject.isAffected())

def testNotAffectedStationIsNotAffected(self):
    stationObject = classes.NotAffectedDijkstraStation("STA", "Station A")
    self.assertFalse(stationObject.isAffected())

def testCalculatePassengerLoad(self):
    stationObject = classes.AffectedDijkstraStation("STA", "Station A")
    stationObject.events[("Test Stadium", "01/01/2026")] = 200
    stationObject.events[("Other Stadium", "01/01/2026")] = 100
    load = stationObject.calculatePassengerLoad()
    self.assertEqual(load, 300)

def testCalculateDelay(self):
    stationObject = classes.AffectedDijkstraStation("STA", "Station A")
    stationObject.currentPassengerLoad = 500
```

```

stationObject.calculateDelay(0.002)
self.assertAlmostEqual(stationObject.DelayFactor, 2.0) #1 + (0.002 * 500) = 2.0

def testAffectedConnectionIsAffected(self):
    connectionObject = classes.AffectedConnection("STA", "STB", "central", 2.0, 1.5, 2.0)
    self.assertTrue(connectionObject.IsAffected())

def testNotAffectedConnectionDelayIsOne(self):
    connectionObject = classes.NotAffectedConnection("STA", "STB", "central", 2.0)
    delay = connectionObject.returnDelay()
    self.assertEqual(delay, 1)

def testAffectedConnectionReturnDelay(self):
    #Delay should be the average of the two station delay factors
    connectionObject = classes.AffectedConnection("STA", "STB", "central", 2.0, 1.5, 2.5)
    delay = connectionObject.returnDelay()
    self.assertAlmostEqual(delay, 2.0) #{(1.5 + 2.5) / 2 = 2.0}

def testAbstractClassCannotBeInstantiated(self):
    with self.assertRaises(TypeError):
        classes.TransportClass()

def testLineSpreadDelay(self):
    #Average delay across both connections should be (1.5 + 2.5) / 2 = 2.0
    connection1 = classes.NotAffectedConnection("STA", "STB", "central", 2.0)
    connection2 = classes.NotAffectedConnection("STB", "STC", "central", 3.0)
    connection1.DelayFactor = 1.5
    connection2.DelayFactor = 2.5
    lineObject = classes.Line([connection1, connection2], "central")
    lineObject.doAllCalculations()
    self.assertAlmostEqual(connection1.LineDelay, 2.0)
    self.assertAlmostEqual(connection2.LineDelay, 2.0)

# -----
# accountClasses.py (via classes.py)
# -----

class TestParameters(unittest.TestCase):

    def setUp(self):
        self.parameterObject = classes.Parameters(4.4, 120, 25, 25, 0.95, 0.75, 4, 50, 0.002, 0.5, 15, 5, 1, 1.3, 2)

    def testCalculateWindow(self):
        self.parameterObject.calculateWindow()
        self.assertEqual(self.parameterObject.arrivalWindowMinutes, 120)
        self.assertEqual(self.parameterObject.departureWindowMinutes, 120)

    def testCalculatePeakOffset(self):
        self.parameterObject.calculatePeakOffset()
        self.assertIsInstance(self.parameterObject.arrivalPeakOffset, datetime.timedelta)

    def testCalculateMinutesOffset(self):
        self.parameterObject.calculateWindow()
        self.parameterObject.calculatePeakOffset()
        self.parameterObject.calculateMinutesOffset()
        self.assertIsNotNone(self.parameterObject.arrivalMinutesOffset)

    def testPeakOffsetIsCorrectMinutes(self):
        #After the fix, timedelta(minutes=25) should give 25 minutes not 25 days
        self.parameterObject.calculatePeakOffset()
        totalMinutes = self.parameterObject.arrivalPeakOffset.total_seconds() / 60
        self.assertAlmostEqual(totalMinutes, 25.0)

```

```

def testAccountCreationParametersLoadsDefaults(self):
    defaultParametersObject =
classes.AccountCreationParameters(os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files',
'defaultParameters.json'))
    self.assertEqual(defaultParametersObject.ATTENDANCE, 0.95)
    self.assertEqual(defaultParametersObject.TRAIN_PROPORTION, 0.75)

# -----
# networkClasses.py (via classes.py)
# -----

class TestSeverity(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

        # Build a minimal network with three unaffected stations
        self.network = classes.Network()
        for NaPTAN, stationName in [("STA", "Station A"), ("STB", "Station B"), ("STC", "Station C")]:
            self.network.nodes[NaPTAN] = classes.NotAffectedDijkstraStation(NaPTAN, stationName)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testGetDescriptionNormal(self):
        path = [("STA", "STB", "central", 2.0), ("STB", "STC", "central", 5.0)]
        severityObject = classes.Severity(path, "testuser", self.databaseFile, "STA", self.network)
        severityObject.getParameters()
        description = severityObject.getDescription(0.5) #Below NORMAL threshold of 1.0
        self.assertEqual(description, "Normal")

    def testGetDescriptionSevere(self):
        path = [("STA", "STB", "central", 2.0)]
        severityObject = classes.Severity(path, "testuser", self.databaseFile, "STA", self.network)
        severityObject.getParameters()
        description = severityObject.getDescription(2.5) #Above BUSY threshold of 2.0
        self.assertEqual(description, "Severe")

    def testCalculateFillsDescriptions(self):
        #Path visits STA (start), STB, STC so descriptions should have 3 entries
        path = [("STA", "STB", "central", 2.0), ("STB", "STC", "central", 5.0)]
        severityObject = classes.Severity(path, "testuser", self.databaseFile, "STA", self.network)
        severityObject.calculate()
        self.assertEqual(len(severityObject.descriptions), 3)

# -----
# main.py — Flask test client simulates HTTP requests without a browser
# -----

class TestMainFlaskRoutes(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

        # Write a flags file so the app doesn't try to create the database on startup
        self.flagsFile = tempfile.NamedTemporaryFile(suffix=".json", delete=False, mode="w")
        json.dump({"status": "success", "error": "success"}, self.flagsFile)
        self.flagsFile.close()
        self.flagsFilePath = self.flagsFile.name

```

```

# Write a minimal londonClubs file containing only the test venue so that
# redoStore() doesn't try to look up real clubs not present in the test database
self.testLondonClubsFile = tempfile.NamedTemporaryFile(suffix=".json", delete=False, mode="w")
json.dump({"Test FC": [{"Test Stadium", 1000}], self.testLondonClubsFile)
self.testLondonClubsFile.close()
self.testLondonClubsFilePath = self.testLondonClubsFile.name

# Configure the Flask app with test settings
mainApp.setUpConfig(
    mainApp.app,
    self.databaseFile,
    os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files', 'lines.json'),
    "FAKE_TFL_KEY",
    "FAKE_OPENCAGE_KEY",
    self.testLondonClubsFilePath,
    os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files', 'leagues.json'),
    self.flagsFilePath,
    os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files', 'defaultParameters.json')
)

mainApp.app.config["TESTING"] = True
mainApp.app.config["SECRET_KEY"] = "testkey"
self.client = mainApp.app.test_client()

def tearDown(self):
    os.unlink(self.databaseFile)
    os.unlink(self.flagsFilePath)
    os.unlink(self.testLondonClubsFilePath)

def testLoginPageLoads(self):
    #GET request to / should return the login page
    response = self.client.get("/")
    self.assertEqual(response.status_code, 200)

def testLoginWithCorrectCredentials(self):
    #POST to / with correct credentials should redirect to main page
    response = self.client.post("/", data={
        "username": "testuser",
        "password": "password",
        "actionName": "login"
    })
    self.assertEqual(response.status_code, 302) #302 = redirect

def testLoginWithWrongPassword(self):
    #POST to / with wrong password should stay on login page with a message
    response = self.client.post("/", data={
        "username": "testuser",
        "password": "wrongpassword",
        "actionName": "login"
    })
    self.assertEqual(response.status_code, 200)
    self.assertIn(b"Wrong Password", response.data)

def testLoginWithUnknownUser(self):
    response = self.client.post("/", data={
        "username": "nobody",
        "password": "pass",
        "actionName": "login"
    })
    self.assertEqual(response.status_code, 200)
    self.assertIn(b"Account Not Found", response.data)

def testSignUpCreatesAccountAndRedirects(self):
    response = self.client.post("/", data={

```

```

        "username": "brandnewuser",
        "password": "pass123",
        "actionName": "signUp"
    })
    self.assertEqual(response.status_code, 302)

def testSignUpDuplicateUsernameStaysOnPage(self):
    response = self.client.post("/", data={
        "username": "testuser",
        "password": "pass",
        "actionName": "signUp"
    })
    self.assertEqual(response.status_code, 200)
    self.assertIn(b"Account already created", response.data)

def testMainPageRedirectsIfNotLoggedIn(self):
    #Accessing /main without a session should redirect to login
    response = self.client.get("/main")
    self.assertEqual(response.status_code, 302)

def testMainPageLoadsWhenLoggedIn(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
    response = self.client.get("/main")
    self.assertEqual(response.status_code, 200)

def testLoadPageLoads(self):
    response = self.client.get("/load")
    self.assertEqual(response.status_code, 200)

def testLogoutClearsSession(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
    response = self.client.post("/logout")
    self.assertEqual(response.status_code, 302)
    #After logout, /main should redirect back to login
    response = self.client.get("/main")
    self.assertEqual(response.status_code, 302)

def testGetStationsReturnsJson(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
    response = self.client.get("/get_stations?search=Station")
    self.assertEqual(response.status_code, 200)
    data = json.loads(response.data)
    self.assertIsInstance(data, list)

def testGetStationsSearchFilters(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
    response = self.client.get("/get_stations?search=Station A")
    data = json.loads(response.data)
    self.assertTrue(any("Station A" in item["text"] for item in data))

def testCheckFlagReturnsJson(self):
    response = self.client.get("/check_flag")
    self.assertEqual(response.status_code, 200)
    data = json.loads(response.data)
    self.assertIn("status", data)

def testLoadDefaultParametersReturnsJson(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
    response = self.client.get("/load_default_parameters")

```

```

self.assertEqual(response.status_code, 200)
data = json.loads(response.data)
self.assertIn("CHANGING_TIME", data)
self.assertIn("ATTENDANCE", data)

def testSaveParametersUpdatesDatabase(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
        #Use the same walking parameters already in the test database so redoStore
        #doesn't fail trying to look up venues that don't exist in the test database
        payload = {
            "CHANGING_TIME": 5.0, "MAX_TIME_WINDOW": 100.0,
            "rawArrivalPeakOffset": 20.0, "rawDeparturePeakOffset": 20.0,
            "ATTENDANCE": 0.9, "TRAIN_PROPORTION": 0.8,
            "SIGMA_FACTOR": 3.0, "MINIMUM_PEOPLE": 40.0,
            "PERSON_DELAY": 0.001, "PROPAGATION_FACTOR": 0.4,
            "WALKING_TIME": 15.0, "WALKING_SPEED": 5.0,
            "NORMAL": 1.0, "SLIGHTLY": 1.5, "BUSY": 2.5
        }
        response = self.client.post("/save_parameters",
            data=json.dumps(payload),
            content_type="application/json"
        )
        self.assertEqual(response.status_code, 200)
        #Verify the value was actually written to the database
        connection = sqlite3.connect(self.databaseFile)
        result = connection.execute("SELECT CHANGING_TIME FROM Users WHERE Username = 'testuser'").fetchone()[0]
        connection.close()
        self.assertEqual(result, 5.0)

def testUpdateFlagsReturnsSuccess(self):
    response = self.client.post("/updateFlags")
    self.assertEqual(response.status_code, 200)
    data = json.loads(response.data)
    self.assertTrue(data["success"])

def testUpdatePageLoadsWhenLoggedIn(self):
    with self.client.session_transaction() as session:
        session["username"] = "testuser"
    response = self.client.get("/update")
    self.assertEqual(response.status_code, 200)

# -----
# main.py — helper functions
# -----

class TestFormatDuration(unittest.TestCase):

    def testLessThanOneHour(self):
        result = mainApp.formatDuration(45)
        self.assertIn("45", result)
        self.assertNotIn("hour", result)

    def testMoreThanOneHour(self):
        result = mainApp.formatDuration(90)
        self.assertIn("hour", result)
        self.assertIn("30", result)

    def testExactlyOneHour(self):
        result = mainApp.formatDuration(60)
        self.assertIn("1", result)
        self.assertIn("hour", result)

```

```

class TestProcessTime(unittest.TestCase):

    def testReturnsCorrectArrivalTime(self):
        duration, arrivalTime = mainApp.processTime(30, "10:00")
        self.assertEqual(arrivalTime, "10:30")

    def testReturnsCorrectDurationString(self):
        duration, arrivalTime = mainApp.processTime(30, "10:00")
        self.assertIn("30", duration)

    def testOvernightJourney(self):
        #Journey starting at 23:30 for 60 minutes should arrive at 00:30
        duration, arrivalTime = mainApp.processTime(60, "23:30")
        self.assertEqual(arrivalTime, "00:30")

class TestGetLines(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsListOfLines(self):
        result = mainApp.getLines("Station A", self.databaseFile)
        self.assertIsInstance(result, list)

    def testCorrectLineReturned(self):
        result = mainApp.getLines("Station A", self.databaseFile)
        self.assertIn("Central Line", result)

    def testUnknownStationReturnsEmpty(self):
        result = mainApp.getLines("Nonexistent Station", self.databaseFile)
        self.assertEqual(result, [])

# -----
# API tests — these require a live internet connection to run
# getTfLData.py and storeStadiumData.py and scraper.py
# -----

class TestFetchStations(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        databaseCreation.createLines(self.databaseFile, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..',
'programme files', 'lines.json'))

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsDictionary(self):
        #Fetches stations for the central line from the TfL API
        result = getTfLData.fetchStations(TfL_API_KEY, ["central"])
        self.assertIsInstance(result, dict)

    def testStationDictionaryNotEmpty(self):
        result = getTfLData.fetchStations(TfL_API_KEY, ["central"])

```

```

self.assertGreater(len(result), 0)

def testStationsHaveCorrectAttributes(self):
    result = getTfLData.fetchStations(TfL_API_KEY, ["central"])
    #Every station object should have a NaPTAN, StationName, Latitude and Longitude
    for stationObject in result.values():
        self.assertIsNotNone(stationObject.NaPTAN)
        self.assertIsNotNone(stationObject.StationName)
        self.assertIsNotNone(stationObject.Latitude)
        self.assertIsNotNone(stationObject.Longitude)

def testBadApiKeyReturnsInt(self):
    #safeGet returns an integer status code when the API call fails
    result = getTfLData.fetchStations("invalid_key_12345", ["central"])
    self.assertIsInstance(result, int)

#fetch_lines() is one of the two functions called by fetchTfLData(), so if these tests
#pass alongside TestFetchStations it also confirms fetchTfLData() works end-to-end
class TestFetchLines(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        databaseCreation.createLines(self.databaseFile, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..',
'programme files', 'lines.json'))
        #fetchLines needs a populated stationDictionary to work with
        self.stationDictionary = getTfLData.fetchStations(TfL_API_KEY, ["central"])

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsDictionary(self):
        result = getTfLData.fetch_lines(self.stationDictionary, TfL_API_KEY, ["central"])
        self.assertIsInstance(result, dict)

    def testCentralLinePresent(self):
        result = getTfLData.fetch_lines(self.stationDictionary, TfL_API_KEY, ["central"])
        self.assertIn("central", result)

    def testBranchesAreListsOfNaPTANs(self):
        result = getTfLData.fetch_lines(self.stationDictionary, TfL_API_KEY, ["central"])
        for branch in result["central"]:
            self.assertIsInstance(branch, list)
            self.assertGreater(len(branch), 0)

class TestGetStationCoordinates(unittest.TestCase):

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)

    def tearDown(self):
        os.unlink(self.databaseFile)

    def testReturnsCoordinates(self):
        result = getTfLData.getStationCoordinates(self.databaseFile, "STA")
        self.assertIsNotNone(result)

    def testCoordinatesAreCorrect(self):
        result = getTfLData.getStationCoordinates(self.databaseFile, "STA")
        latitude, longitude = result
        self.assertAlmostEqual(latitude, 51.50)

```

```
self.assertAlmostEqual(longitude, -0.10)
```

```
class TestGetStationLineRelationships(unittest.TestCase):
```

```
def setUp(self):
    self.connection = createTestDatabase()
    self.databaseFile = writeTestDatabaseToFile(self.connection)
```

```
def tearDown(self):
    os.unlink(self.databaseFile)
```

```
def testRelationshipsArePopulated(self):
    #Wipe existing relationships then repopulate from the Connections table
    connection = sqlite3.connect(self.databaseFile)
    connection.execute("DELETE FROM StationLineRelationships")
    connection.commit()
    connection.close()
    getTfLData.getStationLineRelationships(self.databaseFile)
    connection = sqlite3.connect(self.databaseFile)
    rows = connection.execute("SELECT * FROM StationLineRelationships").fetchall()
    connection.close()
    self.assertGreater(len(rows), 0)
```

```
def testRelationshipsMatchConnections(self):
    #Every NaPTAN in StationLineRelationships should also appear in Connections
    connection = sqlite3.connect(self.databaseFile)
    slrRows = {row[0] for row in connection.execute("SELECT NaPTAN FROM StationLineRelationships").fetchall()}
    connectionRows = {row[0] for row in connection.execute("SELECT StationA FROM Connections").fetchall()}
    connection.close()
    self.assertTrue(slrRows.issubset(connectionRows))
```

*#SaveTfLData() calls fetchTfLData() and getStationLineRelationships() internally, so
#if these tests pass it also confirms both of those functions work correctly end-to-end*

```
class TestSaveTfLData(unittest.TestCase):
```

```
def setUp(self):
    temporaryFile = tempfile.NamedTemporaryFile(suffix=".db", delete=False)
    temporaryFile.close()
    self.databaseFile = temporaryFile.name
    databaseCreation.createDatabase(self.databaseFile)
    databaseCreation.createLines(self.databaseFile, os.path.join(os.path.dirname(os.path.abspath(__file__)), '..',
'programme files', 'lines.json'))
```

```
def tearDown(self):
    os.unlink(self.databaseFile)
```

```
def testSaveTfLDataReturnsTrue(self):
    #Full pipeline — fetches and stores all TfL station and connection data
    result = getTfLData.SaveTfLData(self.databaseFile, TfL_API_KEY)
    self.assertTrue(result)
```

```
def testStationsTablePopulated(self):
    getTfLData.SaveTfLData(self.databaseFile, TfL_API_KEY)
    connection = sqlite3.connect(self.databaseFile)
    count = connection.execute("SELECT COUNT(*) FROM Stations").fetchone()[0]
    connection.close()
    self.assertGreater(count, 0)
```

```
def testConnectionsTablePopulated(self):
    getTfLData.SaveTfLData(self.databaseFile, TfL_API_KEY)
    connection = sqlite3.connect(self.databaseFile)
    count = connection.execute("SELECT COUNT(*) FROM Connections").fetchone()[0]
```

```

connection.close()
self.assertGreater(count, 0)

```

```

class TestGetCoordinates(unittest.TestCase):

```

```

    def testReturnsCoordinatesForKnownStadium(self):
        result = storeStadiumData.getCoordinates(OPENCAGE_API_KEY, "Emirates Stadium")
        self.assertIn("lat", result)
        self.assertIn("lng", result)

```

```

    def testLatitudelsReasonableForLondon(self):
        #All London venues should have latitude roughly between 51 and 52
        result = storeStadiumData.getCoordinates(OPENCAGE_API_KEY, "Stamford Bridge")
        self.assertGreater(result["lat"], 51.0)
        self.assertLess(result["lat"], 52.0)

```

```

    def testLongitudelsReasonableForLondon(self):
        #All London venues should have longitude roughly between -0.5 and 0.1
        result = storeStadiumData.getCoordinates(OPENCAGE_API_KEY, "Stamford Bridge")
        self.assertGreater(result["lng"], -0.5)
        self.assertLess(result["lng"], 0.1)

```

*#getStadiumData() calls getCoordinates() and insertData() for every club in the JSON,
 #so if these tests pass it also confirms both functions work correctly in a real context,
 #in addition to their own direct tests above*

```

class TestGetStadiumData(unittest.TestCase):

```

```

    def setUp(self):
        self.connection = createTestDatabase()
        self.databaseFile = writeTestDatabaseToFile(self.connection)
        self.londonClubsFile = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'programme files', 'londonClubs.json')

```

```

    def tearDown(self):
        os.unlink(self.databaseFile)

```

```

    def testReturnsTrue(self):
        result = storeStadiumData.getStadiumData(self.londonClubsFile, self.databaseFile, OPENCAGE_API_KEY, 1.25)
        self.assertTrue(result)

```

```

    def testVenuesTablePopulated(self):
        storeStadiumData.getStadiumData(self.londonClubsFile, self.databaseFile, OPENCAGE_API_KEY, 1.25)
        connection = sqlite3.connect(self.databaseFile)
        count = connection.execute("SELECT COUNT(*) FROM Venues").fetchone()[0]
        connection.close()
        self.assertGreater(count, 0)

```

```

    def testTeamsTablePopulated(self):
        storeStadiumData.getStadiumData(self.londonClubsFile, self.databaseFile, OPENCAGE_API_KEY, 1.25)
        connection = sqlite3.connect(self.databaseFile)
        count = connection.execute("SELECT COUNT(*) FROM Teams").fetchone()[0]
        connection.close()
        self.assertGreater(count, 0)

```

*#getPageFixtures() calls getDate() internally for every date heading it finds, so if
 #these tests pass it also confirms getDate() works in a real scraping context.
 #scraper.main() is an orchestrator that calls getPageFixtures() for every league and
 #month, so if these tests pass it also confirms scraper.main() works end-to-end*

```

class TestGetPageFixtures(unittest.TestCase):

```

```

    def testReturnsDictionary(self):
        #Scrapes one month of Premier League fixtures from BBC Sport

```

```

fixtures = {}
result = scraper.getPageFixtures(
    "https://www.bbc.co.uk/sport/football/premier-league/scores-fixtures/2026-03?filter=fixtures",
    "2026-03",
    "Premier League",
    fixtures
)
self.assertIsInstance(result, dict)

def testFixturesNotEmpty(self):
    fixtures = {}
    result = scraper.getPageFixtures(
        "https://www.bbc.co.uk/sport/football/premier-league/scores-fixtures/2026-03?filter=fixtures",
        "2026-03",
        "Premier League",
        fixtures
    )
    #There should be at least some fixtures for a given month
    totalFixtures = sum(len(v) for v in result.values())
    self.assertGreater(totalFixtures, 0)

def testFixturesHaveCorrectKeys(self):
    fixtures = {}
    result = scraper.getPageFixtures(
        "https://www.bbc.co.uk/sport/football/premier-league/scores-fixtures/2026-03?filter=fixtures",
        "2026-03",
        "Premier League",
        fixtures
    )
    for dateFixtures in result.values():
        for fixture in dateFixtures:
            self.assertIn("home", fixture)
            self.assertIn("away", fixture)
            self.assertIn("time", fixture)
            self.assertIn("league", fixture)

def testBadUrlReturnsInt(self):
    #safeGet should return 0 if the URL is completely unreachable
    fixtures = {}
    result = scraper.getPageFixtures(
        "https://this.url.does.not.exist.invalid/fixtures",
        "2026-03",
        "Premier League",
        fixtures
    )
    self.assertIsInstance(result, int)

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

```

Run Python tests in test_nea.py
✓ Test Results 8 min 6 sec
✓ Tests passed: 164 of 164 tests - 8 min 6 sec

C:\Users\WP\Downloads\Prototype\venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.3.1.1/plugins/python-ce/helpers/pycharm/_jb_unittest_runner.py" --path "C:\Users\WP\Downloads\Prototype\unit test\test_nea.py"
Testing started at 16:51 ...
Launching unittests with arguments python -m unittest C:\Users\WP\Downloads\Prototype\unit test\test_nea.py in C:\Users\WP\Downloads\Prototype\unit test

Rate limit reached: Attempt 1 of 3
Rate limit reached: Attempt 2 of 3
Rate limit reached: Attempt 3 of 3
Fetched Stations
Server error: Attempt 1 of 2
Fetched Lines
TFL Data Fetched
Stored Stations
Stored connections
Fetched Stations
Fetched Lines
TFL Data Fetched
Stored Stations
Stored connections
Fetched Stations
Fetched Lines
TFL Data Fetched
Stored Stations
Stored connections

Ran 164 tests in 486.174s
OK
Process finished with exit code 0

```

Integration testing

<https://youtu.be/FhCFelha9SU>

Integration testing checks that the individual components of the system work correctly together, with a focus on the interfaces between them rather than the behaviour of any single function in isolation.

The interaction between the crowding algorithm, pathfinding component, and results display is the most important integration point in the system, and is fully demonstrated through tests 19 to 23 of the test plan alongside the system testing video. Those tests show end-to-end journey searches producing correct route comparisons and crowding descriptions, which requires all five components (`getResults.main()`, `crowdingAlgorithm.main()`, `pathFinding.MakeGraph()`, `pathFinding.Dijkstra()`, and the Flask route in `main.py`) to interface correctly.

In addition to those tests and videos, below are three further tests:

Test Reference	Test	Purpose	Test Data	Expected outcome	Actual outcome
A	Log out then log in as a different user with different walking parameters	Test that switching user correctly reconfigures VenueStation Relationships for the new user, confirming the	Two accounts with different WALKING_TIME values	After second login, VenueStation Relationships reflects the second user's parameters, not the first user's	00:00-02:34

		interface between the session, parameter loading, and database state (N)			
B	Click Fetch New Data, wait for completion, verify user parameters are preserved	Test that the database rebuild does not overwrite the current user's parameters, confirming the interface between the fetch route, databaseCreation.main(), and the user data restoration logic (N)	Modify one parameter (CHANGING_TIME = 9.9), click save, and then click Fetch New Data, wait for redirect	After redirect to main page, navigating to settings shows CHANGING_TIME is still 9.9	02:34-05:05
C	Click Fetch New Data, verify new fixture data is present in the database	Test that the scraper runs during the rebuild and the Events table is populated with fresh data, confirming the full acquisition pipeline works end-to-end (N)	Click Fetch New Data	After rebuild, Events table contains upcoming fixtures and journey searches on fixture dates show crowding effects	05:05-07:37

System testing

System testing is a full run through from start to finish of using the system as a standard user. This has been demonstrated in the above tests and testing videos, which show a complete walkthrough of the application. These have been compiled into the following playlist.

<https://www.youtube.com/playlist?list=PLdnZpFTrP5u1rZ2F99BftL63CZarEq9zl>

In addition to those videos, a full walkthrough of the programme has been demonstrated in the following video, which is also present in the above playlist. This demonstrates how a typical user may use the programme.

<https://youtu.be/x7INnkXCpt8>

The system objectives have also been demonstrated via the unit tests and the fact that in the testing videos, the system functions as normal, suggesting all the interfaces are working accordingly.

Acceptance testing

I demonstrated the completed programme to Ms Miah, my end user and asked her to test each of the core features she had requested during the initial interview, and the amendments she proposed after reviewing the prototype. Her feedback and response can be found in the End user and target audience section of the Evaluation section of this document.

Evaluation

Objectives

User objectives:

Objective	Evaluation
1 (a-d)	This objective was fully achieved. The main page provides two Select2 searchable dropdowns populated with every station in the database, allowing the user to find any TfL rail station by typing part of its name. A date input and a time input allow the user to specify their intended departure. On submission, the modified Dijkstra's algorithm runs on the base graph and returns an ordered list of path steps, each containing the start station, end station, line used, and cumulative journey time. The total journey time and estimated arrival time are displayed to the user. The line-change penalty (CHANGING_TIME) is applied every time the algorithm transitions between lines, meaning routes that avoid interchanges are preferred where the time saving is sufficient. This was verified by tests 15, 19, and 23 of the test plan.
2 (a-f)	This objective was fully achieved and represents the most technically complex part of the system. The BBC Sport scraper collects fixture data for all professional English football league tiers for the next twelve months at setup, filtering to London home matches only. For each relevant fixture on the travel date, the crowding algorithm distributes the estimated number of rail users across nearby stations using inverse distance weighting, applies a Gaussian model to estimate the fraction of peak demand present at the journey midpoint, and propagates the crowd recursively through the network with a decay factor. The resulting delay factors are applied to all connections, meaning journey times through affected areas are longer in the crowded graph than in the base graph. The average speed of each line is stored in the Lines table and used when calculating base travel times from geodesic distance. The pathfinding algorithm is then re-run on the crowded graph to produce a second route that may differ from the original if a less delayed alternative exists. This was verified by tests 19, 20, and 21 of the test plan.

3 (a-b)	This objective was fully achieved. Every station displayed on both routes is annotated with a crowding severity label derived from the station's delay factor, which is itself calculated from the estimated number of additional passengers at that station. The four thresholds, NORMAL, SLIGHTLY, and BUSY, are user-configurable, meaning the user can adjust what counts as busy relative to their own tolerance. On a date with no nearby fixtures, all stations show Normal. This was verified by tests 20 and 22 of the test plan.
4 (a-d)	This objective was fully achieved. The settings page displays all 15 parameters in an editable form with JavaScript input sanitisation preventing non-numeric and negative inputs. Saving writes the updated values to the Users table in the database, and any subsequent journey search uses these new values. The Load Default Values button restores all parameters to the values in defaultParameters.json without navigating away from the page. When WALKING_TIME or WALKING_SPEED is changed and saved, the system calls redoStore() to repopulate the VenueStationRelationships table using the new walking distance, ensuring the crowding calculation uses the correct set of nearby stations. This was verified by tests 25 through 31 of the test plan.
5 (a-e)	This objective was fully achieved. Account creation and login are handled by login.py. Passwords are never stored in plaintext, and instead a 16-byte random salt is generated per user using os.urandom(16), and the password is hashed with SHA-256 using the salt. On login, the stored hash is recomputed from the entered password and salt and compared against the stored value. Specific error messages distinguish between a username not found and a wrong password. Attempting to create an account with a username that already exists returns an "Account already created" message. On successful account creation, default parameter values are loaded from defaultParameters.json and stored in the Users table alongside the credentials. All routes other than the login page check for a valid session key and redirect to login if it is absent.

	This was verified by tests 1 through 11 of the test plan and the TestLogin, TestCreateAccount, and TestGenerateSalt unit test groups.
6 (a-e)	This objective was fully achieved. The application has exactly five pages as specified. Station selection uses Select2 dropdowns with AJAX calls to a /get_stations endpoint, returning up to 20 matching stations as the user types. Journey results are rendered on the main page itself next to the search form. Both routes are displayed in parallel containers showing each step with its line, station name, and crowding description, along with total journey time and estimated arrival. The Fetch New Data button on the settings page starts a background thread that backs up user data, rebuilds the database with freshly scraped and fetched data, and restores user data before redirecting back via the loading page. This was verified by tests 12 through 24 and test 33 of the test plan.

System objectives:

Objective	Evaluation
1 (a-g)	This objective was fully achieved. The database schema is in Third Normal Form with junction tables for StationLineRelationships and VenueStationRelationships handling the many-to-many relationships between stations and lines, and between venues and stations respectively. All nine tables are created by databaseCreation.py and populated at setup. Station and connection data is fetched from the TfL API, stadium coordinates from the OpenCage geocoding API, and fixture data by scraping BBC Sport. User credentials and all 15 parameters are stored in the Users table. This was verified by the TestCreateDatabase unit test group and integration tests B and C.
2 (a-b)	This objective was fully achieved. MakeGraph() constructs an adjacency list from the Connections table where each station maps to a list of (neighbour, travel time, line) tuples. Dijkstra() uses a min-heap via Python's heapq module and stores state as (station,

	line) pairs rather than just stations, which allows the line-change penalty to be applied whenever the algorithm moves between lines. The algorithm returns a list of steps in the form [StationA, StationB, LineUsed, CumulativeTime]. This was verified by the TestMakeGraph and TestDijkstra unit test groups.
3 (a-g)	This objective was fully achieved. All seven sub-objectives are implemented in crowdingAlgorithm.py. The journey midpoint is estimated using Dijkstra on the base graph, events within MAX_TIME_WINDOW are selected, passenger demand is distributed using inverse distance weighting and scaled by a Gaussian function, and propagation is carried out recursively with a PROPAGATION_FACTOR decay until crowds fall below MINIMUM_PEOPLE. Station delay factors are calculated from total passenger load and spread uniformly across each line. The algorithm returns a Network object used by the results component. This was verified by the TestInverseWeight, TestGaussianFactor, TestFindEventsInTimeFrame, TestCreateNetwork, TestPropegation, TestProcessAffectedStations, and TestSpreadDelay unit test groups.
4 (a-d)	This objective was fully achieved, as described under User objective 5 above.
5 (a-c)	This objective was fully achieved. getResults.main() produces all three route objects, which are the original fastest route from the base graph, the original route re-evaluated by stripping the affected graph down to only the connections on that route, and a newly optimised route calculated on the full affected graph. Each station on both displayed routes is annotated with a severity description. All three routes are packaged into a Results object returned to the Flask route. This was verified by the TestGetAffectedGraph, TestGetStrippedGraph, and TestMainPagePathClass unit test groups.
6 (a-g)	This objective was fully achieved. All file paths and API keys are stored in Flask's app.config dictionary at startup and referenced by name throughout. The login route checks for the database file on every request. The loading page uses JavaScript setInterval to poll /check_flag every two seconds. All non-login routes check for a session key.

	The <code>/get_stations</code> endpoint filters the Stations table by a search string and returns up to 20 results as JSON. The save parameters route triggers <code>redoStore()</code> if walking parameters have changed. The fetch new data route backs up the Users table, calls <code>databaseCreation.main()</code> in a background thread, and restores user data on completion. This was verified by the <code>TestMainFlaskRoutes</code> unit test group and integration tests A through C.
--	--

End user and target audience

End user

Upon finishing my programme, I sent it over to Ms Miah and asked her to run it locally on her preferred machine so she could try it out. After two months of her testing the programme and using it in her daily commute planning, I invited her to join me in the cafeteria to give me some feedback on it. While conducting this interview I wrote down the conversation in my journal, and I've rewritten it here as before:

Me: "Thanks for meeting me Ms Miah. So you've had a few weeks with it now, what are your overall thoughts?"

Ms Miah: "I've really enjoyed using it actually. I've been checking it before my commute on days when I know there's a match near my route, and it's already saved me from a couple of journeys that would have been much more stressful than they needed to be."

Me: "That's great to hear. Is the route comparison working the way you had in mind?"

Ms Miah: "Yes, exactly how I imagined it. Being able to see both the fastest route normally and the one that avoids the worst crowding side by side is really clear. And the labels regarding station busyness on each station are really helpful. I never had anything like that before. I used to just guess based on experience."

Me: "Do you feel they're accurate to what you've experienced?"

Ms Miah: "Mostly yes, though I did notice on one occasion it said a station would be normal and it was actually slightly busy. But I understand it's working from estimates and assumptions rather than live data, and it's just a prediction and not a crystal ball, so it can't really account for unforeseen circumstances, so I think that's to be expected. It's still far more useful than anything else I've used."

Me: “That’s really useful feedback. Was there anything you felt was missing from the system?”

Ms Miah: “I think a map showing the route geographically would be a nice addition. I’m quite visual and I like to see where I’m going rather than just a list of station names. And it crossed my mind that it might be useful to suggest what time to leave to avoid the worst of the crowds altogether, rather than just offering an alternative route. Though I appreciate those weren’t things I asked for originally, so I understand why they aren’t there.”

Me: “Anything else you’d add?”

Ms Miah: “One thing that occurred to me was that it would be interesting if it could factor in weather as well, since bad weather does tend to push more people onto the tube. But again that feels like quite a different problem, and it’s not something I mentioned at the start. The interface is a little plain visually compared to the apps I normally use, but honestly the functionality is what matters to me and that’s all there. The login is simple which is what I wanted, and I love that it keeps the fixture data up to date automatically. I’d never want to have to input match schedules myself. Overall I’m really pleased with how it’s turned out.”

The feedback was largely positive, with Ms Miah confirming that all of the user objectives set out in the analysis have been met. Notes on suggested improvements and areas for future development can be found in the sections below.

Target audience

Ms Miah distributed the programme to her students, some of whom are also commuters and young professionals who regularly use the TfL rail network, to gather broader feedback from the target audience. 43 students installed and used the programme over the same period and were asked to rate it across five categories on a scale of 1 to 5. Since these students are target audience members rather than the end user, only their average ratings are included here rather than individual responses.

Category	Average Rating (out of 5)
Ease of use	4.1
Usefulness for journey planning	4.3
Clarity of crowding information	4.2
Quality of route suggestions	4.0
Likelihood to use regularly	3.8

The ratings were largely positive across all categories, with usefulness and clarity of crowding information scoring highest, showing that the core purpose of the system was well received by the broader target audience. The lowest score was for likelihood to use regularly, which is understandable given that the system is most relevant on days when matches are taking place near a user's route, which is not an everyday occurrence. This is a limitation of the system's scope rather than a flaw in its implementation, and was not something Ms Miah raised as a concern during her feedback session.

Areas for improvement

- During her feedback session, Ms Miah noted that she would find it helpful to see the suggested route displayed on an actual map rather than just as an ordered list of station names. Currently the system presents both routes as a sequence of steps showing the station name, line used, and crowding description, but gives no geographic context for where those stations are relative to each other or to the user's start and end points.

This feature was not included in the original scope because Ms Miah did not raise it during the initial interview, where the priority was on the crowding data and route comparison rather than visual presentation.

- Ms Miah mentioned that rather than simply offering an alternative route, it would be useful if the system could suggest what time to leave in order to avoid the worst of the crowds altogether. This would involve running the crowding algorithm across a range of departure times around the user's intended journey time and identifying windows where the predicted crowding severity drops below a user-defined threshold. The system could then present these quieter windows alongside the route comparison.

This feature was not included because the crowding algorithm is currently designed to evaluate a single journey midpoint rather than iterate across a range of times. Supporting departure time suggestions would require running the algorithm multiple times per request, potentially ten or more times to cover a meaningful time range, which would significantly increase the response time for each journey search.

- Ms Miah raised the observation that bad weather tends to push more people onto the tube, as those who might otherwise walk or cycle choose to use public transport instead. Incorporating a weather forecast API could allow the system to apply an additional demand multiplier on days where heavy rain is predicted. This would make the crowding estimates more accurate on days when weather-driven demand is likely to be a significant factor independent of any scheduled events.

This feature was not included because it was outside the scope of the original problem, which was specifically focused on event-driven crowding from football

matches. Incorporating weather would also introduce a dependency on a live forecast API being available at query time, adding another potential point of failure to the system. It was also not prioritised given the original requirements.

- The system currently models only TfL rail services. In reality, many journeys involve a combination of rail and bus, or include a walking leg between stations. Adding bus routes to the graph would allow the system to suggest alternatives that avoid crowded rail stations entirely by routing through bus connections.

This feature was not included because the TfL Bus API returns stop and route data in a fundamentally different structure to the rail API. Rail connections between stations are fixed and can be stored as static edges in the graph, but bus routes change more frequently and involve a much larger number of stops, requiring a significantly different data model. Integrating buses would have required building a parallel graph construction pipeline and adapting the pathfinding algorithm to handle mixed mode journeys with different transfer penalties. However, this was beyond the scope of the project and therefore not included.

Areas for future development

- The crowding model currently relies on a set of assumptions about stadium attendance rates, the proportion of attendees who use TfL rail services, and how crowd density spreads through the network over time. These values are stored as user-adjustable parameters and are initialised to defaults derived from publicly available research, but they remain approximations rather than measured values. As Ms Miah noted during feedback, the predictions are mostly accurate but there are occasions where the system underestimates or overestimates crowding at a particular station.

Replacing these assumptions with real historical data would significantly improve the accuracy of the model. TfL publishes anonymised gate count data for some stations under Freedom of Information requests, and studies on crowd behaviour at large events have produced empirically validated models for passenger dispersal. Integrating this data would allow the default parameter values to be grounded in observed rather than estimated behaviour. However, this data is not freely or consistently available through a public API, and requires individual FOI requests and manual processing, which made it impractical to include within the project.

- Ms Miah noted that the interface looks quite plain compared to commercial journey planning applications like Google Maps or Citymapper. The current design is functional and clear but uses minimal styling.

This was not prioritised during development because the focus was on correctness and completeness of the core algorithm rather than visual refinement, and Ms Miah

did not raise aesthetic concerns during the initial requirements interview.

- The scraper currently collects fixture data from BBC Sport for all professional English football leagues with London home teams and their respective stadiums. However, London also hosts a large number of other high attendance events, like concerts at the O2 Arena and Wembley Stadium, major rugby and cricket fixtures at Twickenham and Lord's, and athletics events at the London Stadium, all of which generate significant passenger demand (although usually less than top football games) on the TfL network.

Expanding the Events table to include non-football events would make the crowding model substantially more comprehensive. This was not included in the original scope because Ms Miah's initial requirements were specifically about football matches, and the data acquisition pipeline was designed around BBC Sport's fixture pages. Extending it to other sources would require additional scraping logic for each new data source.

Conclusion

This project has been the most technically ambitious piece of work I have undertaken, and completing it has significantly developed both my programming skills and my understanding of how complex systems are designed and built in practice.

On a practical level, I learned how to build a complete multi-component web application from scratch using Python and Flask, integrating a relational SQLite database, external APIs, a web scraper, and a mathematical model into a single working system. Before this project I had no experience with Flask's routing and session management, and learning to coordinate a background thread for database creation while keeping the UI responsive through polling was one of the more challenging problems I encountered. I also developed a much deeper understanding of graph-based algorithms. Implementing a modified version of Dijkstra's shortest path algorithm with line-change penalties required me to move beyond the standard textbook implementation.

Building the BBC Sport scraper was one of the more unexpectedly difficult parts of the project. Unlike working with a documented API, scraping requires manually inspecting the HTML structure of a webpage and identifying patterns in the markup that are consistent enough to rely on. I spent considerable time loading fixture pages in a browser, reading through the raw HTML, and working out which CSS classes and tag structures consistently contained the data I needed without accidentally picking up unrelated elements. Every time BBC Sport updated their page layout, which happened at least once during development, the patterns I had identified broke silently and I had to go back through the HTML to find the new structure. This taught me to write the scraper defensively, with checks that the expected elements were actually present before attempting to extract data from them, rather than assuming the structure would always be as anticipated.

The crowding model was the part of the project I found most intellectually interesting. Designing a system that uses inverse distance weighting, a Gaussian distribution, and recursive propagation to simulate how crowds spread through a transport network required me to research mathematical modelling techniques I had not encountered before and then translate them into working code. Seeing the algorithm produce genuinely different route suggestions on match days compared to quiet days was a satisfying validation that the model was behaving as intended.

The project also taught me a great deal about the importance of careful debugging and thorough testing. The most significant bug I encountered required a fundamental rethinking of the approach rather than just correcting a line of code. The original implementation of the crowded route calculation worked by taking the old path returned by Dijkstra and multiplying each connection's base travel time by the corresponding delay factor from the affected network. On small test cases with only a few stations this appeared to work correctly, but on real journeys across the full TfL network the crowded journey times were consistently off by a few minutes compared to what the algorithm should have produced. The discrepancy was small enough that it was not immediately obvious something was wrong, and tracking it down required carefully working through what the multiplication was actually doing at each step.

The root cause was that the line change penalty was also being scaled by the delay factor, even though it has nothing to do with crowding on the connections themselves. The penalty was effectively inflated on routes that required line changes. The error grew with each line change in the path, which is why it only became noticeable on longer journeys across the full network rather than in small test cases where line changes were rare.

The fix was to abandon the multiplication approach entirely and instead construct a stripped version of the affected graph containing only the connections present in the original path, then re-run Dijkstra on that stripped graph. This way the pathfinding algorithm applies the line change penalty in its original fixed form during traversal, and the delay is incorporated through the modified edge weights in the graph rather than by multiplication of the result of the old calculation by delay factors. This is the approach used in the final implementation via `getStrippedGraph()` and `getAffectedOldPath()` in `getResults.py`. Arriving at this solution required understanding not just that the numbers were wrong, but precisely why they were wrong and what property of the original approach was causing the error, which in turn pointed directly to the correct fix.

If I were to approach this project again, I would invest more time at the start on the database schema and parameter architecture, as several design decisions had to be revisited mid development when their limitations became apparent. I would also write unit tests from the beginning of each component rather than adding them afterwards, since having tests in place earlier would have caught several of the silent failures much sooner. Having a clearer plan from the beginning would also help me to finish the code in a more timely fashion.

Overall I am proud of what the system achieves. It solves a real problem that my end user experiences in her daily commute, and it does so using genuinely complex technical

components that go well beyond what was strictly required, and it is built in a way that is modular and extensible for the future developments described above. Ms Miah's feedback confirmed that the core objectives have been met, and completing this project has given me the confidence and experience to take on similarly complex problems in the future.

Appendices

Annotated code

dbError.css

```
/* Styles for the Database Error Page */

/* Center everything vertically and horizontally */
body {
  font-family: Arial, sans-serif;
  background-color: #f8f9fa;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh; /* Full viewport height */
  margin: 0; /* Remove default margin */
}

/* Container for the error content */
.container {
  text-align: center;
  background-color: #fff;
  padding: 40px 60px;
  border-radius: 10px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.1);
  max-width: 500px;
}

/* Heading style */
h1 {
  color: #d9534f; /* Red for error */
  margin-bottom: 20px;
}

/* Error message style */
.error-message {
  font-size: 1.2em;
  color: #d9534f;
  margin-bottom: 30px;
}

/* Optional informational messages */
.info-message {
  margin-top: 20px;
  font-size: 1em;
  color: #555;
}

/* Retry button styling */
.retry-button {
  background-color: #0275d8;
  color: white;
  border: none;
  padding: 12px 25px;
  font-size: 1em;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.2s ease;
}
```

```
.retry-button:hover {
  background-color: #025aa5; /* Darker blue on hover */
}
```

loading.css

```
/* Reset default styles and set box-sizing */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Ensure full height for body and html */
body, html {
  height: 100%;
  font-family: Arial, sans-serif;
}

/* Container for the loading screen */
.loadingContainer {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh; /* Full viewport height */
  background-color: white;
}

/* Loading text style with blinking animation */
.loadingText {
  font-size: 2rem;
  font-weight: bold;
  color: black;
  animation: blink 3s infinite;
}

/* Blink keyframes animation */
@keyframes blink {
  0% {
    opacity: 1;
  }
  33% {
    opacity: 0;
  }
  66% {
    opacity: 1;
  }
}
```

login.css

```
/* Page container */
.page {
  display: flex;
  flex-direction: column;
  align-items: center;
  margin-top: 60px;
  font-family: Arial, sans-serif;
}
```

```

/* Buttons styling */
.button {
  padding: 10px;
  font-size: 16px;
  cursor: pointer;
  border: none;
  background-color: green;
  color: white;
  border-radius: 4px;
  transition: background-color 0.3s;
}

.button:hover {
  background-color: seagreen;
}

/* Page title */
.title {
  text-align: center;
  font-size: 32px;
  margin-bottom: 20px;
}

/* Container for form elements */
.container {
  width: 300px;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

/* Text input styling */
.textInput {
  width: 100%;
  padding: 10px;
  box-sizing: border-box;
  font-size: 16px;
  border: 1px solid grey;
  border-radius: 4px;
}

/* Row for buttons */
.buttonRow {
  display: flex;
  justify-content: space-between;
}

.buttonRow .button {
  width: 48%; /* make buttons evenly spaced */
}

/* Form styling */
.loginForm {
  display: flex;
  flex-direction: column;
  gap: 12px;
  width: 100%;
}

/* Error or informational messages */
.message {
  color: red;
  font-size: 14px;
}

```

```
text-align: center;
}
```

mainPageStyle.css

```
/* General page styling */
body {
  margin: 0;
  font-family: Arial, sans-serif;
}

/* Top title section */
.titleSection {
  height: 10vh;
  background-color: #333;
  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 2.5em;
  text-align: center;
}

/* Main container with input and output sections */
.mainContainer {
  display: flex;
  height: 80vh;
}

/* Input section styling */
.inputSection {
  width: 30%;
  padding: 20px;
  background-color: #f4f4f4;
  overflow-y: auto;
}

.theForm {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.inputSection select,
.inputSection input,
.submitButton {
  width: 100%;
  height: 40px;
  padding: 5px 10px;
  font-size: 16px;
  box-sizing: border-box;
}

.dateTimeContainer {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.submitButton {
  cursor: pointer;
}
```

```

}

/* Output section styling */
.outputSection {
  display: flex;
  justify-content: space-between;
  width: 70%;
  padding: 20px;
  background-color: lightgray;
}

.pathContainer {
  width: 48%;
  padding: 10px;
  box-sizing: border-box;
  overflow-y: auto;
}

.path {
  margin-bottom: 10px;
}

.path h6 {
  font-size: 18px;
  font-weight: bold;
  text-align: center;
  text-decoration: underline;
  margin-bottom: 5px;
}

.path ul {
  list-style-type: none;
  padding-left: 0;
  margin-top: 0;
}

.path ul li {
  padding: 5px 0;
}

.timeContainer {
  margin-top: 15px;
  font-size: 14px;
  color: #555;
}

/* Update button fixed position */
.updateButton {
  position: fixed;
  bottom: 20px;
  right: 20px;
  padding: 10px 20px;
  cursor: pointer;
}

/* Select2 plugin customization */
.select2-container {
  width: 100% !important;
}

.select2-container--default .select2-selection--single {
  height: 40px;
  display: flex;
  align-items: center;

```

```

    box-sizing: border-box;
}

.select2-container--default
.select2-selection--single
.select2-selection__rendered {
    line-height: 40px;
    padding-left: 10px;
}

.select2-container--default
.select2-selection--single
.select2-selection__arrow {
    height: 40px;
}

/* Error message styling */
.errorMessage {
    color: red;
    font-size: 14px;
    margin-top: 5px;
}

```

update.css

```

/* General body styling */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    display: flex;
    justify-content: center;
    align-items: flex-start;
    min-height: 100vh;
    background-color: #f0f0f0;
    overflow-y: auto;
}

/* Main container holding the update page content */
.mainContainer {
    width: 100%;
    max-width: 900px;
    padding: 20px;
    background-color: white;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    border-radius: 8px;
    box-sizing: border-box;
    margin-top: 20px;
}

/* Container specific for the update form */
.updatePageContainer {
    width: 100%;
}

/* Parameter table styling */
.parameterTable {
    width: 100%;
    table-layout: auto;
    border-collapse: collapse;
}

```

```

.parameterTable tr {
  width: 100%;
  border-bottom: 1px solid #ddd;
}

/* First column (labels) */
.parameterTable td:first-child {
  width: 40%;
  padding-right: 20px;
  text-align: left;
  border-right: 1px solid #ddd;
  box-sizing: border-box;
}

/* Second column (inputs) */
.parameterTable td:nth-child(2) {
  width: 60%;
  padding: 10px;
  box-sizing: border-box;
}

/* Input fields styling */
.parameterTable input {
  width: 100%;
  padding: 8px;
  margin-top: 5px;
  box-sizing: border-box;
  text-align: left;
}

/* Section for action buttons */
.buttonSection {
  margin-top: 20px;
}

.buttonGroup {
  display: flex;
  justify-content: space-between;
  width: 100%;
  gap: 10px;
}

/* Action buttons styling */
.actionButton {
  flex: 1;
  background-color: grey;
  color: white;
  border: none;
  padding: 10px;
  text-align: center;
  cursor: pointer;
  font-size: 16px;
  transition: background-color 0.3s ease;
}

.actionButton:hover {
  background-color: darkgrey;
}

```

```
// Add an event listener to the "Retry" form on the database error page
document.getElementById("retryForm").addEventListener("submit", async function (event) {
  // Prevent the default form submission behavior (page reload)
  event.preventDefault();

  // Send a POST request to the server to reset or update the flags for database setup
  await fetch("/updateFlags", {
    method: "POST"
  });

  // After updating the flags, redirect the user back to the main page or login
  window.location.href = "/";
});
```

loadingScript.js

```
// Continuously checks the status of the database creation process
function checkFlag() {

  // Send a request to the backend to check the current database status
  fetch('/check_flag')
    .then(r => r.json())
    .then(data => {

      // If database creation was successful
      if (data.status === "success") {
        // Redirect the user back to the page they were on before loading
        window.location.href = data.previousPage;
      }

      // If database creation failed
      else if (data.status === "failed") {
        // Store the error message in sessionStorage so it can be accessed on the error page
        sessionStorage.setItem("db_error", data.error);

        // Redirect the user to the database error page
        window.location.href = "/db_error";
      }

      // If database creation is still in progress
      else {
        // Wait 2 seconds before checking again
        setTimeout(checkFlag, 2000);
      }
    });
}

// Start checking the database status once the page has fully loaded
window.onload = checkFlag;
```

mainPageScript.js

```
// Ensure the page DOM is fully loaded before running any JavaScript
$(document).ready(function(){

  /* Start Station Dropdown (Select2) */

  // Initialise Select2 for the start station input field
```

```

$("#start").select2({

    // Placeholder text shown before the user selects a station
    placeholder: "Select a start station",

    // Configure AJAX-based searching for station names
    ajax: {
        // Backend endpoint that returns matching station names
        url: "/get_stations",

        // Expected response format
        dataType: "json",

        // Delay to prevent excessive requests while typing
        delay: 250,

        // Send the user's search term to the backend
        data: function (parameters) {
            return { search: parameters.term };
        },

        // Convert the backend response into Select2-compatible results
        processResults: function (data) {
            return { results: data };
        },

        // Cache results to improve performance and reduce server load
        cache: true
    }
});

/* End Station Dropdown (Select2) */

// Initialise Select2 for the end station input field
$("#end").select2({

    // Placeholder text shown before the user selects a station
    placeholder: "Select an end station",

    // Configure AJAX-based searching for station names
    ajax: {
        // Backend endpoint that returns matching station names
        url: "/get_stations",

        // Expected response format
        dataType: "json",

        // Delay to prevent excessive requests while typing
        delay: 250,

        // Send the user's search term to the backend
        data: function (parameters) {
            return { search: parameters.term };
        },

        // Convert the backend response into Select2-compatible results
        processResults: function (data) {
            return { results: data };
        },

        // Cache results to improve performance and reduce server load
        cache: true
    }
}

```

```
});
});
```

updatePage.js

```
document.addEventListener("DOMContentLoaded", () => {

  /* Load Default Values */

  // Select the "Load Default Values" button
  const loadDefaultBtn = document.querySelector("button[name='loadDefault']");
  if (loadDefaultBtn) {
    loadDefaultBtn.addEventListener("click", async () => {

      // Fetch default parameter values from the backend
      const response = await fetch("/load_default_parameters");
      const data = await response.json();

      // Populate all parameter input fields with default values
      document.getElementById("param1").value = data.CHANGING_TIME;
      document.getElementById("param2").value = data.MAX_TIME_WINDOW;
      document.getElementById("param3").value = data.rawArrivalPeakOffset;
      document.getElementById("param4").value = data.rawDeparturePeakOffset;
      document.getElementById("param5").value = data.ATTENDANCE;
      document.getElementById("param6").value = data.TRAIN_PROPORTION;
      document.getElementById("param7").value = data.SIGMA_FACTOR;
      document.getElementById("param8").value = data.MINIMUM_PEOPLE;
      document.getElementById("param9").value = data.PERSON_DELAY;
      document.getElementById("param10").value = data.PROPGATION_FACTOR;
      document.getElementById("param11").value = data.WALKING_TIME;
      document.getElementById("param12").value = data.WALKING_SPEED;
      document.getElementById("param13").value = data.NORMAL;
      document.getElementById("param14").value = data.SLIGHTLY;
      document.getElementById("param15").value = data.BUSY;
    });
  }

  /* Save Parameters */

  // Select the "Save" button
  const saveBtn = document.querySelector("button[name='save']");
  if (saveBtn) {
    saveBtn.addEventListener("click", async () => {

      // Collect all parameter values into a payload object
      const payload = {
        CHANGING_TIME: document.getElementById("param1").value,
        MAX_TIME_WINDOW: document.getElementById("param2").value,
        rawArrivalPeakOffset: document.getElementById("param3").value,
        rawDeparturePeakOffset: document.getElementById("param4").value,
        ATTENDANCE: document.getElementById("param5").value,
        TRAIN_PROPORTION: document.getElementById("param6").value,
        SIGMA_FACTOR: document.getElementById("param7").value,
        MINIMUM_PEOPLE: document.getElementById("param8").value,
        PERSON_DELAY: document.getElementById("param9").value,
        PROPGATION_FACTOR: document.getElementById("param10").value,
        WALKING_TIME: document.getElementById("param11").value,
        WALKING_SPEED: document.getElementById("param12").value,
        NORMAL: document.getElementById("param13").value,
        SLIGHTLY: document.getElementById("param14").value,
```

```

        BUSY: document.getElementById("param15").value
    };

    // Send updated parameters to the backend as JSON
    fetch("/save_parameters", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(payload)
    });

    // Redirect back to the main page after saving
    window.location.href = "/main"
    });
}

/* Log Out Button */

// Select the "Log Out" button
const logoutBtn = document.querySelector("button[name='logout']");
if (logoutBtn) {
    logoutBtn.addEventListener("click", async () => {
        // Notify backend to clear user session
        await fetch("/logout", {
            method: "POST"
        });

        // Redirect user to login page
        window.location.href = "/"; // Redirect to login page
    });
}

/* Fetch New Data Button */

// Select the "Fetch New Data" button
const fetchNewDataBtn = document.querySelector("button[name='new']");
if (fetchNewDataBtn) {
    fetchNewDataBtn.addEventListener("click", async () => {

        // Trigger background data fetch on the server
        const response = await fetch("/fetch_new_data", {
            method: "POST"
        });

        // Redirect if the server responds with a redirect
        if (response.redirected) {
            window.location.href = response.url;
        }
    });
}

/* Numeric only enforcement */
document.querySelectorAll("input[type='number']").forEach(input => {
    // Block e, E, +, -
    input.addEventListener("keydown", e => {
        if (["e", "E", "+", "-"].includes(e.key)) {
            e.preventDefault();
        }
    });
});

```

```
// Clean pasted input
input.addEventListener("input", () => {
  input.value = input.value.replace(/^[^0-9.]/g, "");
});
});
});
```

databaseError.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Character encoding to ensure correct text rendering -->
  <meta charset="UTF-8">

  <!-- Title shown in the browser tab -->
  <title>Database Setup Failed</title>

  <!-- CSS file used to style the database error page -->
  <link rel="stylesheet" href="{{ url_for('static', filename='css/dbError.css') }}">
</head>

<body>
  <!-- Main container for the error message and retry option -->
  <div class="container">

    <!-- Main heading indicating database setup failure -->
    <h1>Database Setup Failed</h1>

    <!-- Paragraph used to display a specific error message (passed from Flask) -->
    <p class="error-message" id="error">{{ errorMessage }}</p>

    <!-- Form containing a retry button to attempt database creation again -->
    <form id="retryForm">
      <button type="submit" class="retry-button">Retry</button>
    </form>

  </div>

  <!-- JavaScript file handling retry logic and error-page behaviour -->
  <script src="{{ url_for('static', filename='js/dbError.js') }}"></script>
</body>
</html>
```

loading.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Character encoding to support standard text rendering -->
  <meta charset="UTF-8">

  <!-- Title displayed in the browser tab -->
  <title>Loading</title>

  <!-- CSS file for styling the loading screen -->
  <link rel="stylesheet" href="{{ url_for('static', filename='css/loading.css') }}">
```

```

    <!-- JavaScript file controlling loading behaviour (e.g. polling, redirects) -->
    <script src="{{ url_for('static', filename='js/loadingScript.js') }}"></script>
</head>

<body>
    <!-- Container for the loading screen content -->
    <div class="loadingContainer">

        <!-- Text displayed while the application is loading -->
        <div class="loadingText">Loading...</div>

    </div>
</body>
</html>

```

login.html

```

<!DOCTYPE html>
<html>
<head>
    <!-- Character encoding for proper text rendering -->
    <meta charset="UTF-8">

    <!-- Title shown in the browser tab -->
    <title>Hi</title>

    <!-- Stylesheet for the login page -->
    <link rel="stylesheet" href="{{ url_for('static', filename='css/loginStyle.css') }}">
</head>

<body class="page">

    <!-- Main page title -->
    <h1 class="title">Welcome</h1>
    <!-- Don't have a name inside of programme -- just say welcome -->

    <!-- Container holding the login form and messages -->
    <div class="container">

        <!-- Login / Sign-up form -->
        <form class="loginForm" method="POST">

            <!-- Username input field -->
            <input
                class="textInput"
                type="text"
                name="username"
                placeholder="Username"
                required
            >

            <!-- Password input field -->
            <input
                class="textInput"
                type="password"
                name="password"
                placeholder="Password"
                required
            >

```

```

<!-- Container for login and sign-up buttons -->
<div class="buttonRow">
  <!-- Login button submits the form with actionName=login -->
  <button class="button" type="submit" name="actionName" value="login">
    Login
  </button>

  <!-- Sign-up button submits the form with actionName=signUp -->
  <button class="button" type="submit" name="actionName" value="signUp">
    Sign Up
  </button>
</div>

</form>

<!-- Displays feedback messages from the server (e.g. login errors) -->
{% if message %}
  <div class="message">{{ message }}</div>
{% endif %}

</div>

</body>
</html>

```

mainPage.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Sets character encoding to support modern text -->
  <meta charset="UTF-8">

  <!-- Page title shown in the browser tab -->
  <title>Route Search</title>

  <!-- Main stylesheet for the route search page -->
  <link rel="stylesheet" href="{{ url_for('static', filename='css/mainPageStyle.css') }}">

  <!-- jQuery library used for DOM manipulation and AJAX -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

  <!-- Select2 CSS for enhanced dropdown styling and searching -->
  <link
    href="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/css/select2.min.css"
    rel="stylesheet"
  />

  <!-- Select2 JavaScript library -->
  <script
    src="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/js/select2.min.js"
  ></script>
</head>

<body class="body">

  <!-- Page title section -->
  <div class="titleSection">
    <h1>Route Search</h1>
  </div>

```

```

<!-- Main container holding input and output sections -->
<div class="mainContainer">

    <!-- User input section -->
    <div class="inputSection">
        <!-- Main form used to submit journey details -->
        <form method="POST" class="theForm">

            <!-- Start station dropdown (populated dynamically using Select2) -->
            <select id="start" name="start" required></select>

            <!-- End station dropdown (populated dynamically using Select2) -->
            <select id="end" name="end" required></select>

            <!-- Container for date and time inputs -->
            <div class="dateTimeContainer">
                <!-- Date input for journey date -->
                <input
                    type="date"
                    id="date"
                    name="date"
                    required
                >

                <!-- Time input for journey start time -->
                <input
                    type="time"
                    id="time"
                    name="time"
                    required
                >
            </div>

            <!-- Submit button to trigger route calculation -->
            <button type="submit" class="submitButton">
                Search Route
            </button>

            <!-- Displays validation or logic errors from Flask -->
            {% if error %}
                <p class="errorMessage">{{ error }}</p>
            {% endif %}

            <!-- Displays which lines serve the selected stations -->
            <div class="stationLinesDisplay">
                {% if startLines %}
                    <!-- Lines serving the start station -->
                    <h3>{{ request.form.start }}</h3>
                    <ul class="startStationList">
                        {% for line in startLines %}
                            <li>{{ line }}</li>
                        {% endfor %}
                    </ul>
                {% endif %}

                {% if endLines %}
                    <!-- Lines serving the end station -->
                    <h3>{{ request.form.end }}</h3>
                    <ul class="endStationList">
                        {% for line in endLines %}
                            <li>{{ line }}</li>
                        {% endfor %}
                    </ul>
                {% endif %}
            </div>
        </form>
    </div>
</div>

```

```

    </div>

</form>
</div>

<!-- Output section for displaying journey results -->
<div class="outputSection">

    <!-- Container for the original fastest path -->
    <div class="pathContainer">
        <h2>Original Fastest Path:</h2>

        {% if formattedOldPath %}
            <!-- Loop through each line segment of the original path -->
            {% for path in formattedOldPath %}
                <div class="path">
                    <!-- Line name heading -->
                    <h6 class="lineName">{{ path.LineName }}</h6>

                    <!-- List of stations with crowding descriptions -->
                    <ul>
                        {% for station in path.ListOfStationNamesAndDescriptions %}
                            <li>{{ station[0] }} - {{ station[2] }}</li>
                        {% endfor %}
                    </ul>
                </div>
            {% endfor %}
        {% endif %}

        <!-- Displays total journey time and arrival time -->
        <div class="timeContainer">
            <p>Total Time: {{ oldPathTime }}</p>
            <p>Arrival Time: {{ oldPathArrival }}</p>
        </div>
    </div>

    <!-- Container for the recommended (crowding-aware) path -->
    <div class="pathContainer">
        <h2>Our Recommendation:</h2>

        {% if formattedNewPath %}
            <!-- Loop through each line segment of the recommended path -->
            {% for path in formattedNewPath %}
                <div class="path">
                    <!-- Line name heading -->
                    <h6 class="lineName">{{ path.LineName }}</h6>

                    <!-- List of stations with crowding descriptions -->
                    <ul>
                        {% for station in path.ListOfStationNamesAndDescriptions %}
                            <li>{{ station[0] }} - {{ station[2] }}</li>
                        {% endfor %}
                    </ul>
                </div>
            {% endfor %}
        {% endif %}

        <!-- Displays total journey time and arrival time -->
        <div class="timeContainer">
            <p>Total Time: {{ newPathTime }}</p>
            <p>Arrival Time: {{ newPathArrival }}</p>
        </div>
    </div>
</div>

```

```

</div>

<!-- Button to navigate to the parameter update page -->
<form method="POST">
  <button type="submit" class="updateButton" name="updateButton">Update</button>
</form>

<!-- Page-specific JavaScript for dropdowns and dynamic behaviour -->
<script src="{{ url_for('static', filename='js/mainPageScript.js') }}"></script>
</body>
</html>

```

updatePage.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Character encoding to ensure correct rendering of text -->
  <meta charset="UTF-8">

  <!-- Title shown in the browser tab -->
  <title>Update Page</title>

  <!-- CSS file used to style the update/parameter page -->
  <link rel="stylesheet" href="{{ url_for('static', filename='css/update.css') }}">

  <!-- JavaScript file handling button actions and parameter updates -->
  <script src="{{ url_for('static', filename='js/updatePage.js') }}"></script>
</head>

<body>

  <!-- Main outer container for page layout -->
  <div class="mainContainer">

    <!-- Container specifically for the update page content -->
    <div class="updatePageContainer">

      <!-- Form used to update user-specific parameters -->
      <form method="POST" class="updateForm">

        <!-- Table wrapper for displaying editable parameters -->
        <div class="parameterTable">
          <table>

            <!-- Time penalty applied when changing trains -->
            <tr>
              <td><label for="param1">Time taken to change trains (minutes):</label></td>
              <td><input type="number" id="param1" name="param1" value="{{ CHANGING_TIME }}" required></td>
            </tr>

            <!-- Maximum time window in which an event can affect a journey -->
            <tr>
              <td><label for="param2">Length of effect of event (minutes):</label></td>
              <td><input type="number" id="param2" name="param2" value="{{ MAX_TIME_WINDOW }}" required></td>
            </tr>

            <!-- Offset before event start where arrival peaks usually occur -->
            <tr>
              <td><label for="param3">Peak offset before the game (minutes):</label></td>

```

```

        <td><input type="number" id="param3" name="param3" value="{{ rawArrivalPeakOffset }}" required></td>
    </tr>

    <!-- Offset after event end where departure peaks usually occur -->
    <tr>
        <td><label for="param4">Peak offset after the game (minutes):</label></td>
        <td><input type="number" id="param4" name="param4" value="{{ rawDeparturePeakOffset }}"
required></td>
    </tr>

    <!-- Expected attendance as a proportion of venue capacity -->
    <tr>
        <td><label for="param5">Match attendance (%):</label></td>
        <td><input type="number" id="param5" name="param5" value="{{ ATTENDANCE }}" required></td>
    </tr>

    <!-- Proportion of attendees expected to use TfL rail services -->
    <tr>
        <td><label for="param6">Proportion of people using TfL rail (%):</label></td>
        <td><input type="number" id="param6" name="param6" value="{{ TRAIN_PROPORTION }}" required></td>
    </tr>

    <!-- Sigma scaling factor used in Gaussian crowding calculation -->
    <tr>
        <td><label for="param7">Sigma factor which controls how tightly people cluster around the peak
time:</label></td>
        <td><input type="number" id="param7" name="param7" value="{{ SIGMA_FACTOR }}" required></td>
    </tr>

    <!-- Minimum number of people required for crowding to propagate -->
    <tr>
        <td><label for="param8">Minimum people needed for effect:</label></td>
        <td><input type="number" id="param8" name="param8" value="{{ MINIMUM_PEOPLE }}" required></td>
    </tr>

    <!-- Delay contribution added per person at a station -->
    <tr>
        <td><label for="param9">Delay per person (minutes):</label></td>
        <td><input type="number" id="param9" name="param9" value="{{ PERSON_DELAY }}" required></td>
    </tr>

    <!-- Proportion of people retained when propagating to adjacent stations -->
    <tr>
        <td><label for="param10">People dropped off per hop (%):</label></td>
        <td><input type="number" id="param10" name="param10" value="{{ PROPAGATION_FACTOR }}"
required></td>
    </tr>

    <!-- Average walking time between venues and stations -->
    <tr>
        <td><label for="param11">Average walking distance walking time (minutes):</label></td>
        <td><input type="number" id="param11" name="param11" value="{{ WALKING_TIME }}" required></td>
    </tr>

    <!-- Average walking speed used for distance calculations -->
    <tr>
        <td><label for="param12">Average walking speed (km h-1):</label></td>
        <td><input type="number" id="param12" name="param12" value="{{ WALKING_SPEED }}" required></td>
    </tr>

    <!-- Upper threshold for stations considered to have normal crowding -->
    <tr>
        <td><label for="param13">Max crowding factor threshold for normal crowding:</label></td>
        <td><input type="number" id="param13" name="param13" value="{{ NORMAL }}" required></td>
    </tr>

```

```

    </tr>

    <!-- Upper threshold for stations considered slightly crowded -->
    <tr>
      <td><label for="param14">Max crowding factor threshold for slight levels of crowding:</label></td>
      <td><input type="number" id="param14" name="param14" value="{{ SLIGHTLY }}" required></td>
    </tr>

    <!-- Upper threshold for stations considered busy -->
    <tr>
      <td><label for="param15">Max crowding factor threshold for busy:</label></td>
      <td><input type="number" id="param15" name="param15" value="{{ BUSY }}" required></td>
    </tr>

  </table>
</div>

<!-- Section containing action buttons -->
<div class="buttonSection">
  <div class="buttonGroup">

    <!-- Loads default parameter values from file -->
    <button type="button" class="actionButton" name="loadDefault">
      Load Default Values
    </button>

    <!-- Saves current parameter values to the database -->
    <button type="button" class="actionButton" name="save">
      Save
    </button>

    <!-- Triggers background fetch of new external data -->
    <button type="button" class="actionButton" name="new">
      Fetch New Data
    </button>

    <!-- Logs the user out of the application -->
    <button type="button" class="actionButton" name="logOut">
      Log Out
    </button>

  </div>
</div>

</form>

</div>

</div>

</body>
</html>

```

defaultParameters.json

```

{
  "CHANGING_TIME" : 4.4,
  "MAX_TIME_WINDOW" : 120,
  "rawArrivalPeakOffset" : 25,
  "rawDeparturePeakOffset" : 25,
  "ATTENDANCE" : 0.95,

```

```

"TRAIN_PROPORTION" : 0.75,
"SIGMA_FACTOR" : 4,
"MINIMUM_PEOPLE" : 50,
"PERSON_DELAY" : 0.002,
"PROPAGATION_FACTOR" : 0.5,
"WALKING_TIME" : 15,
"WALKING_SPEED" : 5,
"NORMAL" : 1,
"SLIGHTLY" : 1.3,
"BUSY" : 2
}

```

leagues.json

```

{
  "Premier League": "premier-league",
  "Championship": "championship",
  "League One": "league-one",
  "League Two": "league-two"
}

```

lines.json

```

{
  "bakerloo": ["Bakerloo Line", 27.04],
  "central": ["Central Line", 37.27],
  "circle": ["Circle Line", 23.73],
  "district": ["District Line", 29.19],
  "hammersmith-city": ["Hammersmith and City Line", 25.33],
  "jubilee": ["Jubilee Line", 39.11],
  "metropolitan": ["Metropolitan Line", 45.61],
  "northern": ["Northern Line", 33.28],
  "piccadilly": ["Piccadilly Line", 33.11],
  "victoria": ["Victoria Line", 40.66],
  "waterloo-city": ["Waterloo and City Line", 33.3],
  "dlr": ["DLR", 72.00],
  "elizabeth": ["Elizabeth Line", 51.70],
  "liberty": ["Liberty Line", 36.53],
  "lioness": ["Lioness Line", 33.80],
  "mildmay": ["Mildmay Line", 26.07],
  "windrush": ["Windrush Line", 27.36],
  "weaver": ["Weaver Line", 32.19],
  "suffragette": ["Suffragette Line", 32.19]
}

```

londonClubs.json

```

{
  "Arsenal" : ["Emirates Stadium", 60704],
  "Brentford" : ["Brentford Community Stadium", 18250],
  "Chelsea" : ["Stamford Bridge", 40343],
  "Crystal Palace" : ["Selhurst Park", 25486],
  "Fulham" : ["Craven Cottage", 29600],
  "Tottenham Hotspur" : ["Tottenham Hotspur Stadium", 62850],
  "West Ham United" : ["London Stadium", 62500],
}

```

```

"Charlton Athletic" : ["The Valley, London", 2711],
"Millwall" : ["The Den", 20146],
"Queens Park Rangers" : ["Loftus Road Stadium", 18439],
"AFC Wimbledon" : ["Plough Lane Stadium", 9300],
"Leyton Orient" : ["Brisbane Road Stadium", 9271],
"Barnet" : ["The Hive Stadium", 5100],
"Bromley" : ["Hayes Lane Stadium", 5150]
}

```

accountClasses.py

```

import json

#This class is effectively a container for all the parameters being used, unique to each user, for the calculation of the
journeys
class Parameters():
    def __init__(
        self,
        CHANGING_TIME, # Extra time added to each journey (minutes)
        MAX_TIME_WINDOW, # Maximum time from the event peak to consider crowding (minutes)
        rawArrivalPeakOffset, # Minutes from event start when crowding peaks on arrival
        rawDeparturePeakOffset, # Minutes from event end when crowding peaks on departure
        ATTENDANCE, # Proportion of stadium capacity attending (0-1)
        TRAIN_PROPORTION, # Proportion using TfL rail services (0-1)
        SIGMA_FACTOR, # Controls how tightly people cluster around the peak time — it converts a time window into a
Gaussian spread
        MINIMUM_PEOPLE, # Minimum people threshold for affecting stations
        PERSON_DELAY, # Additional delay per person on journey
        PROPAGATION_FACTOR, # Factor for spreading delay across network
        WALKING_TIME, # Default walking time between station and venue (minutes)
        WALKING_SPEED, # Default walking speed (m/s)
        NORMAL, # Crowding thresholds for classification: normal (below this is normal)
        SLIGHTLY, # Crowding thresholds for classification: slightly busy (below this is slightly busy)
        BUSY # Crowding thresholds for classification: busy (below this is busy and above is severely busy)
    ):

        # Store all parameters in the object
        self.CHANGING_TIME = CHANGING_TIME
        self.MAX_TIME_WINDOW = MAX_TIME_WINDOW
        self.rawArrivalPeakOffset = rawArrivalPeakOffset
        self.rawDeparturePeakOffset = rawDeparturePeakOffset
        self.ATTENDANCE = ATTENDANCE
        self.TRAIN_PROPORTION = TRAIN_PROPORTION
        self.SIGMA_FACTOR = SIGMA_FACTOR
        self.MINIMUM_PEOPLE = MINIMUM_PEOPLE
        self.PERSON_DELAY = PERSON_DELAY
        self.PROPAGATION_FACTOR = PROPAGATION_FACTOR
        self.WALKING_TIME = WALKING_TIME
        self.WALKING_SPEED = WALKING_SPEED
        self.NORMAL = NORMAL
        self.SLIGHTLY = SLIGHTLY
        self.BUSY = BUSY

        # Internal attributes for arrival/departure calculations
        self.arrivalWindowMinutes = None
        self.departureWindowMinutes = None
        self.arrivalPeakOffset = None
        self.departurePeakOffset = None
        self.arrivalMinutesOffset = None
        self.departureMinutesOffset = None

```

```

# Sets the arrival and departure window minutes to MAX_TIME_WINDOW
# Both windows use the same value - the programme is written to allow them to differ in future if needed
def calculateWindow(self):
    self.arrivalWindowMinutes = self.MAX_TIME_WINDOW
    self.departureWindowMinutes = self.MAX_TIME_WINDOW

# Converts raw peak offsets stored in minutes to timedelta objects for use in datetime arithmetic
def calculatePeakOffset(self):
    import datetime
    self.arrivalPeakOffset = datetime.timedelta(minutes=self.rawArrivalPeakOffset)
    self.departurePeakOffset = datetime.timedelta(minutes=self.rawDeparturePeakOffset)

# Converts the peak offsets to numeric minutes for simpler calculations
def calculateMinutesOffset(self):
    self.arrivalMinutesOffset = self.arrivalPeakOffset.total_seconds() / 60
    self.departureMinutesOffset = self.departurePeakOffset.total_seconds() / 60

# This subclass is used when creating a new account
# It loads default parameters from a JSON file so each user starts with reasonable defaults
class AccountCreationParameters(Parameters):
    def __init__(self, defaultParameters):
        # Initialise with placeholder zeros; these will be overwritten by JSON
        super().__init__(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)

        # Load default parameters from JSON configuration file
        with open(defaultParameters, 'r') as file:
            rawData = json.load(file)

        # Dynamically assign attributes from JSON keys
        for key, value in rawData.items():
            setattr(self, key, value)

```

classes.py

```

from transportClasses import *
from scraperClasses import *
from accountClasses import *
from networkClasses import *
from websiteDataClasses import *
#File exists because originally all the classes were in this file, before being split up

```

crowdingAlgorithm.py

```

from pathFinding import MakeGraph, Dijkstra
import datetime
import sqlite3
import math
import classes
import customFunctions

#Packages journey input data into a dictionary for use by downstream functions
def takelnput(date, journeyStart, startStation, endStation):
    data = {

```

```

    "date": date,
    "journeyStart": journeyStart,
    "startStation": startStation,
    "endStation": endStation
}

return data

#Returns the approximate time of the middle of the journey (without consideration of delays and things)
def calculateDefaultMidjourneyTime(baselInputData, databaseFile, CHANGING_TIME):
    journeyStart = baselInputData["journeyStart"]
    startStation = baselInputData["startStation"]
    endStation = baselInputData["endStation"]

    graph = MakeGraph(databaseFile)
    # Run Dijkstra to get the shortest path steps from start to end and the time for the journey (if there was no crowding)
    steps = Dijkstra(graph, startStation, endStation, CHANGING_TIME)
    totalMinutes = steps[-1][3] #steps is in the form [[PreviousStationNaPTAN, NextStationNaPTAN, lineID,
    LengthOfTimeToPoint],...]
    halfMinutes = totalMinutes // 2

    # Convert journey start to datetime object
    timeObject = datetime.datetime.strptime(journeyStart, "%H:%M")

    # Add half journey duration to get midpoint
    timeDelta = datetime.timedelta(minutes=halfMinutes)
    timeObject += timeDelta

    return timeObject

# Returns events that might impact the journey based on a given timeframe
# Takes into account the peak offsets for arrivals and departures
# Returns dictionary of events: {(venue, date): [minuteDifference, startDiff, endDiff]}
def findEventsInTimeFrame(midjourneyTime, databaseFile, baselInputData, MAX_TIME_WINDOW, arrivalPeakOffset,
departurePeakOffset):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    date = baselInputData["date"]
    importantEvents = {} # (venue, date) : minuteDifference - The events which might affect the journey

    #Fetches all the events in that day
    eventsInDay = cursor.execute("""
        SELECT *
        FROM Events
        WHERE Date = ?
        """, (date,)).fetchall()

    #Event is in the form [VenueName, Date, Time, HomeTeam, AwayTeam, EventName, Duration]
    for event in eventsInDay:
        duration = event[6]
        eventTime = datetime.datetime.strptime(event[2], "%H:%M")
        endTime = eventTime + datetime.timedelta(minutes=duration)

        # Calculate difference from journey midpoint to arrival and departure peaks
        startTimeDifference = abs(eventTime - arrivalPeakOffset - midjourneyTime) #Peaks are unlikely to be immediately at
        the start or end of an event
        endTimeDifference = abs((endTime + departurePeakOffset) - midjourneyTime)

        # Choose the smaller of the two differences for maximum impact (the closer events are in time, the bigger their impacts
        are)
        if startTimeDifference > endTimeDifference:
            timeDifference = endTimeDifference

```

```

else:
    timeDifference = startTimeDifference
    minutesDifference = timeDifference.total_seconds() // 60

    # Only consider events within the MAX_TIME_WINDOW
    if minutesDifference <= MAX_TIME_WINDOW:
        venue = event[0]
        startDifferenceMinutes = startTimeDifference.total_seconds() // 60
        endDifferenceMinutes = endTimeDifference.total_seconds() // 60
        importantEvents[(venue, date)] = [minutesDifference, startDifferenceMinutes, endDifferenceMinutes]

connection.close()
return importantEvents

# This function assigns each station a weight so that the number of people at the stations will be more accurately assigned
# stationDistances dictionary format - {NaPTAN: distance}
# Returns dictionary format - {NaPTAN: proportion}
def inverseWeight(stationDistances):
    extra = 1 # Avoid divide by zero errors - if the distance between the station and the venue is 0
    weights = {}
    proportions = {}
    totalWeight = 0

    # Calculate inverse-distance weight for each station
    for station in stationDistances.keys():
        distance = stationDistances[station]
        weight = 1 / (distance + extra)
        totalWeight += weight
        weights[station] = weight

    # Convert weights to proportions of total
    for station in weights.keys():
        proportion = weights[station] / totalWeight
        proportions[station] = proportion

    return proportions

# Fetches stations near a venue and their distance from the venue
# Returns dictionary {NaPTAN: distance}
def returnStationsWithDistances(VenueName, databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    stationsWithDistances = {}

    cursor.execute("""
        SELECT NaPTAN, Distance
        FROM VenueStationRelationships
        WHERE VenueName = ?
    """, (VenueName,))
    rows = cursor.fetchall()

    for stationTuple in rows:
        stationsWithDistances[stationTuple[0]] = stationTuple[1]

    connection.close()
    return stationsWithDistances

# Gaussian factor at peak for crowding
# relativePeak = expected peak
# relativeTimeDifference = time offset from peak
# sigma = standard deviation for the Gaussian curve

```

```

# Returns a value between 0 and 1 representing relative crowding
def gaussianFactorAtPeak(relativePeak, sigma, relativeTimeDifference):
    x = (relativePeak - relativeTimeDifference) / sigma
    returnFactor = math.exp(-0.5 * x * x)
    return returnFactor

#For each event, finds the base number of people at the nearby stations at the middle of the journey
# Returns dictionary {NaPTAN: [arrivalPeople, departurePeople]}
def findBaseNumberOfPeople(
    capacity,
    ATTENDANCE, #The proportion of seats that will actually be filled
    TRAIN_PROPORTION, #The proportion of people using TfL rail
    startDifferenceMinutes, #minutes from journey midpoint to event start
    endDifferenceMinutes, #minutes from journey midpoint to event end
    arrivalWindowMinutes, #Length of time of arrival window where the journey may be affected
    departureWindowMinutes, #Length of time of departure window where the journey may be affected
    arrivalPeakOffset, #Offset because peaks are usually not at exactly the start or end
    departurePeakOffset,
    stationsWithDistances, #Dictionary of stations with the values being the distance between the station and the station
    and the venue
    SIGMA_FACTOR
):

    totalAttendance = capacity * ATTENDANCE #Gets the expected attendance at the venue
    tfUsageTotal = totalAttendance * TRAIN_PROPORTION #Gets the expected total usage of TfL rail

    # Convert distances into proportional distribution of passengers
    proportions = inverseWeight(stationsWithDistances)
    StationsWithPeopleAtPeak = {}
    for stationID in proportions.keys():
        StationsWithPeopleAtPeak[stationID] = proportions[stationID] * tfUsageTotal

    # Adjust for peak offsets
    relativeArrivalPeakTime = startDifferenceMinutes - arrivalPeakOffset
    relativeDepartPeakTime = endDifferenceMinutes + departurePeakOffset

    sigmaArrival = max(1, arrivalWindowMinutes / SIGMA_FACTOR) #max used to make sure sigma is at least bigger or equal
    to 1
    sigmaDeparture = max(1, departureWindowMinutes / SIGMA_FACTOR)

    # Calculates how much of the peak crowd is present at the journey midpoint for both arrival and departure peaks
    # The larger of the two is then used per station, as the journey is most likely to coincide with one peak at a time

    arrivalsFactorAtMid = gaussianFactorAtPeak(relativeArrivalPeakTime, sigmaArrival, 0)
    depratureFactorAtMid = gaussianFactorAtPeak(relativeDepartPeakTime, sigmaDeparture, 0)

    # Compute expected arrivals and departures at each station
    stationsAtMid = {} #Format - {NaPTAN: ArrivalNumbers, DepartureNumbers}
    for stationID in StationsWithPeopleAtPeak.keys():
        arrivalPeople = StationsWithPeopleAtPeak[stationID] * arrivalsFactorAtMid
        departurePeople = StationsWithPeopleAtPeak[stationID] * depratureFactorAtMid
        stationsAtMid[stationID] = [arrivalPeople, departurePeople]

    return stationsAtMid

def findCapacity(databaseFile, VenueName):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("""
        SELECT Capacity
        FROM Venues
        Where VenueName = ?
    """)

```

```

        """', (VenueName,))
        capacity = cursor.fetchall()[0][0]
        connection.close()
        return capacity

# Propagates people counts recursively through the network
# Updates affected stations using AffectedDijkstraStation objects
def propagation(graph, stationID, people, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR):
    stationObject = network.nodes[stationID] #Fetches the Dijkstra station object stored in the network

    # If station is not yet affected, create a new affected object
    if stationObject.IsAffected() == False:
        NaPTAN = stationID
        StationName = stationObject.StationName
        #Above lines fetch the information stored in the old object
        newStationObject = classes.AffectedDijkstraStation(NaPTAN, StationName) #Creates a new affected object
        newStationObject.events[event] = people #Adds the number of people at this station because of this event
        stationObject = newStationObject
        network.nodes[stationID] = stationObject #Replaces old object

    # Check previous people count for this event
    try:
        oldPeople = stationObject.events[event] #This will work If and only if the station was already affected by this event
    except:
        oldPeople = 0 #This will run if the station has been already affected but not by this event so the event can't be found in
        # the object, so the old population will be set as 0

    if oldPeople > people:
        pass #No need to update because a larger wave of people have passed through already. Propagation will not be
        # needed because a larger wave existed which has been propagated already
    else:
        stationObject.events[event] = people
        network.nodes[stationID] = stationObject #Updates the people at this station due to this event at this station
        newPeople = people * PROPEGATION_FACTOR

    # Stop recursion if propagated population is too small
    if newPeople < MINIMUM_PEOPLE:
        pass
    else:
        connections = graph[stationID]
        for connection in connections:
            newStationID = connection[0]
            propagation(graph, newStationID, newPeople, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)

#Creates a parameter object to store the parameters used for calculation
def getParameters(username, databaseFile):
    parameterObject = customFunctions.getParameters(username, databaseFile)
    return parameterObject

# Creates NotAffectedStation and NotAffectedConnection objects which are then stored in the dictionaries
# All this is stored and contained in a network object
def createNetwork(baseGraph, databaseFile, stations, connections, network):
    sqlConnection = sqlite3.connect(databaseFile)
    cursor = sqlConnection.cursor()

    # Create station objects
    for StationA in baseGraph.keys():
        cursor.execute("""
            SELECT StationName
            FROM Stations
            WHERE NaPTAN = ?

```

```

        """', (StationA,))
    row = cursor.fetchall()
    StationName = row[0][0]

    NaPTAN = StationA
    baseStationClass = classes.NotAffectedDijkstraStation(NaPTAN, StationName)
    stations[StationA] = baseStationClass

    # Create connection objects
    for connection in baseGraph[StationA]:
        StationA = StationA
        StationB = connection[0]
        BaseTravelTime = connection[1]
        LineID = connection[2]
        baseConnectionClass = classes.NotAffectedConnection(StationA, StationB, LineID, BaseTravelTime)
        key = (StationA, StationB, LineID)
        connections[key] = baseConnectionClass

    sqlConnection.close()

    network.nodes = stations
    network.edges = connections

#Starts off propagation of populations from each close station - then calls propegation() which then runs recursively
def processEvents(events, databaseFile, parameterObject, baseGraph, network):
    ATTENDANCE = parameterObject.ATTENDANCE
    TRAIN_PROPORTION = parameterObject.TRAIN_PROPORTION
    arrivalWindowMinutes = parameterObject.arrivalWindowMinutes
    departureWindowMinutes = parameterObject.departureWindowMinutes
    arrivalMinutesOffset = parameterObject.arrivalMinutesOffset
    departureMinutesOffset = parameterObject.departureMinutesOffset
    SIGMA_FACTOR = parameterObject.SIGMA_FACTOR
    MINIMUM_PEOPLE = parameterObject.MINIMUM_PEOPLE
    PROPEGATION_FACTOR = parameterObject.PROPEGATION_FACTOR

    for event in events.keys():
        venue = event[0]
        timings = events[event]
        startDifferenceMinutes = timings[1]
        endDifferenceMinutes = timings[2]
        stationsWithDistances = returnStationsWithDistances(venue, databaseFile)
        capacity = findCapacity(databaseFile, venue)
        baseNumberOfPeopleAtStations = findBaseNumberOfPeople(
            capacity,
            ATTENDANCE,
            TRAIN_PROPORTION,
            startDifferenceMinutes,
            endDifferenceMinutes,
            arrivalWindowMinutes,
            departureWindowMinutes,
            arrivalMinutesOffset,
            departureMinutesOffset,
            stationsWithDistances,
            SIGMA_FACTOR
        )

    # Propagate people through the network recursively
    for station in baseNumberOfPeopleAtStations.keys():
        arrivals = baseNumberOfPeopleAtStations[station][0]
        departures = baseNumberOfPeopleAtStations[station][1]
        people = max(arrivals, departures)
        if people >= MINIMUM_PEOPLE:
            propegation(baseGraph, station, people, MINIMUM_PEOPLE, network, event, PROPEGATION_FACTOR)

```

```

#Calls methods to calculate the delay of each station
def processAffectedStations(baseGraph, parameterObject, network):
    PERSON_DELAY = parameterObject.PERSON_DELAY
    for StationID in baseGraph.keys():
        stationObject = network.nodes[StationID]
        if stationObject.IsAffected():
            stationObject.calculatePassengerLoad()
            stationObject.calculateDelay(PERSON_DELAY)

#Calls methods to calculate the delay of each connection
def processAffectedConnections(network):
    connectionObjects = network.edges
    for edgeReference in connectionObjects.keys():
        edge = connectionObjects[edgeReference]
        StationA = edge.StationA
        StationB = edge.StationB
        LineID = edge.LineID
        BaseTravelTime = edge.BaseTravelTime
        StationADelay = network.nodes[StationA].DelayFactor
        StationBDelay = network.nodes[StationB].DelayFactor
        if StationADelay or StationBDelay != 1:
            newConnectionObject = classes.AffectedConnection(StationA, StationB, LineID, BaseTravelTime, StationADelay,
            StationBDelay)
            newConnectionObject.returnDelay()
            network.edges[(StationA, StationB, LineID)] = newConnectionObject

#Delay of each station in the network is calculated, and then the delay of each connection is calculated
def processAffected(baseGraph, parameterObject, network):
    processAffectedStations(baseGraph, parameterObject, network)
    processAffectedConnections(network)

# Main function to compute crowding factors for a journey
# Builds the network, processes events, and applies delays
# Returns network object with updated delays
def getCrowdingFactor(username, databaseFile, date, journeyStart, startStation, endStation):
    stations = {}
    connections = {}

    parameterObject = getParameters(username, databaseFile)
    baseGraph = MakeGraph(databaseFile)
    network = classes.Network()

    createNetwork(baseGraph, databaseFile, stations, connections, network) #Creates NotAffectedStation and
    NotAffectedConnection objects which are then stored in the dictionaries which are then stored in the network object
    baseInputData = takeInput(date, journeyStart, startStation, endStation)
    midTimeObject = calculateDefaultMidjourneyTime(
        baseInputData,
        databaseFile,
        parameterObject.CHANGING_TIME)
    events = findEventsInTimeFrame(
        midTimeObject,
        databaseFile,
        baseInputData,
        parameterObject.MAX_TIME_WINDOW,
        parameterObject.arrivalPeakOffset,
        parameterObject.departurePeakOffset)
    processEvents(events, databaseFile, parameterObject, baseGraph, network)
    processAffected(baseGraph, parameterObject, network)

```

```

#This results in a network object
#The network object has attributes edges and stations
#Stations contain a type of dijkstraStation objects which store delays
#Edges are connections which also store delays
#Station objects within network.nodes are replaced with new affected objects, and their attributes are updated in place,
so the changes persist without needing to return the network

return network

# Adjusts delays across lines to make them consistent along a line
def spreadDelay(network, databaseFile):
    listOfLines = {}
    listOfLinesWithConnectionObjects = {}

    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    #Fetches a list of lines
    cursor.execute("""
        SELECT LineID
        FROM Lines""")
    rows = cursor.fetchall()
    lineIDs = []
    for row in rows:
        lineIDs.append(row[0])
    connection.close()

    #Edits the dictionary so that the keys are the lineIDs
    for lineID in lineIDs:
        listOfLines[lineID] = []

    #Gets all the connections using a line and stores that connection object (memory reference) in the dictionary
    for edgeKey in network.edges.keys():
        lineID = edgeKey[2] #Contains the lineID of the connection
        listOfLines[lineID].append(network.edges[edgeKey])

    #Creates a line object for each line which would contain the list of connection objects belonging to that line, so that
    methods may be called
    for lineID in listOfLines:
        lineObject = classes.Line(listOfLines[lineID], lineID)
        listOfLinesWithConnectionObjects[lineID] = lineObject

    #Calls the methods to calculate the average delay across the entire line, so that the delay is consistent across the line
    (delays tend to affect the entire line as opposed to being local to a station or few)
    for lineObject in listOfLinesWithConnectionObjects.values():
        lineObject.doAllCalculations()

    #Nothing needs to be returned as the objects themselves are not being changed to different objects, only their attributes
    are being changed and therefore their memory reference is the same

# Main for processing a journey
# Returns network object with stations and connections updated with delays
def main(date, journeyStart, startStation, endStation, databaseFile, username):
    affectedNetwork = getCrowdingFactor(username, databaseFile, date, journeyStart, startStation, endStation)
    spreadDelay(affectedNetwork, databaseFile) #For line, the connections belonging to that line will have the same delay
    stored in the LineDelay attribute (local delay is stored in DelayFactor)

    #For testing purposes
    for edgeKey in affectedNetwork.edges.keys():
        connection = affectedNetwork.edges[edgeKey]
        #print(connection.connectionReference, connection.DelayFactor, connection.LineDelay)
    pass

```

```

# For testing purposes
for nodeKey in affectedNetwork.nodes.keys():
    node = affectedNetwork.nodes[nodeKey]
    #print(node.StationName, node.NaPTAN, node.DelayFactor)
    pass

return affectedNetwork

#For testing purposes
if __name__ == "__main__":
    date = "03/12/2025"
    journeyStart = "18:00"
    startStation = "940GZZLULYS"
    endStation = "HUBSRA"
    databaseFile = "final.db"
    username = "Junaet"
    main(date, journeyStart, startStation, endStation, databaseFile, username)

```

customFunctions.py

```

import requests
import time
import sqlite3
import classes

# Performs a safe HTTP GET request with retry logic
# Handles rate limiting, server errors, and lack of internet connection
def safeGet(url, params=None):
    waitTime = 65 # Wait time in seconds when rate-limited or server error occurs
    attempts = 3 # Maximum number of retry attempts
    for attempt in range(attempts):
        try:
            response = requests.get(url, params=params, timeout=5)

            # Handle API rate limiting
            if response.status_code == 429:
                print(f"Rate limit reached: Attempt {attempt + 1} of {attempts}")
                time.sleep(waitTime)
                continue

            # Handle server-side errors
            if response.status_code >= 500 and response.status_code < 600:
                print(f"Server error: Attempt {attempt + 1} of {attempts}")
                time.sleep(waitTime)
                continue

            # Successful or client-side response returned
            return response

        except:
            # Covers timeouts, connection errors, etc.
            pass

    # If all attempts fail, return the status code if available
    try:
        return response.status_code
    except:
        # Occurs if no response was ever received (e.g. no internet)
        return 0

```

```

def getParameters(username, databaseFile):
    # Creates a parameter object to store the parameters used for calculation
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.execute("""
    SELECT
        CHANGING_TIME,
        MAX_TIME_WINDOW,
        rawArrivalPeakOffset,
        rawDeparturePeakOffset,
        ATTENDANCE,
        TRAIN_PROPORTION,
        SIGMA_FACTOR,
        MINIMUM_PEOPLE,
        PERSON_DELAY,
        PROPAGATION_FACTOR,
        WALKING_TIME,
        WALKING_SPEED,
        NORMAL,
        SLIGHTLY,
        BUSY
    FROM Users
    WHERE Username = ?
    """, (username,))

    result = cursor.fetchall()[0]
    connection.close()

    (
        CHANGING_TIME,
        MAX_TIME_WINDOW,
        rawArrivalPeakOffset,
        rawDeparturePeakOffset,
        ATTENDANCE,
        TRAIN_PROPORTION,
        SIGMA_FACTOR,
        MINIMUM_PEOPLE,
        PERSON_DELAY,
        PROPAGATION_FACTOR,
        WALKING_TIME,
        WALKING_SPEED,
        NORMAL,
        SLIGHTLY,
        BUSY
    ) = result

    # Creates a Parameters object which acts as a container for all assumptions
    parameterObject = classes.Parameters(
        CHANGING_TIME,
        MAX_TIME_WINDOW,
        rawArrivalPeakOffset,
        rawDeparturePeakOffset,
        ATTENDANCE,
        TRAIN_PROPORTION,
        SIGMA_FACTOR,
        MINIMUM_PEOPLE,
        PERSON_DELAY,
        PROPAGATION_FACTOR,
        WALKING_TIME,
        WALKING_SPEED,
        NORMAL,

```

```

    SLIGHTLY,
    BUSY
)

# Compute values using the data just stored in the object, to be used later in the programme
parameterObject.calculateWindow()
parameterObject.calculatePeakOffset()
parameterObject.calculateMinutesOffset()

return parameterObject

```

databaseCreation.py

```

import sqlite3
import json
import getTfLData
import storeStadiumData
import scraper

# Creates the SQLite database and all required tables
# If tables already exist, they will not be recreated
def createDatabase(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.executescript("""

CREATE TABLE IF NOT EXISTS Stations (
    NaPTAN TEXT PRIMARY KEY,
    StationName TEXT,
    Latitude REAL,
    Longitude REAL
);

CREATE TABLE IF NOT EXISTS Lines (
    LineID TEXT PRIMARY KEY,
    LineName TEXT,
    AverageSpeed TEXT
);

CREATE TABLE IF NOT EXISTS StationLineRelationships (
    NaPTAN TEXT,
    LineID TEXT,
    PRIMARY KEY (NaPTAN, LineID),
    FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);

CREATE TABLE IF NOT EXISTS Connections (
    StationA TEXT,
    StationB TEXT,
    LineID TEXT,
    BaseTravelTime REAL,
    PRIMARY KEY (StationA, StationB, LineID),
    FOREIGN KEY (StationA) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (StationB) REFERENCES Stations(NaPTAN),
    FOREIGN KEY (LineID) REFERENCES Lines(LineID)
);

CREATE TABLE IF NOT EXISTS Venues (

```

```

VenueName TEXT PRIMARY KEY,
Capacity INTEGER,
Latitude REAL,
Longitude REAL
);

CREATE TABLE IF NOT EXISTS VenueStationRelationships (
    VenueName TEXT,
    NaPTAN TEXT,
    Distance REAL,
    PRIMARY KEY (VenueName, NaPTAN),
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
    FOREIGN KEY (NaPTAN) REFERENCES Stations(NaPTAN)
);

CREATE TABLE IF NOT EXISTS Events (
    VenueName TEXT,
    Date TEXT,
    Time TEXT,
    HomeTeam TEXT,
    AwayTeam TEXT,
    EventName TEXT,
    Duration INTEGER,
    PRIMARY KEY (VenueName, Date),
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName),
    FOREIGN KEY (HomeTeam) REFERENCES Teams(TeamName)
);

CREATE TABLE IF NOT EXISTS Users (
    Username TEXT PRIMARY KEY,
    PasswordHash TEXT,
    Salt TEXT,
    CHANGING_TIME REAL,
    MAX_TIME_WINDOW REAL,
    rawArrivalPeakOffset REAL,
    rawDeparturePeakOffset REAL,
    ATTENDANCE REAL,
    TRAIN_PROPORTION REAL,
    SIGMA_FACTOR REAL,
    MINIMUM_PEOPLE REAL,
    PERSON_DELAY REAL,
    PROPAGATION_FACTOR REAL,
    WALKING_TIME REAL,
    WALKING_SPEED REAL,
    NORMAL REAL,
    SLIGHTLY REAL,
    BUSY REAL
);

CREATE TABLE IF NOT EXISTS Teams (
    TeamName TEXT PRIMARY KEY,
    VenueName TEXT,
    FOREIGN KEY (VenueName) REFERENCES Venues(VenueName)
);

"""

```

```

connection.commit()
connection.close()

```

```

#Puts all the data for me regarding each line into the database
#Line speeds are in kmh^-1
def createLines(databaseFile, jsonFile):

```

```

connection = sqlite3.connect(databaseFile)
cursor = connection.cursor()

# Load line information from JSON file
with open(jsonFile, "r") as file:
    LineInfo = json.load(file)

# Insert or update each line in the Lines table
for lineID, (LineName, AverageSpeed) in LineInfo.items():
    cursor.execute("""
        INSERT OR REPLACE INTO Lines (LineID, LineName, AverageSpeed)
        VALUES (?, ?, ?)
        """, (lineID, LineName, AverageSpeed))

connection.commit()
connection.close()

# Reads default walking parameters from a JSON configuration file
def getWalking(file):
    with open(file, "r") as jsonFile:
        data = json.load(jsonFile)

    # Extract walking speed and walking time parameters
    walkingSpeed = data.get("WALKING_SPEED")
    walkingTime = data.get("WALKING_TIME")
    return walkingSpeed, walkingTime

#Main sequence of steps for building and populating the database
def main(databaseFile, LinesjsonFile, TfL_API_KEY, OPENCAGE_API_KEY, londonjsonFile, leaguesjsonFile,
defaultParametersFile):

    # Load walking parameters used for venue station relationships
    walkingSpeed, walkingTime = getWalking(defaultParametersFile)

    # Create database schema
    createDatabase(databaseFile)
    print("Created Database")

    # Populate Lines table
    createLines(databaseFile, LinesjsonFile)
    print("Created Lines")

    # Fetch and store TfL transport data for me
    tfIDone = getTfLData.main(TfL_API_KEY, databaseFile)
    if type(tfIDone) == int: #When an error has occurred a status code will be returned
        return tfIDone
    pass
    print("Got TFL Data")

    # Geocode stadiums and link them to stations with default walking parameters
    geocodingMessage = storeStadiumData.main(OPENCAGE_API_KEY, londonjsonFile, databaseFile, walkingTime,
walkingSpeed)
    if type(geocodingMessage) != bool:
        return geocodingMessage
    print("Stored Stadium Data")

    #Scrape event data for me
    scraperMessage = scraper.main(databaseFile, leaguesjsonFile)
    if type(scraperMessage) is int:
        return scraperMessage
    print("Done Scraping")

```

```

return "success"

# Run main (remnant of testing)
if __name__ == '__main__':
    message = main("final.db", "lines.json", "0ff5a2076cd640cb957e63d6947efc61", "eb0e2c9b71cc45f7aafe0ae4ecc44cc2",
"londonClubs.json", "leagues.json", "defaultParameters.json")
    print(message)

```

getResults.py

```

import crowdingAlgorithm
import copy # Used for deep copying graphs so original data isn't altered
import pathFinding
import customFunctions
import classes
import sqlite3

# Fetch the NaPTAN (unique station code) from a station name
def getNaPTAN(name, database):
    connection = sqlite3.connect(database)
    cursor = connection.cursor()

    cursor.execute(f"""
        SELECT NaPTAN
        FROM Stations
        WHERE StationName = ?""", (name,))
    result = cursor.fetchall()[0][0]

    connection.close()

    return result

# Create a modified graph where each connection's travel time is multiplied by the delay factor
def GetaffectedGraph(baseGraph, affectedNetwork):
    # Deep copy to avoid modifying the original base graph
    affectedGraph = copy.deepcopy(baseGraph)

    for StationA in affectedGraph.keys():

        connectedStations = affectedGraph[StationA]

        for connection in connectedStations:

            StationB = connection[0]
            LineID = connection[2]
            key = (StationA, StationB, LineID)

            # Multiply old travel time by the LineDelay factor from the affected network
            LineDelay = affectedNetwork.edges[key].LineDelay
            oldTime = connection[1]
            newTime = oldTime * LineDelay
            connection[1] = newTime

    return affectedGraph

# Strip down the affected graph to only include the edges in the old path
# This is used to calculate delays along the original route

```

```

def getStrippedGraph(oldPath, affectedGraph):
    strippedGraph = {}

    for connection in oldPath:
        StationA = connection[0]
        StationB = connection[1]
        LineID = connection[2]

        connectedStations = affectedGraph[StationA]

        for connectedStation in connectedStations:
            connectedStationB = connectedStation[0]
            connectedLineID = connectedStation[2]

            # Only keep the edge if it matches the original path connection
            # Just adjusting the time of individual connections of the base path, doesn't change the time taken to change trains,
            which is a flaw fixed with this implementation
            if connectedStationB == StationB and connectedLineID == LineID:
                strippedGraph[StationA] = [connectedStation]

    return strippedGraph

# Compute the path using only the connections from the old path but considering updated travel times
def getAffectedOldPath(oldPath, affectedGraph, startStation, endStation, CHANGING_TIME):
    strippedGraph = getStrippedGraph(oldPath, affectedGraph)
    returnPath = pathFinding.Dijkstra(strippedGraph, startStation, endStation, CHANGING_TIME)

    return returnPath

# Main function for calling all the functions and calculating new paths and evaluating delays
def main(date, journeyStart, startStationName, endStationName, databaseFile, username):

    # Convert station names to NaPTAN codes
    startStation = getNaPTAN(startStationName, databaseFile)
    endStation = getNaPTAN(endStationName, databaseFile)

    # Generate the affected network (crowding and delays applied)
    affectedNetwork = crowdingAlgorithm.main(date, journeyStart, startStation, endStation, databaseFile, username)
    parameters = customFunctions.getParameters(username, databaseFile)

    # Create the base graph (unaffected network) and then apply delays to get the affected graph
    baseGraph = pathFinding.MakeGraph(databaseFile)
    affectedGraph = GetaffectedGraph(baseGraph, affectedNetwork)

    # Calculate paths
    oldPath = pathFinding.Dijkstra(baseGraph, startStation, endStation, parameters.CHANGING_TIME)
    affectedOldPath = getAffectedOldPath(oldPath, affectedGraph, startStation, endStation, parameters.CHANGING_TIME)
    newPath = pathFinding.Dijkstra(affectedGraph, startStation, endStation, parameters.CHANGING_TIME)

    # Create objects to store severity and description of delays on old vs affected old path
    affectedOldPathDesriptionObject = classes.Severity(affectedOldPath, username, databaseFile, startStation,
    affectedNetwork)
    affectedOldPathDesriptionObject.calculate()
    affectedOldPathDesriptions = affectedOldPathDesriptionObject.descriptions

    # Same for new path
    newPathDesptionObject = classes.Severity(newPath, username, databaseFile, startStation, affectedNetwork)
    newPathDesptionObject.calculate()
    newPathDesriptions = newPathDesptionObject.descriptions

    # Compile all paths and descriptions into a results object
    resultsObject = classes.Results(oldPath, newPath, affectedOldPath, newPathDesriptions, affectedOldPathDesriptions)

```

```

return resultsObject

if __name__ == '__main__':
    date = "03/12/2025"
    journeyStart = "18:30"
    startStation = "Leytonstone Underground Station"
    endStation = "Stonebridge Park"
    databaseFile = "final.db"
    username = "Junaet"

    resultsObject = main(date, journeyStart, startStation, endStation, databaseFile, username)

    print(resultsObject.oldPath)
    print(resultsObject.newPath)
    print(resultsObject.affectedOldPath)
    print(resultsObject.newPathDescriptions)
    print(resultsObject.affectedOldPathDescriptions)

```

getTfLData.py

```

import sqlite3
import classes
from customFunctions import safeGet
from geopy.distance import geodesic

#GLOBAL PARAMETERS
#How long a train waits at a station
WAITING_TIME = 0.5

# Creates a dictionary with the format - {NaPTAN : Station object}
# Fetches all stations served by the given LineIDs from the TfL API
def fetchStations(TfL_API_KEY, lineIDs):
    stationDictionary = {}
    params = {
        "app_key": TfL_API_KEY,
    }

    # Loop through every line stored in the database
    for line in lineIDs:
        url = f"https://api.tfl.gov.uk/Line/{line}/Route/Sequence/inbound"
        response = safeGet(url, params=params)
        if type(response) is int: #Checks if there has been an error
            return response
        data = response.json()
        stations = data["stations"]

        # Create a CreationStation object for every unique station
        for station in stations:
            if station["id"] not in stationDictionary: # So every entry is unique
                NaPTAN = station["id"]
                StationName = station["name"]
                Latitude = station["lat"]
                Longitude = station["lon"]

                # CreationStation extends Station and stores geographic coordinates
                stationDictionary[station["id"]] = classes.CreationStation(NaPTAN, StationName, Latitude, Longitude)

    return stationDictionary

```

```

# Uses a list of LineIDs and a list of Station objects to create a dictionary in the form {LineID: [[Branch1], [Branch2], ..., [Branchn]]}
def fetch_lines(stationDictionary, TfL_API_KEY, lineIDs):
    linesDictionary = {} # Final dictionary that will be returned
    # Some stations are returned strangely by the API (different IDs for different branches)
    # This dictionary maps incorrect IDs to their correct hub NaPTAN IDs
    weirdStations = {} # {incorrectID: correctID}
    params = {
        "app_key": TfL_API_KEY,
    }

    # Loop through each LineID
    for line in lineIDs:
        url = f"https://api.tfl.gov.uk/Line/{line}/Route/Sequence/inbound"
        response = safeGet(url, params=params)
        if type(response) is int: #Checks if there has been an error
            return response
        data = response.json()
        branches = data["orderedLineRoutes"]
        listOflistsOfNAPTAN = []

        # Each branch represents one possible ordered path of stations on the line
        for branchAllData in branches:
            # branchAllData contains additional metadata which is not needed
            branch = branchAllData["naptanIds"] # List of all stations (NaPTAN IDs) of the branch
            for i, id in enumerate(branch):
                if id not in stationDictionary: # All possible unique stations already exist in stationDictionary, but some stations
                    # have other IDs for different modes, which means they need to be filtered out so that there is only one ID per station
                    if id not in weirdStations: # Store of all the stations that have different IDs
                        # Fetch station details to find its hub NaPTAN ID
                        newURL = f"https://api.tfl.gov.uk/StopPoint/{id}" # Fetches all data for me regarding that particular station
                        newResponse = safeGet(newURL, params=params)
                        newData = newResponse.json()
                        newId = newData["hubNaptanCode"] # Collects the ID that represents the entire station and not just that
mode/line
                        # Store mapping so future lookups don't require another API call
                        weirdStations[id] = newId

                        # Replace incorrect ID with correct hub ID
                        id = weirdStations[id]
                        branch[i] = id # Changes the ID of the station in the branch to the correct ID

                    # Add the cleaned branch to the list of branches for this line
                    listOflistsOfNAPTAN.append(branch)

        linesDictionary[line] = listOflistsOfNAPTAN

    return linesDictionary

# Fetches all TfL data for me required to populate the database (that can be gotten from the TfL API)
# Returns stationDictionary and linesDictionary
def fetchTfLData(databaseFile, TfL_API_KEY):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Creates a list of LineIDs
    linesCursorObject = cursor.execute("SELECT LineID FROM Lines")
    lineIDs = []
    for lineTuple in linesCursorObject: # SQL returns data for me in the form (LineID,)
        lineIDs.append(lineTuple[0])

```

```

# Fetch all stations served by the stored lines
stationDictionary = fetchStations(TfL_API_KEY, lineIDs)
print("Fetched Stations")

# Fetch ordered branches for each line
linesDictionary = fetch_lines(stationDictionary, TfL_API_KEY, lineIDs)
print("Fetched Lines")

connection.close()
return stationDictionary, linesDictionary

# Puts every possible Station, Line pair into the StationLineRelationships table
# Acts as the joining table between Stations and Lines
def getStationLineRelationships(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Use the Connections table to get which stations are served by which lines
    rows = cursor.execute("SELECT StationA, LineID from Connections").fetchall()
    for row in rows:
        StationA = row[0]
        LineID = row[1]

    # Ignore as well because station might have more than 1 connection to another station via the same line -- different
    # branch lines are given start to end, which therefore means sometimes multiple of the same connection are attempted to be
    # stored
    cursor.execute("""
        INSERT OR REPLACE INTO StationLineRelationships (NaPTAN, LineID)
        VALUES (?, ?)
        """, (StationA, LineID))

    connection.commit()
    connection.close()

# Fetches the latitude and longitude of a station from the database
def getStationCoordinates(databaseFile, StationID):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("""
        SELECT Latitude, Longitude
        FROM Stations
        WHERE NaPTAN = ?
        """, (StationID, ))
    coordinates = cursor.fetchall()[0]
    connection.close()
    return coordinates

# Stores all fetched TfL data for me into the database
# Populates Stations, Connections, and StationLineRelationships tables
# Print statements for testing purposes
def SaveTfLData(databaseFile, TfL_API_KEY):
    stationDictionary, linesDictionary = fetchTfLData(databaseFile, TfL_API_KEY)

    # Stop execution if an API error occurred
    if type(stationDictionary) is int or type(linesDictionary) is int:
        print(f"A {stationDictionary} error has occurred")
        return stationDictionary

    print("TfL Data Fetched")
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

```

```

# Store all stations in the Stations table
for station in stationDictionary.values():
    NaPTAN = station.NaPTAN
    StationName = station.StationName
    Latitude = station.Latitude
    Longitude = station.Longitude
    cursor.execute("""
        INSERT OR REPLACE INTO Stations (NaPTAN, StationName, Latitude, Longitude)
        VALUES (?, ?, ?, ?)
        """, (NaPTAN, StationName, Latitude, Longitude))
    connection.commit()
print("Stored Stations")

# Store all connections between adjacent stations on each line
for line in linesDictionary.keys():
    LineID = line
    branches = linesDictionary[line]
    for branch in branches:
        # Adjacent stations form edges in the transport graph
        for i in range(len(branch) - 1):
            StationA = branch[i]
            StationB = branch[i + 1]

            # Fetch average speed of the line from the database
            cursor.execute("""
                SELECT AverageSpeed
                From Lines
                WHERE LineID = ?
                """, (LineID,))
            AverageSpeed = float(cursor.fetchall()[0][0])

            # Fetch station coordinates
            stationACoordinates = getStationCoordinates(databaseFile, StationA)
            stationBCoordinates = getStationCoordinates(databaseFile, StationB)

            # Calculate distance and base travel time (no crowding)
            distanceKM = geodesic(stationACoordinates, stationBCoordinates).km

            # Add a default minute penalty to represent the time trains wait at a station
            BaseTravelTime = ((distanceKM / AverageSpeed) * 60) + WAITING_TIME

            # Store connections in both directions
            cursor.execute("""
                INSERT OR REPLACE INTO Connections (StationA, StationB, LineID, BaseTravelTime)
                VALUES (?, ?, ?, ?)
                """, (StationA, StationB, LineID, BaseTravelTime))
            cursor.execute("""
                INSERT OR REPLACE INTO Connections (StationA, StationB, LineID, BaseTravelTime)
                VALUES (?, ?, ?, ?)
                """, (StationB, StationA, LineID, BaseTravelTime))

print("Stored connections")

connection.commit()
connection.close()

# Populate StationLineRelationships using stored connections
getStationLineRelationships(databaseFile)

return True

```

```

#Main function

```

```

def main(TfL_API_KEY, databaseFile):
    code = SaveTfLData(databaseFile, TfL_API_KEY)
    if type(code) is int:
        print(f"A {code} error has occurred")
        return code
    return True

if __name__ == "__main__":
    TfL_API_KEY = "0ff5a2076cd640cb957e63d6947efc61"
    main(TfL_API_KEY, "final.db")
    pass

```

login.py

```

import json
import hashlib
import os
import sqlite3
import classes

# Generates a cryptographically secure random salt
# A unique salt is used per user to protect against rainbow table attacks
def generateSalt():
    return os.urandom(16).hex() #Creates a random 16-byte salt and returns it as hex

# Creates a SHA-256 hash of the password combined with a salt
# Passwords are never stored in plain text
def createHashPassword(password, salt):
    saltedPassword = salt + password
    byteForm = saltedPassword.encode()
    hashObject = hashlib.sha256(byteForm)
    hashedPassword = hashObject.hexdigest()
    return hashedPassword

# Creates a new user account and stores default modelling parameters
# Each user begins with a copy of the default assumptions, which they can later modify
def createAccount(defaultParametersFile, username, password, databaseFile):
    salt = generateSalt()
    hashedPassword = createHashPassword(password, salt)

    # Loads default crowding and delay parameters from a file storing default values
    defaultParametersObject = classes.AccountCreationParameters(defaultParametersFile)

    # Tuple contains login credentials and all model parameters
    parameterTuple = (
        username,
        hashedPassword,
        salt,
        defaultParametersObject.CHANGING_TIME,
        defaultParametersObject.MAX_TIME_WINDOW,
        defaultParametersObject.rawArrivalPeakOffset,
        defaultParametersObject.rawDeparturePeakOffset,
        defaultParametersObject.ATTENDANCE,
        defaultParametersObject.TRAIN_PROPORTION,
        defaultParametersObject.SIGMA_FACTOR,
        defaultParametersObject.MINIMUM_PEOPLE,
        defaultParametersObject.PERSON_DELAY,

```

```

defaultParametersObject.PROPGATION_FACTOR,
defaultParametersObject.WALKING_TIME,
defaultParametersObject.WALKING_SPEED,
defaultParametersObject.NORMAL,
defaultParametersObject.SLIGHTLY,
defaultParametersObject.BUSY
)

connection = sqlite3.connect(databaseFile)
cursor = connection.cursor()

try:
    # Insert user credentials and modelling parameters into Users table
    cursor.execute(
        """
        INSERT INTO Users(
            Username,
            PasswordHash,
            Salt,
            CHANGING_TIME,
            MAX_TIME_WINDOW,
            rawArrivalPeakOffset,
            rawDeparturePeakOffset,
            ATTENDANCE,
            TRAIN_PROPORTION,
            SIGMA_FACTOR,
            MINIMUM_PEOPLE,
            PERSON_DELAY,
            PROPAGATION_FACTOR,
            WALKING_TIME,
            WALKING_SPEED,
            NORMAL,
            SLIGHTLY,
            BUSY
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """
        ,
        parameterTuple
    )
    accountMade = True
except:
    # Likely occurs if username already exists
    accountMade = False

connection.commit()
connection.close()

return accountMade

# Authenticates a user by hashing the entered password with the stored salt
def login(databaseFile, username, password):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.execute("""
        SELECT PasswordHash, Salt
        FROM Users
        WHERE Username = ?
        """, (username,))

    try:
        result = cursor.fetchall()[0]
        accountFound = True

```

```

except:
    accountFound = False

if accountFound:
    storedHash, storedSalt = result
    loginHash = createHashPassword(password, storedSalt)

    if loginHash == storedHash:
        message = "Login Successful"
    else:
        message = "Wrong Password"
else:
    message = "Account Not Found"

connection.close()
return message

# For testing purposes
def main(username, password, defaultParametersFile, databaseFile):
    accountCreated = createAccount(defaultParametersFile, username, password, databaseFile)
    loginSuccessful = login(databaseFile, username, password)
    print(accountCreated)
    print(loginSuccessful)

# For testing purposes
if __name__ == '__main__':
    username = "Jun8"
    password = "h"
    defaultParametersFile = "defaultParameters.json"
    databaseFile = "test.db"
    main(username, password, defaultParametersFile, databaseFile)

```

main.py

```

from flask import Flask, render_template, request, session, redirect, url_for, jsonify
import login
import sqlite3
from datetime import datetime, timedelta
import getResults
import classes
from pathlib import Path
import databaseCreation
import threading
import json
import storeStadiumData
import customFunctions

# Flask app initialisation
app = Flask(__name__)
app.secret_key = "VERY_SECRET_KEY" # Needed for session management

# Makes these accessible anywhere in the app
def setUpConfig(
    app,
    databaseFile,
    LinesjsonFile,
    TfL_API_KEY,

```

```

    OPENCAGE_API_KEY,
    londonjsonFile,
    leaguesjsonFile,
    flagsFile,
    defaultParametersFile
):
    """
    Sets up all configuration variables in Flask's config dictionary.
    """

    app.config["DATABASE_FILE"] = databaseFile
    app.config["LINES_JSON_FILE"] = LinesjsonFile
    app.config["TFL_API_KEY"] = TfL_API_KEY
    app.config["OPENCAGE_API_KEY"] = OPENCAGE_API_KEY
    app.config["LONDONJSON_FILE"] = londonjsonFile
    app.config["LEAGUES_JSON_FILE"] = leaguesjsonFile
    app.config["FLAGS_FILE"] = flagsFile
    app.config["DEFAULT_PARAMETERS_FILE"] = defaultParametersFile

# Login / Signup Page
@app.route('/', methods=['GET', 'POST'])
def loginPage():
    """
    Displays the login page. Handles login and signup requests.
    Creates the database if it doesn't exist.
    """

    databaseFile = app.config["DATABASE_FILE"]
    flagsFile = app.config["FLAGS_FILE"]
    defaultParametersFile = app.config["DEFAULT_PARAMETERS_FILE"]
    londonjsonFile = app.config["LONDONJSON_FILE"]
    message = None

    # Store previous page URL in session (used for redirects later)
    session["previousPage"] = request.url

    # Check if database exists
    databaseFilePath = Path(databaseFile)
    if not databaseFilePath.exists():
        # Create flags file if it doesn't exist
        flagsPath = Path(flagsFile)

        if not flagsPath.exists():
            with open(flagsFile, "w") as file:
                json.dump({"databaseCreated": False}, file)

    # Create database in a separate thread to avoid blocking and to allow for a waiting page to load
    threading.Thread(target=createDatabase).start()
    return redirect(url_for("loadPage"))

# Wipe flags file if database exists
with open(flagsFile, "w") as flag_file:
    pass #Wipe

# Handle POST requests (login/signup)
if request.method == 'POST':
    action = request.form.get('actionName')
    username = request.form.get('username')
    password = request.form.get('password')

    if action == 'login':
        loginMessage = login.login(databaseFile, username, password)
        if loginMessage == "Login Successful":
            session["username"] = username

```

```

        params = customFunctions.getParameters(username, databaseFile)
        storeStadiumData.redoStore(
            params.WALKING_TIME,
            params.WALKING_SPEED,
            londonjsonFile,
            databaseFile
        )

        return redirect(url_for("mainPage"))
    else:
        message = loginMessage
    elif action == 'signUp':
        accountMade = login.createAccount(defaultParametersFile, username, password, databaseFile)
        if accountMade:
            session["username"] = username

            params = customFunctions.getParameters(username, databaseFile)
            storeStadiumData.redoStore(
                params.WALKING_TIME,
                params.WALKING_SPEED,
                londonjsonFile,
                databaseFile
            )

            return redirect(url_for("mainPage"))
        else:
            message = "Account already created"
    return render_template("login.html", message=message)

# Loading Page
@app.route('/load', methods=['GET', 'POST'])
def loadPage():
    """
    Simple page to show when database is being created or updated.
    """
    return render_template("loading.html")

# Main Journey Planning Page
@app.route('/main', methods=['GET', 'POST']) #Need to uncomment out the validation for correct time
def mainPage():
    # Redirect to login if not logged in
    if "username" not in session:
        return redirect(url_for("loginPage"))

    databaseFile = app.config["DATABASE_FILE"]
    username = session["username"]

    # Initialise variables for template
    error = None
    startStationLines = None
    endStationLines = None
    formattedOldPath = None
    formattedNewPath = None
    oldPathTime = None
    oldPathArrival = None
    newPathTime = None
    newPathArrival = None

    validDateCheck = True #Toggle on and off for testing

    if request.method == 'POST':
        if "updateButton" in request.form:

```

```

    # Redirect to update parameters page
    return redirect(url_for("updatePage"))
else:
    # Get form input
    start = request.form.get('start')
    end = request.form.get('end')
    date = request.form.get('date')
    time = request.form.get('time')
    selectedTime = datetime.strptime(f"{date} {time}", "%Y-%m-%d %H:%M")

    if start == end:
        error = "Start and end stations must be different"
    elif validDateCheck and selectedTime < datetime.now():
        error = "Selected time must be in the future"
    else:
        # Format date for getResults
        dateObject = datetime.strptime(date, "%Y-%m-%d")
        passInDate = dateObject.strftime("%d/%m/%Y")

        # Get line information for dropdowns
        startStationLines = getLines(start, databaseFile)
        endStationLines = getLines(end, databaseFile)

        # Calculate journey paths
        resultsObject = getResults.main(passInDate, time, start, end, databaseFile, username)

        # Old Path (Affected)
        affectedOldPath = resultsObject.affectedOldPath
        affectedOldPathDescriptions = resultsObject.affectedOldPathDescriptions
        preFormataffectedOldPathMinutes = affectedOldPath[-1][3] # The time taken to get to the last station on the path,
which is stored as the fourth element [prevStation, currStation, LineUsed, TimeToStation]
        oldPathTime, oldPathArrival = processTime(preFormataffectedOldPathMinutes, time)
        formattedOldPath = formatPathData(affectedOldPath, affectedOldPathDescriptions, start, databaseFile)

        # New Path (Optimised)
        newPath = resultsObject.newPath
        newPathDescriptions = resultsObject.newPathDescriptions
        preFormataffectedNewPathMinutes = newPath[-1][3] # The time taken to get to the last station on the path, which
is stored as the fourth element [prevStation, currStation, LineUsed, TimeToStation]
        newPathTime, newPathArrival = processTime(preFormataffectedNewPathMinutes, time)
        formattedNewPath = formatPathData(newPath, newPathDescriptions, start, databaseFile)

    return render_template(
        "mainPage.html",
        error=error,
        startLines=startStationLines,
        endLines=endStationLines,
        formattedOldPath=formattedOldPath,
        formattedNewPath=formattedNewPath,
        oldPathTime=oldPathTime,
        oldPathArrival=oldPathArrival,
        newPathTime=newPathTime,
        newPathArrival=newPathArrival
    )

# Update User Parameters Page
@app.route('/update', methods=['GET', 'POST'])
def updatePage():
    """
    Displays and allows updating of user-specific parameters.
    """

    # Redirect to login if not logged in

```

```

if "username" not in session:
    return redirect(url_for("loginPage"))

databaseFile = app.config["DATABASE_FILE"]
username = session["username"]

# Retrieve user parameters from database
connection = sqlite3.connect(databaseFile)
cursor = connection.cursor()
cursor.execute("""
    SELECT
        CHANGING_TIME,
        MAX_TIME_WINDOW,
        rawArrivalPeakOffset,
        rawDeparturePeakOffset,
        ATTENDANCE,
        TRAIN_PROPORTION,
        SIGMA_FACTOR,
        MINIMUM_PEOPLE,
        PERSON_DELAY,
        PROPAGATION_FACTOR,
        WALKING_TIME,
        WALKING_SPEED,
        NORMAL,
        SLIGHTLY,
        BUSY
    FROM Users
    WHERE Username = ?
""", (username,))
userData = cursor.fetchone()
connection.close()

# Unpack values for template
(CHANGING_TIME,
MAX_TIME_WINDOW,
rawArrivalPeakOffset,
rawDeparturePeakOffset,
ATTENDANCE,
TRAIN_PROPORTION,
SIGMA_FACTOR,
MINIMUM_PEOPLE,
PERSON_DELAY,
PROPAGATION_FACTOR,
WALKING_TIME,
WALKING_SPEED,
NORMAL,
SLIGHTLY,
BUSY) = userData

return render_template("updatePage.html",
    CHANGING_TIME=CHANGING_TIME,
    MAX_TIME_WINDOW=MAX_TIME_WINDOW,
    rawArrivalPeakOffset=rawArrivalPeakOffset,
    rawDeparturePeakOffset=rawDeparturePeakOffset,
    ATTENDANCE=ATTENDANCE,
    TRAIN_PROPORTION=TRAIN_PROPORTION,
    SIGMA_FACTOR=SIGMA_FACTOR,
    MINIMUM_PEOPLE=MINIMUM_PEOPLE,
    PERSON_DELAY=PERSON_DELAY,
    PROPAGATION_FACTOR=PROPAGATION_FACTOR,
    WALKING_TIME=WALKING_TIME,
    WALKING_SPEED=WALKING_SPEED,
    NORMAL=NORMAL,
    SLIGHTLY=SLIGHTLY,

```

```

        BUSY=BUSY)

# Database Error Page
@app.route("/db_error")
def databaseError():
    """
    Displays a database error page.
    """
    flagsFile = app.config["FLAGS_FILE"]

    with open(flagsFile, "r") as f:
        data = json.load(f)
        errorMessage = data["error"]

    return render_template("databaseError.html") # Can add error message in future

# Database Creation Function
def createDatabase():
    """
    Creates the database using databaseCreation.main().
    Writes success/failure status to flags file.
    """
    databaseFile = app.config["DATABASE_FILE"]
    LinesjsonFile = app.config["LINES_JSON_FILE"]
    TfL_API_KEY = app.config["TFL_API_KEY"]
    OPENCAGE_API_KEY = app.config["OPENCAGE_API_KEY"]
    londonjsonFile = app.config["LONDONJSON_FILE"]
    leaguesjsonFile = app.config["LEAGUES_JSON_FILE"]
    flagsFile = app.config["FLAGS_FILE"]
    defaultParametersFile = app.config["DEFAULT_PARAMETERS_FILE"]

    message = databaseCreation.main(databaseFile, LinesjsonFile, TfL_API_KEY, OPENCAGE_API_KEY, londonjsonFile,
    leaguesjsonFile, defaultParametersFile)

    if message == "success":
        status = "success"
    else:
        # Remove database if creation failed
        status = "failed"
        databaseFile = Path(app.config["DATABASE_FILE"])
        if databaseFile.exists():
            databaseFile.unlink()

    # Write status to flags file
    data = {
        "status": status,
        "error": message
    }

    with open(flagsFile, "w") as file:
        json.dump(data, file)

    print("Database created")

# Format Duration
def formatDuration(minutes):
    """
    Converts minutes into hours and minutes string.
    """
    hours = round(minutes // 60)

```

```

minutesLeft = round(minutes % 60)

if hours > 0:
    return f"{hours} hours and {minutesLeft} minutes"
else:
    return f"{minutesLeft} minutes"

# Process Time
def processTime(totalUnprocessedTime, timeOfJourney):
    """
    Calculates journey duration and arrival time.
    """
    startTime = datetime.strptime(timeOfJourney, "%H:%M")
    journeyTimeObject = timedelta(minutes=round(totalUnprocessedTime))
    arrivalTime = startTime + journeyTimeObject
    arrivalTimeString = arrivalTime.strftime("%H:%M")
    duration = formatDuration(totalUnprocessedTime)
    return duration, arrivalTimeString

# Format Path Data Helper
def formatPathData(affectedPath, affectedPathDescriptions, startStation, databaseFile):
    """
    Converts path tuples into classes.mainPagePathClass objects for display.
    Adds line names, station names, and descriptions.
    """
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Get starting station NaPTAN
    cursor.execute("""
        SELECT NaPTAN
        FROM Stations
        WHERE StationName = ?
    """, (startStation,))
    startStationNaPTAN = cursor.fetchall()[0][0]

    returnAffectedPath = []

    # Initialise first path object
    firstConnection = affectedPath[0]
    firstLine = firstConnection[2]
    currentObject = classes.mainPagePathClass(firstLine)
    currentObject.addStation(startStationNaPTAN)

    # Iterate through affectedPath connections
    for connectionTuple in affectedPath:
        previous, currentStation, line, distance = connectionTuple

        if line == currentObject.LineID:
            currentStation = connectionTuple[1]
            currentObject.addStation(currentStation)
        else:
            returnAffectedPath.append(currentObject)
            currentObject = classes.mainPagePathClass(line)
            currentObject.addStation(currentStation)

    # Append last path object
    returnAffectedPath.append(currentObject)

    # Add names and descriptions
    for object in returnAffectedPath:
        object.getLineName(databaseFile)

```

```

    object.getStationNames(databaseFile)
    object.getDescription(affectedPathDescriptions)

connection.close()
return returnAffectedPath

def getLines(stationName, databaseFile):
    """
    Returns a list of lines a station belongs to.
    """
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.execute("""
        SELECT Lines.LineName
        FROM Lines
        JOIN StationLineRelationships
            ON Lines.LineID = StationLineRelationships.LineID
        JOIN Stations
            ON StationLineRelationships.NaPTAN = Stations.NaPTAN
        WHERE Stations.StationName = ?
        ORDER BY Lines.LineName
    """, (stationName,))

    results = cursor.fetchall()

    connection.close()

    returnList = [resultTuple[0] for resultTuple in results]

    return returnList

# Background Database Update
def backgroundDataFetch():
    """
    Updates the database in the background while preserving user data.
    """
    databaseFile = app.config["DATABASE_FILE"]

    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM Users")
    users_data = cursor.fetchall() # List of tuples with all user rows
    connection.close()

    # Delete old database
    databasePath = Path(databaseFile)
    databasePath.unlink()

    # Recreate database
    createDatabase()

    # Restore user data
    if users_data:
        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()
        cursor.executemany("""
            INSERT OR REPLACE INTO Users (
                Username,
                PasswordHash,
                Salt,
                CHANGING_TIME,

```

```

        MAX_TIME_WINDOW,
        rawArrivalPeakOffset,
        rawDeparturePeakOffset,
        ATTENDANCE,
        TRAIN_PROPORTION,
        SIGMA_FACTOR,
        MINIMUM_PEOPLE,
        PERSON_DELAY,
        PROPAGATION_FACTOR,
        WALKING_TIME,
        WALKING_SPEED,
        NORMAL,
        SLIGHTLY,
        BUSY
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """
    users_data)
connection.commit()
connection.close()

```

Route to Trigger Background Database Fetch

```

@app.route("/fetch_new_data", methods=["POST"])
def fetchNewData():
    threading.Thread(target=backgroundDataFetch).start()
    return redirect(url_for("loadPage"))

```

Save User Parameters

```

@app.route("/save_parameters", methods=["POST"])
def saveParameters():
    """
    Saves user-modified parameters and updates stadium data.
    """
    databaseFile = app.config["DATABASE_FILE"]
    londonjsonFile = app.config["LONDONJSON_FILE"]

```

```

    username = session.get("username")
    data = request.get_json()

```

```

    values = (
        float(data["CHANGING_TIME"]),
        float(data["MAX_TIME_WINDOW"]),
        float(data["rawArrivalPeakOffset"]),
        float(data["rawDeparturePeakOffset"]),
        float(data["ATTENDANCE"]),
        float(data["TRAIN_PROPORTION"]),
        float(data["SIGMA_FACTOR"]),
        float(data["MINIMUM_PEOPLE"]),
        float(data["PERSON_DELAY"]),
        float(data["PROPAGATION_FACTOR"]),
        float(data["WALKING_TIME"]),
        float(data["WALKING_SPEED"]),
        float(data["NORMAL"]),
        float(data["SLIGHTLY"]),
        float(data["BUSY"]),
        username # WHERE clause
    )

```

```

connection = sqlite3.connect(databaseFile)
cursor = connection.cursor()
cursor.execute("""
    UPDATE Users SET
        CHANGING_TIME = ?,
        MAX_TIME_WINDOW = ?,

```

```

        rawArrivalPeakOffset = ?,
        rawDeparturePeakOffset = ?,
        ATTENDANCE = ?,
        TRAIN_PROPORTION = ?,
        SIGMA_FACTOR = ?,
        MINIMUM_PEOPLE = ?,
        PERSON_DELAY = ?,
        PROPAGATION_FACTOR = ?,
        WALKING_TIME = ?,
        WALKING_SPEED = ?,
        NORMAL = ?,
        SLIGHTLY = ?,
        BUSY = ?
    WHERE Username = ?
    """ , values)
connection.commit()
connection.close()

# Recalculate stadium data
walkingTime = float(data["WALKING_TIME"])
walkingSpeed = float(data["WALKING_SPEED"])

storeStadiumData.redoStore(walkingTime, walkingSpeed, londonjsonFile, databaseFile)

return jsonify({"success": True})

# Logout
@app.route("/logout", methods=["POST"])
def logout():
    """
    Logs out the user by removing session.
    """
    session.pop("username", None) # Remove username from session
    return redirect(url_for("loginPage"))

# Load Default Parameters
@app.route("/load_default_parameters", methods=["GET"])
def loadDefaultParameters():
    defaultParametersFile = app.config["DEFAULT_PARAMETERS_FILE"]

    with open(defaultParametersFile, "r") as file:
        data = json.load(file)
    return jsonify(data)

# Flags Management
@app.route("/updateFlags", methods=["POST"])
def updateFlags():
    """
    Updates flags file when database creation starts.
    """
    flagsFile = app.config["FLAGS_FILE"]

    data = {
        "status": "creating",
        "error": "success"
    }

    with open(flagsFile, "w") as file:
        json.dump(data, file)
    return jsonify({"success": True})

```

```

@app.route('/check_flag', methods=['GET'])
def checkFlag():
    """
    Returns current flags status (database creation, error, previous page).
    """
    flagsFile = app.config["FLAGS_FILE"]

    response = {
        "status": "creating",
        "error": None,
        "previousPage": session.get("previousPage")
    }

    try:
        with open(flagsFile, "r") as file:
            response.update(json.load(file))
    except:
        pass

    return jsonify(response)

# Autocomplete Station Search
@app.route("/get_stations")
def get_stations():
    """
    Returns station suggestions for autocomplete.
    """
    databaseFile = app.config["DATABASE_FILE"]
    searchTerm = request.args.get("search", "")

    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    cursor.execute("""
        SELECT StationName
        FROM Stations
        WHERE StationName LIKE ?
        ORDER BY StationName
        LIMIT 20
    """, (f"%{searchTerm}%",))

    rows = cursor.fetchall()
    connection.close()

    results = [
        {"id": StationName, "text": StationName}
        for StationName in rows
    ]

    returnJson = jsonify(results)
    return returnJson

# Call the programme
if __name__ == '__main__':
    databaseFile = "data/theDatabase.db"
    LinesjsonFile = "lines.json"
    TfL_API_KEY = "0ff5a2076cd640cb957e63d6947efc61"
    OPENCAGE_API_KEY = "eb0e2c9b71cc45f7aafe0ae4ecc44cc2"
    londonjsonFile = "londonClubs.json"
    leaguesjsonFile = "leagues.json"
    flagFile = "data/flags.json"

```

```

defaultParametersFile = "defaultParameters.json"
setUpConfig(app, databaseFile, LinesjsonFile, TfL_API_KEY, OPENCAGE_API_KEY, londonjsonFile, leaguesjsonFile, flagFile,
defaultParametersFile)
app.run()

```

networkClasses.py

```

import sqlite3

# Container and to call class methods
class Severity():
    def __init__(self, originalPath, username, databaseFile, start, affectedNetwork):
        self.descriptions = [] # Stores descriptions for each station along the path, e.g., [["StationA", "Normal"], ["StationB", "Busy"]]
        self.originalPath = originalPath # The path returned by Dijkstra [(prevStation, currStation, line, cost), ...]
        self.username = username # Username to fetch user-specific thresholds for crowding
        self.databaseFile = databaseFile # Database file containing the thresholds
        self.start = start # Starting station for the path
        self.affectedNetwork = affectedNetwork # Network object (instance of Network class) containing station nodes with DelayFactor
        self.normal = 0 # Threshold values that will be fetched from the database
        self.slightly = 0
        self.busy = 0

    def getParameters(self):
        connection = sqlite3.connect(self.databaseFile)
        cursor = connection.cursor()

        cursor.execute("""
            SELECT NORMAL, SLIGHTLY, BUSY
            FROM Users
            WHERE Username = ?
            """, (self.username,))
        results = cursor.fetchall()[0]

        connection.close()

        self.normal, self.slightly, self.busy = results

    def getDescription(self, factor):
        # Map a DelayFactor to a human-readable description using thresholds
        if factor > self.busy:
            description = "Severe"
        elif factor > self.slightly:
            description = "Busy"
        elif factor > self.normal:
            description = "Slightly"
        else:
            description = "Normal"

        return description

    def addDescription(self, station):
        firstFactor = self.affectedNetwork.nodes[station].DelayFactor # Get the DelayFactor of the station from the network
        firstDescription = self.getDescription(firstFactor) # Convert factor to description
        self.descriptions.append([station, firstDescription]) # Append [station, description] to the list

```

```

def makePath(self):
    self.addDescription(self.start) # Add the starting station first

    # Iterate over the original path returned by Dijkstra
    # path elements are (prevStation, currStation, lineUsed, cumulativeCost)
    for connectionData in self.originalPath:
        station = connectionData[1] # Current station in the path
        self.addDescription(station)

def calculate(self):
    self.getParameters()
    self.makePath()

#Container for all the results
class Results():
    def __init__(self, oldPath, newPath, affectedOldPath, newPathDescriptions, affectedOldPathDescriptions):
        self.oldPath = oldPath # The original shortest path without considering delays
        self.newPath = newPath # The new path that may be optimised to avoid delays
        self.affectedOldPath = affectedOldPath # The original path but with delays applied (affected)
        self.newPathDescriptions = newPathDescriptions # Descriptions of crowding for the new path
        self.affectedOldPathDescriptions = affectedOldPathDescriptions # Descriptions of crowding for the affected old path

```

pathFinding.py

```

import sqlite3
import collections
import math
import heapq

#Fetches all the unique connections from the database and for every StationA, it adds to a list all StationBs in their own list
including Line and travel time
#Forms dictionary in the form {StationA : [[StationB, BaseTravelTime, LineID], [StationC, ...], ...]}
def MakeGraph(databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    listOfNeighbourStations = {} # {StationA: [[StationB, BaseTravelTime, LineID], ...]}
    rows = cursor.execute("SELECT StationA, StationB, LineID, BaseTravelTime FROM connections").fetchall()

    for StationA, StationB, LineID, BaseTravelTime in rows: #For every unique connection (direction matters)
        if StationA not in listOfNeighbourStations:
            listOfNeighbourStations[StationA] = []
        if [StationB, BaseTravelTime, LineID] not in listOfNeighbourStations[StationA]:
            listOfNeighbourStations[StationA].append([StationB, BaseTravelTime, LineID])

    connection.close()
    return listOfNeighbourStations

# It is assumed that all stations can be accessed from any other station because the graph is made from the data for me
stored in the database which is always complete for the function to be called in the first place
#On the TfL network, every station is eventually connected to every other station
def Dijkstra(graph, startStation, goalStation, CHANGING_TIME, penalty=True):
    distances = collections.defaultdict(lambda: math.inf) #Best known time to each station
    cameFrom = {} #To reconstruct path later
    visited = set() #Stores visited stations

```

```

# Start at the given station, with no line yet chosen
startState = (startStation, None) # This is the state is key because the line change penalty depends on the previous line
distances[startState] = 0

frontier = [(0, startState)] #The first item is always the cheapest station to go to next
goalState = None #Store the final when we reach the goal

while frontier: #This condition means it runs until all the paths have been explored - till the last one
    currentCost, (currentStation, currentLine) = heapq.heappop(frontier) #Deletes the next route at the top so that the next
    one to be explored can be at the top

    # Skip if already fully explored this (station, line) - If at the top of the heap, then it means that it is the shortest ever path
    to the station, which means it is visited
    if (currentStation, currentLine) in visited:
        continue
    visited.add((currentStation, currentLine))

    # Stop if reached the goal station
    if currentStation == goalStation:
        goalState = (currentStation, currentLine)
        break

    # Explore all neighbours of this station
    for nextStation, baseTravelTime, nextLine in graph[currentStation]:
        travelTime = baseTravelTime

        if penalty: #Can be turned off if needed (for testing purposes)
            if currentLine is not None and currentLine != nextLine:
                #Initially no line is chosen, so the penalty is not added for changing lines
                #If the line is switched to travel to the next station, the penalty is added to the cost
                travelTime += CHANGING_TIME

        # New total time to reach the neighbour
        newCost = currentCost + travelTime
        neighbourState = (nextStation, nextLine)

        # Only update if we found a faster way
        if newCost < distances[neighbourState]:
            distances[neighbourState] = newCost
            cameFrom[neighbourState] = (currentStation, currentLine, nextLine)
            heapq.heappush(frontier, (newCost, neighbourState))

# Start from the goal and trace back to the start
pathSteps = []
currentState = goalState

while currentState != startState:
    prevStation, prevLine, lineUsed = cameFrom[currentState]
    currStation, currLine = currentState
    pathSteps.append((prevStation, currStation, lineUsed, distances[(currStation, currLine)]))
    currentState = (prevStation, prevLine)

pathSteps.reverse()
return pathSteps

```

scraper.py

```

import bs4
import datetime

```

```

from customFunctions import safeGet
import re
import classes
import json

#So that 12 months of fixtures can be fetched from the date of request
def getNextTwelveMonths():
    currentDate = datetime.datetime.now()
    months = []
    year = currentDate.year
    month = currentDate.month

    for monthNumber in range(12):
        monthString = str(month)
        if len(monthString) == 1:
            monthString = "0" + monthString
        yearMonth = f"{year}-{monthString}"
        months.append(yearMonth)
        month += 1
        if month > 12:
            month = 1
            year += 1

    return months #results in [yyyy-mm, ...]

#To get the actual numerical date of the month from the string fetched because it is originally in a form like "Saturday 12th
December"
def getDate(monthYear, restOfDate):
    try:
        # Extract the day number from the string
        dayNumber = re.search(r"\d{1,2}", restOfDate).group()
        if len(dayNumber) == 1:
            dayNumber = "0" + dayNumber
        year, month = monthYear.split("-")
        date = f"{dayNumber}-{month}-{year}"
        return date
    except:
        # Returned when the text does not represent a valid date
        return "Error"

# Fetches and parses fixtures from a BBC Sport fixtures page
def getPageFixtures(url, monthYear, tierName, fixtures):
    response = safeGet(url)
    if type(response) is int:
        return response
    soup = bs4.BeautifulSoup(response.text, "html.parser")

    # Check if the page contains a link to today's fixtures
    # If so, ensure today's date exists in the fixtures dictionary
    # This is because they are represented slightly differently in the page (no date, just the word "today")
    today_link = soup.find("a", id="today")
    if today_link:
        currentDate = datetime.datetime.now().strftime("%d-%m-%Y") #Today's date in dd-mm-yyyy format
        try:
            test = fixtures[currentDate] #Basically to check if it exists
        except:
            fixtures[currentDate] = []

    #On the webpage, the fixtures are below each date, so fetch the date first and find all fixtures until the next date, so that
    these fixtures can be stored under the stored date
    for tag in soup.find_all(["h2", "span"]):

```

```

if tag.name == "h2": #This is where the date can be found
    text = tag.get_text(strip=True)
    currentDateText = text
    currentDate = getDate(monthYear, currentDateText)
    # Creates a list to hold fixtures of the current date. If already made by another league, this prevents it being
    overwritten
    try:
        test = fixtures[currentDate]
        if currentDate == "Error":
            raise Exception("Not Date")
    except:
        fixtures[currentDate] = []

elif tag.name == "span" and "versus" in tag.get_text(): #Where fixtures are stored
    if currentDate != "Error":
        text = tag.get_text(strip=True)
        parts = text.split("versus")
        home = parts[0].strip()
        away_time = parts[1].strip()

        # Extract time if present
        if "kick off" in away_time:
            away, time = away_time.split(" kick off ")
            time = time.strip()
        else:
            away, time = away_time, "TBC"

        # Store fixture data for me
        fixtures[currentDate].append({
            "league": tierName,
            "home": home,
            "away": away.strip(),
            "time": time
        })

return fixtures

# Main function for scraping fixtures and storing them in the database
def main(databaseFile, leaguesFile):
    fixtures = {}

    # Load league tiers and their BBC identifiers
    with open(leaguesFile, "r") as leaguesFileObject:
        tiers = json.load(leaguesFileObject)
        dateAdditions = getNextTwelveMonths()

    #Results in a dictionary of lists containing dictionaries in the format : {Date: [{"league": "A", "home": "B", "away": "C",
    "time": "D"}]}
    for tierName in tiers.keys():
        tierID = tiers[tierName]
        for dateAddition in dateAdditions:
            url = f"https://www.bbc.co.uk/sport/football/{tierID}/scores-fixtures/{dateAddition}?filter=fixtures"
            fixtures = getPageFixtures(url, dateAddition, tierName, fixtures)

    # Catch errors returned by safeGet
    if type(fixtures) is int:
        return fixtures

    # Convert raw fixture dictionaries into class objects
    # This allows filtering and database storage using class methods
    dateObjectList = []
    for dateString in fixtures.keys():
        listOfFixtureObjects = []

```

```

for fixtureDictionary in fixtures[dateString]:
    league = fixtureDictionary["league"]
    home = fixtureDictionary["home"]
    away = fixtureDictionary["away"]
    time = fixtureDictionary["time"]

    fixtureObject = classes.Fixture(league, home, away, time)
    listOfFixtureObjects.append(fixtureObject)

dateObject = classes.Date(dateString, listOfFixtureObjects)
dateObjectList.append(dateObject)

# AllDates object contains all Date objects and applies filtering logic
AllDatesObject = classes.AllDates(dateObjectList)

# Remove non-London fixtures using the Teams table
AllDatesObject.findRelevantDates(databaseFile)
AllDatesObject.storeInDatabase(databaseFile)

return True

if __name__ == '__main__':
    message = main("final.db", "leagues.json")
    print(message)

```

scraperClasses.py

```

import sqlite3

#GLOBAL PARAMETERS
#Duration of a match including breaks
DURATION = 120 # Minutes

# Fixture object acts as a container for scraped fixture data for me
class Fixture():
    def __init__(self, league, home, away, time):
        self.league = league # League the fixture belongs to (e.g. Premier League)
        self.home = home # Home team
        self.away = away # Away team
        self.time = time # Kick-off time (or TBC)

#Date object contains the date and a list of fixture objects
#Date object's findRelevantFixtures method deletes all fixtures not in London
class Date():
    def __init__(self, date, fixtures):
        self.date = date # Date in dd-mm-yyyy format
        self.fixtures = fixtures # List of Fixture objects on this date

# Removes all fixtures whose home team is not a London club
def findRelevantFixtures(self, londonClubs):
    relevantFixtures = []

    for fixture in self.fixtures:
        home = fixture.home

```

```

London = False

#Check if home is in London
if home in londonClubs:
    London = True

if London:
    relevantFixtures.append(fixture)

# Replace fixtures list with only relevant fixtures
self.fixtures = relevantFixtures

#Contains a list of date objects
class AllDates():
    def __init__(self, dates):
        self.dates = dates # List of Date objects

    # Filters out dates that do not contain any London fixtures and the fixtures on that date that are not relevant
    def findRelevantDates(self, databaseFile):
        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()
        londonClubs = []

        cursor.execute("""
            SELECT TeamName
            FROM Teams
        """)
        rows = cursor.fetchall()
        for clubTuple in rows:
            londonClubs.append(clubTuple[0])

        relevantDates = []
        for dateObject in self.dates:
            # Filter fixtures within the date
            dateObject.findRelevantFixtures(londonClubs) #Filters out all non-relevant

            # Only keep dates that still contain at least one fixture
            if len(dateObject.fixtures) != 0:
                relevantDates.append(dateObject) #This means at least one event on this date is in London
        self.dates = relevantDates

        connection.close()

# Stores all remaining fixtures into the Events table
def storeInDatabase(self, databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    # Convert date format from dd-mm-yyyy to dd/mm/yyyy
    for dateObject in self.dates:
        Date = dateObject.date
        day, month, year = Date.split("-")
        Date = f"{day}/{month}/{year}"
        for fixture in dateObject.fixtures:
            HomeTeam = fixture.home
            AwayTeam = fixture.away
            Time = fixture.time
            Name = fixture.league

        # Fetch the venue associated with the home team

```

```

        cursor.execute("""
        SELECT VenueName
        FROM Teams
        WHERE TeamName = ?
        """, (HomeTeam,))
        VenueName = cursor.fetchall()[0][0]

        # Store the event in the Events table
        # Duration is fixed as an assumption
        cursor.execute("""
        INSERT OR REPLACE INTO Events (VenueName, Date, Time, HomeTeam, AwayTeam, EventName, Duration)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (VenueName, Date, Time, HomeTeam, AwayTeam, Name, DURATION),)

        connection.commit()

    connection.close()

```

storeStadiumData.py

```

import sqlite3
from opencage.geocoder import OpenCageGeocode
import json
from geopy.distance import geodesic

# Use of the API to get coordinates
def getCoordinates(key, location):
    geocoder = OpenCageGeocode(key)

    # The first result is used as it is assumed to be the most relevant and correct
    results = geocoder.geocode(location)[0][["geometry"]] #Where you can get the coordinates
    return results

# Finds all stations within walking distance of a venue
# Ensures at least one station is always returned (the closest one)
def getCloseStations(allStations, venueCoordinates, walkingDistance):
    stationWithDistance = []
    CloseStationDictionary = {}

    # Calculate distance from the venue to every station
    for stationTuple in allStations:
        stationCoordinates = (stationTuple[1], stationTuple[2])
        distanceKM = geodesic(venueCoordinates, stationCoordinates).km
        stationWithDistance.append((stationTuple[0], distanceKM))

    # Sort stations by distance from the venue
    stationWithDistance.sort(key=lambda x: x[1])

    # Always include the closest station to guarantee at least one station per venue
    firstStation = stationWithDistance.pop(0)
    CloseStationDictionary[firstStation[0]] = firstStation[1]

    # Add all stations within the walking distance threshold
    for station in stationWithDistance:
        if station[1] <= walkingDistance:
            CloseStationDictionary[station[0]] = station[1]

    return CloseStationDictionary

```

```

# Inserts stadium, team, and venue-station relationship data into the database
def insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    latitude = coordinates["lat"]
    longitude = coordinates["lng"]
    venueCoordinates = (latitude, longitude)

    cursor.execute("""
        SELECT NaPTAN, Latitude, Longitude
        FROM Stations
    """)
    allStations = cursor.fetchall()

    # Find stations that are within walking distance of the stadium
    closeStations = getCloseStations(allStations, venueCoordinates, walkingDistance)

    # Store the venue-station relationships and walking distances
    for stationID in closeStations.keys():
        distance = closeStations[stationID]
        cursor.execute("""
            INSERT INTO VenueStationRelationships
            VALUES (?, ?, ?)
            ON CONFLICT (VenueName, NaPTAN) DO
            UPDATE SET
                Distance = excluded.Distance
        """, (stadiumName, stationID, distance))

    # Store team and home venue relationship
    cursor.execute("""
        INSERT INTO Teams (TeamName, VenueName)
        VALUES (?, ?)
        ON CONFLICT (TeamName)
        DO UPDATE SET
            VenueName = excluded.VenueName
    """, (club, stadiumName))

    # Store venue details
    cursor.execute("""
        INSERT INTO Venues (VenueName, Capacity, Latitude, Longitude)
        Values (?, ?, ?, ?)
        ON CONFLICT (VenueName)
        DO UPDATE SET
            Capacity = excluded.Capacity,
            Latitude = excluded.Latitude,
            Longitude = excluded.Longitude
    """, (stadiumName, capacity, latitude, longitude))

    connection.commit()
    connection.close()

# Uses JSON file of club data
#{ "ClubName": [StadiumName, Capacity] }
def getStadiumData(londonClubsFile, databaseFile, key, walkingDistance):
    with open(londonClubsFile, "r") as file:
        londonClubs = json.load(file)

    # Process every London club and its stadium
    for club, data in londonClubs.items():
        stadiumName, capacity = data
        try:

```

```

        coordinates = getCoordinates(key, stadiumName)
    except:
        # Return error message if OpenCage API fails
        return "Opencage Geocoder API"
    insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance)

    return True

# Calculates the maximum walking distance using walking speed and walking time
def calculateWalkingDistance(walkingTime, walkingSpeed):
    # walkingSpeed is in km/h and walkingTime is in minutes
    distance = (walkingSpeed * walkingTime) / 60
    return distance

# Main function for fetching and storing stadium related data for me
def main(key, londonClubsFile, databaseFile, walkingTime, walkingSpeed):
    # Calculate what is considered a reasonable walking distance
    walkingDistance = calculateWalkingDistance(walkingTime, walkingSpeed)

    done = getStadiumData(londonClubsFile, databaseFile, key, walkingDistance)
    print(done)

    return done

# Recalculates and reinserts venue-station relationships when walking parameters change
def redoStore(walkingTime, walkingSpeed, londonClubsFile, databaseFile):
    walkingDistance = calculateWalkingDistance(walkingTime, walkingSpeed)

    with open(londonClubsFile, "r") as file:
        londonClubs = json.load(file)

    # Clear existing venue-station relationships
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()
    cursor.execute("DELETE FROM VenueStationRelationships")
    connection.commit()
    connection.close()

    # Reinsert relationships using updated walking distance
    for club, data in londonClubs.items():
        stadiumName, capacity = data

        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()

        # Fetch stored venue coordinates to avoid unnecessary API calls
        cursor.execute("""
            SELECT Latitude, Longitude
            FROM Venues
            WHERE VenueName = ?
            """, (stadiumName,))

        coordinates = {}
        coordinates["lat"], coordinates["lng"] = cursor.fetchall()[0]

        insertData(databaseFile, club, stadiumName, capacity, coordinates, walkingDistance)

        connection.close()

if __name__ == '__main__':

```

```
key = "eb0e2c9b71cc45f7aafe0ae4ecc44cc2"
londonClubsFile = "londonClubs.json"
databaseFile = "final.db"
main(key, londonClubsFile, databaseFile, 15, 5)
```

transportClasses.py

```
from abc import ABC, abstractmethod

# Abstract base class for all transport-related components
# Holds common attributes used for crowding and delay simulation
class TransportClass(ABC):
    def __init__(self):
        self.currentPassengerLoad = 0 #initially set to 0 and is the extra number of people present due to the events
        self.DelayFactor = 1 # initially set to 1 and is the factor by which the time to travel would be multiplied by to simulate crowding

    @abstractmethod
    def IsAffected(self):
        pass

# Represents a transport line consisting of multiple connections
# Responsible for spreading delay across all connections on the line
class Line():
    def __init__(self, connectionObjects, lineID):
        self.connections = connectionObjects
        self.numberOfConnections = 0
        self.TotalDelay = 0
        self.AverageDelay = 0
        self.lineID = lineID

    # Counts how many connections exist on this line
    def calculateTotalNumberOfConnections(self):
        self.numberOfConnections = len(self.connections)

    # Calculates the total delay factor across all connections
    def calculateTotalDelay(self):
        totalDelay = 0
        for connection in self.connections:
            totalDelay += connection.DelayFactor
        self.TotalDelay = totalDelay

    # Computes the average delay factor for the line
    def calculateAverageDelay(self):
        averageDelay = self.TotalDelay / self.numberOfConnections
        self.AverageDelay = averageDelay

    # Applies the average delay back onto each connection
    def spreadDelay(self):
        for connection in self.connections:
            connection.LineDelay = self.AverageDelay
```

```

# Performs all line-level delay calculations in sequence
def doAllCalculations(self):
    self.calculateTotalNumberOfConnections()
    self.calculateTotalDelay()
    self.calculateAverageDelay()
    self.spreadDelay()

# Base class for all station types
class Station(TransportClass):
    def __init__(self, NaPTAN, StationName):
        super().__init__()
        self.NaPTAN = NaPTAN
        self.StationName = StationName

    def IsAffected(self):
        pass #IsAffected() is intentionally deferred to concrete subclasses - AffectedDijkstraStation returns True,
        NotAffectedDijkstraStation returns False

# Station class used during database creation
# Includes geographic coordinates
class CreationStation(Station):
    def __init__(self, NaPTAN, StationName, Latitude, Longitude):
        super().__init__(NaPTAN, StationName)
        self.Latitude = Latitude
        self.Longitude = Longitude

    def IsAffected(self):
        pass

#Serves as the parent class of the affected and not affected dijkstra station classes
class DijkstraStation(Station):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)

    def IsAffected(self):
        pass #IsAffected() is intentionally deferred to concrete subclasses - AffectedDijkstraStation returns True,
        NotAffectedDijkstraStation returns False

# Represents a station affected by nearby events
class AffectedDijkstraStation(DijkstraStation):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)
        self.events = {} # {(VenueName, Date) : NumberOfPeople}

    def IsAffected(self):
        return True

# Calculates the total passenger load caused by all events
def calculatePassengerLoad(self):
    for event in self.events.keys():
        numberOfPeople = self.events[event]
        self.currentPassengerLoad += numberOfPeople

```

```

    return self.currentPassengerLoad

# Calculates the station delay factor based on passenger load
def calculateDelay(self, PERSON_DELAY):
    extraDelay = PERSON_DELAY * self.currentPassengerLoad
    self.DelayFactor += extraDelay

# Represents a station not affected by any events
class NotAffectedDijkstraStation(DijkstraStation):
    def __init__(self, NaPTAN, StationName):
        super().__init__(NaPTAN, StationName)

    def IsAffected(self):
        return False

# Parent class for all connection types used in Dijkstra's algorithm
class Connection(TransportClass):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime):
        super().__init__()
        self.StationA = StationA
        self.StationB = StationB
        self.LineID = LineID
        self.connectionReference = (self.StationA, self.StationB, self.LineID) #Unique Identifier
        self.BaseTravelTime = BaseTravelTime
        self.LineDelay = 0

    @abstractmethod
    def IsAffected(self):
        pass

    @abstractmethod
    def returnDelay(self):
        pass

# Connection that is not affected by events or crowding
class NotAffectedConnection(Connection):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime):
        super().__init__(StationA, StationB, LineID, BaseTravelTime)

    def IsAffected(self):
        return False

    def returnDelay(self): #Because it isn't affected
        self.DelayFactor = 1
        return 1

# Connection affected by delays at one or both stations
class AffectedConnection(Connection):
    def __init__(self, StationA, StationB, LineID, BaseTravelTime, StationADelayFactor, StationBDelayFactor):
        super().__init__(StationA, StationB, LineID, BaseTravelTime)

```

```

self.StationADelayFactor = StationADelayFactor
self.StationBDelayFactor = StationBDelayFactor

def IsAffected(self):
    return True

# Calculates delay as the average of both station delay factors -- use of polymorphism as the two connection types are handled differently
def returnDelay(self):
    averageDelayFactor = (self.StationADelayFactor + self.StationBDelayFactor) / 2
    self.DelayFactor = averageDelayFactor
    return averageDelayFactor

# Massive container for all the data for me about the graph
class Network():
    def __init__(self):
        self.nodes = {} #Dictionary of station objects with the key being the NaPTAN
        self.edges = {} #Dictionary of connection objects with the key being (StationA, StationB, LineID)
        self.SpreadOutEdges = {} #Used in an earlier version of the delay spreading algorithm - retained in case it is needed again in the future

```

websiteDataClasses.py

```

import sqlite3

# This class is used to format a path segment for display on the main page.
# Each instance represents a continuous segment of a journey on a single line,
# containing the stations travelled through on that line along with crowding descriptions.
class mainPagePathClass():
    def __init__(self, LineID):
        self.LineID = LineID # The internal LineID used in the database
        self.ListOfStations = [] # List of station NaPTANs in the order they are visited on this line
        self.LineName = None # Human-readable line name (e.g. "Central Line")
        self.ListOfStationNames = [] # List of [StationName, NaPTAN]
        self.ListOfStationNamesAndDescriptions = [] # Final display format including crowding description

    # Adds a station (identified by NaPTAN) to the list of stations for this line segment
    def addStation(self, Station):
        self.ListOfStations.append(Station)

    # Fetches the human-readable name of the line from the database and stores it in the LineName attribute
    def getLineName(self, databaseFile):
        connection = sqlite3.connect(databaseFile)
        cursor = connection.cursor()

        cursor.execute("""
            SELECT LineName
            FROM Lines
            WHERE LineID = ?
            """, (self.LineID,))

        LineName = cursor.fetchall()[0][0]

        connection.close()

        self.LineName = LineName

```

Converts the stored list of station NaPTANs into human-readable station names while preserving the NaPTAN for later lookups (e.g. crowding descriptions)

```
def getStationNames(self, databaseFile):
    connection = sqlite3.connect(databaseFile)
    cursor = connection.cursor()

    for i, NaPTAN in enumerate(self.ListOfStations):
        cursor.execute("""
            SELECT StationName
            FROM Stations
            WHERE NaPTAN = ?
            """, (NaPTAN,))
        result = cursor.fetchall()[0][0]

        # Stored as [StationName, NaPTAN] so both display and lookup are possible
        self.ListOfStationNames.append([result, NaPTAN])

    connection.close()

# Matches each station in this path segment with its crowding description
# descriptionList format: [[NaPTAN, Description], ...]
# Final format stored as: [StationName, NaPTAN, Description]
def getDescription(self, descriptionList):
    # Convert list into dictionary for fast lookup by NaPTAN
    descriptionDictionary = {station[0]: station[1] for station in descriptionList}

    for stationData in self.ListOfStationNames:
        description = descriptionDictionary[stationData[1]]
        data = [stationData[0], stationData[1], description]
        self.ListOfStationNamesAndDescriptions.append(data)
```